

Summary of GDB

The purpose of a debugger such as GDB is to allow you to see what is going on “inside” another program while it executes—or what another program was doing at the moment it crashed.

GDB can do four main kinds of things (plus other things in support of these) to help you catch bugs in the act:

- Start your program, specifying anything that might affect its behavior.
- Make your program stop on specified conditions.
- Examine what has happened, when your program has stopped.
- Change things in your program, so you can experiment with correcting the effects of one bug and go on to learn about another.

You can use GDB to debug programs written in C and C++. For more information, see Section 15.4 [Supported Languages], page 233. For more information, see Section 15.4.1 [C and C++], page 233.

Support for D is partial. For information on D, see Section 15.4.2 [D], page 240.

Support for Modula-2 is partial. For information on Modula-2, see Section 15.4.9 [Modula-2], page 246.

Support for OpenCL C is partial. For information on OpenCL C, see Section 15.4.5 [OpenCL C], page 241.

Debugging Pascal programs which use sets, subranges, file variables, or nested functions does not currently work. GDB does not support entering expressions, printing values, or similar features using Pascal syntax.

GDB can be used to debug programs written in Fortran, although it may be necessary to refer to some variables with a trailing underscore.

GDB can be used to debug programs written in Objective-C, using either the Apple/NeXT or the GNU Objective-C runtime.

Free Software

GDB is *free software*, protected by the GNU General Public License (GPL). The GPL gives you the freedom to copy or adapt a licensed program—but every person getting a copy also gets with it the freedom to modify that copy (which means that they must get access to the source code), and the freedom to distribute further copies. Typical software companies use copyrights to limit your freedoms; the Free Software Foundation uses the GPL to preserve these freedoms.

Fundamentally, the General Public License is a license which says that you have these freedoms and that you cannot take these freedoms away from anyone else.

Free Software Needs Free Documentation

The biggest deficiency in the free software community today is not in the software—it is the lack of good free documentation that we can include with the free software. Many of our most important programs do not come with free reference manuals and free introductory

GDB的摘要

调试器（如 GDB）的目的是让你在程序执行时“查看”其内部情况——或者查看另一个程序在崩溃时正在做什么。

GDB 可以做四种主要的事情（以及支持这些事情的其他事情）来帮助你捕捉正在发生的错误：

- 启动你的程序，指定可能影响其行为的任何内容。
- 在指定条件下让你的程序停止。
- 在程序停止时检查发生了什么。
- 修改你的程序，以便你可以通过纠正一个bug的效果来实验，并继续学习关于另一个的内容。

你可以使用 GDB 来调试用C和C++编写的程序。更多信息，请参阅第15.4 [支持的语言], 页233。更多信息，请参阅第15.4.1 [C和C++], 页233。

对D的支持是有限的。有关D的信息，请参阅第15.4.2 [D], 页240。

对Modula-2的支持是有限的。有关 Modula-2，请参阅第15.4.9 [Modula-2], 第246页。

对OpenCL C的支持是有限的。有关OpenCL C的信息，请参阅第15.4.5[OpenCL C], 第241页。

调试使用集合、子范围、文件变量或嵌套函数的Pascal程序目前无法工作。GDB 不支持使用Pascal语法输入表达式、打印值或类似功能。

GDB 可用于调试Fortran编写的程序，尽管可能需要为某些变量添加带下划线的后缀。

GDB 可用于调试使用Apple/NeXT或GNUObjective-C运行时编写的Objective-C程序。

自由软件

GDB 是自由软件，受 GNU 通用公共许可证（GPL）保护。GPL赋予你复制或修改许可程序的自由——但获得副本的每个人也同时获得修改该副本的自由（这意味着他们必须能够获取源代码），以及分发更多副本的自由。典型的软件公司使用版权来限制你的自由；自由软件基金会使用GPL来保护这些自由。

从根本上说，通用公共许可证是一种许可协议，它表明你拥有这些自由，并且你不能从其他人那里剥夺这些自由。

自由软件需要自由文档

今天，自由软件社区最大的缺陷不在于软件本身——而在于缺乏我们可以随自由软件一同提供的优质免费文档。我们许多最重要的程序都没有附带免费的参考手册和免费的入门指南。

texts. Documentation is an essential part of any software package; when an important free software package does not come with a free manual and a free tutorial, that is a major gap. We have many such gaps today.

Consider Perl, for instance. The tutorial manuals that people normally use are non-free. How did this come about? Because the authors of those manuals published them with restrictive terms—no copying, no modification, source files not available—which exclude them from the free software world.

That wasn't the first time this sort of thing happened, and it was far from the last. Many times we have heard a GNU user eagerly describe a manual that he is writing, his intended contribution to the community, only to learn that he had ruined everything by signing a publication contract to make it non-free.

Free documentation, like free software, is a matter of freedom, not price. The problem with the non-free manual is not that publishers charge a price for printed copies—that in itself is fine. (The Free Software Foundation sells printed copies of manuals, too.) The problem is the restrictions on the use of the manual. Free manuals are available in source code form, and give you permission to copy and modify. Non-free manuals do not allow this.

The criteria of freedom for a free manual are roughly the same as for free software. Redistribution (including the normal kinds of commercial redistribution) must be permitted, so that the manual can accompany every copy of the program, both on-line and on paper.

Permission for modification of the technical content is crucial too. When people modify the software, adding or changing features, if they are conscientious they will change the manual too—so they can provide accurate and clear documentation for the modified program. A manual that leaves you no choice but to write a new manual to document a changed version of the program is not really available to our community.

Some kinds of limits on the way modification is handled are acceptable. For example, requirements to preserve the original author's copyright notice, the distribution terms, or the list of authors, are ok. It is also no problem to require modified versions to include notice that they were modified. Even entire sections that may not be deleted or changed are acceptable, as long as they deal with nontechnical topics (like this one). These kinds of restrictions are acceptable because they don't obstruct the community's normal use of the manual.

However, it must be possible to modify all the *technical* content of the manual, and then distribute the result in all the usual media, through all the usual channels. Otherwise, the restrictions obstruct the use of the manual, it is not free, and we need another manual to replace it.

Please spread the word about this issue. Our community continues to lose manuals to proprietary publishing. If we spread the word that free software needs free reference manuals and free tutorials, perhaps the next person who wants to contribute by writing documentation will realize, before it is too late, that only free manuals contribute to the free software community.

If you are writing documentation, please insist on publishing it under the GNU Free Documentation License or another free documentation license. Remember that this decision requires your approval—you don't have to let the publisher decide. Some commercial publishers will use a free license if you insist, but they will not propose the option; it is up

文档是任何软件包的重要组成部分；当重要的自由软件包没有附带免费手册和免费教程时，这就是一个重大缺失。如今我们面临着许多这样的缺失。

以Perl为例。人们通常使用的教程手册都是非自由的。这是怎么回事呢？因为这些手册的作者以限制性条款发布它们——禁止复制、禁止修改、源文件不可用——这使得它们被排除在自由软件世界之外。

这并非第一次发生这种情况，而且也远非最后一次。我们多次听到GNU用户热情地描述他正在编写的手册，这是他对社区的预期贡献，结果却发现自己因为签署了使其成为非自由的出版合同而毁了一切。

自由文档，就像自由软件一样，是关于自由而非价格的问题。非自由手册的问题不在于出版商对印刷副本收费——这本身是没问题的。（自由软件基金会也出售手册的印刷副本。）问题在于使用手册的限制。自由手册以源代码形式提供，并允许你复制和修改。非自由手册不允许这样做。

自由手册的标准与自由软件大致相同。必须允许分发（包括常规的商业分发），以便手册可以随程序的所有副本一起提供，无论是线上还是纸质版。

修改技术内容的使用权也至关重要。当人们修改软件，增加或更改功能时，如果他们认真负责，也会更新手册——这样他们就能为修改后的程序提供准确清晰的文档。如果手册让你不得不编写新的手册来记录程序更改版本，那么它实际上并没有真正为我们社区提供便利。

在处理修改方式上的一些限制是可以接受的。例如，要求保留原始作者的版权声明、分发条款或作者列表，这些都是可以接受的。要求修改版本包含已修改的声明也没有问题。即使某些章节可能无法删除或更改，只要它们处理非技术主题（如本节），也是可以接受的。这类限制是可以接受的，因为它们不会阻碍社区正常使用手册。

然而，必须能够修改手册的所有技术内容，然后通过所有常规渠道以所有常规媒体分发结果。否则，限制阻碍了手册的使用，它不是免费的，我们需要另一份手册来替代它。

请就这个问题进行宣传。我们的社区持续因专有出版而失去手册。如果我们宣传自由软件需要自由参考手册和自由教程，也许下一个想通过编写文档来贡献的人，在为时已晚之前，会意识到只有自由手册才能为自由软件社区做出贡献。

如果你在编写文档，请坚持将其发布在GNU自由文档许可证或其他自由文档许可证下。请记住，这个决定需要你的批准——你不必让出版商来决定。一些商业出版商会在你坚持的情况下使用免费许可证，但他们不会主动提出这个选项；这一切取决于你，需要你提出问题并坚定地说出你的要求。如果你正在处理的出版商拒绝，请尝试其他出版商。如果你不确定所提议的许可证是否为免费许可证，请写信给licensing@gnu.org。

to you to raise the issue and say firmly that this is what you want. If the publisher you are dealing with refuses, please try other publishers. If you're not sure whether a proposed license is free, write to licensing@gnu.org.

You can encourage commercial publishers to sell more free, copylefted manuals and tutorials by buying them, and particularly by buying copies from the publishers that paid for their writing or for major improvements. Meanwhile, try to avoid buying non-free documentation at all. Check the distribution terms of a manual before you buy it, and insist that whoever seeks your business must respect your freedom. Check the history of the book, and try to reward the publishers that have paid or pay the authors to work on it.

The Free Software Foundation maintains a list of free documentation published by other publishers, at <http://www.fsf.org/doc/other-free-books.html>.

Contributors to GDB

Richard Stallman was the original author of GDB, and of many other GNU programs. Many others have contributed to its development. This section attempts to credit major contributors. One of the virtues of free software is that everyone is free to contribute to it; with regret, we cannot actually acknowledge everyone here. The file `ChangeLog` in the GDB distribution approximates a blow-by-blow account.

Changes much prior to version 2.0 are lost in the mists of time.

Plea: Additions to this section are particularly welcome. If you or your friends (or enemies, to be evenhanded) have been unfairly omitted from this list, we would like to add your names!

So that they may not regard their many labors as thankless, we particularly thank those who shepherded GDB through major releases: Andrew Cagney (releases 6.3, 6.2, 6.1, 6.0, 5.3, 5.2, 5.1 and 5.0); Jim Blandy (release 4.18); Jason Molenda (release 4.17); Stan Shebs (release 4.14); Fred Fish (releases 4.16, 4.15, 4.13, 4.12, 4.11, 4.10, and 4.9); Stu Grossman and John Gilmore (releases 4.8, 4.7, 4.6, 4.5, and 4.4); John Gilmore (releases 4.3, 4.2, 4.1, 4.0, and 3.9); Jim Kingdon (releases 3.5, 3.4, and 3.3); and Randy Smith (releases 3.2, 3.1, and 3.0).

Richard Stallman, assisted at various times by Peter TerMaat, Chris Hanson, and Richard Mlynarik, handled releases through 2.8.

Michael Tiemann is the author of most of the GNU C++ support in GDB, with significant additional contributions from Per Bothner and Daniel Berlin. James Clark wrote the GNU C++ demangler. Early work on C++ was by Peter TerMaat (who also did much general update work leading to release 3.0).

GDB uses the BFD subroutine library to examine multiple object-file formats; BFD was a joint project of David V. Henkel-Wallace, Rich Pixley, Steve Chamberlain, and John Gilmore.

David Johnson wrote the original COFF support; Pace Willison did the original support for encapsulated COFF.

Brent Benson of Harris Computer Systems contributed DWARF 2 support.

Adam de Boor and Bradley Davis contributed the ISI Optimum V support. Per Bothner, Noboyuki Hikichi, and Alessandro Forin contributed MIPS support. Jean-Daniel Fekete contributed Sun 386i support. Chris Hanson improved the HP9000 support. Noboyuki

如果你不确定所提议的许可证是否为免费许可证, 请写信给licensing@gnu.org。

你可以通过购买来鼓励商业出版商销售更多免费的、采用Copyleft许可的手册和教程, 特别是购买那些为编写或重大改进这些内容支付费用的出版商的副本。同时, 尽量避免购买非自由的文档。购买手册前, 请检查其分发条款, 并坚持要求任何寻求你生意的人都必须尊重你的自由。检查书籍的历史, 并尝试奖励那些支付费用或向作者支付以使其工作的出版商。

自由软件基金会维护着一个由其他出版商发布的自由文档列表, 网址为 <http://www.fsf.org/doc/other-free-books.html>。

GDB 贡献者

理查德·斯托曼是 GDB 的原始作者, 也是许多其他 GNU 程序的作者。许多其他人也为其开发做出了贡献。本节试图表彰主要贡献者。自由软件的一个优点是每个人都可以为其做出贡献; 我们对此深感遗憾, 无法在此一一列举。GDB 发行版中的 `ChangeLog` 文件大致记录了详细过程。

版本 2.0 之前的许多更改已消散在时光长河中。

恳请: 本节新增内容尤为欢迎。如果你或你的朋友(或为了公平起见, 还包括敌人)未被列入此名单, 我们希望能添加你的名字!

为此, 我们特别感谢那些在重大版本发布中悉心引导 GDB 的人: Andrew Cagney (发布版本 6.3、6.2、6.1、6.0、5.3、5.2、5.1 和 5.0); Jim Blandy (发布版本 4.18); Jason Molenda (发布版本 4.17); Stan Shebs (发布版本 4.14); Fred Fish (发布版本 4.16、4.15、4.13、4.12、4.11、4.10 和 4.9); Stu Grossman 和 John Gilmore (发布版本 4.8、4.7、4.6、4.5 和 4.4); John Gilmore (发布版本 4.3、4.2、4.1、4.0 和 3.9); Jim Kingdon (发布版本 3.5、3.4 和 3.3); 以及 Randy Smith (发布版本 3.2、3.1 和 3.0)。

理查德·斯托曼, 以及彼得·特马特、克里斯·汉森和理查德·姆林尼克, 负责了 2.8 版本之前的发布工作。

迈克尔·蒂曼是 GNU C++ 在 GDB 中的主要作者, 佩尔·博斯纳和丹尼尔·伯林也做出了重要贡献。詹姆斯·克拉克编写了 GNU C++ 反汇编器。早期的 C++ 工作由彼得·特马特(他还负责了大量通用更新工作, 最终促成了 3.0 版本的发布)。

GDB 使用 BFD 子程序库来检查多种目标文件格式; BFD 是一个由大卫·V·亨克尔-华莱士、里奇·皮克利、史蒂夫·钱伯伦和约翰·吉尔莫共同的项目。

大卫·约翰逊编写了原始的 COFF 支持; 佩斯·威廉森负责了封装 COFF 的原始支持。

哈里斯计算机系统公司的布伦特·本森贡献了 DWARF 2 支持。

Adam de Boor 和 Bradley Davis 贡献了 ISI Optimum V 支持。根据 Bothner、Noboyuki Hikichi 和 Alessandro Forin 的贡献, 提供了 MIPS 支持。Jean-Daniel Fekete 贡献了 Sun 386i 支持。Chris Hanson 改进了 HP9000 支持。Noboyuki

Hikichi and Tomoyuki Hasei contributed Sony/News OS 3 support. David Johnson contributed Encore Umax support. Jyrki Kuoppala contributed Altos 3068 support. Jeff Law contributed HP PA and SOM support. Keith Packard contributed NS32K support. Doug Rabson contributed Acorn Risc Machine support. Bob Rusk contributed Harris Nighthawk CX-UX support. Chris Smith contributed Convex support (and Fortran debugging). Jonathan Stone contributed Pyramid support. Michael Tiemann contributed SPARC support. Tim Tucker contributed support for the Gould NP1 and Gould Powernode. Pace Willison contributed Intel 386 support. Jay Vosburgh contributed Symmetry support. Marko Mlinar contributed OpenRISC 1000 support.

Andreas Schwab contributed M68K GNU/Linux support.

Rich Schaefer and Peter Schauer helped with support of SunOS shared libraries.

Jay Fenlason and Roland McGrath ensured that GDB and GAS agree about several machine instruction sets.

Patrick Duval, Ted Goldstein, Vikram Koka and Glenn Engel helped develop remote debugging. Intel Corporation, Wind River Systems, AMD, and ARM contributed remote debugging modules for the i960, VxWorks, A29K UDI, and RDI targets, respectively.

Brian Fox is the author of the readline libraries providing command-line editing and command history.

Andrew Beers of SUNY Buffalo wrote the language-switching code, the Modula-2 support, and contributed the Languages chapter of this manual.

Fred Fish wrote most of the support for Unix System Vr4. He also enhanced the command-completion support to cover C++ overloaded symbols.

Hitachi America (now Renesas America), Ltd. sponsored the support for H8/300, H8/500, and Super-H processors.

NEC sponsored the support for the v850, Vr4xxx, and Vr5xxx processors.

Mitsubishi (now Renesas) sponsored the support for D10V, D30V, and M32R/D processors.

Toshiba sponsored the support for the TX39 Mips processor.

Matsushita sponsored the support for the MN10200 and MN10300 processors.

Fujitsu sponsored the support for SPARClite and FR30 processors.

Kung Hsu, Jeff Law, and Rick Sladkey added support for hardware watchpoints.

Michael Snyder added support for tracepoints.

Stu Grossman wrote gdbserver.

Jim Kingdon, Peter Schauer, Ian Taylor, and Stu Grossman made nearly innumerable bug fixes and cleanups throughout GDB.

The following people at the Hewlett-Packard Company contributed support for the PA-RISC 2.0 architecture, HP-UX 10.20, 10.30, and 11.0 (narrow mode), HP's implementation of kernel threads, HP's aC++ compiler, and the Text User Interface (nee Terminal User Interface): Ben Krepp, Richard Title, John Bishop, Susan Macchia, Kathy Mann, Satish Pai, India Paul, Steve Rehrauer, and Elena Zannoni. Kim Haase provided HP-specific information in this manual.

DJ Delorie ported GDB to MS-DOS, for the DJGPP project. Robert Hoehne made significant contributions to the DJGPP port.

Hikichi 和 Tomoyuki Hasei 贡献了 Sony/News OS 3 支持。David Johnson 贡献了 Encore Umax 支持。Jyrki Kuoppala 贡献了 Altos 3068 支持。Jeff Law 贡献了 HP PA 和 SOM 支持。Keith Packard 贡献了 NS32K 支持。Doug Rabson 贡献了 Acorn Risc Machine 支持。Bob Rusk 贡献了 Harris Nighthawk CX-UX 支持。Chris Smith 贡献了 Convex 支持 (以及 Fortran debugging)。Jonathan Stone 贡献了 Pyramid 支持。迈克尔·蒂曼 (Michael Tiemann) 贡献了 SPARC 支持。蒂姆·塔克 (Tim Tucker) 贡献了 Gould NP1 和 Gould Powernode 的支持。佩斯·威尔逊 (Pace Willison) 贡献了 Intel 386 支持。杰伊·沃斯伯格 (Jay Vosburgh) 贡献了 Symmetry 支持。马克奥·米林纳尔 (Marko Mlinar) 贡献了 OpenRISC 1000 支持。

安德烈亚斯·施瓦布 (Andreas Schwab) 贡献了 M68K GNU/Linux 支持。

理查德·沙费尔 (Rich Schaefer) 和彼得·绍尔 (Peter Schauer) 帮助支持了 SunOS 共享库。

杰伊·芬尔森 (Jay Fenlason) 和罗兰·麦克格雷思 (Roland McGrath) 确保了 GDB 和 GAS 就几种机器指令集达成一致。

帕特里克·杜瓦尔 (Patrick Duval)、特德·戈德斯坦 (Ted Goldstein)、维克拉姆·科卡 (Vikram Koka) 和格伦·恩格尔 (Glenn Engel) 帮助开发了远程调试。英特尔公司 (Intel Corporation)、风洞系统 (Wind River Systems)、AMD 和 ARM 分别贡献了用于 i960、VxWorks、A29K UDI 和 RDI 目标的远程调试模块。

Brian Fox 是 readline 库的作者，这些库提供了命令行编辑和命令历史功能。

Andrew Beers of SUNY Buffalo 编写了语言切换代码、Modula-2 支持，并为手册贡献了《语言》章节。

Fred Fish 编写了 Unix System Vr4 的大部分支持代码。他还增强了命令补全支持，以覆盖 C++ 重载符号。

Hitachi America (现更名为 Renesas America), Ltd. 赞助了 H8/300 的支持，H8/500 以及 Super-H 处理器。

NEC 赞助了 v850、Vr4xxx 和 Vr5xxx 处理器的支持。

三菱 (现瑞萨) 赞助了 D10V、D30V 和 M32R/D 处理器的支持，处理器。

东芝赞助了 TX39 MIPS 处理器的支持。

松下赞助了 MN10200 和 MN10300 处理器的支持。

富士通赞助了 SPARClite 和 FR30 处理器的支持。

孔苏、杰夫·劳和里克·斯拉德基添加了对硬件断点的支持。

迈克尔·斯奈德添加了对跟踪点的支持。

斯图·格罗斯曼编写了 gdbserver。

吉姆·金登、彼得·朔尔、伊恩·泰勒和斯图·格罗斯曼做出了无数 GDB 全面的错误修复和清理。

惠普公司 (Hewlett-Packard Company) 的以下人员为 PA-RISC 2.0 架构、HP-UX 10.20、10.30 和 11.0 (窄模式)、HP 的内核线程实现、HP 的 aC++ 编译器以及文本用户界面 (原终端用户界面) 提供了支持: 本·克雷普 (Ben Krepp)、理查德·泰勒 (Richard Title)、约翰·布莱斯 (John Bishop)、苏珊·麦克奇亚 (Susan Macchia)、凯西·曼 (Kathy Mann)、萨蒂什·派 (Satish Pai)、印度·保罗 (India Paul)、史蒂夫·雷赫劳尔 (Steve Rehrauer) 和叶莲娜·扎诺尼 (Elena Zannoni)。金·哈斯 (Kim Haase) 在本手册中提供了 HP 特定的信息。

DJ Delorie 将 GDB 移植到 MS-DOS, 用于 DJGPP 项目。Robert Hoehne 做出了对 DJGPP 移植的重大贡献。

Cygnus Solutions has sponsored GDB maintenance and much of its development since 1991. Cygnus engineers who have worked on GDB fulltime include Mark Alexander, Jim Blandy, Per Bothner, Kevin Buettner, Edith Epstein, Chris Faylor, Fred Fish, Martin Hunt, Jim Ingham, John Gilmore, Stu Grossman, Kung Hsu, Jim Kingdon, John Metzler, Fernando Nasser, Geoffrey Noer, Dawn Perchik, Rich Pixley, Zdenek Radouch, Keith Seitz, Stan Shebs, David Taylor, and Elena Zannoni. In addition, Dave Brolley, Ian Carmichael, Steve Chamberlain, Nick Clifton, JT Conklin, Stan Cox, DJ Delorie, Ulrich Drepper, Frank Eigler, Doug Evans, Sean Fagan, David Henkel-Wallace, Richard Henderson, Jeff Holcomb, Jeff Law, Jim Lemke, Tom Lord, Bob Manson, Michael Meissner, Jason Merrill, Catherine Moore, Drew Moseley, Ken Raeburn, Gavin Romig-Koch, Rob Savoye, Jamie Smith, Mike Stump, Ian Taylor, Angela Thomas, Michael Tiemann, Tom Tromey, Ron Unrau, Jim Wilson, and David Zuhn have made contributions both large and small.

Andrew Cagney, Fernando Nasser, and Elena Zannoni, while working for Cygnus Solutions, implemented the original GDB/MI interface.

Jim Blandy added support for preprocessor macros, while working for Red Hat.

Andrew Cagney designed GDB's architecture vector. Many people including Andrew Cagney, Stephane Carrez, Randolph Chung, Nick Duffek, Richard Henderson, Mark Kettenis, Grace Sainsbury, Kei Sakamoto, Yoshinori Sato, Michael Snyder, Andreas Schwab, Jason Thorpe, Corinna Vinschen, Ulrich Weigand, and Elena Zannoni, helped with the migration of old architectures to this new framework.

Andrew Cagney completely re-designed and re-implemented GDB's unwinder framework, this consisting of a fresh new design featuring frame IDs, independent frame sniffers, and the sentinel frame. Mark Kettenis implemented the DWARF 2 unwinder, Jeff Johnston the libunwind unwinder, and Andrew Cagney the dummy, sentinel, tramp, and trad unwinders. The architecture-specific changes, each involving a complete rewrite of the architecture's frame code, were carried out by Jim Blandy, Joel Brobecker, Kevin Buettner, Andrew Cagney, Stephane Carrez, Randolph Chung, Orjan Friberg, Richard Henderson, Daniel Jacobowitz, Jeff Johnston, Mark Kettenis, Theodore A. Roth, Kei Sakamoto, Yoshinori Sato, Michael Snyder, Corinna Vinschen, and Ulrich Weigand.

Christian Zankel, Ross Morley, Bob Wilson, and Maxim Grigoriev from Tensilica, Inc. contributed support for Xtensa processors. Others who have worked on the Xtensa port of GDB in the past include Steve Tjiang, John Newlin, and Scott Foehner.

Michael Eager and staff of Xilinx, Inc., contributed support for the Xilinx MicroBlaze architecture.

Initial support for the FreeBSD/mips target and native configuration was developed by SRI International and the University of Cambridge Computer Laboratory under DARPA/AFRL contract FA8750-10-C-0237 ("CTSRD"), as part of the DARPA CRASH research programme.

Initial support for the FreeBSD/riscv target and native configuration was developed by SRI International and the University of Cambridge Computer Laboratory (Department of Computer Science and Technology) under DARPA contract HR0011-18-C-0016 ("ECATS"), as part of the DARPA SSITH research programme.

The original port to the OpenRISC 1000 is believed to be due to Alessandro Forin and Per Bothner. More recent ports have been the work of Jeremy Bennett, Franck Jullien, Stefan Wallentowitz and Stafford Horne.

自1991年以来, 赛格诺斯解决方案公司 (Cygnus Solutions) 一直赞助 GDB 的维护并负责其大部分开发工作。赛格诺斯的全职工程师包括马克·亚历山大 (Mark Alexander)、吉姆·布兰迪 (Jim Blandy)、佩尔·博特纳 (Per Bothner)、凯文·布特纳 (Kevin Buettner)、艾迪斯·爱泼斯坦 (Edith Epstein)、克里斯·费勒 (Chris Faylor)、弗雷德·菲什 (Fred Fish)、马丁·亨特 (Martin Hunt)、吉姆·英厄姆 (Jim Ingham)、约翰·吉尔莫 (John Gilmore)、斯图·格雷曼 (Stu Grossman)、孔·许 (Kung Hsu)、吉姆·金德农 (Jim Kingdon)、约翰·梅茨勒 (John Metzler)、费尔南多·纳斯尔 (Fernando Nasser)、杰弗里·诺尔 (Geoffrey Noer)、道恩·珀奇克 (Dawn Perchik)、里奇·皮克利 (Rich Pixley)、泽丹·拉杜奇 (Zdenek Radouch)、基思·赛茨 (Keith Seitz)、斯坦·谢布斯 (Stan Shebs)、大卫·泰勒 (David Taylor) 和叶莲娜·扎诺尼 (Elena Zannoni)。此外, 还有戴夫·布罗利 (Dave Brolley)、伊恩·卡迈克尔 (Ian Carmichael)、

Steve Chamberlain、Nick Clifton、JT Conklin、Stan Cox、DJ Delorie、Ulrich Drepper、Frank Eigler、Doug Evans、Sean Fagan、David Henkel-Wallace、Richard Henderson、Jeff Holcomb、Jeff Law、Jim Lemke、Tom Lord、Bob Manson、Michael Meissner、Jason Merrill、Catherine Moore、Drew Moseley、Ken Raeburn、Gavin Romig-Koch、Rob Savoye、Jamie Smith、Mike Stump、Ian Taylor、Angela Thomas、Michael Tiemann、Tom Tromey、Ron Unrau、Jim Wilson, 以及DavidZuhn都做出了大小不一的贡献。

Andrew Cagney、Fernando Nasser, 以及Elena Zannoni在为Cygnus Solutions工作时, 实现了最初的 GDB/MI 接口。

Jim Blandy在为Red Hat工作时, 添加了对预处理宏的支持。

Andrew Cagney设计了 GDB的架构向量。Andrew Cagney、Stephane Carrez、Randolph Chung、Nick Duffek、Richard Henderson、Mark Kettenis、Grace Sainsbury、Kei Sakamoto、Yoshinori Sato、Michael Snyder、Andreas Schwab、Jason Thorpe、Corinna Vinschen、Ulrich Weigand, 以及Elena Zannoni等许多人, 帮助将旧的架构迁移到这个新框架。

Andrew Cagney 完全重新设计了 GDB 的 unwinder 框架,

这包括一个全新的设计, 具有框架 ID、独立的框架嗅探器和哨兵框架。Mark Kettenis 实现了 DWARF 2 unwinder, Jeff Johnston 实现了 libunwind unwinder, Andrew Cagney 实现了 dummy、哨兵、tramp 和 trad unwinders。与架构相关的更改, 每个更改都涉及对架构的框架代码的完整重写, 由 Jim Blandy、Joel Brobecker、Kevin Buettner、Andrew Cagney、Stephane Carrez、Randolph Chung、Orjan Friberg、Richard Henderson、Daniel Jacobowitz、Jeff Johnston、Mark Kettenis、Theodore A. Roth、Kei Sakamoto、Yoshinori Sato、Michael Snyder、Corinna Vinschen 和 Ulrich Weigand 负责。

来自 Tensilica, Inc. 的 Christian Zankel、Ross Morley、Bob Wilson 和 Maxim Grigoriev 为 Xtensa 处理器器贡献了支持。过去参与 {v1} Xtensa 端口开发的其他人包括 Steve Tjiang、John Newlin 和 Scott Foehner。

Sato, Michael Snyder, Corinna Vinschen和Ulrich Weigand。

来自 Tensilica, Inc. 的 Christian Zankel、Ross Morley、Bob Wilson 和 Maxim Grigoriev 为 Xtensa 处理器贡献了支持。过去参与 GDB Xtensa 端口开发的其他人包括 Steve Tjiang、John Newlin 和 Scott Foehner。

Michael Eager 和 Xilinx, Inc. 的员工为 Xilinx MicroBlaze 架构提供了支持。

FreeBSD/mips 目标的初始支持和本地配置是由 SRI International 和剑桥大学计算机实验室根据 DARPA/AFRL 合同 FA8750-10-C-0237 ("CTSRD") 开发的, 作为 DARPA CRASH 研究计划的一部分。

FreeBSD/riscv 目标的初始支持和本地配置是由 SRI International 和剑桥大学计算机实验室 (计算机科学与技术系) 根据 DARPA 合同 HR0011-18-C-0016 ("ECATS") 开发的, 作为 DARPA SSITH 研究计划的一部分。

OpenRISC 1000 的原始移植据信是由 Alessandro Forin 和 Per Bothner 完成的。较新的移植工作则由 Jeremy Bennett、Franck Jullien、Stefan Wallentowitz 和 Stafford Horne 负责。

Weimin Pan, David Faust and Jose E. Marchesi contributed support for the Linux kernel BPF virtual architecture. This work was sponsored by Oracle.

Weimin Pan、David Faust 和 Jose E. Marchesi 为 Linux 内核 BPF 虚拟架构贡献了支持。这项工作由 Oracle 赞助。

1 A Sample GDB Session

You can use this manual at your leisure to read all about GDB. However, a handful of commands are enough to get started using the debugger. This chapter illustrates those commands.

In this sample session, we emphasize user input like this: **input**, to make it easier to pick out from the surrounding output.

One of the preliminary versions of GNU m4 (a generic macro processor) exhibits the following bug: sometimes, when we change its quote strings from the default, the commands used to capture one macro definition within another stop working. In the following short m4 session, we define a macro `foo` which expands to `0000`; we then use the m4 built-in `defn` to define `bar` as the same thing. However, when we change the open quote string to `<QUOTE>` and the close quote string to `<UNQUOTE>`, the same procedure fails to define a new synonym `baz`:

```
$ cd gnu/m4
$ ./m4
define(foo,0000)

foo
0000
define(bar,defn('foo'))

bar
0000
changequote(<QUOTE>,<UNQUOTE>)

define(baz,defn(<QUOTE>foo<UNQUOTE>))
baz
Ctrl-d
m4: End of input: 0: fatal error: EOF in string
```

Let us use GDB to try to see what is going on.

```
$ gdb m4
GDB is free software and you are welcome to distribute copies
of it under certain conditions; type "show copying" to see
the conditions.
There is absolutely no warranty for GDB; type "show warranty"
for details.

GDB 18.0.50.20260503-git, Copyright 1999 Free Software Foundation, Inc...
(gdb)
```

GDB reads only enough symbol data to know where to find the rest when needed; as a result, the first prompt comes up very quickly. We now tell GDB to use a narrower display width than usual, so that examples fit in this manual.

```
(gdb) set width 70
```

We need to see how the m4 built-in `changequote` works. Having looked at the source, we know the relevant subroutine is `m4_changequote`, so we set a breakpoint there with the GDB `break` command.

```
(gdb) break m4_changequote
Breakpoint 1 at 0x62f4: file builtin.c, line 879.
```

1 一个示例 GDB 会话

您可以在空闲时使用此手册详细了解 GDB。但是，只需少数几个命令即可开始使用调试器。本章介绍了这些命令。

在此示例会话中，我们强调用户输入，如：输入，以便更容易地从周围输出中识别出来。

GNU m4（一个通用宏处理器）的早期版本之一存在以下错误：有时，当我们将其引号字符串从默认值更改为其他值时，用于在另一个宏定义中捕获宏定义的命令将停止工作。在以下简短的 m4 会话中，我们定义了一个宏 `foo`，它展开为 `0000`；然后我们使用 m4 内置的 `defn` 将 `bar` 定义为相同的内容。但是，当我们更改开引号字符串为 `<QUOTE>` 并将闭引号字符串更改为 `<UNQUOTE>` 时，相同的步骤无法定义新的同义词 `baz`：

```
$ cd gnu/m4
$ ./m4
define(foo,0000)

foo
0000
define(bar,defn( 'foo' ))

bar
0000
changequote(<QUOTE>,<UNQUOTE>)

define(baz,defn(<QUOTE>foo<UNQUOTE>))
baz
Ctrl-d
m4: 输入结束: 0: 致命错误: 字符串中遇到EOF
```

让我们使用 GDB 尝试看看发生了什么。

```
$ gdb m4
GDB is free software and you are welcome to distribute copies
of it under certain conditions; type "show copying" to see
the conditions.
对于 GDB没有任何保证; 输入 "show warranty" 查看详细信息。

GDB 18.0.50.20260503-git, 版权所有 1999 Free Software Foundation, Inc... (gdb)
```

GDB 只读取足够的符号数据以知道在需要时在哪里找到其余部分；因此，第一个提示出现得非常快。我们现在告诉 GDB 使用比平时更窄的显示宽度，以便示例能适应本手册。

```
(gdb) set width 70
```

我们需要看看 m4 内置的 `changequote` 是如何工作的。查看源代码后，我们知道相关的子程序是 `m4_changequote`，所以我们用 GDB `break` 命令在那里设置了一个断点。

```
(gdb) break m4_changequote
断点 1 在 0x62f4 处: 文件 builtin.c, 第 879 行。
```


Using the `run` command, we start `m4` running under GDB control; as long as control does not reach the `m4_changequote` subroutine, the program runs as usual:

```
(gdb) run
Starting program: /work/Editorial/gdb/gnu/m4/m4
define(foo,0000)

foo
0000
```

To trigger the breakpoint, we call `changequote`. GDB suspends execution of `m4`, displaying information about the context where it stops.

```
changequote(<QUOTE>,<UNQUOTE>)

Breakpoint 1, m4_changequote (argc=3, argv=0x33c70)
at builtin.c:879
879      if (bad_argc(TOKEN_DATA_TEXT(argv[0]),argc,1,3))
```

Now we use the command `n` (`next`) to advance execution to the next line of the current function.

```
(gdb) n
882      set_quotes((argc >= 2) ? TOKEN_DATA_TEXT(argv[1])\
: nil,
```

`set_quotes` looks like a promising subroutine. We can go into it by using the command `s` (`step`) instead of `next`. `step` goes to the next line to be executed in *any* subroutine, so it steps into `set_quotes`.

```
(gdb) s
set_quotes (lq=0x34c78 "<QUOTE>", rq=0x34c88 "<UNQUOTE>")
at input.c:530
530      if (lquote != def_lquote)
```

The display that shows the subroutine where `m4` is now suspended (and its arguments) is called a stack frame display. It shows a summary of the stack. We can use the `backtrace` command (which can also be spelled `bt`), to see where we are in the stack as a whole: the `backtrace` command displays a stack frame for each active subroutine.

```
(gdb) bt
#0  set_quotes (lq=0x34c78 "<QUOTE>", rq=0x34c88 "<UNQUOTE>")
at input.c:530
#1  0x6344 in m4_changequote (argc=3, argv=0x33c70)
at builtin.c:882
#2  0x8174 in expand_macro (sym=0x33320) at macro.c:242
#3  0x7a88 in expand_token (obs=0x0, t=209696, td=0xf7ffa30)
at macro.c:71
#4  0x79dc in expand_input () at macro.c:40
#5  0x2930 in main (argc=0, argv=0xf7fffb20) at m4.c:195
```

We step through a few more lines to see what happens. The first two times, we can use `'s'`; the next two times we use `n` to avoid falling into the `xstrdup` subroutine.

```
(gdb) s
0x3b5c 532      if (rquote != def_rquote)
(gdb) s
0x3b80 535      lquote = (lq == nil || *lq == '\0') ? \
def_lquote : xstrdup(lq);
(gdb) n
536      rquote = (rq == nil || *rq == '\0') ? def_rquote\
: xstrdup(rq);
(gdb) n
```

使用 `run` 命令, 我们启动 `m4` 在 GDB 控制下运行; 只要控制没有到达 `m4_changequote` 子程序, 程序就正常运行:

```
(gdb) 运行 启动程序: /
work/Editorial/gdb/gnu/m4/m4 define(foo,0000)

foo
0000
```

要触发断点, 我们调用`changequote`。 GDB 会暂停`m4`的执行, 并显示它停止时的上下文信息。

```
changequote(<QUOTE>,<UNQUOTE>)

Breakpoint 1, m4_changequote (argc=3, argv=0x33c70)
at builtin.c:879
879      if (bad_argc(TOKEN_DATA_TEXT(argv[0]),argc,1,3))
```

现在我们使用命令 `n` (`next`) 将执行前进到当前函数的下一行。

```
(gdb) n
882      set_quotes((argc >= 2) ? TOKEN_DATA_TEXT(argv[1])\
: nil,
```

`set_quotes` 看起来像是一个有前景的子程序。我们可以通过使用命令 `s` (`step`) 而不是 `next` 进入它。 `step` 跳转到子程序的下一行执行, 所以它会进入 `set_quotes`。

```
(gdb) s
set_quotes (lq=0x34c78 "<QUOTE>", rq=0x34c88 "<UNQUOTE>")
at input.c:530
530      if (lquote != def_lquote)
```

显示 `m4` 当前挂起位置 (及其参数) 的界面称为栈帧显示。它显示栈的摘要。我们可以使用 `backtrace` 命令 (也可以拼写为 `bt`), 来查看我们在整个栈中的位置: `backtrace` 命令为每个活动的子程序显示一个栈帧。

```
(gdb) bt #0 set_quotes (lq=0x34c78 "<QUOTE>", rq=0x34c88 "<
UNQUOTE>") at input.c:530 #1 0x6344 in m4_changequote (argc=3,
argv=0x33c70) at builtin.c:882 #2 0x8174 in expand_macro (sym=0
x33320) at macro.c:242 #3 0x7a88 in expand_token (obs=0x0, t
=209696, td=0xf7ffa30) at macro.c:71 #4 0x79dc in expand_input
() at macro.c:40 #5 0x2930 in main (argc=0, argv=0xf7fffb20) at
m4.c:195
```

我们再往前看几行, 看看会发生什么。前两次, 我们可以使用 `'s'`; 接下来的两次, 我们使用 `n` 来避免掉入 `xstrdup` 子例程。

```
(gdb) s 0x3b5c 532 if (rquote!= def_rquote) (gdb) s 0x3b80
535 lquote = (lq ==nil || *lq == '\0' ) ? \def_lquote :
xstrdup(lq); (gdb) n 536 rquote =(rq ==nil || *rq == '\0' ) ? def_
rquote\ : xstrdup(rq); (gdb) n
```



```
538         len_lquote = strlen(rquote);
```

The last line displayed looks a little odd; we can examine the variables `lquote` and `rquote` to see if they are in fact the new left and right quotes we specified. We use the command `p (print)` to see their values.

```
(gdb) p lquote
$1 = 0x35d40 "<QUOTE>"
(gdb) p rquote
$2 = 0x35d50 "<UNQUOTE>"
```

`lquote` and `rquote` are indeed the new left and right quotes. To look at some context, we can display ten lines of source surrounding the current line with the `l (list)` command.

```
(gdb) l
533         xfree(rquote);
534
535         lquote = (lq == nil || *lq == '\0') ? def_lquote\
: xstrdup (lq);
536         rquote = (rq == nil || *rq == '\0') ? def_rquote\
: xstrdup (rq);
537
538         len_lquote = strlen(rquote);
539         len_rquote = strlen(lquote);
540     }
541
542     void
```

Let us step past the two lines that set `len_lquote` and `len_rquote`, and then examine the values of those variables.

```
(gdb) n
539         len_rquote = strlen(lquote);
(gdb) n
540     }
(gdb) p len_lquote
$3 = 9
(gdb) p len_rquote
$4 = 7
```

That certainly looks wrong, assuming `len_lquote` and `len_rquote` are meant to be the lengths of `lquote` and `rquote` respectively. We can set them to better values using the `p` command, since it can print the value of any expression—and that expression can include subroutine calls and assignments.

```
(gdb) p len_lquote==strlen(lquote)
$5 = 7
(gdb) p len_rquote==strlen(rquote)
$6 = 9
```

Is that enough to fix the problem of using the new quotes with the `m4` built-in `defn`? We can allow `m4` to continue executing with the `c (continue)` command, and then try the example that caused trouble initially:

```
(gdb) c
Continuing.

define(baz,defn(<QUOTE>foo<UNQUOTE>))

baz
0000
```

```
538         len_lquote = strlen(rquote);
```

最后一行显示看起来有点奇怪；我们可以检查变量 `lquote` 和 `rquote`，看看它们是否确实是我们指定的新左右引号。我们使用命令 `p (print)` 来查看它们的值。

```
(gdb) p lquote
$1 = 0x35d40 "<QUOTE>"
(gdb) p rquote
$2 = 0x35d50 "<UNQUOTE>"
```

`lquote` 和 `rquote` 确实是新的左右引号。要查看一些上下文，我们可以使用 `l (list)` 命令显示当前行周围的十行源代码。

```
(gdb) l
533         xfree(rquote);
534
535         lquote = (lq == nil || *lq == '\0') ? def_lquote
: xstrdup (lq);
536         rquote = (rq == nil || *rq == '\0') ? def_rquote
: xstrdup (rq);
537
538         len_lquote = strlen(rquote);
539         len_rquote = strlen(lquote);
540     }
541
542     void
```

让我们越过设置 `len_lquote` 和 `len_rquote` 的那两行代码，然后检查一下这些变量的值。

```
(gdb) n
539         len_rquote = strlen(lquote);
(gdb) n
540     }
(gdb) p len_lquote
$3 = 9
(gdb) p len_rquote
$4 = 7
```

这看起来显然是错的，假设 `len_lquote` 和 `len_rquote` 分别表示 `lquote` 和 `rquote` 的长度。我们可以使用 `p` 命令将它们设置为更好的值，因为该命令可以打印任何表达式的值——而该表达式可以包含子程序调用和赋值。

```
(gdb) p len_lquote==strlen(lquote)
$5 = 7
(gdb) p len_rquote==strlen(rquote)
$6 = 9
```

这足以解决使用新引号与 `m4` 内置的 `defn` 的问题吗？我们可以使用 `c (继续)` 命令允许 `m4` 继续执行，然后尝试最初导致问题的示例：

```
(gdb)
c 继续。

define(baz,defn(<QUOTE>foo<UNQUOTE>))

baz
0000
```

Success! The new quotes now work just as well as the default ones. The problem seems to have been just the two typos defining the wrong lengths. We allow `m4` exit by giving it an EOF as input:

```
Ctrl-d
Program exited normally.
```

The message ‘`Program exited normally.`’ is from GDB; it indicates `m4` has finished executing. We can end our GDB session with the GDB `quit` command.

```
(gdb) quit
```

成功！新的引用现在和默认的一样好用。问题似乎只是定义了错误的长度的那两个拼写错误。我们通过给 `m4` 输入 EOF 来允许它退出：

```
Ctrl-d
程序正常退出。
```

消息“程序正常退出。”来自 GDB；它表示 `m4` 已完成执行。我们可以使用 GDB `quit` 命令结束我们的 GDB 会话。

```
(gdb) 退出
```

2 Getting In and Out of GDB

This chapter discusses how to start GDB, and how to get out of it. The essentials are:

- type `'gdb'` to start GDB.
- type `quit`, `exit` or `Ctrl-d` to exit.

2.1 Invoking GDB

Invoke GDB by running the program `gdb`. Once started, GDB reads commands from the terminal until you tell it to exit.

You can also run `gdb` with a variety of arguments and options, to specify more of your debugging environment at the outset.

The command-line options described here are designed to cover a variety of situations; in some environments, some of these options may effectively be unavailable.

The most usual way to start GDB is with one argument, specifying an executable program:

```
gdb program
```

You can also start with both an executable program and a core file specified:

```
gdb program core
```

You can, instead, specify a process ID as a second argument or use option `-p`, if you want to debug a running process:

```
gdb program 1234
gdb -p 1234
```

would attach GDB to process 1234. With option `-p` you can omit the *program* filename.

Taking advantage of the second command-line argument requires a fairly complete operating system; when you use GDB as a remote debugger attached to a bare board, there may not be any notion of “process”, and there is often no way to get a core dump. GDB will warn you if it is unable to attach or to read core dumps.

You can optionally have `gdb` pass any arguments after the executable file to the inferior using `--args` or `--no-escape-args`. These options stop option processing.

```
gdb --args gcc -O2 -c foo.c
```

This will cause `gdb` to debug `gcc`, and to set `gcc`'s command-line arguments (see Section 4.3 [Arguments], page 38) to `'-O2 -c foo.c'`. For the differences between `--args` and `--no-escape-args`, see `[--args and --no-escape-args]`, page 15.

You can run `gdb` without printing the front material, which describes GDB's non-warranty, by specifying `--silent` (or `-q/--quiet`):

```
gdb --silent
```

You can further control how GDB starts up by using command-line options. GDB itself can remind you of the options available.

Type

```
gdb -help
```

to display all available options and briefly describe their use (`'gdb -h'` is a shorter equivalent).

All options and command line arguments you give are processed in sequential order. The order makes a difference when the `'-x'` option is used.

2 进入和退出 GDB

本章讨论如何启动 GDB，以及如何退出。要点如下：

- 输入 `'gdb'` 以启动 GDB。
- 输入 `quit`、`exit` 或 `Ctrl-d` 以退出。

2.1 调用 GDB

通过运行程序 `gdb` 调用 GDB。启动后，GDB 会从终端读取命令，直到你告诉它退出。

你也可以使用各种参数和选项运行 `gdb`，以便从一开始就指定更多调试环境。

这里描述的命令行选项旨在涵盖各种情况；在某些环境中，这些选项中的某些可能实际上无法使用。

GDB 最常用的启动方式是提供一个参数，指定可执行程序：`gdb program`

您也可以同时提供一个可执行程序和一个核心文件：

```
gdb program core
```

或者，您可以指定进程ID作为第二个参数，或使用选项 `-p` 来调试正在运行的进程：

```
gdb program 1234
gdb -p 1234
```

将 GDB 附加到处理 1234。使用选项 `-p` 可以省略程序文件名。

利用第二个命令行参数需要相当完整的操作系统；当你使用 GDB 作为连接到裸板的远程调试器时，可能没有“进程”的概念，而且通常没有办法获取核心转储。GDB 如果无法附加或读取核心转储，会向你发出警告。

你可以选择让 `gdb` 使用 `--args` 或 `--no-escape-args` 将可执行文件之后的任何参数传递给从属进程。这些选项会停止选项处理。

```
gdb --args gcc -O2 -c foo.c
```

这将导致 `gdb` 调试 `gcc`，并将 `gcc` 的命令行参数（参见第 4.3 [Arguments]，页 38）设置为 `'-O2 -c foo.c'`。有关 `--args` 和 `--no-escape-args` 的区别，请参阅 `[--args 和 --no-escape-args]`，页 15。

你可以通过指定 `--silent`（或 `-q/--quiet`）来运行 `gdb`，不打印描述 GDB 不提供担保的前言：

```
gdb --silent
```

您可以通过使用命令行选项进一步控制 GDB 的启动方式。GDB 本身可以提醒您可用的选项。

Type

```
gdb -help
```

以显示所有可用选项并简要描述其用途（`'gdb -h'` 是一个更短的等效选项）。

所有您提供的选项和命令行参数都将按顺序处理。当使用 `'-x'` 选项时，顺序会有所不同。

2.1.1 Choosing Files

When GDB starts, it reads any arguments other than options as specifying an executable file and core file (or process ID). This is the same as if the arguments were specified by the ‘-se’ and ‘-c’ (or ‘-p’) options respectively. (GDB reads the first argument that does not have an associated option flag as equivalent to the ‘-se’ option followed by that argument; and the second argument that does not have an associated option flag, if any, as equivalent to the ‘-c’/‘-p’ option followed by that argument.) If the second argument begins with a decimal digit, GDB will first attempt to attach to it as a process, and if that fails, attempt to open it as a corefile. If you have a corefile whose name begins with a digit, you can prevent GDB from treating it as a pid by prefixing it with ./, e.g. ./12345.

If GDB has not been configured to included core file support, such as for most embedded targets, then it will complain about a second argument and ignore it.

For the ‘-s’, ‘-e’, and ‘-se’ options, and their long form equivalents, the method used to search the file system for the symbol and/or executable file is the same as that used by the `file` command. See Section 18.1 [file], page 293.

Many options have both long and short forms; both are shown in the following list. GDB also recognizes the long forms if you truncate them, so long as enough of the option is present to be unambiguous. (If you prefer, you can flag option arguments with ‘--’ rather than ‘-’, though we illustrate the more usual convention.)

```
-symbols file
-s file    Read symbol table from file file.

-exec file
-e file    Use file file as the executable file to execute when appropriate, and for examining
           pure data in conjunction with a core dump.

-se file    Read symbol table from file file and use it as the executable file.

-core file
-c file    Use file file as a core dump to examine.

-pid number
-p number  Connect to process ID number, as with the attach command.

-command file
-x file    Execute commands from file file. The contents of this file is evaluated exactly
           as the source command would. See Section 23.1.3 [Command files], page 401.

-eval-command command
-ex command
           Execute a single GDB command.

           This option may be used multiple times to call multiple commands. It may also
           be interleaved with ‘-command’ as required.
           gdb -ex 'target sim' -ex 'load' \
             -x setbreakpoints -ex 'run' a.out

-init-command file
-ix file    Execute commands from file file before loading the inferior (but after loading
           gdbinit files). See Section 2.1.3 [Startup], page 17.
```

2.1.1 选择文件

当 GDB 启动时，它会将任何非选项参数视为指定可执行文件和核心文件（或进程ID）。这等同于使用 ‘-se’ 和 ‘-c’（或 ‘-p’）选项分别指定这些参数。（GDB 将第一个没有关联选项标志的参数视为等同于 ‘-se’ 选项后跟该参数；如果有，第二个没有关联选项标志的参数则视为等同于 ‘-c’ / ‘-p’ 选项后跟该参数。）如果第二个参数以十进制数字开头，GDB 将首先尝试将其作为进程附加，如果失败，则尝试将其作为核心文件打开。如果您有一个名称以数字开头的核心文件，可以通过在前面加上 ./ 来阻止 GDB 将其视为 PID，例如 ./12345。

如果 GDB 未配置以包含核心文件支持，例如大多数嵌入式目标，那么它将抱怨第二个参数并忽略它。

对于 ‘-s’、‘-e’ 和 ‘-se’ 选项及其长形式等效选项，用于在文件系统中搜索符号和/或可执行文件的方法与 `file` 命令使用的方法相同。参见第 18.1 [file] 页第 293 页。

许多选项都有长形式和短形式；两者都在以下列表中显示。GDB 也识别如果你截断它们的长形式，只要选项足够长以使其明确无误。（如果你更喜欢，你可以用 ‘--’ 而不是 ‘-’ 来标记选项参数，尽管我们说明更通常的约定。）

```
-symbols file
-s file    从文件file读取符号表。

-exec file
-e file    使用文件 file 作为可执行文件，在适当的时候执行，并与核心转储一起检查纯数
           据。

-se file    从文件 file 读取符号表，并将其用作可执行文件。

-core file
-c file    使用文件 file 作为核心转储来检查。

-pid number
-p number  连接到进程 ID 号，类似于 attach 命令。

-command file
-x file    从文件 file 执行命令。该文件的内容将按源命令完全执行。参见第 23.1 节.3 [命令文
           件] 第 401 页。

-eval-command command
-ex command
           执行单个 GDB 命令。

           此选项可以多次使用以调用多个命令。它也可以根据需要与 “-command” 交错
           使用。
           gdb -ex 'target sim' -ex 'load' \ -x
             setbreakpoints -ex 'run' a.out

-init-command file
-ix file    从文件 file 执行命令，在加载被调试进程之前（但在加载 gdbinit 文件之后）。参见
           第 2.1.3 [启动]，页面 17。
```

`-init-eval-command command`
`-iex command`
Execute a single GDB command before loading the inferior (but after loading gdbinit files). See Section 2.1.3 [Startup], page 17.

`-early-init-command file`
`-eix file` Execute commands from *file* very early in the initialization process, before any output is produced. See Section 2.1.3 [Startup], page 17.

`-early-init-eval-command command`
`-eiex command`
Execute a single GDB command very early in the initialization process, before any output is produced.

`-directory directory`
`-d directory`
Add *directory* to the path to search for source and script files.

`-r`
`-readnow` Read each symbol file’s entire symbol table immediately, rather than the default, which is to read it incrementally as it is needed. This makes startup slower, but makes future operations faster.

`--readnever`
Do not read each symbol file’s symbolic debug information. This makes startup faster but at the expense of not being able to perform symbolic debugging. DWARF unwind information is also not read, meaning backtraces may become incomplete or inaccurate. One use of this is when a user simply wants to do the following sequence: attach, dump core, detach. Loading the debugging information in this case is an unnecessary cause of delay.

2.1.2 Choosing Modes

You can run GDB in various alternative modes—for example, in batch mode or quiet mode.

`-nx`
`-n` Do not execute commands found in any initialization files (see Section 2.1.4 [Initialization Files], page 18).

`-nh` Do not execute commands found in any home directory initialization file (see Section 2.1.4 [Home directory initialization file], page 18). The system wide and current directory initialization files are still loaded.

`-quiet`
`-silent`
`-q` “Quiet”. Do not print the introductory and copyright messages. These messages are also suppressed in batch mode.

This can also be enabled using `set startup-quietly on`. The default is off. Use `show startup-quietly` to see the current setting. Place `set startup-quietly on` into your early initialization file (see Section 2.1.4 [Initialization Files], page 18) to have future GDB sessions startup quietly.

`-init-eval-command command`
`-iex command`
在加载从属环境之前（但在加载 gdbinit 文件之后）执行一个 GDB 命令。参见第 2.1.3 [启动], 页 17。

`-early-init-command file`
`-eix file` 在初始化过程的非常早期执行来自文件的命令，在任何输出产生之前。参见第 2.1.3 [启动], 页第 17 页。

`-early-init-eval-command command`
`-eiex command`
在初始化过程的非常早期执行一个 GDB 命令，在任何输出产生之前。

`-directory directory`
`-d directory`
将目录添加到搜索源文件和脚本文件的路径中。

`-r`
`-readnow` 立即读取每个符号文件的全部符号表，而不是默认的按需增量读取方式。这会使启动变慢，但会使后续操作变快。

`--readnever`
不要读取每个符号文件的符号调试信息。这会使启动更快，但代价是无法执行符号调试。DWARF 回溯信息也不会被读取，这意味着回溯可能变得不完整或不准确。这种情况的一个用途是当用户只想执行以下序列时：连接、转储核心、断开连接。在这种情况下，加载调试信息是造成延迟的不必要原因。

2.1.2 选择模式

你可以以多种替代模式运行 GDB ——例如，以批处理模式或安静模式运行。

`-nx`
`-n` 不要执行在任何初始化文件（参见第 2.1.4[初始化文件],页 18）中找到的命令。

`-nh` 不要执行在任何主目录初始化文件（参见第 2.1.4 [主目录初始化文件], 页 18）中找到的命令。系统范围和当前目录初始化文件仍然会被加载。

`-安静`
`-静音`
`-q` “安静”。不打印引导和版权信息。在批处理模式下，这些信息也会被抑制。

也可以通过设置 `startup-quietly on` 来启用。默认是关闭的。使用 `show startup-quietly` 查看当前设置。将 `set startup-quietly on` 放入早期初始化文件（参见第 2.1.4[初始化文件], 页 18）以使未来的 GDB 会话安静启动。

-batch Run in batch mode. Exit with status 0 after processing all the command files specified with ‘-x’ (and all commands from initialization files, if not inhibited with ‘-n’). Exit with nonzero status if an error occurs in executing the GDB commands in the command files. Batch mode also disables pagination, sets unlimited terminal width and height see Section 22.4 [Screen Size], page 374, and acts as if *set confirm off* were in effect (see Section 22.9 [Messages/Warnings], page 386).

Batch mode may be useful for running GDB as a filter, for example to download and run a program on another computer; in order to make this more useful, the message

Program exited normally.

(which is ordinarily issued whenever a program running under GDB control terminates) is not issued when running in batch mode.

-batch-silent

Run in batch mode exactly like ‘-batch’, but totally silently. All GDB output to **stdout** is prevented (**stderr** is unaffected). This is much quieter than ‘-silent’ and would be useless for an interactive session.

This is particularly useful when using targets that give ‘Loading section’ messages, for example.

Note that targets that give their output via GDB, as opposed to writing directly to **stdout**, will also be made silent.

-return-child-result

The return code from GDB will be the return code from the child process (the process being debugged), with the following exceptions:

- GDB exits abnormally. E.g., due to an incorrect argument or an internal error. In this case the exit code is the same as it would have been without ‘-return-child-result’.
- The user quits with an explicit value. E.g., ‘quit 1’.
- The child process never runs, or is not allowed to terminate, in which case the exit code will be -1.

This option is useful in conjunction with ‘-batch’ or ‘-batch-silent’, when GDB is being used as a remote program loader or simulator interface.

-nowindows

-nw “No windows”. If GDB comes with a graphical user interface (GUI) built in, then this option tells GDB to only use the command-line interface. If no GUI is available, this option has no effect.

-windows

-w If GDB includes a GUI, then this option requires it to be used if possible.

-cd directory

Run GDB using *directory* as its working directory, instead of the current directory.

-batch 以批处理模式运行。处理完所有用 ‘-x’ 指定的命令文件后退出，状态码为 0（如果未用 ‘-n’ 禁用，则还会处理初始化文件中的所有命令）。如果在执行命令文件中的 GDB 命令时发生错误，则退出时状态码非零。批处理模式还会禁用分页，设置无限的终端宽度和高度（参见第 22.4 [屏幕大小], 页 374），并表现得好像 *set confirm off* 生效一样（参见第 22.9 [消息/警告], 页 386）。

批处理模式可用于将 GDB 作为过滤器运行，例如下载并在另一台计算机上运行程序；为了使其更有用，消息

程序正常退出。

(通常在程序在 GDB 控制下终止时发出) 在批处理模式下不会发出。

-batch-silent

以 ‘-batch’ 完全相同的批量模式运行，但完全静默。所有 GDB 输出到标准输出被阻止（标准错误不受影响）。这比 ‘-silent’ 安静得多，对于交互式会话将毫无用处。

当使用会显示 ‘加载部分’ 消息的目标时，这特别有用，例如。

注意：通过 GDB 提供输出的目标（而不是直接写入标准输出）也将被静默处理。

-return-child-result

GDB 的返回代码将是子进程（被调试的进程）的返回代码，但有以下例外：

- GDB 异常退出。例如，由于参数错误或内部错误。在这种情况下，退出代码与不使用 ‘-return-child-result’ 时相同。
- 用户通过显式值退出。例如，‘quit 1’。
- 子进程从未运行，或被禁止终止，在这种情况下退出代码将是 -1。

此选项与 ‘-batch’ 或 ‘-batch-silent’ 一起使用时很有用，当 GDB 被用作远程程序加载器或模拟器接口时。

-nowindows

-nw “无窗口”。如果 GDB 内置了图形用户界面 (GUI)，则此选项指示 GDB 仅使用命令行界面。如果没有 GUI 可用，此选项无效。

-windows

-w 如果 GDB 包含图形用户界面，那么此选项在可能的情况下需要使用它。

-cd directory

使用 GDB 的目录作为其工作目录，而不是当前目录。

-data-directory *directory***-D** *directory*

Run GDB using *directory* as its data directory. The data directory is where GDB searches for its auxiliary files. See Section 18.8 [Data Files], page 312.

-binary-output

Instructs GDB to set the translation mode of its `stdout/stderr` to binary. MS-Windows only. Useful for running the GDB testsuite. By default, GDB opens `stdout/stderr` in text mode, and translates ‘\n’ (LF, a single Line Feed) into ‘\r\n’ (CR-LF, a Carriage Return-Line Feed combination). If this option is set, no translation is done.

-fullname

-f GNU Emacs sets this option when it runs GDB as a subprocess. It tells GDB to output the full file name and line number in a standard, recognizable fashion each time a stack frame is displayed (which includes each time your program stops). This recognizable format looks like two ‘\032’ characters, followed by the file name, line number and character position separated by colons, and a newline. The Emacs-to-GDB interface program uses the two ‘\032’ characters as a signal to display the source code for the frame.

-annotate *level*

This option sets the *annotation level* inside GDB. Its effect is identical to using ‘set annotate level’ (see Chapter 28 [Annotations], page 715). The *annotation level* controls how much information GDB prints together with its prompt, values of expressions, source lines, and other types of output. Level 0 is the normal, level 1 is for use when GDB is run as a subprocess of GNU Emacs, level 3 is the maximum annotation suitable for programs that control GDB, and level 2 has been deprecated.

The annotation mechanism has largely been superseded by GDB/MI (see Chapter 27 [GDB/MI], page 609).

--args

Change interpretation of command line so that arguments following the executable file are passed as command line arguments to the inferior. This option stops option processing.

Arguments supplied using **--args** will have backslashes applied to escape any special shell characters. This ensures that when the inferior starts it is passed arguments exactly as GDB receives them.

For example, consider the following command run under a shell:

```
$ gdb --args ls *.c
```

In this case the shell will expand `*.c` at the time GDB is invoked, not at the time that the inferior is invoked. As a result, if an additional `.c` file is created after GDB is started, but before the inferior is started, then the inferior will not show the file in its output; the list of matching files was resolved at the time GDB was started.

If you quote the `*` character used on the GDB command line argument then this will prevent the shell that starts GDB from expanding the `*.c` pattern, however, this quoting will also be passed to the shell that GDB invokes in order to start

-数据目录 *directory***-D** *directory*

使用 GDB 的目录作为其数据目录。数据目录是 GDB 搜索其辅助文件的地方。参见第 18.8 [数据文件], 页 312。

-二进制输出

指示 GDB 将其 `stdout/stderr` 的翻译模式设置为二进制。仅适用于 MS-Windows。用于运行 GDB 测试套件。默认情况下, GDB 以文本模式打开 `stdout/stderr`, 并将 ‘\n’ (LF, 单个换行符) 翻译为 ‘\r\n’ (CR-LF, 回车换行组合)。如果设置此选项, 则不会进行翻译。

-完整名称**-f**

GNU 当 Emacs 作为子进程运行时, 会设置此选项 GDB。它指示 GDB 每次显示栈帧时 (包括程序停止的每次) 以标准、可识别的格式输出完整文件名和行号。这种可识别的格式看起来像两个 ‘\032’ 字符, 后面跟着文件名、行号和字符位置, 用冒号分隔, 并换行。Emacs 与 -GDB 接口程序使用这两个 ‘\032’ 字符作为显示该栈帧源代码的信号。

-annotate *level*

此选项设置 GDB 内部的注释级别。其效果与使用 ‘set annotate level’ (见第 28 [章 注释], 页 715) 相同。注释级别控制 GDB 与其提示、表达式值、源代码行和其他类型输出一起打印的信息量。级别 0 是正常的, 级别 1 是在 GDB 作为 GNU Emacs 的子进程运行时使用的, 级别 3 是适合控制 GDB 的程序的 最大注释级别, 而级别 2 已被弃用。

注释机制在很大程度上已被 GDB/MI (参见第 27 [章 GDB/MI], 页 609) 所取代。

--args

更改命令行解释, 以便将可执行文件后面的参数作为命令行参数传递给子进程。此选项停止选项处理。

使用 **--args** 提供的参数会对任何特殊 shell 字符进行转义。这确保了当子进程启动时, 它接收到的参数与 GDB 完全一致。

例如, 考虑在 shell 下运行的以下命令:

```
$ gdb --args ls *.c
```

在这种情况下, shell 将在 GDB 被调用时扩展 `*.c`, 而不是在子进程被调用时扩展。因此, 如果 GDB 启动后但在子进程启动之前创建了附加的 `.c` 文件, 那么子进程将不会在其输出中显示该文件; 匹配文件的列表是在 GDB 启动时确定的。

如果你引用了在 GDB 命令行参数中使用的 `*` 字符, 那么这将阻止启动 GDB 的 shell 扩展 `*.c` 模式, 但是, 这种引用也会传递给 GDB 调用的以启动 shell

the inferior (see [set startup-with-shell], page 36), and this will prevent the `*.c` pattern being expanded at this point either:

```
$ gdb --args ls '*.c'
(gdb) show args
Argument list to give program being debugged when it is started is "*.log".
```

If this quoting behaviour does not meet your needs, then you could use `--no-escape-args` instead, which is described below.

`--no-escape-args`

Change interpretation of command line so that arguments following the executable file are passed as command line arguments to the inferior. This option stops option processing.

Unlike `--args`, arguments after the executable name will not have any escaping applied to them. As a result, any special shell characters that are not expanded by the shell that invokes GDB will be expanded by the shell that GDB uses to start the inferior.

```
$ gdb --no-escape-args ls '*.c'
(gdb) show args
Argument list to give program being debugged when it is started is "*.log".
```

`-baud bps`

`-b bps` Set the line speed (baud rate or bits per second) of any serial interface used by GDB for remote debugging.

`-l timeout`

Set the timeout (in seconds) of any communication used by GDB for remote debugging.

`-tty device`

`-t device` Run using *device* for your program's standard input and output.

`-tui`

Activate the *Text User Interface* when starting. The Text User Interface manages several text windows on the terminal, showing source, assembly, registers and GDB command outputs (see Chapter 25 [GDB Text User Interface], page 599). Do not use this option if you run GDB from Emacs (see Chapter 26 [Using GDB under GNU Emacs], page 607).

`-interpreter interp`

Use the interpreter *interp* for interface with the controlling program or device. This option is meant to be set by programs which communicate with GDB using it as a back end. See Chapter 24 [Command Interpreters], page 597.

`'--interpreter=mi'` (or `'--interpreter=mi3'`) causes GDB to use the GDB/MI interface version 3 (see Chapter 27 [The GDB/MI Interface], page 609) included since GDB version 9.1. GDB/MI version 2 (*mi2*), included in GDB 6.0 and version 1 (*mi1*), included in GDB 5.3, are also available. Earlier GDB/MI interfaces are no longer supported.

`-write`

Open the executable and core files for both reading and writing. This is equivalent to the `'set write on'` command inside GDB (see Section 17.6 [Patching], page 287).

的次级 shell (参见 [set startup-with-shell], 页面 36), 这将阻止 `*.c` 模式在此处展开:

```
$ gdb --args ls '*.c'
(gdb) show args
```

启动被调试程序时传递给它的参数列表是 `"*.log"`。如果这种引号行为不符合您的需求, 那么您可以使用 `--no-escape-args`, 它下面有描述。

`--no-escape-args`

更改命令行解释方式, 使得可执行文件之后的参数作为命令行参数传递给子进程。此选项会停止选项处理。

与 `--args` 不同, 可执行文件名之后的参数不会进行任何转义处理。因此, 任何在调用 GDB 的 shell 中未被展开的特殊 shell 字符, 都会在 GDB 用来启动子进程的 shell 中被展开。

```
$ gdb --no-escape-args ls '*.c'
(gdb) show args
启动被调试程序时传递给程序的参数列表是 "*.log".
```

`-baud bps`

`-b bps` 设置 GDB 使用的任何串行接口的线路速度 (波特率或每秒比特数), 用于远程调试。

`-l timeout`

设置 GDB 用于远程调试的任何通信的超时时间 (以秒为单位)。

`-tty device`

`-t device` 使用设备作为程序的标准化输入和输出。

`-tui`

启动时激活文本用户界面。文本用户界面在终端上管理多个文本窗口, 显示源代码、汇编代码、寄存器和 GDB 命令输出 (参见第 25 [GDB 章 文本用户界面], 第 599 页)。如果你从 GDB 在 Emacs 中运行, 则不要使用此选项 (参见第 26 [章 使用 GDB 在 GNU Emacs], 第 607 页)。

`-解释器 interp`

使用解释器 *interp* 与控制程序或设备进行接口。此选项用于由与 GDB 通信的程序设置, 将其用作后端。参见第 24 [命令解释器], 页 597。

`'--interpreter=mi'` (或 `'--interpreter=mi3'`) 使 GDB 使用 GDB/MI 接口版本 3 (参见第 27 [GDB/MI 接口], 页 609) 自 GDB 版本 9.1 起包含。GDB/MI 版本 2 (*mi2*), 包含在 GDB 6.0 中, 版本 1 (*mi1*), 包含在 GDB 5.3 中, 也可用。更早的 GDB/MI 接口不再受支持。

`-write` 打开可执行文件和核心文件进行读写。这相当于在 GDB 中执行 `"set write on"` 命令 (参见第 17.6 [Patching], 页 287 页)。

-statistics

This option causes GDB to print statistics about time and memory usage after it completes each command and returns to the prompt.

-version This option causes GDB to print its version number and no-warranty blurb, and exit.

-configuration

This option causes GDB to print details about its build-time configuration parameters, and then exit. These details can be important when reporting GDB bugs (see Chapter 32 [GDB Bugs], page 729).

2.1.3 What GDB Does During Startup

Here's the description of what GDB does during session startup:

1. Performs minimal setup required to initialize basic internal state.
2. Reads commands from the early initialization file (if any) in your home directory. Only a restricted set of commands can be placed into an early initialization file, see Section 2.1.4 [Initialization Files], page 18, for details.
3. Executes commands and command files specified by the `-eiex` and `-eix` command line options in their specified order. Only a restricted set of commands can be used with `-eiex` and `-eix`, see Section 2.1.4 [Initialization Files], page 18, for details.
4. Sets up the command interpreter as specified by the command line (see Section 2.1.2 [Mode Options], page 13).
5. Reads the system wide initialization file and the files from the system wide initialization directory, see [System Wide Init Files], page 19.
6. Reads the initialization file (if any) in your home directory and executes all the commands in that file, see [Home Directory Init File], page 19.
7. Executes commands and command files specified by the `-iex` and `-ix` options in their specified order. Usually you should use the `-ex` and `-x` options instead, but this way you can apply settings before GDB init files get executed and before inferior gets loaded.
8. Processes command line options and operands.
9. Reads and executes the commands from the initialization file (if any) in the current working directory as long as `set auto-load local-gdbinit` is set to `on` (see Section 22.8.1 [Init File in the Current Directory], page 383). This is only done if the current directory is different from your home directory. Thus, you can have more than one init file, one generic in your home directory, and another, specific to the program you are debugging, in the directory where you invoke GDB. See [Init File in the Current Directory during Startup], page 20.
10. If the command line specified a program to debug, or a process to attach to, or a core file, GDB loads any auto-loaded scripts provided for the program or for its loaded shared libraries. See Section 22.8 [Auto-loading], page 381.

If you wish to disable the auto-loading during startup, you must do something like the following:

```
$ gdb -iex "set auto-load python-scripts off" myprogram
```

Option `-ex` does not work because the auto-loading is then turned off too late.

- 统计信息

此选项会导致 GDB 在每个命令执行完毕并返回提示符后，打印关于时间和内存使用情况的统计信息。

- 版本号 此选项会导致 GDB 打印其版本号和免责声明，然后退出。

- 配置信息

此选项会导致 GDB 打印其构建时配置参数的详细信息，然后退出。当报告 GDB 错误时（见第 32 [章 GDB 错误]，页面 729），这些详细信息可能很重要。

2.1.3 GDB 启动时做什么

这是 GDB 在会话启动期间执行的操作描述：

1. 执行初始化基本内部状态所需的最小设置。
2. 读取您主目录中早期初始化文件（如果有）中的命令。只有一组受限的命令可以放入早期初始化文件，请参阅第2.1节.4 [初始化文件],页18，了解详情。
3. 按指定顺序执行由命令行选项 `-eiex` 和 `-eix` 指定的命令和命令文件。只有一组受限的命令可与 `-eiex` 和 `-eix` 一起使用，请参阅第2.1节.4 [初始化文件],页18，了解详情。
4. 根据命令行设置命令解释器（请参阅第2.1.2节.[模式选项],页13）。
5. 读取系统范围的初始化文件和系统范围初始化目录中的文件，请参阅 [系统范围初始化文件],页19。
6. 读取您主目录中的初始化文件（如果有），并执行该文件中的所有命令，参见 [主目录初始化文件], 页 19。
7. 按指定顺序执行由 `-iex` 和 `-ix` 选项指定的命令和命令文件。通常您应该使用 `-ex` 和 `-x` 选项，但这种方式可以在 GDB 初始化文件被执行以及 inferior 被加载之前应用设置。
8. 处理命令行选项和操作数。
9. 读取并执行当前工作目录中的初始化文件（如果有）的命令，只要 `set auto-load local-gdbinit` 设置为 `on`（参见第 22.8.1 [当前目录初始化文件], 页 383）。只有当当前目录与您的主目录不同时才会执行此操作。因此，您可以拥有多个初始化文件，一个通用文件位于您的主目录中，而另一个则位于您调用 GDB 的目录中，专门针对您正在调试的程序。参见 [启动时当前目录中的初始化文件], 页 20。
10. 如果命令行指定了要调试的程序，或要附加的进程，或核心文件，GDB 会加载为该程序或其加载的共享库提供的任何自动加载脚本。参见第 22.8 [自动加载], 页 381。

如果您希望在启动时禁用自动加载，您必须执行类似以下操作：

```
$ gdb -iex "set auto-load python-scripts off" myprogram
```

选项 `-ex` 无效，因为自动加载太晚被关闭了。

11. Executes commands and command files specified by the ‘-ex’ and ‘-x’ options in their specified order. See Section 23.1.3 [Command Files], page 401, for more details about GDB command files.
12. Reads the command history recorded in the *history file*. See Section 22.3 [Command History], page 372, for more details about the command history and the files where GDB records it.

2.1.4 Initialization Files

During startup (see Section 2.1.3 [Startup], page 17) GDB will execute commands from several initialization files. These initialization files use the same syntax as *command files* (see Section 23.1.3 [Command Files], page 401) and are processed by GDB in the same way.

To display the list of initialization files loaded by GDB at startup, in the order they will be loaded, you can use `gdb --help`.

The *early initialization* file is loaded very early in GDB’s initialization process, before the interpreter (see Chapter 24 [Interpreters], page 597) has been initialized, and before the default target (see Chapter 19 [Targets], page 313) is initialized. Only `set` or `source` commands should be placed into an early initialization file, and the only `set` commands that can be used are those that control how GDB starts up.

Commands that can be placed into an early initialization file will be documented as such throughout this manual. Any command that is not documented as being suitable for an early initialization file should instead be placed into a general initialization file. Command files passed to `--early-init-command` or `-eix` are also early initialization files, with the same command restrictions. Only commands that can appear in an early initialization file should be passed to `--early-init-eval-command` or `-eiex`.

In contrast, the *general initialization* files are processed later, after GDB has finished its own internal initialization process, any valid command can be used in these files.

Throughout the rest of this document the term *initialization file* refers to one of the general initialization files, not the early initialization file. Any discussion of the early initialization file will specifically mention that it is the early initialization file being discussed.

As the system wide and home directory initialization files are processed before most command line options, changes to settings (e.g. ‘`set complaints`’) can affect subsequent processing of command line options and operands.

The following sections describe where GDB looks for the early initialization and initialization files, and the order that the files are searched for.

2.1.4.1 Home directory early initialization files

GDB initially looks for an early initialization file in the users home directory¹. There are a number of locations that GDB will search in the home directory, these locations are searched in order and GDB will load the first file that it finds, and subsequent locations will not be checked.

On non-macOS hosts the locations searched are:

- The file `gdb/gdbearlyinit` within the directory pointed to by the environment variable `XDG_CONFIG_HOME`, if it is defined.

¹ On DOS/Windows systems, the home directory is the one pointed to by the `HOME` environment variable.

11. 执行 “-ex” 和 “-x” 选项指定的命令和命令文件，并按指定顺序执行。有关 GDB 命令文件]，的更多详细信息，请参阅第 23.1.3 [命令文件]，页 401。

12. 读取历史文件中记录的命令历史。参见第 22.3 [节命令历史]，第 372 页，有关命令历史和文件位置的更多信息 GDB 记录它。

2.1.4 初始化文件

在启动过程中（参见第 2.1.3 [启动]，页 17）GDB 将执行来自多个初始化文件的命令。这些初始化文件使用与命令文件相同的语法（参见第 23.1.3 [命令文件]，页 401），并以相同的方式由 GDB 处理。

要显示由 GDB 在启动时加载的初始化文件列表，并按加载顺序，可以使用 `gdb --help`。

早期初始化文件在 GDB 的初始化过程中非常早地被加载，在解释器（参见第 24 [章]，解释器 19 [页 597]）初始化之前，以及在默认目标（参见第 19 [章]，目标 GDB 页 313）初始化之前。早期初始化文件中应仅放置 `set` 或 `source` 命令，并且只能使用那些控制 GDB 如何启动的 `set` 命令。

可以放入早期初始化文件中的命令将在本手册中相应说明。任何未说明适用于早期初始化文件的命令，应放入通用初始化文件中。传递给 `--early-init-command` 或 `-eix` 的命令文件也是早期初始化文件，具有相同的命令限制。只有可以出现在早期初始化文件中的命令，才应传递给 `--early-init-eval-command` 或 `-eiex`。

相比之下，通用初始化文件会在 GDB 完成其自身的内部初始化过程之后进行处理，在这些文件中可以使用任何有效的命令。

在本文档的其余部分中，“初始化文件”一词指的是通用初始化文件之一，而不是早期初始化文件。任何关于早期初始化文件的讨论都会特别说明正在讨论的是早期初始化文件。

由于系统级和主目录初始化文件会在大多数命令行选项之前进行处理，因此设置更改（例如 ‘`set complaints`’）可能会影响后续命令行选项和操作数的处理。

以下部分描述了 GDB 查找早期初始化文件和初始化文件的位置，以及文件搜索的顺序。

2.1.4.1 主目录早期初始化文件

GDB 首先会在用户的主目录¹中查找早期初始化文件。GDB 会按顺序搜索主目录中的多个位置，GDB 会加载找到的第一个文件，后续位置将不会被检查。

在非 macOS 主机上，搜索的位置有：

- 环境变量 `XD_CONFIG_HOME` 指向的目录中，如果存在文件 `gdb/gdbearlyinit`，则该文件将被使用。

¹ 在 DOS/Windows 系统上，主目录是 `HOME` 环境变量指向的目录。

- The file `.config/gdb/gdbearlyinit` within the directory pointed to by the environment variable `HOME`, if it is defined.
- The file `.gdbearlyinit` within the directory pointed to by the environment variable `HOME`, if it is defined.

By contrast, on macOS hosts the locations searched are:

- The file `Library/Preferences/gdb/gdbearlyinit` within the directory pointed to by the environment variable `HOME`, if it is defined.
- The file `.gdbearlyinit` within the directory pointed to by the environment variable `HOME`, if it is defined.

It is possible to prevent the home directory early initialization file from being loaded using the `-nx` or `-nh` command line options, see Section 2.1.2 [Choosing Modes], page 13.

2.1.4.2 System wide initialization files

There are two locations that are searched for system wide initialization files. Both of these locations are always checked:

system.gdbinit

This is a single system-wide initialization file. Its location is specified with the `--with-system-gdbinit` configure option (see Section C.6 [System-wide configuration], page 775). It is loaded first when GDB starts, before command line options have been processed.

system.gdbinit.d

This is the system-wide initialization directory. Its location is specified with the `--with-system-gdbinit-dir` configure option (see Section C.6 [System-wide configuration], page 775). Files in this directory are loaded in alphabetical order immediately after `system.gdbinit` (if enabled) when GDB starts, before command line options have been processed. Files need to have a recognized scripting language extension (`.py/.scm`) or be named with a `.gdb` extension to be interpreted as regular GDB commands. GDB will not recurse into any subdirectories of this directory.

It is possible to prevent the system wide initialization files from being loaded using the `-nx` command line option, see Section 2.1.2 [Choosing Modes], page 13.

2.1.4.3 Home directory initialization file

After loading the system wide initialization files GDB will look for an initialization file in the users home directory². There are a number of locations that GDB will search in the home directory, these locations are searched in order and GDB will load the first file that it finds, and subsequent locations will not be checked.

On non-Apple hosts the locations searched are:

```
$XDG_CONFIG_HOME/gdb/gdbinit
$HOME/.config/gdb/gdbinit
$HOME/.gdbinit
```

While on Apple hosts the locations searched are:

² On DOS/Windows systems, the home directory is the one pointed to by the `HOME` environment variable.

- 目录中指向由环境变量 `HOME` 指向的 `.config/gdb/gdbearlyinit` 文件如果它被定义了，如果已定义。
- 环境变量指向的目录中 `.gdbearlyinit` 文件 `HOME`，如果它被定义了。

相比之下，在 macOS 主机上搜索的位置是：

- 环境变量 `HOME` 所指向的目录中的 `Library/Preferences/gdb/gdbearlyinit` 文件，如果它已被定义。
- 环境变量 `HOME` 所指向的目录中的 `.gdbearlyinit` 文件，如果它被定义。

可以使用 `-nx` 或 `-nh` 命令行选项来阻止加载主目录的早期初始化文件，参见第 2.1 节. 2 [选择模式]，第 13 页。

2.1.4.2 全局初始化文件

系统全局初始化文件会在两个位置进行搜索。这两个位置总是会进行检查：

system.gdbinit

这是一个全局系统初始化文件。其位置通过 `--with-system-gdbinit` 配置选项指定（参见第 C.6 [系统配置]，节第 775 页）。在 GDB 启动时，它将首先被加载，此时命令行选项尚未处理。

system.gdbinit.d

这是一个全局系统初始化目录。其位置通过 `--with-system-gdbinit-dir` 配置选项指定（参见第 C.6 [系统配置]，节第 775 页）。此目录中的文件会在 6 [启动时，在命令行选项处理之前，按字母顺序立即加载（如果已启用）。文件需要具有已识别的脚本语言扩展 (`.py/.scm`) 或以 `.gdb` 扩展名命名。

脚本语言扩展 (`.py/.scm`) 或以 `.gdb` 扩展名命名。应将其解释为添加到此目录的任何子目录中。

可以使用 `-nx` 命令行选项来阻止加载系统范围的初始化文件，参见第 2.1.2 [选择模式]，页 13。

2.1.4.3 主目录初始化文件

加载系统范围的初始化文件后，GDB 将查找用户主目录中的初始化文件²。GDB 将在主目录中搜索多个位置，这些位置按顺序搜索，GDB 将加载找到的第一个文件，后续位置将不会被检查。

在非 Apple 主机上，搜索的位置是：

```
$XDG_CONFIG_HOME/gdb/gdbinit
$HOME/.config/gdb/gdbinit
$HOME/.gdbinit
```

在 Apple 主机上，搜索的位置有：

² 在 DOS/Windows 系统上，主目录是 `HOME` 环境变量指向的那个目录。

```
$HOME/Library/Preferences/gdb/gdbinit
$HOME/.gdbinit
```

It is possible to prevent the home directory initialization file from being loaded using the ‘-nx’ or ‘-nh’ command line options, see Section 2.1.2 [Choosing Modes], page 13.

The DJGPP port of GDB uses the name `gdb.ini` instead of `.gdbinit` or `gdbinit`, due to the limitations of file names imposed by DOS filesystems. The Windows port of GDB uses the standard name, but if it finds a `gdb.ini` file in your home directory, it warns you about that and suggests to rename the file to the standard name.

2.1.4.4 Local directory initialization file

GDB will check the current directory for a file called `.gdbinit`. It is loaded last, after command line options other than ‘-x’ and ‘-ex’ have been processed. The command line options ‘-x’ and ‘-ex’ are processed last, after `.gdbinit` has been loaded, see Section 2.1.1 [Choosing Files], page 12.

If the file in the current directory was already loaded as the home directory initialization file then it will not be loaded a second time.

It is possible to prevent the local directory initialization file from being loaded using the ‘-nx’ command line option, see Section 2.1.2 [Choosing Modes], page 13.

2.2 Quitting GDB

```
quit [expression]
exit [expression]
```

q To exit GDB, use the `quit` command (abbreviated `q`), the `exit` command, or type an end-of-file character (usually `Ctrl-d`). If you do not supply *expression*, GDB will terminate normally; otherwise it will terminate using the result of *expression* as the error code.

An interrupt (often `Ctrl-c`) does not exit from GDB, but rather terminates the action of any GDB command that is in progress and returns to GDB command level. It is safe to type the interrupt character at any time because GDB does not allow it to take effect until a time when it is safe.

If you have been using GDB to control an attached process or device, you can release it with the `detach` command (see Section 4.7 [Debugging an Already-running Process], page 41).

2.3 Shell Commands

If you need to execute occasional shell commands during your debugging session, there is no need to leave or suspend GDB; you can just use the `shell` command.

```
shell command-string
!command-string
```

Invoke a shell to execute *command-string*. Note that no space is needed between `!` and *command-string*. On GNU and Unix systems, the environment variable `SHELL`, if it exists, determines which shell to run. Otherwise GDB uses the default shell (`/bin/sh` on GNU and Unix systems, `cmd.exe` on MS-Windows, `COMMAND.COM` on MS-DOS, etc.).

```
$HOME/Library/Preferences/gdb/gdbinit
$HOME/.gdbinit
```

可以使用 ‘-nx’ 或 ‘-nh’ 命令行选项来阻止加载主目录初始化文件，参见第2.1节.2 [选择模式], 页13。

GDB 的DJGPP端口由于DOS文件系统对文件名施加的限制，使用`gdb.ini`名称而不是`gdbinit`或`gdbinit`。GDB的Windows端口使用标准名称，但如果它在您的主目录中找到一个`gdb.ini`文件，它会警告您并建议将文件重命名为标准名称。

2.1.4.4 本地目录初始化文件

GDB 将检查当前目录中是否存在名为`.gdbinit`的文件。它在处理完除‘-x’和‘-ex’以外的命令行选项后最后加载。命令行选项‘-x’和‘-ex’在加载`gdbinit`之后最后处理，参见第2.1.1[选择文件],页第12页。

如果当前目录中的文件已被作为主目录初始化文件加载，则它不会再次加载。

使用 ‘-nx’ 命令行选项可以防止加载本地目录初始化文件，参见第2.1节.2 [选择模式], 页13。

2.2 退出 GDB

```
quit [expression]
exit [expression]
```

q 要退出 GDB，使用`quit`命令（缩写`q`）、`exit`命令，或输入文件结束字符（通常是 `Ctrl-d`）。如果不提供表达式,GDB 将正常终止；否则将使用表达式结果作为错误码终止。

中断（通常 `Ctrl-c`）不会从 GDB 退出，而是终止正在进行的任何 GDB 命令，并返回到 GDB 命令级别。在任意时间输入中断字符是安全的，因为 `Ctrl-c` 不会在安全时点之前让它生效。因为 GDB 不允许它在安全时点之前生效，所以可以在任何时间输入中断字符。

如果你一直在使用 GDB 控制连接的处理器设备，可以使用`detach`命令（参见第4.7 [调试正在运行的进程],页41）释放它。

2.3 Shell 命令

如果你需要在调试过程中偶尔执行 shell 命令，无需离开或挂起 GDB；你只需使用 `shell` 命令即可。

```
shell command-string
!command-string
```

调用外壳执行命令字符串。注意！和命令字符串之间不需要空格。在GNU和 Unix系统上，如果环境变量`SHELL`存在，它将决定运行哪个外壳。否则 GDB 使用默认外壳（GNU和Unix系统上是`/bin/sh`，MS-Windows上是`cmd.exe`，

`COMMAND.COM` 在 MS-DOS 等系统上）。

You may also invoke shell commands from expressions, using the `$_shell` convenience function. See [\$_shell convenience function], page 179.

The utility `make` is often needed in development environments. You do not have to use the `shell` command for this purpose in GDB:

`make make-args`
Execute the `make` program with the specified arguments. This is equivalent to `'shell make make-args'`.

`pipe [command] | shell_command`
`| [command] | shell_command`
`pipe -d delim command delim shell_command`
`| -d delim command delim shell_command`
Executes `command` and sends its output to `shell_command`. Note that no space is needed around `|`. If no `command` is provided, the last command executed is repeated.

In case the `command` contains a `|`, the option `-d delim` can be used to specify an alternate delimiter string `delim` that separates the `command` from the `shell_command`.

Example:

```
(gdb) p var
$1 = {
  black = 144,
  red = 233,
  green = 377,
  blue = 610,
  white = 987
}
(gdb) pipe p var|wc
      7      19      80
(gdb) |p var|wc -l
7
(gdb) p /x var
$4 = {
  black = 0x90,
  red = 0xe9,
  green = 0x179,
  blue = 0x262,
  white = 0x3db
}
(gdb) ||grep red
red => 0xe9,
(gdb) | -d ! echo this contains a | char\n ! sed -e 's/|/PIPE/'
this contains a PIPE char
(gdb) | -d xxx echo this contains a | char!\n xxx sed -e 's/|/PIPE/'
this contains a PIPE char!
(gdb)
```

The convenience variables `$_shell_exitcode` and `$_shell_exitsignal` can be used to examine the exit status of the last shell command launched by `shell`, `make`, `pipe` and `|`. See Section 10.12 [Convenience Variables], page 173.

你也可以从表达式中调用外壳命令，使用`$_`外壳便捷函数。参见 [外壳便捷函数],页179。

GDB 开发环境中经常需要使用 `make` 工具。在 GDB 中，您不必使用 `shell` 命令来实现这一目的：

`make make-args`
使用指定的参数执行 `make` 程序。这相当于 `'shell make make-args'`。

`pipe [command] | shell_command`
`| [command] | shell_command`
`pipe -d delim command delim shell_command`
`| -d delim command delim shell_command`
执行命令并将输出发送到 `shell` 命令。请注意，在 `|` 周围不需要空格。如果没有提供命令，则会重复执行最后一个命令。

如果命令中包含 `|`，可以使用选项 `-d delim` 指定一个替代的分隔符字符串 `delim`，该字符串用于分隔命令和 `shell` 命令。

示例：

```
(gdb) p var
$1 = {
  black = 144,
  red = 233,
  green = 377,
  blue = 610,
  white = 987
}
(gdb) pipe p var|wc
      7      19      80
(gdb) |p var|wc -l
7
(gdb) p /x var
$4 = {
  black = 0x90,
  red = 0xe9,
  green = 0x179,
  blue = 0x262,
  white = 0x3db
}
(gdb) ||grep red
red => 0xe9,
(gdb) | -d ! echo this contains a | char\n ! sed -e 's/|/PIPE/'
this contains a PIPE char
(gdb) | -d xxx echo this contains a | char!\n xxx sed -e 's/|/PIPE/'
this contains a PIPE char!
(gdb)
```

变量 `$_shell_exitcode` 和 `$_shell_exitsignal` 可用于检查由 `shell`、`make`、`pipe` 和 `|` 启动的最后一个 `shell` 命令的退出状态。参见第 10.12 [Convenience Variables], 页 173。

2.4 Logging Output

You may want to save the output of GDB commands to a file. There are several commands to control GDB's logging.

- `set logging enabled [on|off]`
Enable or disable logging.
- `set logging file file`
Change the name of the current logfile. The default logfile is `gdb.txt`.
- `set logging overwrite [on|off]`
By default, GDB will append to the logfile. Set `overwrite` if you want `set logging enabled on` to overwrite the logfile instead.
- `set logging redirect [on|off]`
By default, GDB output will go to both the terminal and the logfile. Set `redirect` if you want output to go only to the log file.
- `set logging debugredirect [on|off]`
By default, GDB debug output will go to both the terminal and the logfile. Set `debugredirect` if you want debug output to go only to the log file.
- `show logging`
Show the current values of the logging settings.

You can also redirect the output of a GDB command to a shell command. See `[pipe]`, page 21.

2.4 日志输出

您可能想将 GDB 命令的输出保存到文件中。有多个命令可用于控制 GDB的日志记录。

- 设置日志启用 [关闭|禁用]
启用或禁用日志
 - 设置日志文件 *file*
更改当前日志文件名称。默认日志文件为gdb.txt。
 - 设置日志覆盖 [开启|关闭]
默认情况下， GDB 会追加到日志文件。设置o 覆盖，如果你想要设置日志启用覆盖，以覆盖日志文件。
 - 设置日志重定向 [开启|关闭]
默认情况下， GDB 输出会同时发送到终端和日志文件。 Set 如果您希望输出仅定向到日志文件，请重定向。
 - `set logging debugredirect [on|off]`
默认情况下， GDB 调试输出会同时发送到终端和日志文件。如果希望调试输出仅发送到日志文件，请设置 `debugredirect`。
 - 显示日志
显示当前的日志设置值。
- 你也可以将 GDB命令的输出重定向到 shell 命令。参见 `[管道]`,页 21。

3 GDB Commands

You can abbreviate a GDB command to the first few letters of the command name, if that abbreviation is unambiguous; and you can repeat certain GDB commands by typing just RET. You can also use the TAB key to get GDB to fill out the rest of a word in a command (or to show you the alternatives available, if there is more than one possibility).

3.1 Command Syntax

A GDB command is a single line of input. There is no limit on how long it can be. It starts with a command name, which is followed by arguments whose meaning depends on the command name. For example, the command `step` accepts an argument which is the number of times to step, as in `'step 5'`. You can also use the `step` command with no arguments. Some commands do not allow any arguments.

GDB command names may always be truncated if that abbreviation is unambiguous. Other possible command abbreviations are listed in the documentation for individual commands. In some cases, even ambiguous abbreviations are allowed; for example, `s` is specially defined as equivalent to `step` even though there are other commands whose names start with `s`. You can test abbreviations by using them as arguments to the `help` command.

A blank line as input to GDB (typing just RET) means to repeat the previous command. Certain commands (for example, `run`) will not repeat this way; these are commands whose unintentional repetition might cause trouble and which you are unlikely to want to repeat. User-defined commands can disable this feature; see Section 23.1.1 [Define], page 397.

The `list` and `x` commands, when you repeat them with RET, construct new arguments rather than repeating exactly as typed. This permits easy scanning of source or memory.

GDB can also use RET in another way: to partition lengthy output, in a way similar to the common utility `more` (see Section 22.4 [Screen Size], page 374). Since it is easy to press one RET too many in this situation, GDB disables command repetition after any command that generates this sort of display.

Any text from a `#` to the end of the line is a comment; it does nothing. This is useful mainly in command files (see Section 23.1.3 [Command Files], page 401).

The `Ctrl-o` binding is useful for repeating a complex sequence of commands. This command accepts the current line, like RET, and then fetches the next line relative to the current line from the history for editing.

3.2 Command Settings

Many commands change their behavior according to command-specific variables or settings. These settings can be changed with the `set` subcommands. For example, the `print` command (see Chapter 10 [Examining Data], page 143) prints arrays differently depending on settings changeable with the commands `set print elements NUMBER-OF-ELEMENTS` and `set print array-indexes`, among others.

You can change these settings to your preference in the `gdbinit` files loaded at GDB startup. See Section 2.1.3 [Startup], page 17.

The settings can also be changed interactively during the debugging session. For example, to change the limit of array elements to print, you can do the following:

```
(gdb) set print elements 10
```

3 GDB命令

如果该缩写是明确的，你可以将 GDB 命令缩写到命令名的头几个字母；并且你可以通过仅按 RET 键来重复某些 GDB 命令。你也可以使用 TAB 键来让 GDB 补全命令中剩余的部分（如果存在多个可能性，则显示可用的替代选项）。

3.1 命令语法

一个 GDB 命令是一行输入。它的长度没有限制。它以命令名称开头，后面跟着的参数含义取决于命令名称。例如，命令 `step` 接受一个参数，表示步进次数，如 `'step 5'`。你也可以不带参数使用 `step` 命令。有些命令不允许任何参数。

GDB 命令名称如果缩写是明确的，总是可以缩短。其他可能的命令缩写列在每个单独命令的文档中。在某些情况下，即使缩写是模糊的也是允许的；例如，`s` 被特别定义为等同于 `step`，尽管有其他命令的名称以 `s` 开头。你可以通过将它们作为 `help` 命令的参数来测试缩写。

将一个空行作为输入到 GDB（仅按回车键）表示重复上一个命令。某些命令（例如，`run`）不会以这种方式重复；这些是无意中重复可能会引起问题的命令，而你不太可能想要重复它们。用户定义的命令可以禁用此功能；参见第 23.1.1 [定义], 页 397。

列表和 `x` 命令，当你用 RET 重复它们时，会构建新的参数，而不是完全重复你输入的内容。这允许轻松扫描源代码或内存。

GDB 也可以以另一种方式使用 RET：将长输出分割，类似于常见的实用程序 `more`（见第 22.4 [屏幕大小], 页 374）。由于在这种情况下很容易按过多的 RET，GDB 禁用了在生成此类显示的任何命令之后重复命令。

从 `a #` 到行尾的任何文本都是注释；它什么也不做。这主要在命令文件中很有用（见第 23.1.3 [命令文件], 页 401）。

`Ctrl-o` 绑定对于重复复杂的命令序列很有用。此命令接受当前行，像 RET 一样，然后从历史记录中获取相对于当前行的下一行以进行编辑。

3.2 命令设置

许多命令根据特定的命令变量或设置更改它们的行为。这些设置可以使用 `set` 子命令更改。例如，`print` 命令（见第 10 [检查数据], 章页 143）根据可使用 `set print elements NUMBER-OF-ELEMENTS` 和 `set print array-indexes` 等命令更改的设置以不同的方式打印数组。

您可以在 GDB 启动时加载的 `gdbinit` 文件中更改这些设置以符合您的偏好。参见第 2.1.3 [启动], 页 17。

在调试会话期间，设置也可以交互式更改。例如，要更改打印数组元素的限制，您可以执行以下操作：

```
(gdb) set print elements 10
```



```
(gdb) print some_array
$1 = {0, 10, 20, 30, 40, 50, 60, 70, 80, 90...}
```

The above `set print elements 10` command changes the number of elements to print from the default of 200 to 10. If you only intend this limit of 10 to be used for printing `some_array`, then you must restore the limit back to 200, with `set print elements 200`.

Some commands allow overriding settings with command options. For example, the `print` command supports a number of options that allow overriding relevant global print settings as set by `set print` subcommands. See [print options], page 143. The example above could be rewritten as:

```
(gdb) print -elements 10 -- some_array
$1 = {0, 10, 20, 30, 40, 50, 60, 70, 80, 90...}
```

Alternatively, you can use the `with` command to change a setting temporarily, for the duration of a command invocation.

```
with setting [value] [-- command]
w setting [value] [-- command]
```

Temporarily set *setting* to *value* for the duration of *command*.

setting is any setting you can change with the `set` subcommands. *value* is the value to assign to *setting* while running *command*.

If no *command* is provided, the last command executed is repeated.

If a *command* is provided, it must be preceded by a double dash (`--`) separator. This is required because some settings accept free-form arguments, such as expressions or filenames.

For example, the command

```
(gdb) with print array on -- print some_array
```

is equivalent to the following 3 commands:

```
(gdb) set print array on
(gdb) print some_array
(gdb) set print array off
```

The `with` command is particularly useful when you want to override a setting while running user-defined commands, or commands defined in Python or Guile. See Chapter 23 [Extending GDB], page 397.

```
(gdb) with print pretty on -- my_complex_command
```

To change several settings for the same command, you can nest `with` commands. For example, `with language ada -- with print elements 10` temporarily changes the language to Ada and sets a limit of 10 elements to print for arrays and strings.

3.3 Command Completion

GDB can fill in the rest of a word in a command for you, if there is only one possibility; it can also show you what the valid possibilities are for the next word in a command, at any time. This works for GDB commands, GDB subcommands, command options, and the names of symbols in your program.

Press the `TAB` key whenever you want GDB to fill out the rest of a word. If there is only one possibility, GDB fills in the word, and waits for you to finish the command (or press `RET` to enter it). For example, if you type

```
(gdb) print some_array
$1 = {0, 10, 20, 30, 40, 50, 60, 70, 80, 90...}
```

上述的打印元素10命令将打印元素的数量从默认的200更改为10。如果你只想将这个10的限制用于打印某些_数组，那么你必须将限制恢复为200，使用`set print elements 200`。

某些命令允许使用命令选项覆盖设置。例如，打印命令支持多个选项，允许覆盖由`set print`子命令设置的相应全局打印设置。参见[打印选项]，页143。上面的示例可以重写为：

```
(gdb) print -elements 10 -- 某些_数组
$1 = {0, 10, 20, 30, 40, 50, 60, 70, 80, 90...}
```

或者，你也可以使用 `with` 命令来临时更改设置，仅在命令调用期间生效。

```
with setting [value] [-- command]
w setting [value] [-- command]
```

临时将设置值设置为命令持续时间。

setting 是你可以使用 `set` 子命令更改的任何设置。value 是在运行命令期间分配给 *setting* 的值。

如果没有提供命令，则重复执行最后一个已执行的命令。

如果提供了命令，它必须由一个双破折号(`--`)分隔符前缀。这是必需的，因为某些设置接受自由形式的参数，例如表达式或文件名。

例如，命令

```
(gdb) with print array on -- print some_array
```

相当于以下 3 个命令：

```
(gdb) set print array on
(gdb) print some_array
(gdb) set print array off
```

当您希望在运行用户自定义命令或Python或Guile定义的命令时覆盖设置时，``with``命令特别有用。参见第 23 [章],扩展GDB],页397。

```
(gdb) with print pretty on -- my_complex_命令
```

要为同一个命令更改多个设置，您可以嵌套``with``命令。例如，``with language ada -- with print elements 10``会临时将语言更改为Ada，并为数组和字符串设置打印元素的上限为10。

3.3 命令补全

GDB 可以在命令中为您补全剩余的单词，如果只有一个可能；它还可以随时显示命令中下一个单词的有效可能性。这适用于 GDB 命令、GDB 子命令、命令选项以及您程序中符号的名称。

按 `TAB` 键，当你想 GDB 补全单词时。如果只有一个可能，GDB 会自动补全该单词，并等待你完成命令（或按 `RET` 键确认）。例如，如果你输入

```
(gdb) info breTAB
```

GDB fills in the rest of the word ‘breakpoints’, since that is the only `info` subcommand beginning with ‘bre’:

```
(gdb) info breakpoints
```

You can either press `RET` at this point, to run the `info breakpoints` command, or backspace and enter something else, if ‘breakpoints’ does not look like the command you expected. (If you were sure you wanted `info breakpoints` in the first place, you might as well just type `RET` immediately after ‘`info bre`’, to exploit command abbreviations rather than command completion).

If there is more than one possibility for the next word when you press `TAB`, GDB sounds a bell. You can either supply more characters and try again, or just press `TAB` a second time; GDB displays all the possible completions for that word. For example, you might want to set a breakpoint on a subroutine whose name begins with ‘`make_`’, but when you type `b make_TAB` GDB just sounds the bell. Typing `TAB` again displays all the function names in your program that begin with those characters, for example:

```
(gdb) b make_TAB
```

GDB sounds bell; press `TAB` again, to see:

<code>make_a_section_from_file</code>	<code>make_environ</code>
<code>make_abs_section</code>	<code>make_function_type</code>
<code>make_blockvector</code>	<code>make_pointer_type</code>
<code>make_cleanup</code>	<code>make_reference_type</code>
<code>make_command</code>	<code>make_symbol_completion_list</code>

```
(gdb) b make_
```

After displaying the available possibilities, GDB copies your partial input (‘`b make_`’ in the example) so you can finish the command.

If the command you are trying to complete expects either a keyword or a number to follow, then ‘NUMBER’ will be shown among the available completions, for example:

```
(gdb) print -elements TABTAB
NUMBER      unlimited
(gdb) print -elements
```

Here, the option expects a number (e.g., 100), not literal `NUMBER`. Such metasyntactical arguments are always presented in uppercase.

If you just want to see the list of alternatives in the first place, you can press `M-?` rather than pressing `TAB` twice. `M-?` means *META ?*. You can type this either by holding down a key designated as the *META* shift on your keyboard (if there is one) while typing `?`, or as `ESC` followed by `?`.

If the number of possible completions is large, GDB will print as much of the list as it has collected, as well as a message indicating that the list may be truncated.

```
(gdb) b mTABTAB
main
<... the rest of the possible completions ...>
*** List may be truncated, max-completions reached. ***
(gdb) b m
```

This behavior can be controlled with the following commands:

```
set max-completions limit
set max-completions unlimited
```

Set the maximum number of completion candidates. GDB will stop looking for more completions once it collects this many candidates. This is useful when

```
(gdb) info breTAB
```

GDB 会补全单词 ‘breakpoints’，因为这是唯一以 ‘bre’ 开头的 `info` 子命令：

```
(gdb) info breakpoints
```

你可以选择此时按 `RET` 键来运行 `info breakpoints` 命令，或者按退格键输入其他内容，如果 ‘breakpoints’ 看起来不是你期望的命令。（如果你一开始就确定想要 `info breakpoints`，不妨在 ‘`info bre`’ 后立即按 `RET` 键，以利用命令缩写而不是命令补全）。

当你按 `TAB` 键时，如果下一个单词有多个可能，GDB 会发出提示音。你可以输入更多字符再试一次，或者再次按 `TAB` 键；GDB 会显示该单词的所有可能补全。例如，你可能想设置一个以 ‘`make_`’ 开头的子程序断点，但当你输入 `bmake_TAB` GDB 时只会发出提示音。再次按 `TAB` 键会显示你程序中以这些字符开头的所有函数名，例如：

```
(gdb) b make_TAB
```

GDB sounds bell; press `TAB` again, to see:

<code>make_a_section_from_file</code>	<code>make_environ</code>
<code>make_abs_section</code>	<code>make_function_type</code>
<code>make_blockvector</code>	<code>make_pointer_type</code>
<code>make_cleanup</code>	<code>make_reference_type</code>
<code>make_command</code>	<code>make_symbol_completion_list</code>

```
(gdb) b make_
```

显示可用可能性后，GDB 会复制你的部分输入（例如示例中的 ‘`b make_`’），以便你完成命令。

如果你尝试完成的命令期望跟一个关键字或数字，那么 ‘NUMBER’ 将会在可用的补全选项中显示，例如：

```
(gdb) print -elements TABTAB
NUMBER      unlimited
(gdb) print -elements
```

在这里，选项期望一个数字（例如，100），而不是字面意义上的 `NUMBER`。这种元语法参数总是以大写形式呈现。

如果你只想先看到替代选项列表，可以按 `M-?` 而不是按 `TAB` 两次。`M-?` 意味着 *META ?*。你可以通过按住键盘上指定的 *META* Shift 键（如果有的话）同时输入 `?`，或者按 `ESC` 后跟 `?` 来输入。

如果可能的补全选项数量很大，GDB 会打印出它已经收集到的列表部分，以及一条指示列表可能被截断的消息。

```
(gdb) b mTABTAB
main
<...其余可能的补全...>
*** List may be truncated, max-completions reached. ***
(gdb) b m
```

此行为可通过以下命令控制：

```
set max-completions limit
set max-completions unlimited
```

设置最大补全候选数。GDB 在收集到这种数量的候选后，将停止寻找更多补全。这在

completing on things like function names as collecting all the possible candidates can be time consuming. The default value is 200. A value of zero disables tab-completion. Note that setting either no limit or a very large limit can make completion slow.

show max-completions

Show the maximum number of candidates that GDB will collect and show during completion.

Sometimes the string you need, while logically a “word”, may contain parentheses or other characters that GDB normally excludes from its notion of a word. To permit word completion to work in this situation, you may enclose words in ‘ (single quote marks) in GDB commands.

A likely situation where you might need this is in typing an expression that involves a C++ symbol name with template parameters. This is because when completing expressions, GDB treats the ‘<’ character as word delimiter, assuming that it’s the less-than comparison operator (see Section 15.4.1.1 [C and C++ Operators], page 233).

For example, when you want to call a C++ template function interactively using the `print` or `call` commands, you may need to distinguish whether you mean the version of `name` that was specialized for `int`, `name<int>()`, or the version that was specialized for `float`, `name<float>()`. To use the word-completion facilities in this situation, type a single quote ‘ at the beginning of the function name. This alerts GDB that it may need to consider more information than usual when you press `TAB` or `M-?` to request word completion:

```
(gdb) p 'func<M-?
func<int>()    func<float>()
(gdb) p 'func<
```

When setting breakpoints however (see Section 9.2 [Location Specifications], page 128), you don’t usually need to type a quote before the function name, because GDB understands that you want to set a breakpoint on a function:

```
(gdb) b func<M-?
func<int>()    func<float>()
(gdb) b func<
```

This is true even in the case of typing the name of C++ overloaded functions (multiple definitions of the same function, distinguished by argument type). For example, when you want to set a breakpoint you don’t need to distinguish whether you mean the version of `name` that takes an `int` parameter, `name(int)`, or the version that takes a `float` parameter, `name(float)`.

```
(gdb) b bubble(M-?
bubble(int)    bubble(double)
(gdb) b bubble(douM-?
bubble(double)
```

See [quoting names], page 265, for a description of other scenarios that require quoting.

For more information about overloaded functions, see Section 15.4.1.3 [C++ Expressions], page 236. You can use the command `set overload-resolution off` to disable overload resolution; see Section 15.4.1.7 [GDB Features for C++], page 237.

完成诸如函数名之类的事情时，收集所有可能的候选者可能会很耗时。默认值是 200。值为零会禁用 Tab 补全。请注意，设置无限制或非常大的限制都可能导致补全变慢。

显示最大补全数

显示 GDB 在补全过程中收集和显示的最大候选数。

有时，您需要的字符串在逻辑上是一个“单词”，但可能包含括号或其他字符，而 GDB 通常将这些字符排除在其单词概念之外。为了在这种情况下允许单词补全正常工作，您可以在 GDB 命令中用单引号 (‘) 将单词括起来。

您可能需要这种情况的一个典型场景是在输入涉及 C++ 符号名称和模板参数的表达式时。这是因为当补全表达式时，GDB 将 ‘<’ 字符视为单词分隔符，假设它是小于比较运算符（见第 15.4.1.1 [C 和 C++ 运算符], 页 233）。

例如，当您想使用 `print` 或 `call` 命令交互式调用 C++ 模板函数时，您可能需要区分您是指专门为 `int` 定制的名称版本，即 `name<int>()`，还是指专门为 `float` 定制的名称 `<float>()` 版本。要在这种情况下使用单词补全功能，请在函数名称的开头输入一个单引号'。这会提醒 GDB 在您按下 `TAB` 或 `M-?` 请求单词补全时，它可能需要考虑比平时更多的信息：

```
(gdb) p 'func<M-?
func<int>()    func<float>()
(gdb) p 'func<
```

然而，在设置断点时（参见第 9.2 [位置规范], 页 128），通常不需要在函数名前输入引号，因为 GDB 会理解你想在函数上设置断点：

```
(gdb) b func<M-?
func<int>()    func<float>()
(gdb) b func<
```

即使是在输入 C++ 重载函数的名称时也是如此（相同函数的多重定义，通过参数类型区分）。例如，当你想设置断点时，不需要区分你是想设置接受 `int` 参数的版本 `name(int)`，还是接受 `float` 参数的版本 `name(float)`。

```
(gdb) b bubble(M-?
bubble(int)    bubble(double)
(gdb) b bubble(douM-?
bubble(double)
```

参见 [引用名称], 页 265，了解需要引用的其他场景的描述。

有关重载函数的更多信息，请参阅第 15.4.1 节.3 [C++ 表达式], 第 236 页。您可以使用命令 `set overload-resolution off` 禁用重载解析；请参阅第 15.4.1.7 [GDB C++ 特性], 第 237 页。

When completing in an expression which looks up a field in a structure, GDB also tries¹ to limit completions to the field names available in the type of the left-hand-side:

```
(gdb) p gdb_stdout.M-?
magic          to_fputs          to_rewind
to_data        to_isatty        to_write
to_delete      to_put          to_write_async_safe
to_flush       to_read
```

This is because the `gdb_stdout` is a variable of the type `struct ui_file` that is defined in GDB sources as follows:

```
struct ui_file
{
    int *magic;
    ui_file_flush_ftype *to_flush;
    ui_file_write_ftype *to_write;
    ui_file_write_async_safe_ftype *to_write_async_safe;
    ui_file_fputs_ftype *to_fputs;
    ui_file_read_ftype *to_read;
    ui_file_delete_ftype *to_delete;
    ui_file_isatty_ftype *to_isatty;
    ui_file_rewind_ftype *to_rewind;
    ui_file_put_ftype *to_put;
    void *to_data;
}
```

3.4 Filenames As Command Arguments

When passing filenames (or directory names) as arguments to a command, if the filename argument does not include any whitespace, double quotes, or single quotes, then for all commands the filename can be written as a simple string, for example:

```
(gdb) file /path/to/some/file
```

If the filename does include whitespace, double quotes, or single quotes, then GDB has two approaches for how these filenames should be formatted; which format to use depends on which command is being used.

Most GDB commands don't require, or support, quoting and escaping. These commands treat any text after the command name, that is not a command option (see Section 3.5 [Command Options], page 28), as the filename, even if the filename contains whitespace or quote characters. In the following example the user is adding `/path/that contains/two spaces/` to the auto-load safe-path (see [add-auto-load-safe-path], page 384):

```
(gdb) add-auto-load-safe-path /path/that contains/two spaces/
```

A small number of commands require that filenames containing whitespace or quote characters are either quoted, or have the special characters escaped with a backslash. Commands that support this style are marked as such in the manual, any command not marked as accepting quoting and escaping of its filename argument, does not accept this filename argument style.

For example, to load the file `/path/with spaces/to/a file` with the `file` command (see Section 18.1 [Commands to Specify Files], page 293), you can escape the whitespace characters with a backslash:

```
(gdb) file /path/with\ spaces/to/a\ file
```

¹ The completer can be confused by certain kinds of invalid expressions. Also, it only examines the static type of the expression, not the dynamic type.

当在查找结构中字段的表达式中完成时， GDB 还会尝试¹将完成限制为左侧类型中可用的字段名称：

```
(gdb) p gdb_stdout.M-?
magic          to_fputs          to_rewind
to_data        to_isatty        to_write
to_delete      to_put          to_write_async_safe
to_flush       to_read
```

这是因为`gdb_stdout`是一个类型为`struct ui_file`的结构体变量，它在GDB 源代码中定义如下：

```
struct ui_file
{
    int *magic;
    ui_file_flush_ftype *to_flush;
    ui_file_write_ftype *to_write;
    ui_file_write_async_safe_ftype *to_write_async_safe;
    ui_file_fputs_ftype *to_fputs;
    ui_file_read_ftype *to_read;
    ui_file_delete_ftype *to_delete;
    ui_file_isatty_ftype *to_isatty;
    ui_file_rewind_ftype *to_rewind;
    ui_file_put_ftype *to_put;
    void *to_data;
}
```

3.4 文件名作为命令参数

当将文件名（或目录名）作为命令参数传递时，如果文件名参数不包含任何空白字符、双引号或单引号，那么对于所有命令，文件名都可以写成一个简单的字符串，例如：

```
(gdb) file /path/to/some/file
```

如果文件名包含空白字符、双引号或单引号，那么 GDB 有两种方法来格式化这些文件名；使用哪种格式取决于正在使用的命令。

大多数 GDB 命令不需要或支持引号和转义。这些命令将命令名之后的所有文本（即不是命令选项（参见第3.5 [命令选项], 页28）的内容）视为文件名，即使文件名包含空白字符或引号字符。在以下示例中，用户正在将包含两个空格的路径`/path/that contains/two spaces/`添加到自动加载安全路径（参见[add-auto-load-safe-path],页384）：

```
(gdb) add-auto-load-safe-path /path/that contains/two spaces/
```

少数命令要求包含空格或引号字符的文件名要么被引号括起来，要么用反斜杠对特殊字符进行转义。支持这种格式的命令在手册中会有标注，任何未标注为接受文件名参数引号和转义的命令，都不接受这种文件名参数格式。

例如，要使用文件命令（参见第18.1 [指定文件], 页293）加载包含空格的文件 `/path/withspaces/to/a file`，可以用反斜杠对空格字符进行转义：

```
(gdb) file /路径/包含空格/到一个文件
```

¹ 补全器可能会被某些类型的无效表达式混淆。此外，它只检查表达式的静态类型，而不是动态类型。

Alternatively the entire filename can be wrapped in either single or double quotes, in which case no backslashes are needed, for example:

```
(gdb) symbol-file "/path/with spaces/to/a file"
(gdb) exec-file '/path/with spaces/to/a file'
```

It is possible to include a quote character within a quoted filename by escaping it with a backslash, for example, within a filename surrounded by double quotes, a double quote character should be escaped with a backslash, but a single quote character should not be escaped. Within a single quoted string a single quote character needs to be escaped, but a double quote character does not.

A literal backslash character can also be included by escaping it with a backslash.

3.5 Command options

Some commands accept options starting with a leading dash. For example, `print -pretty`. Similarly to command names, you can abbreviate a GDB option to the first few letters of the option name, if that abbreviation is unambiguous, and you can also use the `TAB` key to get GDB to fill out the rest of a word in an option (or to show you the alternatives available, if there is more than one possibility).

Some commands take raw input as argument. For example, the `print` command processes arbitrary expressions in any of the languages supported by GDB. With such commands, because raw input may start with a leading dash that would be confused with an option or any of its abbreviations, e.g. `print -p` (short for `print -pretty` or printing negative `p`?), if you specify any command option, then you must use a double-dash (`--`) delimiter to indicate the end of options.

Some options are described as accepting an argument which can be either `on` or `off`. These are known as *boolean options*. Similarly to boolean settings commands—`on` and `off` are the typical values, but any of `1`, `yes` and `enable` can also be used as “true” value, and any of `0`, `no` and `disable` can also be used as “false” value. You can also omit a “true” value, as it is implied by default.

For example, these are equivalent:

```
(gdb) print -object on -pretty off -element unlimited -- *myptr
(gdb) p -o -p 0 -e u -- *myptr
```

You can discover the set of options some command accepts by completing `on -` after the command name. For example:

```
(gdb) print -TABTAB
-address          -max-depth          -object          -static-members
-array            -memory-tag-violations -pretty          -symbol
-array-indexes    -nibbles            -raw-values      -union
-elements         -null-stop          -repeats         -vtbl
```

Completion will in some cases guide you with a suggestion of what kind of argument an option expects. For example:

```
(gdb) print -elements TABTAB
NUMBER          unlimited
```

Here, the option expects a number (e.g., 100), not literal `NUMBER`. Such metasyntactical arguments are always presented in uppercase.

(For more on using the `print` command, see Chapter 10 [Examining Data], page 143.)

或者，整个文件名可以包含单引号或双引号，在这种情况下不需要反斜杠，例如：

```
(gdb) symbol-file "/path/with spaces/to/a file"
(gdb) exec-file '/path/with spaces/to/a file'
```

可以在带引号的文件名中包含引号字符，方法是使用反斜杠转义它，例如，在双引号包围的文件名中，双引号字符需要用反斜杠转义，而单引号字符不需要转义。在单引号字符串中，单引号字符需要转义，而双引号字符不需要。

字面反斜杠字符也可以通过使用反斜杠转义来包含。

3.5 命令选项

某些命令接受以连字符开头的选项。例如，`print -pretty`。与命令名称类似，如果该缩写是明确的，你可以将 GDB 选项缩写为选项名称的前几个字母，还可以使用 `TAB` 键来补全选项的其余部分（或者显示可用的替代选项，如果有多个可能性）。

某些命令接受原始输入作为参数。例如，`print` 命令处理由 GDB 支持的任何语言中的任意表达式。对于此类命令，因为原始输入可能以连字符开头，而连字符会被误解为选项或其任何缩写，例如 `print -p`（短按 `print -pretty` 或打印负 `p`？），如果你指定了任何命令选项，则必须使用双连字符 (`--`) 分隔符来指示选项的结束。

某些选项被描述为接受一个参数，该参数可以是开或关。这些选项被称为布尔选项。与布尔设置命令类似——开和关是典型值，但 `1`、`yes` 和 `enable` 也可以用作“真”值，而 `0`、`no` 和 `disable` 也可以用作“假”值。您还可以省略“真”值，因为它默认为真。

例如，这些是等效的：

```
(gdb) 打印 -对象 on -格式 off -元素无限 -- *myptr (gdb) p -o -p 0 -e u --
*myptr
```

您可以通过在命令名后补全 `-on` 来发现某个命令接受的选项集。例如：

```
(gdb) 打印 -TABTAB -地址 -最大深度 -对象 -静态成员 -数组 -内存标签违规 -美化 -
符号 -数组索引 -十六进制 -原始值 -联合 -元素 -空停止 -重复 -vtbl
```

在某些情况下，完成功能会提示你某个选项期望的参数类型。例如：

```
(gdb) print -elements
TABTAB NUMBER unlimited
```

这里，选项期望的是一个数字（例如，100），而不是字面值 `NUMBER`。这类元语法参数始终以大写形式呈现。

(关于如何使用打印命令，请参阅第 10 [检查数据] 页 143。)

3.6 Getting Help

You can always ask GDB itself for information on its commands, using the command `help`.

`help`

`h` You can use `help` (abbreviated `h`) with no arguments to display a short list of named classes of commands:

```
(gdb) help
List of classes of commands:

aliases -- User-defined aliases of other commands
breakpoints -- Making program stop at certain points
data -- Examining data
files -- Specifying and examining files
internals -- Maintenance commands
obscure -- Obscure features
running -- Running the program
stack -- Examining the stack
status -- Status inquiries
support -- Support facilities
tracepoints -- Tracing of program execution without
                stopping the program
user-defined -- User-defined commands

Type "help" followed by a class name for a list of
commands in that class.
Type "help" followed by command name for full
documentation.
Command name abbreviations are allowed if unambiguous.
(gdb)
```

`help class`

Using one of the general help classes as an argument, you can get a list of the individual commands in that class. If a command has aliases, the aliases are given after the command name, separated by commas. If an alias has default arguments, the full definition of the alias is given after the first line. For example, here is the help display for the class `status`:

```
(gdb) help status
Status inquiries.

List of commands:

info, inf, i -- Generic command for showing things
                about the program being debugged
info address, iain -- Describe where symbol SYM is stored.
                alias iain = info address main
info all-registers -- List of all registers and their contents,
                    for selected stack frame.
...
show, info set -- Generic command for showing things
                about the debugger

Type "help" followed by command name for full
documentation.
Command name abbreviations are allowed if unambiguous.
(gdb)
```

3.6 获取帮助

你可以随时使用命令 `help` 向 GDB 查询其命令的信息。

`help`

`h` 您可以使用不带参数的 `help` (缩写为 `h`) 来显示命令的简短分类列表:

```
(gdb) help
List of classes of commands:

aliases -- User-defined aliases of other commands
breakpoints -- Making program stop at certain points
data -- Examining data
files -- Specifying and examining files
internals -- Maintenance commands
obscure -- Obscure features
running -- Running the program
stack -- Examining the stack
status -- Status inquiries
support -- Support facilities
tracepoints -- Tracing of program execution without
                stopping the program
user-defined -- User-defined commands

Type "help" followed by a class name for a list of
commands in that class.
Type "help" followed by command name for full
documentation.
Command name abbreviations are allowed if unambiguous.
(gdb)
```

`help class`

使用通用帮助类作为参数, 你可以获取该类中各个命令的列表。如果一个命令有别名, 别名会以逗号分隔地显示在命令名称之后。如果一个别名有默认参数, 该别名的完整定义会显示在第一行之后。例如, 以下是类 `status` 的帮助显示:

```
(gdb) help status
Status inquiries.

命令列表:

info, inf, i -- Generic command for showing things
                about the program being debugged
info address, iain -- Describe where symbol SYM is stored.
                alias iain = info address main
info all-registers -- List of all registers and their contents,
                    for selected stack frame.
...
show, info set -- Generic command for showing things
                about the debugger

Type "help" followed by command name for full
documentation.
Command name abbreviations are allowed if unambiguous.
(gdb)
```


help command

With a command name as **help** argument, GDB displays a short paragraph on how to use that command. If that command has one or more aliases, GDB will display a first line with the command name and all its aliases separated by commas. This first line will be followed by the full definition of all aliases having default arguments. When asking the help for an alias, the documentation for the aliased command is shown.

A user-defined alias can optionally be documented using the **document** command (see Section 23.1.1 [Define], page 397). GDB then considers this alias as different from the aliased command: this alias is not listed in the aliased command help output, and asking help for this alias will show the documentation provided for the alias instead of the documentation of the aliased command.

apropos [-v] regexp

The **apropos** command searches through all of the GDB commands and aliases, and their documentation, for the regular expression specified in *args*. It prints out all matches found. The optional flag **-v**, which stands for **verbose**, indicates to output the full documentation of the matching commands and highlight the parts of the documentation matching *regexp*. For example:

```
apropos alias
```

results in:

```
alias -- Define a new command that is an alias of an existing command
aliases -- User-defined aliases of other commands
```

while

```
apropos -v cut.*thread apply
```

results in the below output, where **cut for 'thread apply'** is highlighted if styling is enabled.

```
taas -- Apply a command to all threads (ignoring errors
and empty output).
Usage: taas COMMAND
shortcut for 'thread apply all -s COMMAND'
```

```
tfaas -- Apply a command to all frames of all threads
(ignoring errors and empty output).
Usage: tfaas COMMAND
shortcut for 'thread apply all -s frame apply all -s COMMAND'
```

complete args

The **complete args** command lists all the possible completions for the beginning of a command. Use *args* to specify the beginning of the command you want completed. For example:

```
complete i
```

results in:

```
if
ignore
info
inspect
```

This is intended for use by GNU Emacs.

帮助 command

使用命令名称作为帮助参数，GDB 会显示一个简短段落，说明如何使用该命令。如果该命令有一个或多个别名，GDB 会显示第一行包含命令名称及其所有别名（以逗号分隔）。这行内容之后会显示所有具有默认参数的别名的完整定义。当你请求别名的帮助时，会显示被别名指向的命令的文档。

用户定义的别名可以可选地使用文档命令进行说明（参见第23.1.1 [Define], 页397）。GDB 将此别名视为与别名命令不同：此别名不会列在别名命令的帮助输出中，并且查询此别名的帮助将显示为别名的文档，而不是别名命令的文档。

apropos [-v] regexp

apropos命令会搜索所有 GDB 命令和别名的文档，查找*args*中指定的正则表达式。它会打印出所有找到的匹配项。可选标志 **-v**，代表 **详细**，指示输出匹配命令的完整文档并突出显示匹配正则表达式的文档部分。例如：

```
apropos alias
```

导致：

```
alias -- 定义一个新命令，该命令是现有命令的别名
aliases -- 其他命令的用户自定义别名
```

while

```
apropos -v cut.*thread apply
```

导致以下输出，如果启用样式，则 **“cut for thread apply”** 将被突出显示。

```
taas -- 将命令应用于所有线程（忽略错误
and empty output）。
Usage: taas COMMAND
快捷键为 'thread apply all -s COMMAND'
```

```
tfaas -- 将命令应用于所有线程的所有帧
（忽略错误和空输出）。
Usage: tfaas COMMAND
快捷键为 'thread apply all -s frame apply all -s COMMAND'
```

完成 args

完整的 **args** 命令会列出所有可能的命令开头补全选项。使用*args*指定你想补全的命令开头。例如：

```
complete i
```

结果为：

```
if i
ignore
info
inspect
```

这是为 GNU Emacs 使用的。

In addition to **help**, you can use the GDB commands **info** and **show** to inquire about the state of your program, or the state of GDB itself. Each command supports many topics of inquiry; this manual introduces each of them in the appropriate context. The listings under **info** and under **show** in the Command, Variable, and Function Index point to all the sub-commands. See [Command and Variable Index], page 961.

- info

This command (abbreviated **i**) is for describing the state of your program. For example, you can show the arguments passed to a function with **info args**, list the registers currently in use with **info registers**, or list the breakpoints you have set with **info breakpoints**. You can get a complete list of the **info** sub-commands with **help info**.
- set

You can assign the result of an expression to an environment variable with **set**. For example, you can set the GDB prompt to a \$-sign with **set prompt \$**.
- show

In contrast to **info**, **show** is for describing the state of GDB itself. You can change most of the things you can **show**, by using the related command **set**; for example, you can control what number system is used for displays with **set radix**, or simply inquire which is currently in use with **show radix**.

To display all the settable parameters and their current values, you can use **show** with no arguments; you may also use **info set**. Both commands produce the same display.

Here are several miscellaneous **show** subcommands, all of which are exceptional in lacking corresponding **set** commands:

- show version

Show what version of GDB is running. You should include this information in GDB bug-reports. If multiple versions of GDB are in use at your site, you may need to determine which version of GDB you are running; as GDB evolves, new commands are introduced, and old ones may wither away. Also, many system vendors ship variant versions of GDB, and there are variant versions of GDB in GNU/Linux distributions as well. The version number is the same as the one announced when you start GDB.
- show copying

info copying

Display information about permission for copying GDB.
- show warranty

info warranty

Display the GNU “NO WARRANTY” statement, or a warranty, if your version of GDB comes with one.
- show configuration

Display detailed information about the way GDB was configured when it was built. This displays the optional arguments passed to the **configure** script and also configuration parameters detected automatically by **configure**. When reporting a GDB bug (see Chapter 32 [GDB Bugs], page 729), it is important to include this information in your report.

除了帮助，您还可以使用 GDB 命令的 **info** 和 **show** 来查询您的程序状态，或 GDB 本身的状态。每个命令都支持许多查询主题；本手册在适当的上下文中介绍了每一个。在 Command、Variable 和 Function 索引中的 **info** 和 **show** 下列出了所有子命令。参见 [命令和变量索引], 页 961。

- info

此命令（缩写为*i*）用于描述您的程序状态。例如，您可以使用**info args**显示传递给函数的参数，使用**info registers**列出当前正在使用的寄存器，或使用**info breakpoints**列出您设置的断点。您可以使用**help info**获取**info**子命令的完整列表。
- set

您可以使用**set**将表达式的结果分配给环境变量。例如，您可以使用**set prompt \$**将 GDB 提示符设置为\$符号。
- show

与**info**不同，**show**用于描述 GDB 本身的状态。您可以通过使用相关的命令**set**来更改大多数可以显示的内容；例如，您可以使用**set radix**控制用于显示的数制，或使用**show radix**查询当前正在使用的数制。

要显示所有可设置的参数及其当前值，您可以使用不带参数的**show**；您也可以使用**info set**。这两个命令产生相同的显示。

这里有一些杂项显示子命令，它们都缺少相应的设置命令：

- show version

显示正在运行的 GDB 版本。你应该在 GDB 故障报告中包含此信息。如果你的站点使用多个 GDB 版本，你可能需要确定你正在运行哪个 GDB 版本；随着 GDB 的发展，会引入新命令，而旧命令可能会逐渐消失。此外，许多系统供应商会提供 GDB 的变体版本，GNU/Linux 发行版中也有 GDB 的变体版本。版本号与启动 GDB 时宣布的版本号相同。
- show copying

info copying

显示关于复制 GDB 权限的信息。
- 显示保修

保修信息

显示 GNU “不提供担保” 声明，或保修，如果你的 GDB 版本附带保修。
- 显示配置

显示 GDB 在构建时配置的详细信息。这会显示传递给 **configure** 脚本的可选参数，以及 **configure** 自动检测到的配置参数。在报告 GDB 错误（参见第 32 [章 GDB 错误], 页 729）时，在您的报告中包含此信息非常重要。

This commands also displays the version number of the Readline library that GDB is using. After the version number will appear either ‘(system)’ indicating GDB is using the shared system Readline library, or ‘(internal)’ indicating GDB is using a statically linked in version of the Readline library.

此命令还会显示 GDB 正在使用的 Readline 库的版本号。版本号之后将显示 “（系统）” 表示 GDB 正在使用共享的系统 Readline 库，或 “（内部）” 表示 GDB 正在使用静态链接的 Readline 库版本。

4 Running Programs Under GDB

When you run a program under GDB, you must first generate debugging information when you compile it.

You may start GDB with its arguments, if any, in an environment of your choice. If you are doing native debugging, you may redirect your program's input and output, debug an already running process, or kill a child process.

4.1 Compiling for Debugging

In order to debug a program effectively, you need to generate debugging information when you compile it. This debugging information is stored in the object file; it describes the data type of each variable or function and the correspondence between source line numbers and addresses in the executable code.

To request debugging information, specify the `-g` option when you run the compiler.

Programs that are to be shipped to your customers are compiled with optimizations, using the `-O` compiler option. However, some compilers are unable to handle the `-g` and `-O` options together. Using those compilers, you cannot generate optimized executables containing debugging information.

GCC, the GNU C/C++ compiler, supports `-g` with or without `-O`, making it possible to debug optimized code. We recommend that you *always* use `-g` whenever you compile a program. You may think your program is correct, but there is no sense in pushing your luck. For more information, see Chapter 11 [Optimized Code], page 197.

Older versions of the GNU C compiler permitted a variant option `-gg` for debugging information. GDB no longer supports this format; if your GNU C compiler has this option, do not use it.

GDB knows about preprocessor macros and can show you their expansion (see Chapter 12 [Macros], page 201). Most compilers do not include information about preprocessor macros in the debugging information if you specify the `-g` flag alone. Version 3.1 and later of GCC, the GNU C compiler, provides macro information if you are using the DWARF debugging format, and specify the option `-g3`.

See Section “Options for Debugging Your Program or GCC” in *Using the GNU Compiler Collection (GCC)*, for more information on GCC options affecting debug information.

You will have the best debugging experience if you use the latest version of the DWARF debugging format that your compiler supports. DWARF is currently the most expressive and best supported debugging format in GDB.

GDB 下运行 4 个程序

在 GDB 下运行程序时，您必须在编译时首先生成调试信息。

您可以选择在您喜欢的环境中以 GDB 及其参数（如果有）启动。如果您正在进行原生调试，可以重定向程序的输入和输出，调试正在运行的进程，或杀死子进程。

4.1 调试编译

为了有效地调试程序，您需要在编译时生成调试信息。这些调试信息存储在对象文件中；它描述了每个变量或函数的数据类型以及源代码行号与可执行代码地址之间的对应关系。

要请求调试信息，请在运行编译器时指定 `-g` 选项。

要发往客户手中的程序会使用 `-O` 编译器选项进行优化编译。然而，某些编译器无法同时处理 `-g` 和 `-O` 选项。在这些编译器上，你无法生成包含调试信息的优化可执行文件。

GCC，即 GNU C/C++ 编译器，支持带或不带 `-O` 的 `-g` 选项，从而可以调试优化代码。我们建议你在编译任何程序时始终使用 `-g` 选项。你的程序可能正确，但冒险行事毫无意义。更多信息，请参阅第 11 [章优化代码],页197。

旧版本的 GNU C 编译器允许使用 `-gg` 选项来生成调试信息。GDB 不再支持这种格式；如果你的 GNU C 编译器有此选项，请勿使用。

GDB 了解预处理宏，并能显示它们的展开结果（见第12章[宏], 页201）。大多数编译器在仅指定-g选项时，不会在调试信息中包含预处理宏信息。GCC的 GNU C编译器3.1及以上版本，在使用DWARF调试格式并指定-g3选项时，会提供宏信息。

请参阅《使用 GNU 编译器集合 (GCC) 中的“调试您的程序或 GCC 的选项”一节，以获取有关影响调试信息的 GCC 选项的更多信息。

如果你使用编译器支持的最新的 DWARF 调试格式，你将获得最佳的调试体验。DWARF 目前是 GDB 中表达最丰富且支持最好的调试格式。

4.2 Starting your Program

run

r Use the **run** command to start your program under GDB. You must first specify the program name with an argument to GDB (see Chapter 2 [Getting In and Out of GDB], page 11), or by using the **file** or **exec-file** command (see Section 18.1 [Commands to Specify Files], page 293).

If you are running your program in an execution environment that supports processes, **run** creates an inferior process and makes that process run your program. In some environments without processes, **run** jumps to the start of your program. Other targets, like ‘remote’, are always running. If you get an error message like this one:

```
The "remote" target does not support "run".
Try "help target" or "continue".
```

then use **continue** to run your program. You may need **load** first (see [load], page 315).

The execution of a program is affected by certain information it receives from its superior. GDB provides ways to specify this information, which you must do *before* starting your program. (You can change it after starting your program, but such changes only affect your program the next time you start it.) This information may be divided into four categories:

The arguments.

Specify the arguments to give your program as the arguments of the **run** command. If a shell is available on your target, the shell is used to pass the arguments, so that you may use normal conventions (such as wildcard expansion or variable substitution) in describing the arguments. In Unix systems, you can control which shell is used with the **SHELL** environment variable. If you do not define **SHELL**, GDB uses the default shell (/bin/sh). You can disable use of any shell with the **set startup-with-shell** command (see below for details).

The environment.

Your program normally inherits its environment from GDB, but you can use the GDB commands **set environment** and **unset environment** to change parts of the environment that affect your program. See Section 4.4 [Your Program’s Environment], page 38.

The working directory.

You can set your program’s working directory with the command **set cwd**. If you do not set any working directory with this command, your program will inherit GDB’s working directory if native debugging, or the remote server’s working directory if remote debugging. See Section 4.5 [Your Program’s Working Directory], page 40.

The standard input and output.

Your program normally uses the same device for standard input and standard output as GDB is using. You can redirect input and output in the **run** command line, or you can use the **tty** command to set a different device for your program. See Section 4.6 [Your Program’s Input and Output], page 41.

4.2 启动你的程序

run

r 使用 **run** 命令在 GDB 下启动你的程序。你必须首先用参数指定程序名称给 GDB (参见第 2 [章《如何进入和退出 GDB》, 第 11 页), 或者使用 **file** 或 **exec-file** 命令 (参见第 18.1 [节《指定文件命令》, 第 293 页)。

如果你在支持进程的执行环境中运行你的程序, **run**会创建一个子进程并让该进程运行你的程序。在一些没有进程的环境里, **run**会跳转到你的程序起始处。其他目标 (如 ‘remote’) 总是处于运行状态。如果你收到类似以下的错误消息:

```
目标 "remote" 不支持 "run"。
尝试 "help target" 或 "continue"。
```

那么请使用**continue**来运行你的程序。你可能需要先加载 (参见 [load], 第315页)。

程序的执行会受到它从其上级接收的某些信息的影响。GDB 提供指定这些信息的方法, 你必须在启动程序之前这样做。(你可以在启动程序后更改它, 但此类更改仅在下次启动程序时才影响你的程序。) 这些信息可以分为四类:

参数。

指定作为运行命令参数传递给你的程序的参数。如果目标上可用 shell, 则使用 shell 传递参数, 因此你可以在描述参数时使用正常约定 (例如通配符扩展或变量替换)。在 Unix 系统中, 你可以使用 **SHELL** 环境变量控制使用的 shell。如果你未定义 **SHELL**, GDB 使用默认 shell (/bin/sh)。你可以使用 **set startup-with-shell** 命令禁用使用任何 shell (有关详细信息, 请参阅下文)。

环境。

你的程序通常从 GDB继承其环境, 但你可以使用 GDB 命令 **set environment** 和 **unset environment** 来更改影响你的程序的环境部分。见第 4.4 [页的程序环境],。

工作目录。

您可以使用命令 **set cwd** 设置程序的当前工作目录。如果您使用此命令未设置任何工作目录, 则程序将继承 GDB 的当前工作目录 (如果是本地调试), 或远程服务器的当前工作目录 (如果是远程调试)。参见第 4.5 [您的程序的工作目录],页 40。

标准输入和输出。

您的程序通常使用与 GDB 相同的设备作为标准输入和标准输出。您可以在运行命令行中重定向输入和输出, 或者使用**tty**命令为您的程序设置不同的设备。参见第4.6 [您的程序的输入和输出],页41。

Warning: While input and output redirection work, you cannot use pipes to pass the output of the program you are debugging to another program; if you attempt this, GDB is likely to wind up debugging the wrong program.

When you issue the `run` command, your program begins to execute immediately. See Chapter 5 [Stopping and Continuing], page 59, for discussion of how to arrange for your program to stop. Once your program has stopped, you may call functions in your program, using the `print` or `call` commands. See Chapter 10 [Examining Data], page 143.

If the modification time of your symbol file has changed since the last time GDB read its symbols, GDB discards its symbol table, and reads it again. When it does this, GDB tries to retain your current breakpoints.

start The name of the main procedure can vary from language to language. With C or C++, the main procedure name is always `main`, but other languages such as Ada do not require a specific name for their main procedure. The debugger provides a convenient way to start the execution of the program and to stop at the beginning of the main procedure, depending on the language used.

The `'start'` command does the equivalent of setting a temporary breakpoint at the beginning of the main procedure and then invoking the `'run'` command.

Some programs contain an *elaboration* phase where some startup code is executed before the main procedure is called. This depends on the languages used to write your program. In C++, for instance, constructors for static and global objects are executed before `main` is called. It is therefore possible that the debugger stops before reaching the main procedure. However, the temporary breakpoint will remain to halt execution.

Specify the arguments to give to your program as arguments to the `'start'` command. These arguments will be given verbatim to the underlying `'run'` command. Note that the same arguments will be reused if no argument is provided during subsequent calls to `'start'` or `'run'`.

It is sometimes necessary to debug the program during elaboration. In these cases, using the `start` command would stop the execution of your program too late, as the program would have already completed the elaboration phase. Under these circumstances, either insert breakpoints in your elaboration code before running your program or use the `starti` command.

starti The `'starti'` command does the equivalent of setting a temporary breakpoint at the first instruction of a program's execution and then invoking the `'run'` command. For programs containing an elaboration phase, the `starti` command will stop execution at the start of the elaboration phase.

`set exec-wrapper wrapper`
`show exec-wrapper`
`unset exec-wrapper`

When `'exec-wrapper'` is set, the specified wrapper is used to launch programs for debugging. GDB starts your program with a shell command of the form `exec wrapper program`. Quoting is added to `program` and its arguments, but not to `wrapper`, so you should add quotes if appropriate for your shell. The wrapper runs until it executes your program, and then GDB takes control.

警告：虽然输入和输出重定向可以工作，但您不能使用管道将正在调试的程序输出传递给另一个程序；如果您尝试这样做，GDB 很可能会调试错误程序。

当你发出run命令时，你的程序会立即开始执行。参见第 5 [停止和继续], 页59，了解如何安排你的程序停止。一旦你的程序停止，你可以使用print或call命令调用程序中的函数。参见第 10 [检查数据],页143。

如果你的符号文件的修改时间自上次 GDB 读取其符号以来已更改，GDB 会丢弃其符号表并重新读取。当它这样做时，GDB 尝试保留你当前的断点。

start 主程序的名字可能因语言而异。对于C或C++，主程序名始终是main，但其他语言如Ada则不需要为其主程序指定特定名称。调试器提供了一种方便的方法来启动程序的执行，并在主程序开始时停止，具体取决于所使用的语言。

'start' 命令执行的效果相当于在主程序开头设置一个临时断点，然后调用 'run' 命令。

某些程序包含一个细化阶段，在主程序被调用之前会执行一些启动代码。这取决于编写你程序所使用的语言。例如，在 C++ 中，静态和全局对象的构造函数会在 main 被调用之前执行。因此，调试器可能在到达主程序之前就停止了。但是，临时断点仍然会保持以停止执行。

将传递给你的程序的参数指定为 'start' 命令的参数。这些参数将原封不动地传递给底层的 'run' 命令。请注意，如果没有在后续的 'start' 或 'run' 调用期间提供参数，相同的参数将被重复使用。

有时需要在细化阶段调试程序。在这些情况下，使用 start 命令会导致程序执行停止得太晚，因为程序已经完成了细化阶段。在这种情况下，要么在运行程序之前在你的细化代码中插入断点，要么使用 starti 命令。

starti “starti” 命令相当于在程序的执行第一条指令处设置一个临时断点，然后调用 “run” 命令。对于包含展开阶段的程序，starti命令会在展开阶段开始时停止执行。

设置exec-wrapper wrapper
显示exec-wrapper
取消设置exec-wrapper

当设置 `'exec-wrapper'` 时，指定的包装器将用于启动用于调试的程序。

GDB 使用形如 `execwrapper program` 的 shell 命令启动您的程序。程序及其参数会被引号处理，但包装器不会，因此如果您的 shell 需要，您应该适当添加引号。包装器会一直运行直到执行您的程序，然后 GDB 接管控制权。

You can use any program that eventually calls `execve` with its arguments as a wrapper. Several standard Unix utilities do this, e.g. `env` and `nohup`. Any Unix shell script ending with `exec "$@"` will also work.

For example, you can use `env` to pass an environment variable to the debugged program, without setting the variable in your shell's environment:

```
(gdb) set exec-wrapper env 'LD_PRELOAD=libtest.so'
(gdb) run
```

This command is available when debugging locally on most targets, excluding DJGPP, Cygwin, and MS Windows.

```
set startup-with-shell
set startup-with-shell on
set startup-with-shell off
show startup-with-shell
```

On Unix systems, by default, if a shell is available on your target, GDB) uses it to start your program. Arguments of the `run` command are passed to the shell, which does variable substitution, expands wildcard characters and performs redirection of I/O. In some circumstances, it may be useful to disable such use of a shell, for example, when debugging the shell itself or diagnosing startup failures such as:

```
(gdb) run
Starting program: ./a.out
During startup program terminated with signal SIGSEGV, Segmentation fault.
```

which indicates the shell or the wrapper specified with `'exec-wrapper'` crashed, not your program. Most often, this is caused by something odd in your shell's non-interactive mode initialization file—such as `.cshrc` for C-shell, `$.zshenv` for the Z shell, or the file specified in the `BASH_ENV` environment variable for BASH.

```
set auto-connect-native-target
set auto-connect-native-target on
set auto-connect-native-target off
show auto-connect-native-target
```

By default, if the current inferior is not connected to any target yet (e.g., with `target remote`), the `run` command starts your program as a native process under GDB, on your local machine. If you're sure you don't want to debug programs on your local machine, you can tell GDB to not connect to the native target automatically with the `set auto-connect-native-target off` command.

If `on`, which is the default, and if the current inferior is not connected to a target already, the `run` command automatically connects to the native target, if one is available.

If `off`, and if the current inferior is not connected to a target already, the `run` command fails with an error:

```
(gdb) run
Don't know how to run. Try "help target".
```

If the current inferior is already connected to a target, GDB always uses it with the `run` command.

您可以使用任何最终以 `execve` 调用其参数的包装器程序。一些标准的 Unix 实用程序会这样做，例如 `env` 和 `nohup`。任何以 `exec "$@"` 结尾的 Unix shell 脚本也会工作。

例如，您可以使用 `env` 将环境变量传递给被调试程序，而无需在您的 shell 环境中设置该变量：

```
(gdb) set exec-wrapper env 'LD_PRELOAD=libtest.so'
(gdb) run
```

当在大多数目标上本地调试时，此命令可用，但排除 DJGPP、Cygwin 和 MSWindows。

```
设置 startup-with-shell
设置 startup-with-shell on
设置 startup-with-shell off
显示 startup-with-shell
```

在 Unix 系统上，默认情况下，如果目标系统上可用，GDB) 会使用它来启动您的程序。运行命令的参数会传递给 shell，shell 会进行变量替换、展开通配符并执行 I/O 重定向。在某些情况下，禁用这种 shell 使用可能很有用，例如，在调试 shell 本身或诊断启动失败时，如：

```
(gdb) run
Starting program: ./a.out
启动程序因信号 SIGSEGV 终止，段错误。
```

这表明 shell 或使用 `'exec-wrapper'` 指定的包装器崩溃了，而不是您的程序。通常，这是由您的 shell 的非交互模式初始化文件中的某些异常引起的——例如 C-shell 的 `.cshrc`、Z shell 的 `$.zshenv` 或 `BASH_ENV` 环境变量中指定的文件。

```
设置 auto-connect-native-target
设置 auto-connect-native-target 开启
关闭自动连接原生目标
显示自动连接原生目标
```

默认情况下，如果当前调试目标尚未连接到任何目标（例如使用远程目标），则运行命令会在 GDB 下以原生进程方式启动您的程序，在您的本地机器上。如果您确定不想在本地机器上调试程序，可以使用 `set auto-connect-native-target off` 命令让 GDB 不自动连接到原生目标。

如果开启（默认设置），并且当前调试目标尚未连接到任何目标，则运行命令会自动连接到可用的原生目标。

如果关闭，并且当前调试目标尚未连接到任何目标，则运行命令会因错误而失败：

```
(gdb) run
不知道如何运行。请尝试 "help target".
```

如果当前从属项已经连接到目标，GDB 始终在运行命令中使用它。

In any case, you can explicitly connect to the native target with the `target native` command. For example,

```
(gdb) set auto-connect-native-target off
(gdb) run
Don't know how to run. Try "help target".
(gdb) target native
(gdb) run
Starting program: ./a.out
[Inferior 1 (process 10421) exited normally]
```

In case you connected explicitly to the `native` target, GDB remains connected even if all inferiors exit, ready for the next `run` command. Use the `disconnect` command to disconnect.

Examples of other commands that likewise respect the `auto-connect-native-target` setting: `attach`, `info proc`, `info os`.

```
set disable-randomization
set disable-randomization on
```

This option (enabled by default in GDB) will turn off the native randomization of the virtual address space of the started program. This option is useful for multiple debugging sessions to make the execution better reproducible and memory addresses reusable across debugging sessions.

This feature is implemented only on certain targets, including GNU/Linux. On GNU/Linux you can get the same behavior using

```
(gdb) set exec-wrapper setarch 'uname -m' -R
```

```
set disable-randomization off
```

Leave the behavior of the started executable unchanged. Some bugs rear their ugly heads only when the program is loaded at certain addresses. If your bug disappears when you run the program under GDB, that might be because GDB by default disables the address randomization on platforms, such as GNU/Linux, which do that for stand-alone programs. Use `set disable-randomization off` to try to reproduce such elusive bugs.

On targets where it is available, virtual address space randomization protects the programs against certain kinds of security attacks. In these cases the attacker needs to know the exact location of a concrete executable code. Randomizing its location makes it impossible to inject jumps misusing a code at its expected addresses.

Prelinking shared libraries provides a startup performance advantage but it makes addresses in these libraries predictable for privileged processes by having just unprivileged access at the target system. Reading the shared library binary gives enough information for assembling the malicious code misusing it. Still even a prelinked shared library can get loaded at a new random address just requiring the regular relocation process during the startup. Shared libraries not already prelinked are always loaded at a randomly chosen address.

Position independent executables (PIE) contain position independent code similar to the shared libraries and therefore such executables get loaded at a randomly chosen address upon startup. PIE executables always load even already

无论如何，您都可以使用目标原生命令显式连接到原生目标。例如，

```
(gdb) set auto-connect-native-target off
(gdb) run
不知道如何运行。尝试 "help target"。
(gdb) target native
(gdb) run
正在启动程序: ./a.out
[从属程序 1 (进程 10421) 正常退出]
```

如果你明确连接到原生目标，GDB 即使所有从属程序退出，仍然保持连接，准备执行下一个运行命令。使用 `disconnect` 命令断开连接。

其他同样尊重 `auto-connect-native-` 目标设置的命令示例：`attach`、`info proc`、`info os`。

```
set disable-randomization
set disable-randomization on
```

此选项（在 GDB 中默认启用）将关闭启动程序的虚拟地址空间的本地随机化。此选项对于多个调试会话很有用，可以更好地重现执行并使内存地址跨调试会话可重用。

此功能仅在特定目标上实现，包括 GNU/Linux。在 GNU/Linux 上，您可以使用以下方式获得相同的行为：

```
(gdb) 设置执行包装器 setarch 'uname -m' -R
```

关闭随机化

保持启动的可执行文件行为不变。某些错误只有在程序加载到特定地址时才会出现。如果你在 GDB 下运行程序时错误消失，那可能是因为 GDB 默认禁用了地址随机化，这在 GNU/Linux 等为独立程序进行地址随机化的平台上是如此。使用 `set disable-randomization off` 尝试重现这种难以捉摸的错误。

在可用目标上，虚拟地址空间随机化可以保护程序免受某些类型的安全攻击。在这些情况下，攻击者需要知道具体可执行代码的确切位置。随机化其位置使得利用其预期地址注入跳转变得不可能。

预链接共享库提供了启动性能优势，但它通过在目标系统上仅具有非特权访问权限，使得这些库中的地址对特权进程可预测。读取共享库二进制文件提供了足够的信息来组装利用它的恶意代码。即使预链接的共享库也可以在新的随机地址上加载，这只需要在启动期间进行常规的重定位过程。尚未预链接的共享库始终加载到随机选择的地址。

位置独立可执行文件（PIE）包含与共享库类似的独立代码位置代码，因此此类可执行文件在启动时会在随机选择的地址上加载。PIE 可执行文件始终会加载，即使已经

prelinked shared libraries at a random address. You can build such executable using `gcc -fPIE -pie`.

Heap (malloc storage), stack and custom mmap areas are always placed randomly (as long as the randomization is enabled).

show disable-randomization

Show the current setting of the explicit disable of the native randomization of the virtual address space of the started program.

4.3 Your Program's Arguments

The arguments to your program can be specified by the arguments of the `run` command. They are passed to a shell, which expands wildcard characters and performs redirection of I/O, and thence to your program. Your `SHELL` environment variable (if it exists) specifies what shell GDB uses. If you do not define `SHELL`, GDB uses the default shell (`/bin/sh` on Unix).

On non-Unix systems, the program is usually invoked directly by GDB, which emulates I/O redirection via the appropriate system calls, and the wildcard characters are expanded by the startup code of the program, not by the shell.

`run` with no arguments uses the same arguments used by the previous `run`, or those set by the `set args` command.

set args Specify the arguments to be used the next time your program is run. If `set args` has no arguments, `run` executes your program with no arguments. Once you have run your program with arguments, using `set args` before the next `run` is the only way to run it again without arguments.

It is possible to set arguments containing a newline character. This can be done by enclosing an argument within a quotes. For example, start by entering:

```
(gdb) set args "ab
```

and then enter the newline, GDB gives a secondary prompt `>` and allows you to continue entering the arguments:

```
(gdb) set args "ab
>cd"
(gdb) show args
Argument list to give program being debugged when it is started is "'ab
cd"'.
```

On Unix systems both single or double quotes can be used around arguments containing a newline. On MS-Windows, only double quotes can be used.

show args Show the arguments to give your program when it is started.

4.4 Your Program's Environment

The *environment* consists of a set of environment variables and their values. Environment variables conventionally record such things as your user name, your home directory, your terminal type, and your search path for programs to run. Usually you set up environment variables with the shell and they are inherited by all the other programs you run. When

在随机地址上加载预链接共享库。您可以使用 `gcc -fPIE -pie` 构建此类可执行文件。

堆 (malloc 存储)、栈和自定义 mmap 区域始终随机放置 (只要随机化功能已启用)。

显示禁用随机化

显示原生随机化的显式禁用当前设置

启动程序的虚拟地址空间

4.3 您程序的参数

你的程序参数可以通过运行命令的参数指定。它们会被传递给一个 shell, 该 shell 会展开通配符并执行 I/O 重定向, 然后传递给你的程序。你的 `SHELL` 环境变量 (如果存在) 指定了 GDB 使用的 shell。如果你没有定义 `SHELL`, GDB 会使用默认 shell (Unix 上的 `/bin/sh`)。

在非 Unix 系统上, 程序通常由 GDB 直接调用, 该程序通过适当的系统调用模拟 I/O 重定向, 并且通配符由程序的启动代码展开, 而不是由 shell 展开。

不带参数运行时, 会使用上次运行时使用的相同参数, 或由 `set args` 命令设置的参数。

`set args` 指定下次运行你的程序时要使用的参数。如果 `set args` 没有参数, `run` 会用不带参数的方式执行你的程序。一旦你用参数运行了程序, 在下次运行前使用 `set args` 是唯一一种不带参数再次运行它的方法。

可以设置包含换行符的参数。这可以通过将参数用引号括起来来完成。例如, 先输入:

```
(gdb) set args "ab
```

然后输入换行符, GDB 会给出一个次要提示 `>` 并允许你继续输入参数:

```
(gdb) set args "ab
>cd"
(gdb) show args
Argument list to give program being debugged when it is started is "'ab
cd"'.
```

在 Unix 系统中, 包含换行符的参数周围可以使用单引号或双引号。在 MS-Windows 中, 只能使用双引号。

`show args` 显示启动时给程序的参数。

4.4 你的程序的运行环境

环境由一组环境变量及其值组成。环境变量通常记录诸如您的用户名、主目录、终端类型以及程序搜索路径等信息。通常您通过 shell 设置环境变量, 并且它们会被您运行的所有其他程序继承。当

debugging, it can be useful to try running your program with a modified environment without having to start GDB over again.

path directory

Add *directory* to the front of the PATH environment variable (the search path for executables) that will be passed to your program. The value of PATH used by GDB does not change. You may specify several directory names, separated by whitespace or by a system-dependent separator character (':' on Unix, ';' on MS-DOS and MS-Windows). If *directory* is already in the path, it is moved to the front, so it is searched sooner.

You can use the string '\$cwd' to refer to whatever is the current working directory at the time GDB searches the path. If you use '.' instead, it refers to the directory where you executed the **path** command. GDB replaces '.' in the *directory* argument (with the current path) before adding *directory* to the search path.

show paths

Display the list of search paths for executables (the PATH environment variable).

show environment [varname]

Print the value of environment variable *varname* to be given to your program when it starts. If you do not supply *varname*, print the names and values of all environment variables to be given to your program. You can abbreviate **environment** as **env**.

set environment varname [=value]

Set environment variable *varname* to *value*. The value changes for your program (and the shell GDB uses to launch it), not for GDB itself. The *value* may be any string; the values of environment variables are just strings, and any interpretation is supplied by your program itself. The *value* parameter is optional; if it is eliminated, the variable is set to a null value.

For example, this command:

```
set env USER = foo
```

tells the debugged program, when subsequently run, that its user is named 'foo'. (The spaces around '=' are used for clarity here; they are not actually required.)

Note that on Unix systems, GDB runs your program via a shell, which also inherits the environment set with **set environment**. If necessary, you can avoid that by using the 'env' program as a wrapper instead of using **set environment**. See [set exec-wrapper], page 35, for an example doing just that.

Environment variables that are set by the user are also transmitted to **gdbserver** to be used when starting the remote inferior. see [QEnvironmentHexEncoded], page 813.

unset environment varname

Remove variable *varname* from the environment to be passed to your program. This is different from 'set env *varname* ='; **unset environment** removes the variable from the environment, rather than assigning it an empty value.

调试时，尝试使用修改后的环境运行您的程序（而无需重新启动 GDB）可能很有用。

路径 directory

将目录添加到 PATH 环境变量（可执行文件的搜索路径）的前面，该变量将传递给您的程序。GDB 使用的 PATH 值不会改变。您可以指定多个目录名，用空格或系统相关的分隔符字符（Unix 上为 ':'，MS-DOS 和 MS-Windows 上为 ';'）分隔。如果目录已经在路径中，它会被移到前面，以便更早地被搜索。

您可以使用字符串 '\$cwd' 来引用 GDB 搜索路径时当前的当前工作目录。如果您使用 '.' 代替，它指的是您执行 path 命令的目录。GDB 在将目录添加到搜索路径之前，会用当前路径替换目录参数中的 '.'。

显示路径

显示可执行文件（PATH 环境变量）的搜索路径列表。

显示环境 [varname]

打印环境变量 *varname* 的值，该值将在程序启动时传递给程序。如果您不提供 *varname*，将打印传递给程序的所有环境变量的名称和值。可以将环境缩写为 **env**。

设置环境 varname [=value]

将环境变量 *varname* 设置为 *value*。该值将更改您的程序（以及启动它的 shell GDB）的值，而不是 GDB 本身的值。值可以是任何字符串；环境变量的值只是字符串，任何解释都由您的程序本身提供。值参数是可选的；如果省略它，则变量将设置为空值。

For example, this command:

```
set env USER = foo
```

告诉被调试的程序，在随后运行时，其用户名为“foo”。（这里的“=”周围的空格用于清晰表示；实际上它们不是必需的。）

请注意，在 Unix 系统上，GDB 通过 shell 运行您的程序，该 shell 也会继承使用 **set environment** 设置的环境。如有必要，您可以通过使用“env”程序作为包装器来避免这种情况，而不是使用 **set environment**。有关示例，请参阅 [set exec-wrapper]，页 35，该示例正是这样做的。

用户设置的环境变量也会传递给 **gdbserver**，在启动远程子进程时使用。请参考 [QEnvironmentHexEncoded]，页面第 813 页。

unset 环境 varname

从传递给程序的环境中移除变量 *varname*。这与 'set env *varname* =' 不同；**unset 环境** 会从环境中移除变量，而不是将其赋值为空值。

Environment variables that are unset by the user are also unset on `gdbserver` when starting the remote inferior. see [QEnvironmentUnset], page 814.

Warning: On Unix systems, GDB runs your program using the shell indicated by your SHELL environment variable if it exists (or `/bin/sh` if not). If your SHELL variable names a shell that runs an initialization file when started non-interactively—such as `.cshrc` for C-shell, `$.zshenv` for the Z shell, or the file specified in the `BASH_ENV` environment variable for BASH—any variables you set in that file affect your program. You may wish to move setting of environment variables to files that are only run when you sign on, such as `.login` or `.profile`.

4.5 Your Program's Working Directory

Each time you start your program with `run`, the inferior will be initialized with the current working directory specified by the `set cwd` command. If no directory has been specified by this command, then the inferior will inherit GDB's current working directory as its working directory if native debugging, or it will inherit the remote server's current working directory if remote debugging.

set cwd [*directory*]

Set the inferior's working directory to *directory*, which will be `glob`-expanded in order to resolve tildes (`~`). If no argument has been specified, the command clears the setting and resets it to an empty state. This setting has no effect on GDB's working directory, and it only takes effect the next time you start the inferior. The `~` in *directory* is a short for the *home directory*, usually pointed to by the `HOME` environment variable. On MS-Windows, if `HOME` is not defined, GDB uses the concatenation of `HOMEDRIVE` and `HOMEPATH` as fallback.

You can also change GDB's current working directory by using the `cd` command. See [cd command], page 40.

show cwd Show the inferior's working directory. If no directory has been specified by `set cwd`, then the default inferior's working directory is the same as GDB's working directory.

cd [*directory*]

Set the GDB working directory to *directory*. If not given, *directory* uses `'~'`.

The GDB working directory serves as a default for the commands that specify files for GDB to operate on. See Section 18.1 [Commands to Specify Files], page 293. See [set cwd command], page 40.

pwd Print the GDB working directory.

It is generally impossible to find the current working directory of the process being debugged (since a program can change its directory during its run). If you work on a system where GDB supports the `info proc` command (see Section 21.1.2 [Process Information], page 339), you can use the `info proc` command to find out the current working directory of the debuggee.

用户未设置的环境变量在启动远程子进程时也会在 `gdbserver` 上被移除。参见 [QEnvironmentUnset], 页面第 814 页。

警告: 在Unix系统上, GDB 会使用您的SHELL环境变量指定的shell来运行您的程序(如果存在, 否则使用/bin/sh)。如果您的SHELL变量指定了一个在非交互式启动时运行初始化文件的shell——例如C-shell的.cshrc、Z shell的\$.zshenv, 或用于BASH的BASH_ENV环境变量指定的文件——那么您在该文件中设置的任何变量都会影响您的程序。您可能希望将环境变量的设置移动到仅在您登录时运行的文件中, 例如.login或.profile。

4.5 您的程序的工作目录

每次您使用`run`启动程序时, 都会使用 `set cwd` 命令指定的当前工作目录来初始化子进程。如果该命令未指定目录, 那么如果使用本地调试, 子进程将继承 GDB的当前工作目录作为其工作目录; 如果使用远程调试, 则子进程将继承远程服务器的当前工作目录。

设置当前工作目录 [*directory*]

将子进程的工作目录设置为目录, 该目录将按顺序进行通配符扩展以解析波浪号 (`~`)。如果没有指定参数, 该命令将清除设置并将其重置为空状态。此设置对 GDB 的工作目录无效, 并且仅在下次启动子进程时生效。目录中的 `~` 是家目录的缩写, 通常指向 `HOME` 环境变量。在 MS-Windows 上, 如果 `HOME` 未定义, GDB 使用 `HOMEDRIVE` 和 `HOMEPATH` 的连接作为备用。

您也可以通过使用 `cd` 命令更改 GDB 的当前工作目录。参见 [cd 命令], 页 40。

显示当前工作目录 显示子进程的工作目录。如果没有通过 `set cwd` 指定目录, 则默认子进程的工作目录与 GDB 的工作目录相同。

cd [*directory*]

将 GDB 工作目录设置为目录。如果未指定, 目录使用 `'~'`。

GDB 工作目录是用于指定文件供 GDB 操作的命令的默认目录。参见第18.1 [指定文件命令], 页293。参见 [设置当前工作目录命令], 页40。

pwd 打印 GDB 工作目录。

通常情况下, 无法找到正在调试的进程的当前工作目录(因为程序在运行时可以更改其目录)。如果你在支持 `info proc` 命令的系统上工作(参见第 21.1.2 [进程信息], 页 339), 你可以使用 `info proc` 命令来获取被调试进程的当前工作目录。

4.6 Your Program’s Input and Output

By default, the program you run under GDB does input and output to the same terminal that GDB uses. GDB switches the terminal to its own terminal modes to interact with you, but it records the terminal modes your program was using and switches back to them when you continue running your program.

info terminal

Displays information recorded by GDB about the terminal modes your program is using.

You can redirect your program’s input and/or output using shell redirection with the **run** command. For example,

```
run > outfile
```

starts your program, diverting its output to the file **outfile**.

Another way to specify where your program should do input and output is with the **tty** command. This command accepts a file name as argument, and causes this file to be the default for future **run** commands. It also resets the controlling terminal for the child process, for future **run** commands. For example,

```
tty /dev/ttyb
```

directs that processes started with subsequent **run** commands default to do input and output on the terminal **/dev/ttyb** and have that as their controlling terminal.

An explicit redirection in **run** overrides the **tty** command’s effect on the input/output device, but not its effect on the controlling terminal.

When you use the **tty** command or redirect input in the **run** command, only the input *for your program* is affected. The input for GDB still comes from your terminal. **tty** is an alias for **set inferior-tty**.

You can use the **show inferior-tty** command to tell GDB to display the name of the terminal that will be used for future runs of your program.

set inferior-tty [tty]

Set the tty for the program being debugged to *tty*. Omitting *tty* restores the default behavior, which is to use the same terminal as GDB.

show inferior-tty

Show the current tty for the program being debugged.

4.7 Debugging an Already-running Process

attach process-id

This command attaches to a running process—one that was started outside GDB. (**info files** shows your active targets.) The command takes as argument a process ID. The usual way to find out the *process-id* of a Unix process is with the **ps** utility, or with the ‘**jobs -l**’ shell command.

attach does not repeat if you press RET a second time after executing the command.

4.6 您程序的输入和输出

默认情况下，您在 GDB 下运行的程序会将输入和输出到 GDB 使用的同一个终端。GDB 会切换终端到它自己的终端模式来与您交互，但它会记录您的程序所使用的终端模式，并在您继续运行程序时切换回这些模式。

信息终端

显示 GDB 记录的关于您程序正在使用的终端模式的信息。

您可以使用 **run** 命令的 **shell** 重定向功能来重定向程序的输入和/或输出。例如，

```
run >outfile
```

启动您的程序，并将输出重定向到文件 **outfile**。

另一种指定您的程序应进行输入和输出的方式是使用 **tty** 命令。该命令接受一个文件名作为参数，并将此文件设置为未来 **run** 命令的默认值。它还会重置子进程的控制终端，用于未来的 **run** 命令。例如，

```
tty /dev/ttyb
```

指示后续使用 **run** 命令启动的程序默认在终端 **/dev/ttyb** 上进行输入和输出，并将该终端作为其控制终端。

在 **run** 命令中的显式重定向会覆盖 **tty** 命令对输入/输出设备的影响，但不会影响其对控制终端的影响。

当您使用 **tty** 命令或在 **run** 命令中重定向输入时，仅影响您的程序的输入。GDB 的输入仍然来自您的终端。**tty** 是 **set inferior-tty** 的别名。

您可以使用 **show inferior-tty** 命令来告诉 GDB 显示未来运行您的程序时将使用的终端名称。

set inferior-tty [tty]

将正在调试的程序使用的终端设置为 *tty*。省略 *tty* 会恢复默认行为，即使用与 GDB 相同的终端。

show inferior-tty

显示正在调试的程序当前的终端。

4.7 调试已运行进程

附加 *process-id*

此命令附加到一个正在运行的进程——该进程是在外部启动的GDB。（信息文件显示你的活动目标。）该命令以进程ID作为参数。通常使用**ps**实用程序或‘**jobs -l**’ shell命令来获取Unix进程的进程ID。

执行命令后，如果第二次按下 RET，附加操作不会重复。

To use **attach**, your program must be running in an environment which supports processes; for example, **attach** does not work for programs on bare-board targets that lack an operating system. You must also have permission to send the process a signal.

When you use **attach**, the debugger finds the program running in the process first by looking in the current working directory, then (if the program is not found) by using the source file search path (see Section 9.5 [Specifying Source Directories], page 133). You can also use the **file** command to load the program. See Section 18.1 [Commands to Specify Files], page 293.

If the debugger can determine that the executable file running in the process it is attaching to does not match the current **exec-file** loaded by GDB, the option **exec-file-mismatch** specifies how to handle the mismatch. GDB tries to compare the files by comparing their build IDs (see [build ID], page 303), if available.

set exec-file-mismatch 'ask|warn|off'

Whether to detect mismatch between the current executable file loaded by GDB and the executable file used to start the process. If **'ask'**, the default, display a warning and ask the user whether to load the process executable file; if **'warn'**, just display a warning; if **'off'**, don't attempt to detect a mismatch. If the user confirms loading the process executable file, then its symbols will be loaded as well.

show exec-file-mismatch

Show the current value of **exec-file-mismatch**.

The first thing GDB does after arranging to debug the specified process is to stop it. You can examine and modify an attached process with all the GDB commands that are ordinarily available when you start processes with **run**. You can insert breakpoints; you can step and continue; you can modify storage. If you would rather the process continue running, you may use the **continue** command after attaching GDB to the process.

detach When you have finished debugging the attached process, you can use the **detach** command to release it from GDB control. Detaching the process continues its execution. After the **detach** command, that process and GDB become completely independent once more, and you are ready to **attach** another process or start one with **run**. **detach** does not repeat if you press **RET** again after executing the command.

If you exit GDB while you have an attached process, you detach that process. If you use the **run** command, you kill that process. By default, GDB asks for confirmation if you try to do either of these things; you can control whether or not you need to confirm by using the **set confirm** command (see Section 22.9 [Optional Warnings and Messages], page 386).

4.8 Killing the Child Process

kill Kill the child process in which your program is running under GDB.

This command is useful if you wish to debug a core dump instead of a running process. GDB ignores any core dump file while your program is running.

On some operating systems, a program cannot be executed outside GDB while you have breakpoints set on it inside GDB. You can use the **kill** command in this situation to permit running your program outside the debugger.

要使用attach，你的程序必须在支持进程的环境下运行；例如，attach不适用于没有操作系统的裸板目标上的程序。你还必须有权限向进程发送信号。

当你使用attach时，调试器首先会在当前工作目录中查找正在运行的程序，如果程序未找到，则使用源文件搜索路径（参见第9.5 [指定源目录], 页133）。你也可以使用file命令加载程序。参见第18.1 [指定文件的命令], 页293。

如果调试器可以确定进程正在运行的可执行文件与 GDB当前加载的exec-file不匹配，则选项exec-file-mismatch指定如何处理不匹配。GDB 尝试通过比较它们的构建ID（参见[构建ID], 页303，如果可用）来比较文件。

set exec-file-mismatch 'ask|warn|off'

是否检测 GDB当前加载的可执行文件与启动进程所使用的可执行文件之间是否存在不匹配。如果为 **'ask'**，默认值，显示警告并询问用户是否要加载进程的可执行文件；如果为 **'warn'**，仅显示警告；如果为 **'off'**，则不尝试检测不匹配。如果用户确认加载进程的可执行文件，则其符号也将被加载。

显示执行文件不匹配

Show the current value of **exec-file-mismatch**.

在安排调试指定进程后，GDB 首先会停止该进程。您可以使用 **run** 命令启动进程时通常可用的所有 GDB 命令来检查和修改附加的进程。您可以插入断点；您可以单步执行和继续；您可以修改存储。如果您希望进程继续运行，可以在将 GDB 附加到进程后使用 **continue** 命令。

detach 完成附加进程的调试后，您可以使用 **detach** 命令将其从 GDB 控制中释放。解除附加进程会继续其执行。**detach** 命令执行后，该进程和 GDB 再次完全独立，您可以附加另一个进程或使用 **run** 命令启动一个。如果执行命令后再次按下 **RET**，**detach** 不会重复执行。

如果您在附加进程时退出 GDB，则会解除附加该进程。如果您使用 **run** 命令，则会杀死该进程。默认情况下，GDB 如果您尝试执行这些操作，会请求确认；您可以通过使用 **set confirm** 命令（见第 22.9 [可选警告和消息], 页 386）来控制是否需要确认。

4.8 终止子进程

kill 在 GDB 中终止你的程序正在运行的子进程。

如果你希望调试核心转储而不是运行中的进程，这个命令很有用。GDB 在你的程序运行时忽略任何核心转储文件。

在某些操作系统中，当你在 GDB 中为其设置了断点时，程序无法在 GDB 外部执行。在这种情况下，你可以使用 **kill** 命令来允许程序在调试器外部运行。

The `kill` command is also useful if you wish to recompile and relink your program, since on many systems it is impossible to modify an executable file while it is running in a process. In this case, when you next type `run`, GDB notices that the file has changed, and reads the symbol table again (while trying to preserve your current breakpoint settings).

4.9 Debugging Multiple Inferiors Connections and Programs

GDB lets you run and debug multiple programs in a single session. In addition, GDB on some systems may let you run several programs simultaneously (otherwise you have to exit from one before starting another). On some systems GDB may even let you debug several programs simultaneously on different remote systems. In the most general case, you can have multiple threads of execution in each of multiple processes, launched from multiple executables, running on different machines.

GDB represents the state of each program execution with an object called an *inferior*. An inferior typically corresponds to a process, but is more general and applies also to targets that do not have processes. Inferiors may be created before a process runs, and may be retained after a process exits. Inferiors have unique identifiers that are different from process ids. Usually each inferior will also have its own distinct address space, although some embedded targets may have several inferiors running in different parts of a single address space. Each inferior may in turn have multiple threads running in it.

The commands `info inferiors` and `info connections`, which will be introduced below, accept a space-separated *ID list* as their argument specifying one or more elements on which to operate. A list element can be either a single non-negative number, like ‘5’, or an ascending range of such numbers, like ‘5-7’. A list can consist of any combination of such elements, even duplicates or overlapping ranges are valid. E.g. ‘1 4-6 5 4-4’ or ‘1 2 4-7’.

To find out what inferiors exist at any moment, use `info inferiors`:

info inferiors

Print a list of all inferiors currently being managed by GDB. By default all inferiors are printed, but the ID list *id...* can be used to limit the display to just the requested inferiors.

GDB displays for each inferior (in this order):

- 1. the inferior number assigned by GDB
- 2. the target system’s inferior identifier
- 3. the target connection the inferior is bound to, including the unique connection number assigned by GDB, and the protocol used by the connection.
- 4. the name of the executable the inferior is running.

If an inferior is currently debugging a core file, then an additional line will be printed after the line describing the inferior, this additional line gives the file name of the core file being debugged.

An asterisk ‘*’ preceding the GDB inferior number indicates the current inferior.

For example,

(gdb) info inferiors			
Num	Description	Connection	Executable
* 1	process 3401	1 (native)	goodbye
2	process 2307	2 (extended0remote host:10000)	hello

如果你希望重新编译和链接你的程序，`kill` 命令也非常有用，因为在许多系统中，当可执行文件正在进程运行时，无法修改它。在这种情况下，当你下次键入 `run` 时，GDB 会注意到文件已更改，并重新读取符号表（同时尝试保留你当前的断点设置）。

4.9 调试多个子进程连接和程序

GDB 允许你在单个会话中运行和调试多个程序。此外，GDB 在某些系统上甚至可以让你同时运行多个程序（否则你必须先退出一个才能开始另一个）。在某些系统上，GDB 甚至可以让你在不同的远程系统上同时调试多个程序。在最一般的情况下，你可以在多个进程中的每个进程中有多个执行线程，这些进程由多个可执行文件启动，并在不同的机器上运行。

GDB 使用一个称为子进程的对象来表示每个程序执行的状态。子进程通常对应于一个进程，但更通用，也适用于没有进程的目标。子进程可以在进程运行之前创建，并且在进程退出后仍然保留。子进程具有唯一的标识符，这些标识符与进程 ID 不同。通常每个子进程都会有自己独立的地址空间，尽管某些嵌入式目标可能允许多个子进程在单个地址空间的不同部分运行。每个子进程可以进一步有多个线程在其中运行。

下面将介绍的命令 `info inferiors` 和 `info connections` 会接受一个空格分隔的 ID 列表作为参数，用于指定要操作的一个或多个元素。列表元素可以是单个非负数，如 ‘5’，也可以是这种数字的升序范围，如 ‘5-7’。列表可以包含这些元素的任意组合，即使包含重复项或重叠范围也是有效的。例如 ‘1 4-6 5 4-4’ 或 ‘1 2 4-7’。

要查询当前时刻存在的 inferior，请使用 `info inferiors`：

info inferiors

打印由 GDB 当前管理的所有 inferior 列表。默认情况下会打印所有 inferior，但可以使用 ID 列表 *id...* 限制显示为仅请求的 inferior。

GDB 会为每个 inferior（按此顺序）显示：

- 1. 由 GDB分配的较低编号
- 2. 目标系统的较低标识符
- 3. 较低程序绑定到的目标连接，包括由 GDB分配的唯一连接编号，以及连接使用的协议。
- 4. 较低程序正在运行的可执行文件名称。

如果较低程序当前正在调试核心文件，那么在描述较低程序的行之后会打印出额外的行，该额外行会给出正在调试的核心文件名称。

在 GDB 较低编号前前置的星号 ‘*’ 表示当前较低程序。

例如，

(gdb) info inferiors			
Num	描述	连接	可执行文件
* 1	进程 3401	1 (本地)	再见
2	处理 2307	2 (扩展0远程主机:10000)	你好

Adding an additional inferior that is debugging a core file causes the additional core file line to be displayed, for example

```
(gdb) info inferiors
Num  Description      Connection      Executable
* 1   process 3401     1 (native)      goodbye
  2   process 2307     2 (extended-remote host:10000) hello
  3   process 1578     3 (core)        broken
      core file core.1578
```

To get information about the current inferior, use `inferior`:

`inferior` Shows information about the current inferior.

For example,

```
(gdb) inferior
[Current inferior is 1 [process 3401] (helloworld)]
```

To find out what open target connections exist at any moment, use `info connections`:

`info connections`

Print a list of all open target connections currently being managed by GDB. By default all connections are printed, but the ID list `id...` can be used to limit the display to just the requested connections.

GDB displays for each connection (in this order):

1. the connection number assigned by GDB.
2. the protocol used by the connection.
3. a textual description of the protocol used by the connection.

An asterisk ‘*’ preceding the connection number indicates the connection of the current inferior.

For example,

```
(gdb) info connections
Num  What      Description
* 1   extended-remote host:10000 Extended remote serial target in gdb-specific protocol
  2   native      Native process
  3   core        Local core dump file
```

To switch focus between inferiors, use the `inferior` command:

`inferior infno`

Make inferior number *infno* the current inferior. The argument *infno* is the inferior number assigned by GDB, as shown in the first field of the ‘`info inferiors`’ display.

The debugger convenience variable ‘`$_inferior`’ contains the number of the current inferior. You may find this useful in writing breakpoint conditional expressions, command scripts, and so forth. See Section 10.12 [Convenience Variables], page 173, for general information on convenience variables.

You can get multiple executables into a debugging session via the `add-inferior` and `clone-inferior` commands. On some systems GDB can add inferiors to the debug session automatically by following calls to `fork` and `exec`. To remove inferiors from the debugging session use the `remove-inferiors` command.

添加一个额外的inferior，该inferior正在调试corefile，会导致额外的corefile行被显示，例如

```
(gdb) info inferiors
Num  描述      连接      可执行文件
* 1   进程 3401   1 (本地)   再见
  2   处理 2307   2 (扩展-远程主机:10000) 你好
  3   处理 1578   3 (核心)   损坏
      核心文件 core.1578
```

要获取当前调试目标的有关信息，请使用 `inferior`：

`inferior` 显示当前调试目标的信息。

例如，

```
(gdb) inferior
[当前inferior是 1 [进程 3401](helloworld)]
```

要查询当前时刻存在的所有打开目标连接，请使用`info connections`：

`info connections`

打印由 GDB当前管理的所有打开目标连接的列表。默认情况下会打印所有连接，但可以使用ID列表id...来限制显示仅为您请求的连接。

GDB 为每个连接（按此顺序）显示：

1. 由 GDB分配的连接编号。
2. 该连接使用的协议。
3. 该连接使用的协议的文本描述。

连接编号前的星号 ‘*’ 表示当前子进程的连接。

例如，

```
(gdb) info connections
Num  What      描述
* 1   远程主机:10000 gdb特定协议中的扩展远程串行目标
  2   本地      本地进程
  3   core      本地核心转储文件
```

在下属之间切换焦点，请使用 `inferior` 命令：

`inferior infno`

将 inferior 编号 infno 设置为当前下属。参数 infno 是由 GDB 分配的下属编号，如 “`info inferiors`” 显示的第一字段所示。

调试器便利变量 ‘`$_inferior`’ 包含当前下属的编号。在编写断点条件表达式、命令脚本等时，您可能会发现这个变量很有用。有关便利变量的详细信息，请参阅第 10.12 [Convenience Variables], 页 173。

您可以通过 `add-inferior` 和 `clone-inferior` 命令将多个可执行文件添加到调试会话中。在某些系统上，GDB 可以通过 `fork` 和 `exec` 调用自动将 inferior 添加到调试会话中。要从调试会话中移除 inferior，请使用 `remove-inferiors` 命令。

`add-inferior [ -copies n ] [ -exec executable ] [-no-connection ]`

Adds *n* inferiors to be run using *executable* as the executable; *n* defaults to 1. If no executable is specified, the inferiors begins empty, with no program. You can still assign or change the program assigned to the inferior at any time by using the `file` command with the executable name as its argument.

By default, the new inferior begins connected to the same target connection as the current inferior. For example, if the current inferior was connected to `gdbserver` with `target remote`, then the new inferior will be connected to the same `gdbserver` instance. The ‘`-no-connection`’ option starts the new inferior with no connection yet. You can then for example use the `target remote` command to connect to some other `gdbserver` instance, use `run` to spawn a local program, etc.

Not all connections can be shared between inferiors. For example, the `target core` target is unique for each inferior. That is, multiple inferiors can use `target core` at the same time, but each `target core` is different. If you try to `add-inferior`, and the current inferior is `target core`, then GDB will give a warning and create the new inferior without a connection, like this

```
(gdb) file test1
Reading symbols from test1...
(gdb) target core core.test1.433190
[New LWP 433190]
Core was generated by './test1'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0 0x0000000000401111 in foo () at test1.c:6
6 return *global_ptr;
(gdb) add-inferior
[New inferior 2]
warning: can't share connection 1 (core) between inferiors
Added inferior 2
```

Another target that cannot be shared is `target remote` (see Section 20.1 [Types of Remote Connections], page 317). A remote target doesn’t allow new inferiors to be started, as such creating a new, non-running, inferior that shares a remote connection doesn’t make sense. As with the core target above, if you try to `add-inferior`, and the current inferior is `target remote`, then GDB will give a warning and create the new inferior without a connection.

`clone-inferior [ -copies n ] [ -no-connection ] [ infno ]`

Adds *n* inferiors ready to execute the same program as inferior *infno*; *n* defaults to 1, and *infno* defaults to the number of the current inferior. This command copies the values of the *args*, *inferior-tty* and *cwd* properties from the current inferior to the new one. It also propagates changes the user made to environment variables using the `set environment` and `unset environment` commands. This is a convenient command when you want to run another instance of the inferior you are debugging.

```
(gdb) info inferiors
Num Description Connection Executable
* 1 process 29964 1 (native) helloworld
(gdb) clone-inferior
Added inferior 2.
1 inferiors added.
```

`add-inferior [ -copies n ] [ -exec executable ] [-no-connection ]`

将 *n* 个 inferior 添加到使用 *executable* 作为可执行文件的运行中；*n* 默认为 1。如果没有指定可执行文件，则 inferior 将为空，没有程序。您仍然可以通过使用以可执行文件名作为参数的 `file` 命令，随时分配或更改分配给 inferior 的程序。

默认情况下，新的从属进程会连接到与当前从属进程相同的靶机连接。例如，如果当前从属进程通过 `target remote` 连接到 `gdbserver`，那么新的从属进程将会连接到同一个 `gdbserver` 实例。`-no-connection` 选项会以无连接状态启动新的从属进程。然后你可以例如使用 `target remote` 命令连接到其他 `gdbserver` 实例，使用 `run` 命令来启动本地程序等。

并非所有连接都可以在从属之间共享。例如，每个从属的目标核心都是唯一的。也就是说，多个从属可以同时使用目标核心，但每个目标核心是不同的。如果你尝试添加从属，并且当前从属是目标核心，那么 GDB 会发出警告，并创建一个没有连接的新从属，就像这样

```
(gdb) 文件 test1 正在读取符号... (gdb) 目标
corecore.test1.433190[新 LWP 433190]核心是由 './test1' 生成
的。程序因信号 SIGSEGV, 段错误而终止。#0
0x0000000000401111 在 test1.c:6 的 foo () 中 6 return *global_
ptr; (gdb) add-inferior[新从属 2]警告：无法在从属之间共享连接
1 (core) 添加从属 2
```

另一个不能共享的目标是远程目标（参见第20.1 [远程连接类型], 页317）。远程目标不允许启动新的从属进程，因此创建一个共享远程连接但未运行的新从属进程没有意义。与上面的核心目标类似，如果你尝试添加从属进程，而当前从属进程是远程目标，那么 GDB 会发出警告并创建新的从属进程但不建立连接。

`clone-inferior [ -copies n ] [ -no-connection ] [ infno ]`

为 inferior *infno* 准备好执行相同程序的 *n* 个从属进程；*n* 默认为 1, *i* *nfno* 默认为当前从属进程的编号。此命令将当前从属进程的 *args*、*inferior-tty* 和 *cwd* 属性的值复制到新从属进程。它还传播用户使用 `set environment` 和 `unset environment` 命令对环境变量所做的更改。当你想运行正在调试的从属进程的另一个实例时，这是一个方便的命令。

```
(gdb) info inferiors Num 描述 连接 可执行 *1
process29964 1(native) helloworld (gdb)
clone-inferior 添加了从属 2。1个从属已添加。
```



```
(gdb) info inferiors
Num  Description      Connection  Executable
* 1   process 29964    1 (native) helloworld
  2   <null>          1 (native) helloworld
```

You can now simply switch focus to inferior 2 and run it.

Like `add-inferior`, `clone-inferior` shares the connection with the inferior *infno*. If the `-no-connection` option is given, then the new inferior will be created without a connection. If the connection of inferior *infno* can't be shared, then GDB will give a warning, and the new inferior will be created without a connection.

remove-inferiors infno...

Removes the inferior or inferiors *infno*... It is not possible to remove an inferior that is running with this command. For those, use the `kill` or `detach` command first.

To quit debugging one of the running inferiors that is not the current inferior, you can either detach from it by using the `detach inferior` command (allowing it to run independently), or kill it using the `kill inferiors` command:

detach inferior infno...

Detach from the inferior or inferiors identified by GDB inferior number(s) *infno*... Note that the inferior's entry still stays on the list of inferiors shown by `info inferiors`, but its Description will show '`<null>`'.

kill inferiors infno...

Kill the inferior or inferiors identified by GDB inferior number(s) *infno*... Note that the inferior's entry still stays on the list of inferiors shown by `info inferiors`, but its Description will show '`<null>`'.

After the successful completion of a command such as `detach`, `detach inferiors`, `kill` or `kill inferiors`, or after a normal process exit, the inferior is still valid and listed with `info inferiors`, ready to be restarted.

To be notified when inferiors are started or exit under GDB's control use `set print inferior-events`:

```
set print inferior-events
set print inferior-events on
set print inferior-events off
```

The `set print inferior-events` command allows you to enable or disable printing of messages when GDB notices that new inferiors have started or that inferiors have exited or have been detached. By default, these messages will be printed.

show print inferior-events

Show whether messages will be printed when GDB detects that inferiors have started, exited or have been detached.

Many commands will work the same with multiple programs as with a single program: e.g., `print myglobal` will simply display the value of `myglobal` in the current inferior.

```
(gdb) info inferiors
Num  描述      连接      可执行文件
* 1   进程 29964  1 (本地)  helloworld
  2   <null>    1 (本地)  helloworld
```

现在您可以简单地切换到从属 2 并运行它。

像 `add-inferior` 一样, `clone-inferior` 与 inferior *infno* 共享连接。如果给出 `-no-connection` 选项, 则新 inferior 将不会创建连接。如果 inferior *infno* 的连接无法共享, 则 GDB 会发出警告, 新 inferior 将不会创建连接。

移除从属进程 infno...

移除从属进程或从属进程信息。... 不可能移除正在使用该命令运行的从属进程。对于这些, 请先使用 `kill` 或 `detach` 命令。

要退出当前不是当前从属进程的正在运行的从属进程, 您可以使用 `detachinferior` 命令将其分离 (允许其独立运行), 或使用 `killinferiors` 命令将其杀死:

分离从属进程 infno...

从由 GDB 从属进程编号 *infno*... 注意, 从属进程的条目仍然保留在 `info inferiors` 显示的从属进程列表中, 但其描述将显示为 '`<null>`'。

杀死从属进程 infno...

杀死由 GDB inferior number(s) *infno*. 标识的劣等者。注意, 劣等者的条目仍会保留在 `info inferiors` 显示的劣等者列表中, 但其描述将显示为 '`<null>`'。

在成功完成 `detach`、`detach 劣等者`、`kill` 或 `kill 劣等者` 等命令, 或正常进程退出后, 劣等者仍然有效, 并显示在 `info inferiors` 列表中, 准备重新启动。

要在 GDB 控制下启动或退出劣等者时收到通知, 请使用 `setprint inferior-events`:

```
set print inferior-events
set print inferior-events on
set print inferior-events off
```

`set print inferior-events` 命令允许您启用或禁用打印消息, 当 GDB 检测到新子进程启动或子进程退出或已分离时。默认情况下, 这些消息将被打印。

show print inferior-events

显示当 GDB 检测到子进程启动、退出或已分离时是否打印消息。

许多命令在多个程序和单个程序中都能以相同的方式工作: 例如, `printmyglobal` 将简单地显示当前子进程中的 `myglobal` 的值。

Occasionally, when debugging GDB itself, it may be useful to get more info about the relationship of inferiors, programs, address spaces in a debug session. You can do that with the `maint info program-spaces` command.

```
maint info program-spaces
Print a list of all program spaces currently being managed by GDB.
GDB displays for each program space (in this order):
1. the program space number assigned by GDB
2. the name of the executable loaded into the program space, with e.g., the
   file command.
```

An asterisk ‘*’ preceding the GDB program space number indicates the current program space.

In addition, below each program space line, GDB prints extra information that isn’t suitable to display in tabular form. For example, the list of inferiors bound to the program space.

```
(gdb) maint info program-spaces
  Id  Executable
* 1   hello
  2   goodbye
    Bound inferiors: ID 1 (process 21561)
```

Here we can see that no inferior is running the program `hello`, while process 21561 is running the program `goodbye`. On some targets, it is possible that multiple inferiors are bound to the same program space. The most common example is that of debugging both the parent and child processes of a `vfork` call. For example,

```
(gdb) maint info program-spaces
  Id  Executable
* 1   vfork-test
    Bound inferiors: ID 2 (process 18050), ID 1 (process 18045)
```

Here, both inferior 2 and inferior 1 are running in the same program space as a result of inferior 1 having executed a `vfork` call.

4.9.1 Inferior-Specific Breakpoints

When debugging multiple inferiors, you can choose whether to set breakpoints for all inferiors, or for a particular inferior.

```
break locspec inferior inferior-id
break locspec inferior inferior-id if ...
```

`locspec` specifies a code location or locations in your program. See Section 9.2 [Location Specifications], page 128, for details.

Use the qualifier ‘`inferior inferior-id`’ with a breakpoint command to specify that you only want GDB to stop when a particular inferior reaches this breakpoint. The `inferior-id` specifier is one of the inferior identifiers assigned by GDB, shown in the first column of the ‘`info inferiors`’ output.

If you do not specify ‘`inferior inferior-id`’ when you set a breakpoint, the breakpoint applies to *all* inferiors of your program.

偶尔，在调试 GDB 本身时，获取有关调试会话中子进程、程序、地址空间之间关系的更多信息可能很有用。您可以使用 `maint info program-spaces` 命令来完成此操作。

维护信息 程序空间

打印由 GDB 当前管理的所有程序空间列表。

GDB 为每个程序空间（按此顺序）显示：

- 由 GDB 分配的程序空间编号
- 装载到程序空间的可执行文件名称，例如使用文件命令。

在 GDB 程序空间编号前有一个星号 ‘*’ 表示当前程序空间。

此外，在每个程序空间行下方，GDB 会打印不适合以表格形式显示的额外信息。例如，绑定到程序空间的子进程列表。

```
(gdb) maint info program-spaces
  Id  Executable
* 1   hello
  2   goodbye
    Bound inferiors: ID 1 (process 21561)
```

这里我们可以看到没有子进程正在运行程序`hello`，而进程21561正在运行程序`goodbye`。在某些目标上，可能有多个子进程绑定到同一个程序空间。最常见的例子是同时调试`vfork`调用时的父进程和子进程。例如，

```
(gdb) maint info program-spaces
  Id  Executable
* 1   vfork-test
    绑定的子进程: ID 2 (进程18050) , ID 1 (进程18045)
```

在这里，子进程2和子进程1都运行在同一个程序空间中，因为子进程1执行了 `vfork`调用。

4.9.1 子进程特定断点

在调试多个子进程时，你可以选择是否为所有子进程设置断点，或为特定子进程设置断点。

断点 `locspec` 子进程 `inferior-id`

断点 `locspec` 子进程 `inferior-id` 如果...

`locspec`指定你的程序中的代码位置或位置。有关详细信息，请参阅第9.2[位置规范],页128。

使用断点命令与限定符 ‘子进程 `inferior-id`’ 一起，指定你只希望在特定子进程到达此断点时才停止 GDB。子进程ID指定符是 GDB分配的子进程标识符之一，显示在 ‘`info inferiors`’ 输出的第一列中。

如果你在设置断点时没有指定 ‘`inferior inferior-id`’，则该断点将应用于你的程序的所有子进程。

You can use the `inferior` qualifier on conditional breakpoints as well; in this case, place ‘`inferior inferior-id`’ before or after the breakpoint condition, like this:

```
(gdb) break frik.c:13 inferior 2 if bartab > lim
```

Inferior-specific breakpoints are automatically deleted when the corresponding inferior is removed from GDB. For example:

```
(gdb) remove-inferiors 2
Inferior-specific breakpoint 3 deleted - inferior 2 has been removed.
```

A breakpoint can’t be both inferior-specific and thread-specific (see Section 5.5.4 [Thread-Specific Breakpoints], page 99), or task-specific (see Section 15.4.10.7 [Ada Tasks], page 256); using more than one of the `inferior`, `thread`, or `task` keywords when creating a breakpoint will give an error.

4.10 Debugging Programs with Multiple Threads

In some operating systems, such as GNU/Linux and Solaris, a single program may have more than one *thread* of execution. The precise semantics of threads differ from one operating system to another, but in general the threads of a single program are akin to multiple processes—except that they share one address space (that is, they can all examine and modify the same variables). On the other hand, each thread has its own registers and execution stack, and perhaps private memory.

GDB provides these facilities for debugging multi-thread programs:

- automatic notification of new threads
- ‘`thread thread-id`’, a command to switch among threads
- ‘`info threads`’, a command to inquire about existing threads
- ‘`thread apply [thread-id-list | all] args`’, a command to apply a command to a list of threads
- thread-specific breakpoints
- ‘`set print thread-events`’, which controls printing of messages on thread start and exit.
- ‘`set libthread-db-search-path path`’, which lets the user specify which `libthread_db` to use if the default choice isn’t compatible with the program.

The GDB thread debugging facility allows you to observe all threads while your program runs—but whenever GDB takes control, one thread in particular is always the focus of debugging. This thread is called the *current thread*. Debugging commands show program information from the perspective of the current thread.

Whenever GDB detects a new thread in your program, it displays the target system’s identification for the thread with a message in the form ‘`[New systag]`’, where *systag* is a thread identifier whose form varies depending on the particular system. For example, on GNU/Linux, you might see

```
[New Thread 0x41e02940 (LWP 25582)]
```

when GDB notices a new thread. In contrast, on other systems, the *systag* is simply something like ‘`process 368`’, with no further qualifier.

你也可以在条件断点上使用子进程限定符；在这种情况下，将 ‘`inferior inferior-id`’ 放在断点条件之前或之后，例如：

```
(gdb) break frik.c:13 inferior 2 if bartab >lim
```

当对应的 inferior 从 GDB 中移除时，特定于 inferior 的断点会自动删除。例如：

```
(gdb) remove-inferiors 2
特定于 inferior 的断点 3 已删除 - inferior 2 已被移除。
```

一个断点不能同时是特定于 inferior、特定于线程（参见第 5.5.4 特定于线程的断点], 页 99）或特定于任务（参见第 15.4.10.7 [Ada 任务], 页 256）；在创建断点时使用多个 inferior、线程或任务关键字将导致错误。

4.10 使用多线程调试程序

在 GNU/Linux 和 Solaris 等一些操作系统中，单个程序可能拥有多个执行线程。不同操作系统的线程精确语义有所不同，但通常来说，单个程序中的线程类似于多个进程——只是它们共享一个地址空间（也就是说，它们都可以检查和修改相同的变量）。另一方面，每个线程都有自己的寄存器和执行栈，或许还有私有内存。

GDB 为调试多线程程序提供以下功能：

- 自动通知新线程
- ‘切换线程 `thread-id`’，一个在多个线程之间切换的命令
- ‘线程信息’，一个查询现有线程的命令
- ‘`thread apply [thread-id-list | all] args`’，一个将命令应用于线程列表的命令
- 线程特定断点
- ‘`set print thread-events`’，用于控制线程启动和退出时消息的打印
- ‘`set libthread-db-search-path path`’，允许用户指定要使用的 `libthread_db`，如果默认选项与程序不兼容

The GDB 线程调试功能允许你在程序运行时观察所有线程——但每当 GDB 接管控制时，总有一个特定的线程是调试的重点。这个线程称为当前线程。调试命令会从当前线程的角度显示程序信息。

每当 GDB 检测到你的程序中的新线程时，它会使用 ‘`[New systag]`’ 格式的消息显示目标系统的线程标识符，其中 *systag* 是一个线程标识符，其形式因系统而异。例如，在 GNU/Linux 上，你可能会看到

```
[新线程 0x41e02940 (LWP 25582)]
```

当 GDB 发现一个新线程时。相比之下，在其他系统上，*systag* 只是一些像“进程 368”的东西，没有进一步的限定。

For debugging purposes, GDB associates its own thread number —always a single integer—with each thread of an inferior. This number is unique between all threads of an inferior, but not unique between threads of different inferiors.

You can refer to a given thread in an inferior using the qualified *inferior-num.thread-num* syntax, also known as *qualified thread ID*, with *inferior-num* being the inferior number and *thread-num* being the thread number of the given inferior. For example, thread 2.3 refers to thread number 3 of inferior 2. If you omit *inferior-num* (e.g., `thread 3`), then GDB infers you're referring to a thread of the current inferior.

Until you create a second inferior, GDB does not show the *inferior-num* part of thread IDs, even though you can always use the full *inferior-num.thread-num* form to refer to threads of inferior 1, the initial inferior.

Some commands accept a space-separated *thread ID list* as argument. A list element can be:

1. A thread ID as shown in the first field of the 'info threads' display, with or without an inferior qualifier. E.g., '2.1' or '1'.
2. A range of thread numbers, again with or without an inferior qualifier, as in *inf.thr1-thr2* or *thr1-thr2*. E.g., '1.2-4' or '2-4'.
3. All threads of an inferior, specified with a star wildcard, with or without an inferior qualifier, as in *inf.** (e.g., '1.*') or ***. The former refers to all threads of the given inferior, and the latter form without an inferior qualifier refers to all threads of the current inferior.

For example, if the current inferior is 1, and inferior 7 has one thread with ID 7.1, the thread list '1 2-3 4.5 6.7-9 7.*' includes threads 1 to 3 of inferior 1, thread 5 of inferior 4, threads 7 to 9 of inferior 6 and all threads of inferior 7. That is, in expanded qualified form, the same as '1.1 1.2 1.3 4.5 6.7 6.8 6.9 7.1'.

In addition to a *per-inferior* number, each thread is also assigned a unique *global* number, also known as *global thread ID*, a single integer. Unlike the thread number component of the thread ID, no two threads have the same global ID, even when you're debugging multiple inferiors.

From GDB's perspective, a process always has at least one thread. In other words, GDB assigns a thread number to the program's "main thread" even if the program is not multi-threaded.

The debugger convenience variables '\$_thread' and '\$_gthread' contain, respectively, the per-inferior thread number and the global thread number of the current thread. You may find this useful in writing breakpoint conditional expressions, command scripts, and so forth. The convenience variable '\$_inferior_thread_count' contains the number of live threads in the current inferior. See Section 10.12 [Convenience Variables], page 173, for general information on convenience variables.

When running in non-stop mode (see Section 5.5.2 [Non-Stop Mode], page 97), where new threads can be created, and existing threads exit, at any time, '\$_inferior_thread_count' could return a different value each time it is evaluated.

If GDB detects the program is multi-threaded, it augments the usual message about stopping at a breakpoint with the ID and name of the thread that hit the breakpoint.

```
Thread 2 "client" hit Breakpoint 1, send_message () at client.c:68
```

为了调试目的，GDB 会将其自己的线程号——始终是一个整数——与每个子进程的线程关联。这个号码在同一个子进程的所有线程之间是唯一的，但在不同子进程的线程之间不是唯一的。

您可以使用 qualified-inferior-num.thread-num 语法引用 inferior 中的指定线程，也称为限定线程 ID，其中 inferior-num 是 inferior 编号，thread-num 是给定 inferior 的线程编号。例如，线程 2.3 引用 inferior 2 的线程编号 3。如果您省略 inferior-num（例如，线程 3），则 GDB 会推断您正在引用当前 inferior 的线程。

直到您创建第二个 inferior，GDB 不会显示线程 ID 的 inferior-num 部分，即使您始终可以使用完整的 inferior-num.thread-num 形式来引用初始 inferior 1 的线程。

某些命令接受空格分隔的线程 ID 列表作为参数。列表元素可以是：

1. 与“info threads”显示的第一字段中所示的线程 ID 相同，可以带有或不带有 inferior 限定符。例如，“2.1”或“1”。
2. 一系列线程编号，同样可以带有或不带有下限限定符，例如 *inf.thr1-thr2* 或 *thr1-thr2*。例如 '1.2-4' 或 '2-4'。
3. 所有下限线程，使用星号通配符指定，可以带有或不带有下限限定符，例如 *inf.**（例如 '1.*'）或 ***。前者指给定下限的所有线程，后者不带下限限定符指当前下限的所有线程。

例如，如果当前下限是1，且下限7有一个ID为7.1的线程，则线程列表 '1 2-3 4.5 6.7-9 7.*' 包括下限1的1到3号线程、下限4的5号线程、下限6的7到9号线程以及下限7的所有线程。也就是说，在展开限定形式下，与 '1.1 1.2 1.3 4.5 6.7 6.8 6.9 7.1' 相同。

除了每个 Inferior 的线程号外，每个线程还被分配了一个唯一的全局编号，也称为全局线程 ID，是一个整数。与线程 ID 中的线程号部分不同，即使你正在调试多个 Inferior，也没有两个线程的全局 ID 是相同的。

从 GDB 的角度来看，一个进程始终至少有一个线程。换句话说，GDB 即使程序不是多线程的，也会为程序的“主线程”分配一个线程号。

调试器便利变量 '\$_thread' 和 '\$_gthread' 分别包含当前线程的每个 Inferior 线程号和全局线程号。在编写断点条件表达式、命令脚本等时，你可能会发现这些变量很有用。便利变量 '\$_inferior_thread_count' 包含当前 Inferior 中活动线程的数量。有关便利变量的详细信息，请参阅第 10.12 [便利变量]，页 173。

在非停止模式（参见第 5.5.2 [非停止模式]，页 97）下运行时（在这种模式下可以创建新线程，并且现有线程可以随时退出），'\$_inferior_thread_count' 在每次被评估时可能会返回不同的值。

如果 GDB 检测到程序是多线程的，它会将通常关于在断点处停止的消息，额外增加被触发断点的线程的ID和名称。

```
线程 2 "client" 触发断点 1, 发送-消息 () 在 client.c:68
```

Likewise when the program receives a signal:

Thread 1 "main" received signal SIGINT, Interrupt.

info threads [-gid] [-stopped] [-running] [thread-id-list]

Display information about one or more threads. With no arguments displays information about all threads. You can specify the list of threads that you want to display using the thread ID list syntax (see [thread ID lists], page 49).

GDB displays for each thread (in this order):

- 1. the per-inferior thread number assigned by GDB
- 2. the global thread number assigned by GDB, if the ‘-gid’ option was specified
- 3. the target system’s thread identifier (systag)
- 4. the thread’s name, if one is known. A thread can either be named by the user (see **thread name**, below), or, in some cases, by the program itself.
- 5. the current stack frame summary for that thread

An asterisk ‘*’ to the left of the GDB thread number indicates the current thread.

For example,

```
(gdb) info threads
Id Target Id      Frame
* 1  process 35 thread 13  main (argc=1, argv=0x7ffffff8)
  2  process 35 thread 23  0x34e5 in sigpause ()
  3  process 35 thread 27  0x34e5 in sigpause ()
    at threadtest.c:68
```

If you’re debugging multiple inferiors, GDB displays thread IDs using the qualified *inferior-num.thread-num* format. Otherwise, only *thread-num* is shown.

If you specify the ‘-stopped’ option, GDB filters the output of the command to print the stopped threads only. Similarly, if you specify the ‘-running’ option, GDB filters the output to print the running threads only. These options can be helpful to reduce the output list if there is a large number of threads. If you specify both options, GDB prints both stopped and running threads.

If you specify the ‘-gid’ option, GDB displays a column indicating each thread’s global thread ID:

```
(gdb) info threads
Id  GId Target Id      Frame
1.1 1   process 35 thread 13  main (argc=1, argv=0x7ffffff8)
1.2 3   process 35 thread 23  0x34e5 in sigpause ()
1.3 4   process 35 thread 27  0x34e5 in sigpause ()
* 2.1 2   process 65 thread 1   main (argc=1, argv=0x7ffffff8)
```

On Solaris, you can display more information about user threads with a Solaris-specific command:

maint info sol-threads

Display info on Solaris user threads.

thread thread-id

Make thread ID *thread-id* the current thread. The command argument *thread-id* is the GDB thread ID, as shown in the first field of the ‘**info threads**’ display, with or without an inferior qualifier (e.g., ‘2.1’ or ‘1’).

同样，当程序收到一个信号时：

线程 1 "main" 接收到信号 SIGINT，中断。

info 线程 [-gid] [-已停止] [-正在运行] [thread-id-list]

显示一个或多个线程的信息。如果没有参数，将显示所有线程的信息。您可以使用线程 ID 列表语法指定要显示的线程列表（参见 [线程 ID 列表], 页 49）。

GDB 为每个线程（按此顺序）显示：

- 1. 由 GDB 分配的每个子线程编号
- 2. 由 GDB 指定的全局线程编号，如果指定了 ‘-gid’ 选项
- 3. 目标系统的线程标识符 (systag)
- 4. 如果知道，线程的名称。线程可以由用户命名（见下文线程名称），或者在某些情况下，由程序本身命名。
- 5. 该线程的当前栈帧摘要

在 GDB 线程编号左侧的星号 ‘*’ 表示当前线程。

例如，

```
(gdb) info threads
Id  目标 ID      <frame>框架</frame>
* 1  进程 35 线程 13  main (argc=1, argv=0x7ffffff8)
  2  进程 35 线程 23  0x34e5 在 sigpause () 中
  3  进程 35 线程 27  0x34e5 在 sigpause () 中
    在 threadtest.c:68 处
```

如果你正在调试多个子进程，GDB 会使用 qualified inferior-num.thread-num 格式显示线程 ID。否则，仅显示 thread-num。

如果你指定了 ‘-stopped’ 选项，GDB 会过滤命令的输出，仅打印已停止的线程。类似地，如果你指定了 ‘-running’ 选项，GDB 会过滤输出，仅打印正在运行的线程。这些选项有助于减少输出列表，如果线程数量很大。如果你同时指定了这两个选项，GDB 会打印所有已停止和正在运行的线程。

如果你指定 ‘-gid’ 选项，GDB 会显示一个列，指示每个线程的全局线程ID：

```
(gdb) info threads
Id  GId 目标ID      帧
1.1 1   进程 35 线程 13  main (argc=1, argv=0x7ffffff8)
1.2 3   进程 35 线程 23  0x34e5 在 sigpause () 中
1.3 4   进程 35 线程 27  0x34e5 在 sigpause () 中
* 2.1 2   进程 65 线程 1   main (argc=1, argv=0x7ffffff8)
```

在 Solaris 上，您可以使用 Solaris 特定的命令显示更多关于用户线程的信息：

维护信息 sol-threads

显示 Solaris 用户线程的信息。

线程 thread-id

将线程 ID thread-id 设置为当前线程。命令参数 thread-id 是 GDB 线程 ID，如 ‘info threads’ 显示的第一字段所示，可带或不带子限定符（例如 ‘2.1’ 或 ‘1’）。

GDB responds by displaying the system identifier of the thread you selected, and its current stack frame summary:

```
(gdb) thread 2
[Switching to thread 2 (Thread 0xb7fdab70 (LWP 12747))]
#0 some_function (ignore=0x0) at example.c:8
8 printf ("hello\n");
```

As with the ‘[New ...]’ message, the form of the text after ‘Switching to’ depends on your system’s conventions for identifying threads.

thread apply [*thread-id-list* | all [-ascending]] [*flag*]... *command*

The **thread apply** command allows you to apply the named *command* to one or more threads. Specify the threads that you want affected using the thread ID list syntax (see [thread ID lists], page 49), or specify **all** to apply to all threads. To apply a command to all threads in descending order, type **thread apply all command**. To apply a command to all threads in ascending order, type **thread apply all -ascending command**.

The *flag* arguments control what output to produce and how to handle errors raised when applying *command* to a thread. *flag* must start with a - directly followed by one letter in **qcs**. If several flags are provided, they must be given individually, such as **-c -q**.

By default, GDB displays some thread information before the output produced by *command*, and an error raised during the execution of a *command* will abort **thread apply**. The following flags can be used to fine-tune this behavior:

- c The flag **-c**, which stands for ‘continue’, causes any errors in *command* to be displayed, and the execution of **thread apply** then continues.
- s The flag **-s**, which stands for ‘silent’, causes any errors or empty output produced by a *command* to be silently ignored. That is, the execution continues, but the thread information and errors are not printed.
- q The flag **-q** (‘quiet’) disables printing the thread information.

Flags **-c** and **-s** cannot be used together.

taas [*option*]... *command*

Shortcut for **thread apply all -s** [*option*]... *command*. Applies *command* on all threads, ignoring errors and empty output.

The **taas** command accepts the same options as the **thread apply all** command. See [thread apply all], page 51.

tfaas [*option*]... *command*

Shortcut for **thread apply all -s -- frame apply all -s** [*option*]... *command*. Applies *command* on all frames of all threads, ignoring errors and empty output. Note that the flag **-s** is specified twice: The first **-s** ensures that **thread apply** only shows the thread information of the threads for which **frame apply** produces some output. The second **-s** is needed to ensure that **frame apply** shows the frame information of a frame only if the *command* successfully produced some output.

GDB 通过显示您选择的线程的系统标识符及其当前栈帧摘要来响应：

```
(gdb) thread 2
[切换到线程2 (线程 0xb7fdab70 (LWP 12747))]
#0 some_函数 (ignore=0x0) at example.c:8
8 printf ("hello\n");
```

与 ‘[New ...]’ 消息一样，‘切换到’ 之后的文本形式取决于您的系统识别线程的约定。

thread apply [*thread-id-list* | all [-ascending]] [*flag*]... *command*

线程应用命令允许您将命名命令应用于一个或多个线程。使用线程ID列表语法（参见 [线程ID列表], 第49页）指定要影响的线程，或指定**all**以应用于所有线程。要以降序将命令应用于所有线程，请输入 **threadapply all command**；要以升序将命令应用于所有线程，请输入 **thread apply all -ascending command**。

标志参数控制生成何种输出以及如何处理将命令应用于线程时引发的错误。标志必须以-开头，直接后跟qcs中的一个字母。如果提供了多个标志，必须单独给出，例如-c -q。

默认情况下，GDB在命令生成的输出之前显示一些线程信息，并且如果在命令执行过程中引发错误，线程应用将中止。可以使用以下标志来微调此行为：

- c 标志-c，代表“继续”，会导致命令中的任何错误都被显示，然后线程应用继续执行。
 - s 标志-s，代表‘安静’，会导致命令产生的任何错误或空输出被静默忽略。也就是说，执行会继续，但不会打印线程信息和错误。
 - q 标志-q（‘安静模式’）会禁用打印线程信息。
- 标志 -c 和 -s 不能同时使用。

taas [*option*]... *command*

线程应用所有 -s [*option*]... *command* 的快捷方式。对所有线程应用命令，忽略错误和空输出。
taas 命令接受与 thread apply all 命令相同的选项。参见[thread apply all], 页面 51。

tfaas [*option*]... *command*

线程应用全部 -s -- 帧应用全部 -s [*option*]... *command*。该命令对所有线程的所有帧执行，忽略错误和空输出。
该线程仅显示那些在执行frame apply时产生输出的线程的线程信息。第二个 -s 参数用于确保frame apply仅在命令成功产生输出时显示帧信息。

It can for example be used to print a local variable or a function argument without knowing the thread or frame where this variable or argument is, using:

```
(gdb) tfaas p some_local_var_i_do_not_remember_where_it_is
```

The **tfaas** command accepts the same options as the **frame apply** command. See Section 8.5 [frame apply], page 122.

thread name [name]

This command assigns a name to the current thread. If no argument is given, any existing user-specified name is removed. The thread name appears in the ‘info threads’ display.

On some systems, such as GNU/Linux, GDB is able to determine the name of the thread as given by the OS. On these systems, a name specified with ‘thread name’ will override the system-give name, and removing the user-specified name will cause GDB to once again display the system-specified name.

thread find [regex]

Search for and display thread ids whose name or *systag* matches the supplied regular expression.

As well as being the complement to the ‘thread name’ command, this command also allows you to identify a thread by its target *systag*. For instance, on GNU/Linux, the target *systag* is the LWP id.

```
(gdb) thread find 26688
Thread 4 has target id 'Thread 0x41e02940 (LWP 26688)'
(gdb) info thread 4
   Id   Target Id      Frame
   4    Thread 0x41e02940 (LWP 26688) 0x00000031ca6cd372 in select ()
```

set print thread-events

set print thread-events on

set print thread-events off

The **set print thread-events** command allows you to enable or disable printing of messages when GDB notices that new threads have started or that threads have exited. By default, these messages will be printed if detection of these events is supported by the target. Note that these messages cannot be disabled on all targets.

show print thread-events

Show whether messages will be printed when GDB detects that threads have started and exited.

See Section 5.5 [Stopping and Starting Multi-thread Programs], page 95, for more information about how GDB behaves when you stop and start programs with multiple threads.

See Section 5.1.2 [Setting Watchpoints], page 68, for information about watchpoints in programs with multiple threads.

set libthread-db-search-path [path]

If this variable is set, *path* is a colon-separated list of directories GDB will use to search for **libthread_db**. If you omit *path*, ‘libthread-db-search-path’ will be reset to its default value (\$*sdir*:*\$pdir* on GNU/Linux and Solaris systems). Internally, the default value comes from the **LIBTHREAD_DB_SEARCH_PATH** macro.

例如，它可用于在不知道变量或参数所在的线程或帧的情况下打印局部变量或函数参数，使用方式如下：

```
(gdb) tfaas p some_local_var_i_do_not_remember_where_it_is
```

tfaas 命令与 frameapply 命令接受相同的选项。参见第 8.5 [frame apply], 页 122 页。

线程名称 [name]

此命令为当前线程分配一个名称。如果未提供参数，则会移除任何现有的用户指定名称。线程名称会显示在 “info threads” 输出中。

在某些系统上，例如 GNU/Linux，GDB 能够确定操作系统提供的线程名称。在这些系统上，使用 “thread name” 指定的名称将覆盖系统提供的名称，移除用户指定名称将导致 GDB 重新显示系统提供的名称。

查找线程 [regex]

搜索并显示名称或系统标签与提供的正则表达式匹配的线程 ID。

除了作为 ‘线程名称’ 命令的补充之外，此命令还允许您通过其目标系统标签来识别线程。例如，在 GNU/Linux 上，目标系统标签是 LWP ID。

```
(gdb) thread find 26688
Thread 4 has target id 'Thread 0x41e02940 (LWP 26688)'
(gdb) info thread 4
   Id   Target Id      Frame
   4    Thread 0x41e02940 (LWP 26688) 0x00000031ca6cd372 in select ()
```

设置打印线程事件

设置打印线程事件 开启

设置打印线程事件 关闭

set print thread-events 命令允许您启用或禁用 GDB 检测到新线程启动或线程退出时打印消息的功能。默认情况下，如果目标支持检测这些事件，则会打印这些消息。请注意，这些消息无法在所有目标上禁用。

show print thread-events

显示 GDB 检测到线程启动和退出时是否将打印消息。

参见第 5 部分.5 [停止和启动多线程程序], 页 95，了解更多关于 GDB 在您停止和启动多线程程序时如何行为的信息。

参见第 5.1 部分.2 [设置断点], 页 68，了解更多关于多线程程序中断点的信息。

set libthread-db-search-path [path]

如果设置此变量，*path* 将是 GDB 用于搜索 **libthread_db** 的以冒号分隔的目录列表。如果你省略 *path*，‘libthread-db-search-path’ 将被重置为其默认值（在 GNU/Linux 和 Solaris 系统上是 \$*sdir*:*\$pdir*）。内部默认值来自 **LIBTHREAD_DB_SEARCH_PATH** 宏。

On GNU/Linux and Solaris systems, GDB uses a “helper” `libthread_db` library to obtain information about threads in the inferior process. GDB will use ‘`libthread-db-search-path`’ to find `libthread_db`. GDB also consults first if inferior specific thread debugging library loading is enabled by ‘`set auto-load libthread-db`’ (see Section 22.8.2 [libthread_db.so.1 file], page 383).

A special entry ‘`$sdir`’ for ‘`libthread-db-search-path`’ refers to the default system directories that are normally searched for loading shared libraries. The ‘`$sdir`’ entry is the only kind not needing to be enabled by ‘`set auto-load libthread-db`’ (see Section 22.8.2 [libthread_db.so.1 file], page 383).

A special entry ‘`$pdir`’ for ‘`libthread-db-search-path`’ refers to the directory from which `libpthread` was loaded in the inferior process.

For any `libthread_db` library GDB finds in above directories, GDB attempts to initialize it with the current inferior process. If this initialization fails (which could happen because of a version mismatch between `libthread_db` and `libpthread`), GDB will unload `libthread_db`, and continue with the next directory. If none of `libthread_db` libraries initialize successfully, GDB will issue a warning and thread debugging will be disabled.

Setting `libthread-db-search-path` is currently implemented only on some platforms.

```
show libthread-db-search-path
    Display current libthread_db search path.

set debug libthread-db
show debug libthread-db
    Turns on or off display of libthread_db-related events. Use 1 to enable, 0 to
    disable.

set debug threads [on|off]
show debug threads
    When ‘on’ GDB will print additional messages when threads are created and
    deleted.
```

For some debugging targets, GDB has support for accessing variables that reside in Thread Local Storage (TLS). TLS variables are similar to global variables, except that each thread has its own copy of the variable. While often used in multi-threaded programs, TLS variables can also be used in programs without threads. The C library variable `errno` is, perhaps, the most prominent example of a TLS variable that is frequently used in non-threaded programs. For targets where GDB does not have good TLS support, printing or changing the value of `errno` might not be directly possible.

GNU/Linux and FreeBSD targets have support for accessing TLS variables. On GNU/Linux, the helper library, `libthread_db`, is used to help resolve the addresses of TLS variables. Some FreeBSD and some GNU/Linux targets also have GDB-internal TLS resolution code. GNU/Linux targets will attempt to use the TLS address lookup functionality provided by `libthread_db`, but will fall back to using its internal TLS support when `libthread_db` is not available. This can happen in cross-debugging scenarios or when debugging programs that are linked in such a way that `libthread_db` support

在 GNU/Linux 和 Solaris 系统上，GDB 使用一个“辅助” `libthread_db` 库来获取子进程中线程的信息。GDB 将使用 ‘`libthread-db-search-path`’ 来查找 `libthread_db`。GDB 还会首先咨询，如果通过 ‘`set auto-load libthread-db`’ 启用了子进程特定线程调试库加载（参见第 22.8.2 [libthread_db.so.1 文件], 页 383）。

‘`libthread-db-search-path`’ 的特殊条目 ‘`$sdir`’ 指向默认系统目录，这些目录通常用于加载共享库。‘`$sdir`’ 条目是唯一不需要通过 ‘`set auto-load libthread-db`’ （参见第22.8.2 [libthread_db.so.1文件], 页383）启用的方式。

‘`libthread-db-search-path`’ 的特殊条目 ‘`$pdir`’ 指向在从属进程中加载 `libpthread` 的目录。对于在任何上述目录中找到的 `libthread_db` 库 GDB，GDB 会尝试使用当前从属进程初始化它。如果此初始化失败（这可能是因为 `libthread_db` 和 `libpthread` 之间存在版本不匹配），GDB 将卸载 `libthread_db`，并继续下一个目录。如果没有任何 `libthread_db` 库初始化成功，GDB 将发出警告，并且线程调试将被禁用。

设置 `libthread-db-search-path` 目前仅在部分平台上实现。

```
显示 libthread-db-search-path
    显示当前的 libthread_db 搜索路径。

设置 debug libthread-db
显示 debug libthread-db
    开启或关闭 libthread_db 相关事件的显示。使用 1 启用，0 禁用。

设置调试线程 [开启|关闭]
显示调试线程
    当 ‘开启’ GDB 时，会在创建和删除线程时打印额外信息。
```

对于某些调试目标，GDB 支持访问位于线程本地存储（TLS）中的变量。TLS 变量类似于全局变量，但每个线程都有自己的变量副本。虽然通常用于多线程程序，但 TLS 变量也可以用于没有线程的程序。C 库变量 `errno` 可能是非线程程序中频繁使用的 TLS 变量的最突出例子。对于 GDB 没有良好 TLS 支持的目标，可能无法直接打印或更改 `errno` 的值。

GNU/Linux 和 FreeBSD 目标支持访问 TLS 变量。在 GNU/Linux，使用辅助库 `libthread_db` 来帮助解析 TLS 变量的地址。一些 FreeBSD 和一些 GNU/Linux 目标也有 GDB-内部的 TLS 解析代码。GNU/Linux 目标将尝试使用 TLS 地址查找

`libthread_db` 提供的功能，但在 `libthread_db` 不可用时将回退使用其内部 TLS 支持。这可能在交叉调试场景中发生，或当调试以某种方式链接的程序时，导致 `libthread_db` 支持不可用

is unavailable – this includes statically linked programs, linking against GLIBC versions earlier than 2.34, but not with `libpthread`, and use of other (non-GLIBC) C libraries.

```
maint set force-internal-tls-address-lookup
maint show force-internal-tls-address-lookup
```

Turns on or off forced use of GDB-internal TLS address lookup code. Use `on` to enable and `off` to disable. The default for this setting is `off`.

When disabled, GDB will attempt to use a helper `libthread_db` library if possible, but will fall back to use of its own internal TLS address lookup mechanisms if necessary.

When enabled, GDB will only use GDB's internal TLS address lookup mechanisms, if they exist.

This command is only available for GNU/Linux targets. Its primary use is for testing – certain tests in the GDB test suite use this command to force use of internal TLS address lookup.

4.11 Debugging Forks

On most systems, GDB has no special support for debugging programs which create additional processes using the `fork` function. When a program forks, GDB will continue to debug the parent process and the child process will run unimpeded. If you have set a breakpoint in any code which the child then executes, the child will get a `SIGTRAP` signal which (unless it catches the signal) will cause it to terminate.

However, if you want to debug the child process there is a workaround which isn't too painful. Put a call to `sleep` in the code which the child process executes after the fork. It may be useful to sleep only if a certain environment variable is set, or a certain file exists, so that the delay need not occur when you don't want to run GDB on the child. While the child is sleeping, use the `ps` program to get its process ID. Then tell GDB (a new invocation of GDB if you are also debugging the parent process) to attach to the child process (see Section 4.7 [Attach], page 41). From that point on you can debug the child process just like any other process which you attached to.

On some systems, GDB provides support for debugging programs that create additional processes using the `fork` or `vfork` functions. On GNU/Linux platforms, this feature is supported with kernel version 2.5.46 and later.

The fork debugging commands are supported in native mode and when connected to `gdbserver` in either `target remote` mode or `target extended-remote` mode.

By default, when a program forks, GDB will continue to debug the parent process and the child process will run unimpeded.

If you want to follow the child process instead of the parent process, use the command `set follow-fork-mode`.

```
set follow-fork-mode mode
```

Set the debugger response to a program call of `fork` or `vfork`. A call to `fork` or `vfork` creates a new process. The `mode` argument can be:

parent The original process is debugged after a fork. The child process runs unimpeded. This is the default.

——这包括静态链接的程序、链接 GLIBC 版本早于 2.34 的程序，但不包括与 `libpthread` 链接的情况，以及使用其他（非 GLIBC）C 库

```
maint set force-internal-tls-address-lookup
maint show force-internal-tls-address-lookup
```

开启或关闭强制使用 GDB-内部 TLS 地址查找代码。使用 `on` 来启用，使用 `off` 来禁用。此设置的默认值是 `off`。

当禁用时，GDB 将尝试使用 helper `libthread_db` 库（如果可能），但在必要时会回退到使用自己的内部 TLS 地址查找机制。

当启用时，GDB 将仅使用 GDB 的内部 TLS 地址查找机制（如果存在）。

此命令仅适用于 GNU/Linux 目标。其主要用途是测试——GDB 测试套件中的某些测试使用此命令来强制使用内部 TLS 地址查找。

4.11 调试分叉进程

在大多数系统上，GDB 对使用 `fork` 函数创建额外进程的程序没有特殊的调试支持。当程序分叉时，GDB 会继续调试父进程，而子进程将不受阻碍地运行。如果你在子进程随后执行的任何代码中设置了断点，子进程将收到一个 `SIGTRAP` 信号，除非它捕获了该信号，否则这将导致它终止。

但是，如果你想调试子进程，有一个不太麻烦的解决方法。在子进程分叉后执行的代码中放入一个 `sleep` 调用。如果只在设置了某个环境变量或某个文件存在时才需要延迟，这可能会很有用，这样当你不想在子进程上运行 GDB 时就不需要延迟。在子进程休眠期间，使用 `ps` 程序获取其进程 ID。然后告诉 GDB（如果你也在调试父进程，则使用 GDB 的新实例）将调试器附加到子进程（参见第 4.7 [Attach], 页 41）。从那时起，你可以像附加到任何其他进程一样调试子进程。

在某些系统上，GDB 支持调试使用 `fork` 或 `vfork` 函数创建额外进程的程序。在 GNU/Linux 平台上，此功能从内核版本 2.5.46 及更高版本开始支持。

`fork` 调试命令在原生模式下以及连接到 `gdbserver` 时（无论是目标远程模式还是目标扩展远程模式）都受支持。

默认情况下，当程序进行 `fork` 时，GDB 会继续调试父进程，而子进程将不受影响地运行。

如果您想跟踪子进程而不是父进程，请使用 `set follow-fork-mode` 命令。

```
set follow-fork-mode mode
```

设置调试器对 `fork` 或 `vfork` 的程序调用做出响应。对 `fork` 的调用或者 `vfork` 创建一个新进程。模式参数可以是：

父节点 在 `fork` 后，原始进程会被调试。这是默认行为。

child	The new process is debugged after a fork. The parent process runs unimpeded.
--------------	--

```
show follow-fork-mode
```

Display the current debugger response to a `fork` or `vfork` call.

On Linux, if you want to debug both the parent and child processes, use the command `set detach-on-fork`.

```
set detach-on-fork mode
```

Tells gdb whether to detach one of the processes after a fork, or retain debugger control over them both.

on The child process (or parent process, depending on the value of **follow-fork-mode**) will be detached and allowed to run independently. This is the default.

off Both processes will be held under the control of GDB. One process (child or parent, depending on the value of `follow-fork-mode`) is debugged as usual, while the other is held suspended.

```
show detach-on-fork
```

Show whether detach-on-fork mode is on/off.

If you choose to set `'detach-on-fork'` mode off, then GDB will retain control of all forked processes (including nested forks). You can list the forked processes under the control of GDB by using the `info inferiors` command, and switch from one fork to another by using the `inferior` command (see Section 4.9 [Debugging Multiple Inferiors Connections and Programs], page 43).

To quit debugging one of the forked processes, you can either detach from it by using the `detach inferiors` command (allowing it to run independently), or kill it using the `kill inferiors` command. See Section 4.9 [Debugging Multiple Inferiors Connections and Programs], page 43.

If you ask to debug a child process and a `vfork` is followed by an `exec`, GDB executes the new target up to the first breakpoint in the new target. If you have a breakpoint set on `main` in your original program, the breakpoint will also be set on the child process's `main`.

On some systems, when a child process is spawned by `vfork`, you cannot debug the child or parent until an `exec` call completes.

If you issue a **run** command to GDB after an **exec** call executes, the new target restarts. To restart the parent process, use the **file** command with the parent executable name as its argument. By default, after an **exec** call executes, GDB discards the symbols of the previous executable image. You can change this behavior with the **set follow-exec-mode** command.

```
set follow-exec-mode mode
```

Set debugger response to a program call of `exec`. An `exec` call replaces the program image of a process.

follow-exec-mode can be:

new GDB creates a new inferior and rebinds the process to this new inferior. The program the process was running before the **exec** call can be restarted afterwards by restarting the original inferior.

子进程在创建后会被调试。父进程可以无阻碍运行。

显示follow-fork模式

显示对fork或vfork调用的当前调试器响应。

在Linux上, 如果您想同时调试父进程和子进程, 请使用命令set detach-on-fork。

```
set detach-on-fork mode
```

告诉gdb在fork后是否应该分离其中一个进程，或者保留调试器对它们都有控制权。

on 子进程（或父进程，取决于 follow-fork-mode 的值）将被分离，并允许独立运行。这是默认行为。

off 两个进程都将受 GDB 控制。其中一个进程（子进程或父进程，取决于 follow-fork-mode 的值）按常规进行调试，而另一个进程则被挂起。

显示 detach-on-fork

显示 detach-on-fork 模式是否开启。

如果你选择关闭 ‘detach-on-fork’ 模式，则 GDB 将保留对所有分叉进程的控制（包括嵌套分叉）。你可以通过使用 `info inferiors` 命令列出由 GDB 控制的 fork 进程，并通过使用 `inferiors` 命令在分叉进程之间切换（参见第 4.9 [调试多个 `inferiors` 连接和程序], 页 43）。

要退出调试其中一个分叉进程，你可以通过使用`detach inferiors`命令将其分离（允许其独立运行），或使用`kill inferiors`命令将其终止。参见第4.9 [调试多个inferiors连接和程序], 页43。

如果你要求调试子进程，并且vfork后跟exec，则 GDB 会执行新目标直到新目标中的第一个断点。如果你在原始程序中为main设置了断点，该断点也将子在进程的main中设置。

在某些系统上，当通过 `vfork` 创建子进程时，直到 `exec` 调用完成之前，你无法调试子进程或父进程。

如果你在 `exec` 调用执行后向 `GDB` 发出运行命令，新的目标会重新启动。要重新启动父进程，请使用父进程可执行文件名作为参数的 `file` 命令。默认情况下，`exec` 调用执行后，`GDB` 会丢弃先前可执行映像的符号。你可以使用 `set follow-exec-mode` 命令更改此行为。

```
set follow-exec-mode mode
```

将调试器响应设置为 exec 程序调用。
进程的映像。

follow-exec-mode 可以是:

new GDB 创建一个新的从属进程并将当前进程绑定到这个新的从属进程。在执行exec调用之前进程所运行的程序之后可以通过重启原始从属进程来重新启动。

For example:

```
(gdb) info inferiors
(gdb) info inferior
  Id  Description  Executable
* 1   <null>      prog1
(gdb) run
process 12020 is executing new program: prog2
Program exited normally.
(gdb) info inferiors
  Id  Description  Executable
  1   <null>      prog1
* 2   <null>      prog2
```

same

GDB keeps the process bound to the same inferior. The new executable image replaces the previous executable loaded in the inferior. Restarting the inferior after the `exec` call, with e.g., the `run` command, restarts the executable the process was running after the `exec` call. This is the default mode.

For example:

```
(gdb) info inferiors
  Id  Description  Executable
* 1   <null>      prog1
(gdb) run
process 12020 is executing new program: prog2
Program exited normally.
(gdb) info inferiors
  Id  Description  Executable
* 1   <null>      prog2
```

`follow-exec-mode` is supported in native mode and `target extended-remote` mode.

You can use the `catch` command to make GDB stop whenever a `fork`, `vfork`, or `exec` call is made. See Section 5.1.3 [Setting Catchpoints], page 71.

4.12 Setting a *Bookmark* to Return to Later

On certain operating systems¹, GDB is able to save a *snapshot* of a program’s state, called a *checkpoint*, and come back to it later.

Returning to a checkpoint effectively undoes everything that has happened in the program since the `checkpoint` was saved. This includes changes in memory, registers, and even (within some limits) system state. Effectively, it is like going back in time to the moment when the checkpoint was saved.

Thus, if you’re stepping through a program and you think you’re getting close to the point where things go wrong, you can save a checkpoint. Then, if you accidentally go too far and miss the critical statement, instead of having to restart your program from the beginning, you can just go back to the checkpoint and start again from there.

This can be especially useful if it takes a lot of time or steps to reach the point where you think the bug occurs.

To use the `checkpoint/restart` method of debugging:

¹ Currently, only GNU/Linux.

例如: (gdb) info inferiors (gdb) info inferior Id 描述 可
执行文件 * 1 <null> prog1 (gdb) run
process12020 isexecutingnewprogram:prog2 程序正常
退出。(gdb) info inferiors Id 描述 可执行文件1 <null
> prog1 * 2 <null> prog2

same

GDB 使进程绑定到同一个子进程。新的可执行文件会替换子进程中加载的旧可执行文件。在exec调用后重启子进程，例如使用run命令，会重启exec调用后进程正在运行的可执行文件。这是默认模式。

例如:

```
(gdb) info inferiors
  Id  描述      可执行文件
* 1   <null>      prog1
(gdb) run
进程 12020 正在执行新程序: prog2
程序正常退出。
(gdb) info inferiors
  Id  描述      可执行文件
* 1   <null>      prog2
```

在原生模式和目标扩展远程模式下都支持 follow-exec 模式。

您可以使用 `catch` 命令，在发生 `fork`、`vfork` 或 `exec` 调用时让 GDB 停止。参见第 5.1.3 [设置 Catchpoints],页 71。

4.12 设置书签以便稍后返回

在某些操作系统上¹, GDB 能够保存程序状态的快照，称为检查点，并稍后返回该检查点。

返回检查点实际上会撤销自检查点保存以来程序中发生的一切。这包括内存变化、寄存器变化，甚至在某些限制范围内还包括系统状态。实际上，这就像回到检查点保存的那个时刻。

因此，如果你正在单步执行程序，并且认为已经接近问题发生的地方，你可以保存一个检查点。然后，如果你不小心走得太远而错过了关键语句，你不必从头开始重新启动程序，只需返回检查点并从那里重新开始即可。

如果到达你认为出现错误的地方需要花费大量时间或步骤，这尤其有用。

要使用检查点/重启方法进行调试:

¹ 目前，仅支持 GNU/Linux。

checkpoint

Save a snapshot of the debugged program’s current execution state. The `checkpoint` command takes no arguments, but each checkpoint is assigned a small integer id, similar to a breakpoint id.

info checkpoints

List the checkpoints that have been saved in the current debugging session. For each checkpoint, the following information will be listed:

Checkpoint ID
Active state indicator
Process ID
Code Address
Source line, or label

Checkpoint IDs will be displayed as either a non-negative integer or in the form *i.n*, where *i* is the inferior number, a positive integer, as shown by the command `info inferiors`, and *n*, a non-negative integer, is the checkpoint number for that inferior. The single non-negative integer form is used when there is only one inferior. The *i.n* form is used when there are multiple inferiors.

The active state indicator is a single letter, either ‘y’ or ‘n’, indicating yes or no. Only one checkpoint per inferior may be active at once. The active checkpoint in the current inferior is also shown by a ‘*’ at the start of the line. Checkpoints whose active state is ‘n’ can be switched to using the `restart` command or deleted using the `delete checkpoint` command.

restart checkpoint-id

Restore the program state that was saved as checkpoint number *checkpoint-id*. All program variables, registers, stack frames etc. will be returned to the values that they had when the checkpoint was saved. In essence, gdb will “wind back the clock” to the point in time when the checkpoint was saved. The checkpoint number *checkpoint-id* is specified in the same form as that output by the `info checkpoints` command.

Note that breakpoints, GDB variables, command history etc. are not affected by restoring a checkpoint. In general, a checkpoint only restores things that reside in the program being debugged, not in the debugger.

delete checkpoint checkpoint-id

Delete the previously-saved checkpoint identified by *checkpoint-id*.

Returning to a previously saved checkpoint will restore the user state of the program being debugged, plus a significant subset of the system (OS) state, including file pointers. It won’t “un-write” data from a file, but it will rewind the file pointer to the previous location, so that the previously written data can be overwritten. For files opened in read mode, the pointer will also be restored so that the previously read data can be read again.

Of course, characters that have been sent to a printer (or other external device) cannot be “snatched back”, and characters received from eg. a serial device can be removed from internal program buffers, but they cannot be “pushed back” into the serial pipeline, ready to be received again. Similarly, the actual contents of files that have been changed cannot be restored (at this time).

检查点

保存被调试程序的当前执行状态快照。检查点命令不带参数，但每个检查点会被分配一个小的整型id，类似于断点id。

信息检查点

列出当前调试会话中已保存的检查点。对于每个检查点，将列出以下信息：

Checkpoint ID
活动状态指示器
进程 ID
代码地址
源行号或标签

检查点ID将显示为非负整数，或以 *i.n* 的形式显示，其中 *i* 是由命令 `info inferiors` 显示的正整数（下级编号），*n* 是该下级的非负整数检查点编号。当只有一个下级时，使用单个非负整数形式。当有多个下级时，使用 *i.n* 形式。

活动状态指示符是一个字母，为 'y' 或 'n'，分别表示是或否。每个下级一次只能有一个活动检查点。当前下级中的活动检查点也会在线首显示一个 '*'。活动状态为 'n' 的检查点可以使用 `restart` 命令切换为活动状态，或使用 `delete checkpoint` 命令删除。

restart checkpoint-id

恢复保存为检查点编号 *checkpoint-id* 的程序状态。所有程序变量、寄存器、栈帧等都将恢复到检查点保存时的值。本质上，gdb 会“倒转时钟”到检查点保存的时间点。检查点编号 *checkpoint-id* 的形式与 `info checkpoints` 命令输出的形式相同。

请注意，断点、GDB 变量、命令历史等在恢复检查点时不受影响。通常情况下，检查点仅恢复被调试程序中的内容，而不是调试器中的内容。

删除检查点 checkpoint-id

删除先前保存的、由检查点 ID 标识的检查点。

返回先前保存的检查点将恢复被调试程序的用户状态，以及系统（操作系统）状态的一个重要子集，包括文件指针。它不会“撤销”文件中的数据，但会将文件指针回滚到之前的位置，以便之前写入的数据可以被覆盖。对于以读取模式打开的文件，指针也将被恢复，以便之前读取的数据可以再次读取。

当然，已发送到打印机（或其他外部设备）的字符无法“收回”，而从串行设备等接收到的字符可以从内部程序缓冲区中移除，但它们无法“推回”到串行管道中，以便再次接收。类似地，已更改的文件的实际内容无法恢复（此时）。

However, within those constraints, you actually can “rewind” your program to a previously saved point in time, and begin debugging it again — and you can change the course of events so as to debug a different execution path this time.

Finally, there is one bit of internal program state that will be different when you return to a checkpoint — the program’s process id. Each checkpoint will have a unique process id (or *pid*), and each will be different from the program’s original *pid*. If your program has saved a local copy of its process id, this could potentially pose a problem.

4.12.1 A Non-obvious Benefit of Using Checkpoints

On some systems such as GNU/Linux, address space randomization is performed on new processes for security reasons. This makes it difficult or impossible to set a breakpoint, or watchpoint, on an absolute address if you have to restart the program, since the absolute location of a symbol will change from one execution to the next.

A checkpoint, however, is an *identical* copy of a process. Therefore if you create a checkpoint at (eg.) the start of main, and simply return to that checkpoint instead of restarting the process, you can avoid the effects of address randomization and your symbols will all stay in the same place.

然而，在这些限制范围内，你实际上可以“回滚”程序到一个先前保存的时间点，并重新开始调试它——而且你可以改变事件的走向，以便这次调试不同的执行路径。

最后，当你返回到检查点时，有一个内部程序状态会不同——那就是程序的进程ID。每个检查点都会有唯一的进程ID（或pid），并且每个都会与程序原始的进程ID不同。如果你的程序保存了其进程ID的本地副本，这可能会成为一个潜在的问题。

4.12.1 使用检查点的一个非显而易见的优势

在某些系统上，例如 GNU/Linux，出于安全原因，对新进程执行地址空间随机化。这使得如果你必须重新启动程序，很难或不可能在绝对地址上设置断点或观察点，因为符号的绝对位置在每次执行中都会改变。

然而，检查点是一个进程的完全副本。因此，如果你在（例如）主函数开始时创建一个检查点，然后直接返回该检查点而不是重新启动进程，你可以避免地址随机化的影响，并且你的符号将保持在相同的位置。

5 Stopping and Continuing

The principal purposes of using a debugger are so that you can stop your program before it terminates; or so that, if your program runs into trouble, you can investigate and find out why.

Inside GDB, your program may stop for any of several reasons, such as a signal, a breakpoint, or reaching a new line after a GDB command such as `step`. You may then examine and change variables, set new breakpoints or remove old ones, and then continue execution. Usually, the messages shown by GDB provide ample explanation of the status of your program—but you can also explicitly request this information at any time.

info program

Display information about the status of your program: whether it is running or not, what process it is, and why it stopped.

5.1 Breakpoints, Watchpoints, and Catchpoints

A *breakpoint* makes your program stop whenever a certain point in the program is reached. For each breakpoint, you can add conditions to control in finer detail whether your program stops. You can set breakpoints with the `break` command and its variants (see Section 5.1.1 [Setting Breakpoints], page 60), to specify the place where your program should stop by line number, function name or exact address in the program.

On some systems, you can set breakpoints in shared libraries before the executable is run.

A *watchpoint* is a special breakpoint that stops your program when the value of an expression changes. The expression may be a value of a variable, or it could involve values of one or more variables combined by operators, such as `'a + b'`. This is sometimes called *data breakpoints*. You must use a different command to set watchpoints (see Section 5.1.2 [Setting Watchpoints], page 68), but aside from that, you can manage a watchpoint like any other breakpoint: you enable, disable, and delete both breakpoints and watchpoints using the same commands.

You can arrange to have values from your program displayed automatically whenever GDB stops at a breakpoint. See Section 10.8 [Automatic Display], page 158.

A *catchpoint* is another special breakpoint that stops your program when a certain kind of event occurs, such as the throwing of a C++ exception or the loading of a library. As with watchpoints, you use a different command to set a catchpoint (see Section 5.1.3 [Setting Catchpoints], page 71), but aside from that, you can manage a catchpoint like any other breakpoint. (To stop when your program receives a signal, use the `handle` command; see Section 5.4 [Signals], page 92.)

GDB assigns a number to each breakpoint, watchpoint, or catchpoint when you create it; these numbers are successive integers starting with one. In many of the commands for controlling various features of breakpoints you use the breakpoint number to say which breakpoint you want to change. Each breakpoint may be *enabled* or *disabled*; if disabled, it has no effect on your program until you enable it again.

Some GDB commands accept a space-separated list of breakpoints on which to operate. A list element can be either a single breakpoint number, like `'5'`, or a range of such numbers,

5 暂停与继续

使用调试器的最主要目的是在程序终止之前停止你的程序；或者，如果你的程序遇到问题，你可以调查并找出原因。

在 GDB 中，您的程序可能因多种原因停止，例如收到信号、触发断点，或在执行完 GDB 命令（如 `step`）后到达新行。此时，您可以检查和修改变量、设置新的断点或删除旧的断点，然后继续执行。通常，GDB 显示的消息足以解释程序的状态——但您也可以随时显式请求这些信息。

info program

显示程序状态信息：它是否正在运行、是什么进程，以及为什么停止。

5.1 断点、观察点和捕获点

断点会让您的程序在达到程序中的某个特定点时停止。对于每个断点，您可以添加条件来更精细地控制程序是否停止。您可以使用 `break` 命令及其变体（参见第 5.1.1[设置断点]，页 60）来设置断点，通过行号、函数名或程序中的确切地址指定程序应停止的位置。

在某些系统上，您可以在可执行文件运行之前在共享库中设置断点。

观察点是一种特殊的断点，当表达式的值发生变化时，它会停止您的程序。该表达式可以是变量的值，也可以是使用运算符组合一个或多个变量的值，例如 `'a + b'`。这有时被称为数据断点。您必须使用不同的命令来设置观察点（参见第 5.1.2[设置观察点]，页 68），但除此之外，您可以像管理任何其他断点一样管理观察点：您使用相同的命令来启用、禁用和删除断点和观察点。

您可以安排在程序每次 GDB 停在断点时自动显示值。参见第 10.8 [自动显示]，页 158。

捕获点是另一种特殊的断点，当发生特定类型的事件时，它会停止您的程序，例如 C++ 异常的抛出或库的加载。与观察点一样，您使用不同的命令来设置捕获点（参见第 5.1.3 [设置捕获点]，页 71），但除此之外，您可以像管理任何其他断点一样管理捕获点。（要让您的程序在接收到信号时停止，请使用 `handle` 命令；参见第 5.4 [信号]，页 92。）

GDB 在创建断点、观察点或捕获点时，会为每个点分配一个编号；这些编号是连续的整数，从 1 开始。在许多用于控制断点各种特性的命令中，您使用断点编号来指定要修改的断点。每个断点可以启用或禁用；如果禁用，它对您的程序没有影响，直到您再次启用它。

一些 GDB 命令接受空格分隔的断点列表进行操作。列表元素可以是单个断点编号，如 `'5'`，也可以是一组这样的编号，

like ‘5-7’. When a breakpoint list is given to a command, all breakpoints in that list are operated on.

5.1.1 Setting Breakpoints

Breakpoints are set with the `break` command (abbreviated `b`). The debugger convenience variable ‘`$bpnum`’ records the number of the breakpoint you’ve set most recently:

```
(gdb) b main
Breakpoint 1 at 0x11c6: file zeoes.c, line 24.
(gdb) p $bpnum
$1 = 1
```

A breakpoint may be mapped to multiple code locations for example with inlined functions, Ada generics, C++ templates or overloaded function names. GDB then indicates the number of code locations in the breakpoint command output:

```
(gdb) b some_func
Breakpoint 2 at 0x1179: some_func. (3 locations)
(gdb) p $bpnum
$2 = 2
(gdb)
```

When your program stops on a breakpoint, the convenience variables ‘`$_hit_bpnum`’ and ‘`$_hit_locno`’ are respectively set to the number of the encountered breakpoint and the number of the breakpoint’s code location:

```
Thread 1 "zeoes" hit Breakpoint 2.1, some_func () at zeoes.c:8
8  printf("some func\n");
(gdb) p $_hit_bpnum
$5 = 2
(gdb) p $_hit_locno
$6 = 1
(gdb)
```

Note that ‘`$_hit_bpnum`’ and ‘`$bpnum`’ are not equivalent: ‘`$_hit_bpnum`’ is set to the breakpoint number **last hit**, while ‘`$bpnum`’ is set to the breakpoint number **last set**.

If the encountered breakpoint has only one code location, ‘`$_hit_locno`’ is set to 1:

```
Breakpoint 1, main (argc=1, argv=0x7fffffffe018) at zeoes.c:24
24  if (argc > 1)
(gdb) p $_hit_bpnum
$3 = 1
(gdb) p $_hit_locno
$4 = 1
(gdb)
```

The ‘`$_hit_bpnum`’ and ‘`$_hit_locno`’ variables can typically be used in a breakpoint command list. (see Section 5.1.7 [Breakpoint Command Lists], page 79). For example, as part of the breakpoint command list, you can disable completely the encountered breakpoint using `disable $_hit_bpnum` or disable the specific encountered breakpoint location using `disable $_hit_bpnum.$_hit_locno`. If a breakpoint has only one location, ‘`$_hit_locno`’ is set to 1 and the commands `disable $_hit_bpnum` and `disable $_hit_bpnum.$_hit_locno` both disable the breakpoint.

You can also define aliases to easily disable the last hit location or last hit breakpoint:

```
(gdb) alias lld = disable $_hit_bpnum.$_hit_locno
(gdb) alias lbd = disable $_hit_bpnum
```

如 ‘5-7’。当向命令提供断点列表时，该列表中的所有断点都会被操作。

5.1.1 设置断点

断点通过`break`命令（缩写为`b`）设置。调试器的便利变量 ‘`$bpnum`’ 记录了你最近设置的断点编号：

```
(gdb) b main
Breakpoint 1 at 0x11c6: file zeoes.c, line 24.
(gdb) p $bpnum
$1 = 1
```

断点可以映射到多个代码位置，例如内联函数、Ada泛型、C++ 模板或重载函数名。

GDB 则表示断点命令输出中的代码位置数量：

```
(gdb) b some_func
Breakpoint 2 at 0x1179: some_func. (3 locations)
(gdb) p $bpnum
$2 = 2
(gdb)
```

当你的程序停在断点时，便利变量 ‘`$_hit_bpnum`’ 和 ‘`$_hit_locno`’ 分别被设置为遇到的断点编号和断点的代码位置编号：

```
Thread 1 "zeoes" hit Breakpoint 2.1, some_func () at zeoes.c:8
8  printf("some func\n");
(gdb) p $_hit_bpnum
$5 = 2
(gdb) p $_hit_locno
$6 = 1
(gdb)
```

请注意 ‘`$_hit_bpnum`’ 和 ‘`$bpnum`’ 并不等价：‘`$_hit_bpnum`’ 被设置为最近遇到的断点编号，而 ‘`$bpnum`’ 被设置为最近设置的断点编号。

如果遇到的断点只有一个代码位置，‘`$_hit_locno`’ 会被设置为1：

```
Breakpoint 1, main (argc=1, argv=0x7fffffffe018) at zeoes.c:24
24  if (argc > 1)
(gdb) p $_hit_bpnum
$3 = 1
(gdb) p $_hit_locno
$4 = 1
(gdb)
```

‘`$_hit_bpnum`’ 和 ‘`$_hit_locno`’ 变量通常可以在断点命令列表中使用。（参见第5.1.7 [断点命令列表]，页79）。例如，作为断点命令列表的一部分，你可以完全禁用遇到的断点使用 `disable $_hit_bpnum`，或者禁用遇到的特定断点位置使用 `disable $_hit_bpnum.$_hit_locno`。如果一个断点只有一个位置，‘`$_hit_locno`’ 会被设置为1，并且命令 `disable $_hit_bpnum` 和 `disable $_hit_bpnum.$_hit_locno` 都会禁用断点。

您还可以定义别名，以便轻松禁用最后命中位置或最后命中断点：

```
(gdb) alias lld = disable $_hit_bpnum.$_hit_locno
(gdb) alias lbd = disable $_hit_bpnum
```

break locspec

Set a breakpoint at all the code locations in your program that result from resolving the given *locspec*. *locspec* can specify a function name, a line number, an address of an instruction, and more. See Section 9.2 [Location Specifications], page 128, for the various forms of *locspec*. The breakpoint will stop your program just before it executes the instruction at the address of any of the breakpoint's code locations.

When using source languages that permit overloading of symbols, such as C++, a function name may refer to more than one symbol, and thus more than one place to break. See Section 10.2 [Ambiguous Expressions], page 148, for a discussion of that situation.

It is also possible to insert a breakpoint that will stop the program only if a specific thread (see Section 5.5.4 [Thread-Specific Breakpoints], page 99), specific inferior (see Section 4.9.1 [Inferior-Specific Breakpoints], page 47), or a specific task (see Section 15.4.10.7 [Ada Tasks], page 256) hits that breakpoint.

break

When called without any arguments, **break** sets a breakpoint at the next instruction to be executed in the selected stack frame (see Chapter 8 [Examining the Stack], page 115). In any selected frame but the innermost, this makes your program stop as soon as control returns to that frame. This is similar to the effect of a **finish** command in the frame inside the selected frame—except that **finish** does not leave an active breakpoint. If you use **break** without an argument in the innermost frame, GDB stops the next time it reaches the current location; this may be useful inside loops.

GDB normally ignores breakpoints when it resumes execution, until at least one instruction has been executed. If it did not do this, you would be unable to proceed past a breakpoint without first disabling the breakpoint. This rule applies whether or not the breakpoint already existed when your program stopped.

break ... if cond

Set a breakpoint with condition *cond*; evaluate the expression *cond* each time the breakpoint is reached, and stop only if the value is nonzero—that is, if *cond* evaluates as true. ‘...’ stands for one of the possible arguments described above (or no argument) specifying where to break. See Section 5.1.6 [Break Conditions], page 78, for more information on breakpoint conditions.

The breakpoint may be mapped to multiple locations. If the breakpoint condition *cond* is invalid at some but not all of the locations, the locations for which the condition is invalid are disabled. For example, GDB reports below that two of the three locations are disabled.

```
(gdb) break func if a == 10
warning: failed to validate condition at location 0x11ce, disabling:
No symbol "a" in current context.
warning: failed to validate condition at location 0x11b6, disabling:
No symbol "a" in current context.
Breakpoint 1 at 0x11b6: func. (3 locations)
```

Locations that are disabled because of the condition are denoted by an uppercase N in the output of the **info breakpoints** command:

```
(gdb) info breakpoints
```

break locspec

在您的程序中所有由解析给定 *locspec* 产生的代码位置设置断点。locspec 可以指定函数名、行号、指令地址等。参见第 9.2 [位置特定说明], 页 128, 了解 locspec 的各种形式。该断点将在程序执行任何断点代码位置的指令地址之前停止您的程序。

当使用允许符号重载的源语言（如C++）时，函数名可能引用多个符号，因此可能有多个停止点。有关这种情况的讨论，请参阅第10.2 [模糊表达式], 页 148。

还可以插入一个断点，只有当特定的线程（参见第5.5.4 [线程特定断点], 页 99）、特定的目标（参见第4.9.1 [目标特定断点], 页47）或特定的任务（参见第15.4.10.7 [Ada任务], 页256）触发该断点时，程序才会停止。

break

如果没有参数调用break，它会在选定栈帧中下一个要执行的指令处设置断点（参见第 8 [检查栈], 页115）。在任何选定的非最内层栈帧中，这会使程序在控制返回到该栈帧时立即停止。这类似于选定栈帧内部栈帧中finish命令的效果——但finish不会留下一个活动的断点。如果你在最内层栈帧中使用没有参数的break，GDB 会在下次到达当前位置时停止；这在循环内部可能很有用。

GDB 通常在恢复执行时忽略断点，直到至少执行了一条指令。如果它不这样做，你将无法在不先禁用断点的情况下继续执行到断点处。当你的程序停止时，这个规则是否适用，无论断点是否存在。

break ... if cond

设置一个带条件的断点 cond；每次断点被触发时，评估表达式 cond，只有当值为非零时——即 cond 评估为真时——才停止。‘...’ 代表上述可能参数之一（或无参数）指定断点位置。有关断点条件的更多信息，请参阅第 5.1.6 [断点条件], 页 78。

断点可能映射到多个位置。如果断点条件 cond 在某些位置无效但在所有位置都有效，则条件无效的位置将被禁用。例如，GDB 报告如下，其中三个位置中的两个被禁用。

```
(gdb) break func if a == 10
warning: failed to validate condition at location 0x11ce, disabling:
No symbol "a" in current context.
warning: failed to validate condition at location 0x11b6, disabling:
No symbol "a" in current context.
断点 1 在 0x11b6 处: 函数 (3 个位置)
```

由于条件禁用的位置在 **info breakpoints** 命令的输出中以大写字母 N 标识：

```
(gdb) info breakpoints
```


Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	y	<MULTIPLE>	
	stop only if a == 10				
1.1		N*		0x00000000000011b6 in ...	
1.2		y		0x00000000000011c2 in ...	
1.3		N*		0x00000000000011ce in ...	
(*): Breakpoint condition is invalid at this location.					

If the breakpoint condition *cond* is invalid in the context of *all* the locations of the breakpoint, GDB refuses to define the breakpoint. For example, if variable *foo* is an undefined variable:

```
(gdb) break func if foo
No symbol "foo" in current context.
```

`break ... -force-condition if cond`

There may be cases where the condition *cond* is invalid at all the current locations, but the user knows that it will be valid at a future location; for example, because of a library load. In such cases, by using the `-force-condition` keyword before `'if'`, GDB can be forced to define the breakpoint with the given condition expression instead of refusing it.

```
(gdb) break func -force-condition if foo
warning: failed to validate condition at location 1, disabling:
No symbol "foo" in current context.
warning: failed to validate condition at location 2, disabling:
No symbol "foo" in current context.
warning: failed to validate condition at location 3, disabling:
No symbol "foo" in current context.
Breakpoint 1 at 0x1158: test.c:18. (3 locations)
```

This causes all the present locations where the breakpoint would otherwise be inserted, to be disabled, as seen in the example above. However, if there exist locations at which the condition is valid, the `-force-condition` keyword has no effect.

tbreak *args*

Set a breakpoint enabled only for one stop. The *args* are the same as for the **break** command, and the breakpoint is set in the same way, but the breakpoint is automatically deleted after the first time your program stops there. See Section 5.1.5 [Disabling Breakpoints], page 76.

hbreak *args*

Set a hardware-assisted breakpoint. The *args* are the same as for the **break** command and the breakpoint is set in the same way, but the breakpoint requires hardware support and some target hardware may not have this support. The main purpose of this is EPROM/ROM code debugging, so you can set a breakpoint at an instruction without changing the instruction. This can be used with the new trap-generation provided by SPARClite DSU and most x86-based targets. These targets will generate traps when a program accesses some data or instruction address that is assigned to the debug registers. However the hardware breakpoint registers can take a limited number of breakpoints. For example, on the DSU, only two data breakpoints can be set at a time, and GDB will reject this command if more than two are used. Delete or disable unused hardware breakpoints before setting new ones (see Section 5.1.5 [Dis-

Num	Type	启用地址	What
1	断点 仅当a == 10时停止	保持y <多个>	
1.1		N* 0x00000000000011b6 in ...	
1.2		y 0x00000000000011c2 in ...	
1.3		N* 0x00000000000011ce in ...	
(*): 在此位置断点条件无效。			

如果断点条件 *cond* 在所有断点位置上均无效， GDB 将拒绝定义断点。例如，如果变量 *foo* 是一个未定义的变量：

```
(gdb) break func if foo
当前上下文中没有符号 "foo".
```

`break ... -force-condition if cond`

可能存在这种情况：条件 *cond* 在所有当前位置上均无效，但用户知道它将在未来某个位置有效；例如，由于库加载的原因。在这种情况下，通过在 `'if'` 之前使用 `-force-condition` 关键字， GDB 可以强制使用给定的条件表达式来定义断点，而不是拒绝它。

```
(gdb) break func -force-condition if foo
警告：在位置 1 失败验证条件，禁用：
当前上下文中没有符号 "foo".
warning: failed to validate condition at location 2, disabling:
No symbol "foo" in current context.
警告：在位置 3 失败验证条件，禁用：
No symbol "foo" in current context.
断点 1 在 0x1158: test.c:18。 (3 个位置)
```

这会导致所有原本可以插入断点的现有位置被禁用，如上例所示。但是，如果存在条件有效时的位置，`-force-condition` 关键字将无效。

tbreak *args*

设置一个仅在程序停止一次时启用的断点。参数与 **break** 命令相同，断点设置方式也相同，但断点会在程序第一次停止该位置时自动删除。参见第 5.1.5 [禁用断点],页 76。

hbreak *args*

设置硬件辅助断点。参数与 **break** 命令相同，断点设置方式也相同，但该断点需要硬件支持，且某些目标硬件可能不支持此功能。主要用途是 EPROM/ROM 代码调试，因此你可以在不修改指令的情况下在指令处设置断点。这可与 SPARClite DSU 提供的新异常生成功能以及大多数 x86 架构目标一起使用。这些目标会在程序访问分配给调试寄存器的某些数据或指令地址时生成异常。但是硬件断点寄存器可以设置有限数量的断点。例如，在 DSU 上，一次只能设置两个数据断点，如果使用超过两个，GDB 将拒绝此命令。在设置新断点之前，请删除或禁用未使用的硬件断点（见第 5.1.5 [禁用断点GDB 页 76]）。参见第 5.1.5 [断点条件页 78]。对于远程目标，你可以限制 将使用的硬件断点数量，参见 `set remote hardware-breakpoint-limit`页 329。

abling Breakpoints], page 76). See Section 5.1.6 [Break Conditions], page 78. For remote targets, you can restrict the number of hardware breakpoints GDB will use, see [set remote hardware-breakpoint-limit], page 329.

thbreak args

Set a hardware-assisted breakpoint enabled only for one stop. The *args* are the same as for the **hbreak** command and the breakpoint is set in the same way. However, like the **tbreak** command, the breakpoint is automatically deleted after the first time your program stops there. Also, like the **hbreak** command, the breakpoint requires hardware support and some target hardware may not have this support. See Section 5.1.5 [Disabling Breakpoints], page 76. See also Section 5.1.6 [Break Conditions], page 78.

rbreak regex

Set breakpoints on all functions matching the regular expression *regex*. This command sets an unconditional breakpoint on all matches, printing a list of all breakpoints it set. Once these breakpoints are set, they are treated just like the breakpoints set with the **break** command. You can delete them, disable them, or make them conditional the same way as any other breakpoint.

In programs using different languages, GDB chooses the syntax to print the list of all breakpoints it sets according to the ‘**set language**’ value: using ‘**set language auto**’ (see Section 15.1.3 [Set Language Automatically], page 230) means to use the language of the breakpoint’s function, other values mean to use the manually specified language (see Section 15.1.2 [Set Language Manually], page 230).

The syntax of the regular expression is the standard one used with tools like **grep**. Note that this is different from the syntax used by shells, so for instance **foo*** matches all functions that include an **fo** followed by zero or more **os**. There is an implicit **.*** leading and trailing the regular expression you supply, so to match only functions that begin with **foo**, use **^foo**.

When debugging C++ programs, **rbreak** is useful for setting breakpoints on overloaded functions that are not members of any special classes.

The **rbreak** command can be used to set breakpoints in **all** the functions in a program, like this:

```
(gdb) rbreak .
```

rbreak file:regex

If **rbreak** is called with a filename qualification, it limits the search for functions matching the given regular expression to the specified *file*. This can be used, for example, to set breakpoints on every function in a given file:

```
(gdb) rbreak file.c:.
```

The colon separating the filename qualifier from the regex may optionally be surrounded by spaces.

info breakpoints [list...]

info break [list...]

Print a table of all breakpoints, watchpoints, tracepoints, and catchpoints set and not deleted. Optional argument *n* means print information only about the

对于远程目标，你可以限制 GDB 将使用的硬件断点数量，参见 [set remote hardware-breakpoint-limit], 页 329。

tbreak args

设置一个仅用于一次停止的硬件辅助断点。参数与 **hbreak** 命令相同，断点设置方式也相同。但是，与 **tbreak** 命令一样，断点在程序第一次停止时自动删除。此外，与 **hbreak** 命令一样，断点需要硬件支持，而某些目标硬件可能不支持此功能。参见第 5.1.5 [禁用断点], 页 76。另见第 5.1.6 [断点条件], 页 78。

rbreak regex

在所有匹配正则表达式 *regex* 的函数上设置断点。此命令对所有匹配项设置无条件断点，并打印出设置的断点列表。设置这些断点后，它们与使用 **break** 命令设置的断点一样处理。您可以像其他任何断点一样删除它们、禁用它们或使它们成为有条件的。

在多语言程序中，GDB 根据 “set language” 值选择打印所有设置断点列表的语法：使用 “set language auto”（参见第 15.1.3 [自动设置语言], 页 230）意味着使用断点函数的语言，其他值意味着使用手动指定的语言（参见第 15.1.2 [手动设置语言], 页 230）。

正则表达式的语法与 **grep** 等工具使用的标准语法相同。请注意，这与 shell 使用的语法不同，因此例如 **foo*** 会匹配所有包含 **fo** 后跟零个或多个 **o** 的函数。你提供的正则表达式前后隐含着 **.***，因此若要匹配仅以 **foo** 开头的函数，请使用 **^foo**。

在调试 C++ 程序时，**rbreak** 可用于在非任何特殊类成员的重载函数上设置断点。

rbreak 命令可用于在程序中的所有函数上设置断点，例如：

```
(gdb) rbreak .
```

rbreak file:regex

如果 **rbreak** 命令带有文件名限定符调用，它会将匹配给定正则表达式的函数的搜索范围限制在指定文件中。这可用于例如在给定文件中的每个函数上设置断点：

```
(gdb) rbreak file.c:.
```

文件名限定符与正则表达式之间的冒号可以可选地被空格包围。

info 断点 [list...]

info 断点 [list...]

打印所有已设置且未被删除的断点、观察点、跟踪点和捕获点。可选参数 *n* 表示仅打印关于每个断点的信息

specified breakpoint(s) (or watchpoint(s) or tracepoint(s) or catchpoint(s)). For each breakpoint, following columns are printed:

<i>Breakpoint Numbers</i>	
<i>Type</i>	Breakpoint, watchpoint, tracepoint, or catchpoint.
<i>Disposition</i>	
	Whether the breakpoint is marked to be disabled or deleted when hit.
<i>Enabled or Disabled</i>	
	Enabled breakpoints are marked with ‘y’. ‘n’ marks breakpoints that are not enabled.
<i>Address</i>	Where the breakpoint is in your program, as a memory address. For a pending breakpoint whose address is not yet known, this field will contain ‘<PENDING>’. Such breakpoint won’t fire until a shared library that has the symbol or line referred by breakpoint is loaded. See below for details. A breakpoint with several locations will have ‘<MULTIPLE>’ in this field—see below for details.
<i>What</i>	Where the breakpoint is in the source for your program, as a file and line number. For a pending breakpoint, the original string passed to the breakpoint command will be listed as it cannot be resolved until the appropriate shared library is loaded in the future.

If a breakpoint is conditional, there are two evaluation modes: “host” and “target”. If mode is “host”, breakpoint condition evaluation is done by GDB on the host’s side. If it is “target”, then the condition is evaluated by the target. The `info break` command shows the condition on the line following the affected breakpoint, together with its condition evaluation mode in between parentheses.

Breakpoint commands, if any, are listed after that. A pending breakpoint is allowed to have a condition specified for it. The condition is not parsed for validity until a shared library is loaded that allows the pending breakpoint to resolve to a valid location.

`info break` with a breakpoint number *n* as argument lists only that breakpoint. The convenience variable `$_` and the default examining-address for the `x` command are set to the address of the last breakpoint listed (see Section 10.6 [Examining Memory], page 154).

`info break` displays a count of the number of times the breakpoint has been hit. This is especially useful in conjunction with the `ignore` command. You can ignore a large number of breakpoint hits, look at the breakpoint info to see how many times the breakpoint was hit, and then run again, ignoring one less than that number. This will get you quickly to the last hit of that breakpoint.

For a breakpoints with an enable count (`xref`) greater than 1, `info break` also displays that count.

GDB allows you to set any number of breakpoints at the same place in your program. There is nothing silly or meaningless about this. When the breakpoints are conditional, this is even useful (see Section 5.1.6 [Break Conditions], page 78).

指定的断点（或观察点、跟踪点或捕捉点）。对于随后将打印以下列信息：

断点编号	
<i>Type</i>	断点、观察点、跟踪点或捕捉点。
状态	
	当命中断点时，是否标记为禁用或删除。
启用或禁用	
	已启用的断点用 ‘y’ 标记。 ‘n’ 标记未启用的断点这些断点未启用。
地址	断点在您的程序中的位置，作为内存地址。对于地址尚未知的待处理断点，此字段将包含 ‘<PENDING>’。此类断点在引用的符号或行所在的共享库加载之前不会触发。详情见下文。具有多个位置的断点在此字段中将包含 ‘<MULTIPLE>’ ——详情见下文。
<i>What</i>	
	断点在您的程序源代码中的位置，作为文件和行号。对于待处理断点，将列出传递给断点命令的原始字符串，因为它无法解析直到未来加载适当的共享库为止。

如果断点有条件，则有两种评估模式：“主机”和“目标”。如果模式是“主机”，则断点条件评估由 GDB 在主机端进行。如果是“目标”，则条件由目标进行评估。`info break` 命令显示受影响断点下一行的条件，并在括号中显示其条件评估模式。

断点命令（如有）列在后面。一个挂起的断点允许为其指定条件。直到加载允许挂起断点解析到有效位置的共享库后，才会解析该条件的有效性。

`info break` 命令以断点编号 *n* 作为参数，仅列出该断点。便利变量 `$_` 和 `x` 命令的默认检查地址设置为最后列出的断点的地址（参见第 10.6[检查内存]，页 154）。

`info break` 显示断点被触发的次数。这在与其他命令（如 `ignore`）结合使用时特别有用。您可以忽略大量断点触发，查看断点信息以了解断点被触发的次数，然后再次运行，忽略比该次数少 1 的次数。这将帮助您快速到达该断点的最后一次触发。对于具有启用计数（`xref`）大于 1 的断点，`info break` 还显示该计数。

GDB 允许你在程序同一位置设置任意数量的断点。这没有任何荒谬或不合理之处。当断点是条件性的时，这甚至更有用（参见第5.1.6 [断点条件]，页78）。

It is possible that a single logical breakpoint is set at several code locations in your program. See Section 9.2 [Location Specifications], page 128, for examples.

A breakpoint with multiple code locations is displayed in the breakpoint table using several rows—one header row, followed by one row for each code location. The header row has ‘<MULTIPLE>’ in the address column. Each code location row contains the actual address, source file, source line and function of its code location. The number column for a code location is of the form *breakpoint-number.location-number*.

For example:

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	y	<MULTIPLE>	
	stop only if i==1				
	breakpoint already hit 1 time				
1.1		y		0x080486a2 in void foo<int>() at t.cc:8	
1.2		y		0x080486ca in void foo<double>() at t.cc:8	

You cannot delete the individual locations from a breakpoint. However, each location can be individually enabled or disabled by passing *breakpoint-number.location-number* as argument to the **enable** and **disable** commands. It’s also possible to **enable** and **disable** a range of *location-number* locations using a *breakpoint-number* and two *location-numbers*, in increasing order, separated by a hyphen, like *breakpoint-number.location-number1-location-number2*, in which case GDB acts on all the locations in the range (inclusive). Disabling or enabling the parent breakpoint (see Section 5.1.5 [Disabling], page 76) affects all of the locations that belong to that breakpoint.

Locations that are enabled while their parent breakpoint is disabled won’t trigger a break, and are denoted by y- in the **Enb** column. For example:

(gdb) info breakpoints					
Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	n	<MULTIPLE>	
1.1			y-	0x00000000000011b6 in ...	
1.2			y-	0x00000000000011c2 in ...	
1.3			n	0x00000000000011ce in ...	

It’s quite common to have a breakpoint inside a shared library. Shared libraries can be loaded and unloaded explicitly, and possibly repeatedly, as the program is executed. To support this use case, GDB updates breakpoint locations whenever any shared library is loaded or unloaded. Typically, you would set a breakpoint in a shared library at the beginning of your debugging session, when the library is not loaded, and when the symbols from the library are not available. When you try to set breakpoint, GDB will ask you if you want to set a so called *pending breakpoint*—breakpoint whose address is not yet resolved.

After the program is run, whenever a new shared library is loaded, GDB reevaluates all the breakpoints. When a newly loaded shared library contains the symbol or line referred to by some pending breakpoint, that breakpoint is resolved and becomes an ordinary breakpoint. When a library is unloaded, all breakpoints that refer to its symbols or source lines become pending again.

This logic works for breakpoints with multiple locations, too. For example, if you have a breakpoint in a C++ template function, and a newly loaded shared library has an instantiation of that template, a new location is added to the list of locations for the breakpoint.

一个逻辑断点可能被设置在你的程序多个代码位置。参见第9.2 [位置规范], 页128, 了解示例。

带有多个代码位置的断点会在断点表中显示为多行——一行标题行，以及每条代码位置的一行。标题行的地址列包含 ‘<多>’。每条代码位置行包含该代码位置的地址、源文件、源行号和函数。代码位置的编号列格式为断点编号.位置编号。

例如：

Num	Type	Disp	Enb	地址	What
1	断点	保持	y	<多个>	
	仅当 i==1 时停止				
	断点已触发 1 次				
1.1		y		0x080486a2 在 void foo<int>() 函数中，位于 t.cc:8 行	
1.2		y		0x080486ca 在 void foo<double>() 函数中，位于 t.cc:8 行	

您无法从断点中删除单个位置。但是，可以通过将断点编号.位置编号作为参数传递给 **enable** 和 **disable** 命令来单独启用或禁用每个位置。还可以使用断点编号和两个按升序排列的位置编号（用连字符分隔），例如 *breakpoint-number.location-number1-location-number2*，此时 GDB 会作用于该范围内的所有位置（包含首尾）。禁用或启用父断点（参见第 5.1.5 [禁用], 页 76）会影响属于该断点的所有位置。

当其父级断点处于禁用状态时，启用的位置不会触发断点，并在 **Enb** 列中用 y- 标记。例如：

(gdb) info breakpoints					
Num	Type	Disp	Enb	地址	What
1	断点	keep	n	<多个>	
1.1			y-	0x00000000000011b6 在 ...	
1.2			y-	0x00000000000011c2 在 ...	
1.3			n	0x00000000000011ce 在 ...	

在共享库中设置断点是很常见的。共享库可以在程序执行时显式加载和卸载，甚至可能多次。为了支持这种用例，GDB 会在任何共享库加载或卸载时更新断点位置。通常，你会在调试会话开始时，在共享库尚未加载且库的符号不可用时设置断点。当你尝试设置断点时，GDB 会询问你是否要设置一个所谓的挂起断点——地址尚未解析的断点。

程序运行后，每当加载新的共享库时，GDB 会重新评估所有断点。当新加载的共享库包含某些挂起断点所引用的符号或行时，该断点会被解析并成为普通断点。当库被卸载时，所有引用其符号或源行的断点会再次变为挂起状态。

这种逻辑也适用于具有多个位置点的断点。例如，如果你在 C++ 模板函数中有一个断点，并且一个新加载的共享库中有该模板的实例，则会在断点的位置列表中添加一个新的位置。

Except for having unresolved address, pending breakpoints do not differ from regular breakpoints. You can set conditions or commands, enable and disable them and perform other breakpoint operations.

GDB provides some additional commands for controlling what happens when the ‘**break**’ command cannot resolve the location spec to any code location in your program (see Section 9.2 [Location Specifications], page 128):

set breakpoint pending auto

This is the default behavior. When GDB cannot resolve the location spec, it queries you whether a pending breakpoint should be created.

set breakpoint pending on

This indicates that when GDB cannot resolve the location spec, it should create a pending breakpoint without confirmation.

set breakpoint pending off

This indicates that pending breakpoints are not to be created. If GDB cannot resolve the location spec, it aborts the breakpoint creation with an error. This setting does not affect any pending breakpoints previously created.

show breakpoint pending

Show the current behavior setting for creating pending breakpoints.

The settings above only affect the **break** command and its variants. Once a breakpoint is set, it will be automatically updated as shared libraries are loaded and unloaded.

For some targets, GDB can automatically decide if hardware or software breakpoints should be used, depending on whether the breakpoint address is read-only or read-write. This applies to breakpoints set with the **break** command as well as to internal breakpoints set by commands like **next** and **finish**. For breakpoints set with **hbreak**, GDB will always use hardware breakpoints.

You can control this automatic behavior with the following commands:

set breakpoint auto-hw on

This is the default behavior. When GDB sets a breakpoint, it will try to use the target memory map to decide if software or hardware breakpoint must be used.

set breakpoint auto-hw off

This indicates GDB should not automatically select breakpoint type. If the target provides a memory map, GDB will warn when trying to set software breakpoint at a read-only address.

GDB normally implements breakpoints by replacing the program code at the breakpoint address with a special instruction, which, when executed, given control to the debugger. By default, the program code is so modified only when the program is resumed. As soon as the program stops, GDB restores the original instructions. This behavior guards against leaving breakpoints inserted in the target should gdb abruptly disconnect. However, with slow remote targets, inserting and removing breakpoint can reduce the performance. This behavior can be controlled with the following commands::

除了地址未解析外，待处理断点与普通断点没有区别。你可以设置条件或命令，启用和禁用它们，并执行其他断点操作。

GDB 提供了一些附加命令，用于控制当 ‘**break**’ 命令无法将位置规范解析到您的程序中的任何代码位置时发生的情况（参见第9.2 [位置规范], 页128）：

set breakpoint pending auto

这是默认行为。当 GDB 无法解析位置规范时，它会询问您是否应创建挂起的断点。

set breakpoint pending on

这表示当 GDB 无法解析位置规范时，它应在不确认的情况下创建挂起的断点。

设置断点挂起关闭

这表示不会创建挂起的断点。如果 GDB 无法解析位置规范，它会以错误终止断点创建。此设置不会影响先前已创建的任何挂起断点。

显示断点挂起

显示创建挂起断点的当前行为设置。

上述设置仅影响 **break** 命令及其变体。一旦设置断点，它将在共享库加载和卸载时自动更新。

对于某些目标，GDB 可以自动决定使用硬件断点还是软件断点，具体取决于断点地址是只读还是可写。这适用于使用 **break** 命令设置的断点，也适用于由 **next** 和 **finish** 等命令设置的内部断点。对于使用 **hbreak** 设置的断点，GDB 总是使用硬件断点。

您可以通过以下命令控制此自动行为：

set breakpoint auto-hw on

这是默认行为。当 GDB 设置断点时，它将尝试使用目标内存映射来决定是使用软件断点还是硬件断点。

set breakpoint auto-hw off

这表示 GDB 不应自动选择断点类型。如果目标提供了内存映射，GDB 在尝试在只读地址设置软件断点时会发出警告。

GDB 通常通过用特殊指令替换断点地址处的程序代码来实现断点，当执行该指令时，会交给调试器控制。默认情况下，程序代码仅在程序恢复时被这样修改。一旦程序停止，GDB就会恢复原始指令。这种行为可以防止在目标中留下断点，以防gdb突然断开连接。然而，对于慢速远程目标，插入和删除断点可能会降低性能。这种行为可以通过以下命令进行控制：

set breakpoint always-inserted off

All breakpoints, including newly added by the user, are inserted in the target only when the target is resumed. All breakpoints are removed from the target when it stops. This is the default mode.

set breakpoint always-inserted on

Causes all breakpoints to be inserted in the target at all times. If the user adds a new breakpoint, or changes an existing breakpoint, the breakpoints in the target are updated immediately. A breakpoint is removed from the target only when breakpoint itself is deleted.

GDB handles conditional breakpoints by evaluating these conditions when a breakpoint breaks. If the condition is true, then the process being debugged stops, otherwise the process is resumed.

If the target supports evaluating conditions on its end, GDB may download the breakpoint, together with its conditions, to it.

This feature can be controlled via the following commands:

set breakpoint condition-evaluation host

This option commands GDB to evaluate the breakpoint conditions on the host's side. Unconditional breakpoints are sent to the target which in turn receives the triggers and reports them back to GDB for condition evaluation. This is the standard evaluation mode.

set breakpoint condition-evaluation target

This option commands GDB to download breakpoint conditions to the target at the moment of their insertion. The target is responsible for evaluating the conditional expression and reporting breakpoint stop events back to GDB whenever the condition is true. Due to limitations of target-side evaluation, some conditions cannot be evaluated there, e.g., conditions that depend on local data that is only known to the host. Examples include conditional expressions involving convenience variables, complex types that cannot be handled by the agent expression parser and expressions that are too long to be sent over to the target, specially when the target is a remote system. In these cases, the conditions will be evaluated by GDB.

set breakpoint condition-evaluation auto

This is the default mode. If the target supports evaluating breakpoint conditions on its end, GDB will download breakpoint conditions to the target (limitations mentioned previously apply). If the target does not support breakpoint condition evaluation, then GDB will fallback to evaluating all these conditions on the host's side.

GDB itself sometimes sets breakpoints in your program for special purposes, such as proper handling of `longjmp` (in C programs). These internal breakpoints are assigned negative numbers, starting with `-1`; `'info breakpoints'` does not display them. You can see these breakpoints with the GDB maintenance command `'maint info breakpoints'` (see [maint info breakpoints], page 777).

设置断点始终插入关闭

所有断点，包括用户新添加的，都仅在目标恢复时插入。当目标停止时，所有断点都会从目标中移除。这是默认模式。

设置断点始终插入开启

导致所有断点始终插入到目标中。如果用户添加新的断点，或修改现有断点，目标中的断点会立即更新。断点只有在自身被删除时才会从目标中移除。

GDB 通过在断点触发时评估这些条件来处理条件断点。如果条件为真，则被调试的进程停止，否则进程继续。

如果目标支持在其端评估条件，GDB 可以将其断点（包括其条件）下载到目标端。

此功能可通过以下命令进行控制：

set breakpoint condition-evaluation host

此选项命令 GDB 在主机端评估断点条件。无条件断点会发送到目标，目标接收触发器并报告给GDB进行条件评估。这是标准的评估模式。

set breakpoint condition-evaluation target

此选项命令 GDB 在插入断点时将断点条件下载到目标。目标负责评估条件表达式，并在条件为真时将断点停止事件报告给 GDB。由于目标端评估的限制，某些条件无法在那里评估，例如依赖于仅主机端知道的本地数据的条件。包括涉及便利变量的条件表达式、代理表达式解析器无法处理的复杂类型以及过长无法发送到目标的表达式，特别是当目标为远程系统时。在这些情况下，条件将由 GDB进行评估。

设置断点条件自动评估

这是默认模式。如果目标端支持评估断点条件，GDB 将向目标端下载断点条件（之前提到的限制适用）。如果目标端不支持断点条件评估，则 GDB 将回退到在主机端评估所有这些条件。

GDB 它本身有时会为特殊目的在您的程序中设置断点，例如正确处理 `longjmp`（在 C 程序中）。这些内部断点被分配负数编号，从 `-1` 开始；`'info breakpoints'` 命令不会显示它们。您可以使用 GDB 维护命令 `'maint info breakpoints'`（参见[maint info breakpoints],第 777 页）来查看这些断点。

5.1.2 Setting Watchpoints

You can use a watchpoint to stop execution whenever the value of an expression changes, without having to predict a particular place where this may happen. (This is sometimes called a *data breakpoint*.) The expression may be as simple as the value of a single variable, or as complex as many variables combined by operators. Examples include:

- A reference to the value of a single variable.
- An address cast to an appropriate data type. For example, `*(int *)0x12345678` will watch a 4-byte region at the specified address (assuming an `int` occupies 4 bytes).
- An arbitrarily complex expression, such as `a*b + c/d`. The expression can use any operators valid in the program's native language (see Chapter 15 [Languages], page 229).

You can set a watchpoint on an expression even if the expression can not be evaluated yet. For instance, you can set a watchpoint on `*global_ptr` before `global_ptr` is initialized. GDB will stop when your program sets `global_ptr` and the expression produces a valid value. If the expression becomes valid in some other way than changing a variable (e.g. if the memory pointed to by `*global_ptr` becomes readable as the result of a `malloc` call), GDB may not stop until the next time the expression changes.

Depending on your system, watchpoints may be implemented in software or hardware. GDB does software watchpointing by single-stepping your program and testing the variable's value each time, which is hundreds of times slower than normal execution. (But this may still be worth it, to catch errors where you have no clue what part of your program is the culprit.)

On some systems, such as most PowerPC or x86-based targets, GDB includes support for hardware watchpoints, which do not slow down the running of your program.

`watch [-l|-location] expr [thread thread-id] [mask maskvalue] [task task-id]`

Set a watchpoint for an expression. GDB will break when the expression `expr` is written into by the program and its value changes. The simplest (and the most popular) use of this command is to watch the value of a single variable:

```
(gdb) watch foo
```

If the command includes a `[thread thread-id]` argument, GDB breaks only when the thread identified by `thread-id` changes the value of `expr`. If any other threads change the value of `expr`, GDB will not break. Note that watchpoints restricted to a single thread in this way only work with Hardware Watchpoints.

Similarly, if the `task` argument is given, then the watchpoint will be specific to the indicated Ada task (see Section 15.4.10.7 [Ada Tasks], page 256).

Ordinarily a watchpoint respects the scope of variables in `expr` (see below). The `-location` argument tells GDB to instead watch the memory referred to by `expr`. In this case, GDB will evaluate `expr`, take the address of the result, and watch the memory at that address. The type of the result is used to determine the size of the watched memory. If the expression's result does not have an address, then GDB will print an error.

The `[mask maskvalue]` argument allows creation of masked watchpoints, if the current architecture supports this feature (e.g., PowerPC Embedded architecture, see Section 21.3.8 [PowerPC Embedded], page 354.) A *masked watchpoint* specifies a mask in addition to an address to watch. The mask specifies that

5.1.2 设置断点

您可以使用观察点来在表达式的值发生变化时停止执行，而无需预测这种情况可能发生的特定位置。（这有时被称为数据断点。）该表达式可以简单到是一个单个变量的值，也可以复杂到是多个由运算符组合的变量。例如：

- 单个变量的值引用。
- 一个转换为适当数据类型的地址。例如，`*(int *)0x12345678` 将监视指定地址处的4字节区域（假设int占用4字节）。
- 一个任意复杂的表达式，例如 `a*b + c/d`。该表达式可以使用程序本机语言中任何有效的运算符（见第•语言 + 页229）。程序本机语言中有效的生成器（见第 15 [语言], 页229）。

即使表达式还不能被计算，你也可以为其设置断点。例如，在 `global_ptr` 初始化之前，你可以为其设置断点。GDB 将在你的程序设置 `global_ptr` 并且表达式产生有效值时停止。如果表达式以除变量更改以外的其他方式变为有效（例如，如果 `*global_ptr` 指向的内存因malloc调用而变得可读），GDB 可能不会在表达式下次变化之前停止。

根据你的系统，断点可能以软件或硬件的方式实现。GDB 通过单步执行你的程序并每次测试变量的值来实现软件断点，这比正常执行慢数百倍。（但即使如此，也可能值得，以捕获你不知道程序哪部分是问题的错误。）

在一些系统上，例如大多数PowerPC或x86架构的目标，GDB 包含对硬件断点的支持，这不会减慢你的程序运行。

`watch [-l|-location] expr [thread thread-id] [mask maskvalue] [task task-id]`

为表达式设置一个断点。GDB 将在表达式 `expr` 是 written into by the program and its value changes. The simplest (and the most popular) use of this command is to watch the value of a single variable:

```
(gdb) watch foo
```

如果命令包含一个 `[线程 thread-id]` 参数，GDB 仅在由 `thread-id` 标识的线程更改 `expr` 的值时才会中断。如果任何其他线程更改 `expr` 的值，GDB 不会中断。请注意，以这种方式仅限于单个线程的断点仅适用于硬件断点。

同样地，如果给出了任务参数，那么观察点将针对指定的Ada任务（参见第 15.4.10.7 [Ada任务], 页256）。

通常情况下，一个断点会尊重 `expr` 中的变量作用域（见下文）。`-location` 参数指示 GDB 而不是监视 `expr`，而是监视 `expr` 引用的内存。在这种情况下，GDB 会计算 `expr`，获取结果的地址，并监视该地址处的内存。结果类型用于确定

监视内存的大小。如果表达式的结果没有地址，然后 GDB 将打印错误。

The `[mask maskvalue]` 参数允许创建掩码监视点，如果当前架构支持此功能（例如 PowerPC 嵌入式架构，参见第 21.3.8 [PowerPC 嵌入式], 页 354。）掩码监视点除了指定监视的地址外，还会指定一个掩码。该掩码指定在匹配被监视进程访问的地址与监视点地址时，应忽略地址中某些位（即掩码中置零的位）。因此，掩码监视点可以同时监视多个地址——那些未掩码的位与监视点地址中未掩码的位相同的地址。掩码参数隐含 `-location` 选项。示例：

some bits of an address (the bits which are reset in the mask) should be ignored when matching the address accessed by the inferior against the watchpoint address. Thus, a masked watchpoint watches many addresses simultaneously—those addresses whose unmasked bits are identical to the unmasked bits in the watchpoint address. The `mask` argument implies `-location`. Examples:

```
(gdb) watch foo mask 0xffff00ff
(gdb) watch *0xdeadbeef mask 0xffffffff00
```

```
rwatch [-l|-location] expr [thread thread-id] [mask maskvalue]
```

Set a watchpoint that will break when the value of `expr` is read by the program.

```
awatch [-l|-location] expr [thread thread-id] [mask maskvalue]
```

Set a watchpoint that will break when `expr` is either read from or written into by the program.

```
info watchpoints [list...]
```

This command prints a list of watchpoints, using the same format as `info break` (see Section 5.1.1 [Set Breaks], page 60).

If you watch for a change in a numerically entered address you need to dereference it, as the address itself is just a constant number which will never change. GDB refuses to create a watchpoint that watches a never-changing value:

```
(gdb) watch 0x600850
Cannot watch constant value 0x600850.
(gdb) watch *(int *) 0x600850
Watchpoint 1: *(int *) 6293584
```

GDB sets a *hardware watchpoint* if possible. Hardware watchpoints execute very quickly, and the debugger reports a change in value at the exact instruction where the change occurs. If GDB cannot set a hardware watchpoint, it sets a software watchpoint, which executes more slowly and reports the change in value at the next *statement*, not the instruction, after the change occurs.

You can force GDB to use only software watchpoints with the `set can-use-hw-watchpoints 0` command. With this variable set to zero, GDB will never try to use hardware watchpoints, even if the underlying system supports them. (Note that hardware-assisted watchpoints that were set *before* setting `can-use-hw-watchpoints` to zero will still use the hardware mechanism of watching expression values.)

```
set can-use-hw-watchpoints
```

Set whether or not to use hardware watchpoints.

```
show can-use-hw-watchpoints
```

Show the current mode of using hardware watchpoints.

For remote targets, you can restrict the number of hardware watchpoints GDB will use, see [set remote hardware-breakpoint-limit], page 329.

When you issue the `watch` command, GDB reports

```
Hardware watchpoint num: expr
```

if it was able to set a hardware watchpoint.

Currently, the `awatch` and `rwatch` commands can only set hardware watchpoints, because accesses to data that don't change the value of the watched expression cannot be

某些地址位（在掩码中被重置的位）在将调试目标访问的地址与监视点地址进行匹配时应被忽略。因此，一个带掩码的监视点可以同时监视许多地址——那些未掩码的位与监视点地址中未掩码的位相同的地址。掩码参数隐含 `-location`。示例：

```
(gdb) watch foo mask 0xffff00ff
(gdb) watch *0xdeadbeef mask 0xffffffff00
```

```
rwatch [-l|-location] expr [thread thread-id] [mask maskvalue]
```

设置一个断点，当程序读取`expr`的值时将触发中断。

```
awatch [-l|-location] expr [thread thread-id] [mask maskvalue]
```

设置一个断点，当程序读取或写入`expr`时将触发中断。

```
info 断点 [list...]
```

此命令将打印断点列表，格式与`info break`相同（参见第5.1.1 [设置断点],页60）。

如果你监视一个数值输入的地址变化，需要对其进行解引用，因为地址本身只是一个常数，永远不会改变。GDB 拒绝创建一个监视永不变值的断点：

```
(gdb) watch 0x600850
不能监视常数 0x600850.
(gdb) watch *(int *) 0x600850
Watchpoint 1: *(int *) 6293584
```

GDB 在可能的情况下设置硬件断点。硬件断点执行非常快，调试器会在发生变化的精确指令处报告值的变化。如果 GDB 无法设置硬件断点，它会设置软件断点，软件断点执行较慢，并在变化发生后的下一条语句（而不是指令）处报告值的变化。

你可以使用 `set can-use-hw-watchpoints 0` 命令强制 GDB 仅使用软件断点。将此变量设置为 0 后，GDB 将永远不会尝试使用硬件断点，即使底层系统支持它们。（注意：在将 `can-use-hw-watchpoints` 设置为 0 之前设置的硬件辅助断点仍然会使用硬件机制监视表达式值。）

```
设置 can-use-hw-watchpoints
```

设置是否使用硬件断点。

```
显示 can-use-hw-watchpoints
```

显示当前硬件断点使用的模式。

对于远程目标，您可以限制硬件断点 GDB 将使用的数量，请参阅 [set remote hardware-breakpoint-limit], 页面 329。

当您发出 `watch` 命令时，GDB 会报告

```
硬件看门狗 num: expr
```

如果它能够设置一个硬件看门狗。

目前，`awatch` 和 `rwatch` 命令只能设置硬件断点，因为无法在不检查每条正在执行的指令的情况下检测到对未改变被观察表达式值的访问

detected without examining every instruction as it is being executed, and GDB does not do that currently. If GDB finds that it is unable to set a hardware breakpoint with the `awatch` or `rwatch` command, it will print a message like this:

```
Expression cannot be implemented with read/access watchpoint.
```

Sometimes, GDB cannot set a hardware watchpoint because the data type of the watched expression is wider than what a hardware watchpoint on the target machine can handle. For example, some systems can only watch regions that are up to 4 bytes wide; on such systems you cannot set hardware watchpoints for an expression that yields a double-precision floating-point number (which is typically 8 bytes wide). As a work-around, it might be possible to break the large region into a series of smaller ones and watch them with separate watchpoints.

If you set too many hardware watchpoints, GDB might be unable to insert all of them when you resume the execution of your program. Since the precise number of active watchpoints is unknown until such time as the program is about to be resumed, GDB might not be able to warn you about this when you set the watchpoints, and the warning will be printed only when the program is resumed:

```
Hardware watchpoint num: Could not insert watchpoint
```

If this happens, delete or disable some of the watchpoints.

Watching complex expressions that reference many variables can also exhaust the resources available for hardware-assisted watchpoints. That's because GDB needs to watch every variable in the expression with separately allocated resources.

If you call a function interactively using `print` or `call`, any watchpoints you have set will be inactive until GDB reaches another kind of breakpoint or the call completes.

GDB automatically deletes watchpoints that watch local (automatic) variables, or expressions that involve such variables, when they go out of scope, that is, when the execution leaves the block in which these variables were defined. In particular, when the program being debugged terminates, *all* local variables go out of scope, and so only watchpoints that watch global variables remain set. If you rerun the program, you will need to set all such watchpoints again. One way of doing that would be to set a code breakpoint at the entry to the `main` function and when it breaks, set all the watchpoints.

In multi-threaded programs, watchpoints will detect changes to the watched expression from every thread.

Warning: In multi-threaded programs, software watchpoints have only limited usefulness. If GDB creates a software watchpoint, it can only watch the value of an expression *in a single thread*. If you are confident that the expression can only change due to the current thread's activity (and if you are also confident that no other thread can become current), then you can use software watchpoints as usual. However, GDB may not notice when a non-current thread's activity changes the expression. (Hardware watchpoints, in contrast, watch an expression in all threads.)

See [set remote hardware-watchpoint-limit], page 329.

而 GDB 目前不会这样做。如果 GDB 发现使用 `awatch` 或 `rwatch` 命令无法设置硬件断点，它将打印类似以下的消息：

```
无法使用读/访问断点实现该表达式。
```

有时，GDB 无法设置硬件断点，因为被监视表达式的数据类型比目标机器上的硬件断点所能处理的范围更宽。例如，某些系统只能监视宽度最多为4字节的区域；在这样的系统上，您无法为产生双精度浮点数（通常为8字节宽）的表达式设置硬件断点。作为解决方法，可以将大区域拆分为一系列较小的区域，并使用单独的断点进行监视。

如果您设置了过多的硬件断点，GDB 在您恢复程序执行时可能无法插入所有断点。由于活动断点的确切数量直到程序即将恢复时才可知晓，GDB 在您设置断点时可能无法警告您，而警告信息仅在程序恢复时才会打印：

```
硬件断点 num: 无法插入断点
```

如果出现这种情况，请删除或禁用一些断点。

监视引用多个变量的复杂表达式也可能耗尽硬件辅助断点可用的资源。这是因为 GDB 需要为表达式中的每个变量分配单独的资源进行监视。

如果你使用 `print` 或 `call` 交互式调用函数，你设置的任何断点都将处于非活动状态，直到 GDB 触发另一种类型的断点或调用完成。

GDB 会在局部（自动）变量或涉及此类变量的表达式超出作用域时自动删除断点，即当执行离开这些变量定义的代码块时。特别是，当被调试的程序终止时，所有局部变量都会超出作用域，因此只有监视全局变量的断点会保持设置。如果你重新运行程序，你需要再次设置所有此类断点。一种方法是在主函数入口处设置代码断点，当它触发断点时，再设置所有断点。

在多线程程序中，断点器将检测所有线程对监视表达式的更改。

*警告：*在多线程程序中，软件断点器只有有限的用途。如果 GDB 创建了一个软件断点器，它只能监视单个线程中表达式的值。如果你确信该表达式只能由于当前线程的活动而更改（并且你也确信没有其他线程会成为当前线程），那么你可以像往常一样使用软件断点器。然而，GDB 可能不会注意到非当前线程的活动更改了表达式。（相比之下，硬件断点器会监视所有线程中的表达式。）

请参阅 [设置远程硬件断点限制], 第 329 页。

5.1.3 Setting Catchpoints

You can use *catchpoints* to cause the debugger to stop for certain kinds of program events, such as C++ exceptions or the loading of a shared library. Use the `catch` command to set a catchpoint.

`catch event`

Stop when *event* occurs. The *event* can be any of the following:

```
throw [regex]
rethrow [regex]
catch [regex]
```

The throwing, re-throwing, or catching of a C++ exception.

If *regex* is given, then only exceptions whose type matches the regular expression will be caught.

The convenience variable `$_exception` is available at an exception-related catchpoint, on some systems. This holds the exception being thrown.

There are currently some limitations to C++ exception handling in GDB:

- The support for these commands is system-dependent. Currently, only systems using the ‘gnu-v3’ C++ ABI (see Section 22.7 [ABI], page 380) are supported.
- The regular expression feature and the `$_exception` convenience variable rely on the presence of some SDT probes in `libstdc++`. If these probes are not present, then these features cannot be used. These probes were first available in the GCC 4.8 release, but whether or not they are available in your GCC also depends on how it was built.
- The `$_exception` convenience variable is only valid at the instruction at which an exception-related catchpoint is set.
- When an exception-related catchpoint is hit, GDB stops at a location in the system library which implements runtime exception support for C++, usually `libstdc++`. You can use `up` (see Section 8.3 [Selection], page 119) to get to your code.
- If you call a function interactively, GDB normally returns control to you when the function has finished executing. If the call raises an exception, however, the call may bypass the mechanism that returns control to you and cause your program either to abort or to simply continue running until it hits a breakpoint, catches a signal that GDB is listening for, or exits. This is the case even if you set a catchpoint for the exception; catchpoints on exceptions are disabled within interactive calls. See Section 17.5 [Calling], page 283, for information on controlling this with `set unwind-on-terminating-exception`.
- You cannot raise an exception interactively.
- You cannot install an exception handler interactively.

5.1.3 设置捕获点

您可以使用断点来让调试器在特定类型的程序事件时停止，例如 C++ 异常或共享库的加载。使用 `catch` 命令来设置断点。

`catch event`

当事件发生时停止。该事件可以是以下任何一种：

```
throw [regex]
rethrow [regex]
catch [regex]
```

C++ 异常的抛出、重新抛出或捕获。

如果提供了正则表达式，则仅捕获类型与正则表达式匹配的异常。

在关联异常的捕获点处，某些系统上可用便利变量 `$_exception`，它保存着被抛出的异常。

目前C++ 异常处理在GDB中存在一些限制：

- 这些命令的支持取决于系统。目前，仅支持使用 ‘gnu-v3’ C++ ABI（参见第22.7 [ABI],页380页）的系统。
- 正则表达式功能和 `$_exception` 便利变量依赖于 `libstdc++` 中存在某些 SDT 探针。如果这些探针不存在，则无法使用这些功能。这些探针首次出现在 GCC 4.8 版本中，但它们是否在您的 GCC 中可用，还取决于它是如何构建的。
- `$_exception` 便利变量仅在设置了关联异常捕获点的指令处有效。
- 当关联异常的捕获点被触发时，GDB 会停止在实现 C++ 运行时异常支持的系统库位置，通常是 `libstdc++`。您可以使用 `up`（参见第 8.3 [选择],页119）来进入您的代码。
- 如果你交互式调用一个函数，GDB 通常在函数执行完毕后会返回控制权给你。但如果调用引发异常，调用可能会绕过返回控制权的机制，导致你的程序要么中止，要么继续运行直到遇到断点、捕获到正在监听信号或退出。即使你为异常设置了捕获点，这种情况也会发生；在交互式调用中，异常的捕获点被禁用。参见第17节.5 [调用], 页283，了解如何使用 `set unwind-on-terminating-exception` 控制此行为。
- 你无法交互式引发异常。
- 你无法交互式安装异常处理器。

exception [*name*]
An Ada exception being raised. If an exception name is specified at the end of the command (eg `catch exception Program_Error`), the debugger will stop only when this specific exception is raised. Otherwise, the debugger stops execution when any Ada exception is raised.

When inserting an exception catchpoint on a user-defined exception whose name is identical to one of the exceptions defined by the language, the fully qualified name must be used as the exception name. Otherwise, GDB will assume that it should stop on the pre-defined exception rather than the user-defined one. For instance, assuming an exception called `Constraint_Error` is defined in package `Pck`, then the command to use to catch such exceptions is `catch exception Pck.Constraint_Error`.

The convenience variable `$_ada_exception` holds the address of the exception being thrown. This can be useful when setting a condition for such a catchpoint.

exception unhandled
An exception that was raised but is not handled by the program. The convenience variable `$_ada_exception` is set as for `catch exception`.

handlers [*name*]
An Ada exception being handled. If an exception name is specified at the end of the command (eg `catch handlers Program_Error`), the debugger will stop only when this specific exception is handled. Otherwise, the debugger stops execution when any Ada exception is handled.

When inserting a handlers catchpoint on a user-defined exception whose name is identical to one of the exceptions defined by the language, the fully qualified name must be used as the exception name. Otherwise, GDB will assume that it should stop on the pre-defined exception rather than the user-defined one. For instance, assuming an exception called `Constraint_Error` is defined in package `Pck`, then the command to use to catch such exceptions handling is `catch handlers Pck.Constraint_Error`.

The convenience variable `$_ada_exception` is set as for `catch exception`.

assert
A failed Ada assertion. Note that the convenience variable `$_ada_exception` is *not* set by this catchpoint.

exec
A call to `exec`.

syscall
syscall [*name* | *number* | *group:groupname* | *g:groupname*] ...
A call to or return from a system call, a.k.a. *syscall*. A syscall is a mechanism for application programs to request a service from the

异常 [*name*]
一个Ada异常被抛出。如果命令末尾指定了异常名称（例如 `catch exception Program_Error`），调试器只有在抛出此特定异常时才会停止。否则，调试器会在抛出任何Ada异常时停止执行。

当在名称与语言预定义异常相同的用户自定义异常上插入异常捕获点时，必须使用完全限定名作为异常名。否则，GDB 会假定它应该停止在预定义异常而不是用户自定义异常上。例如，假设在包 `Pck`中定义了一个名为`Constraint_Error`的异常，那么捕获此类异常的命令是 `catchexception Pck.Constraint_Error`。

便利变量`$_ada_exception`保存正在抛出的异常的地址。这在设置此类捕获点的条件时很有用。

异常未处理
一个被抛出但未被程序处理的异常。便利变量`$_ada_exception`的设置方式与`catch`异常相同。

处理器 [*name*]
正在处理 Ada 异常。如果在命令末尾指定了异常名称（例如 `catch handlers Program_Error`），调试器将仅在处理该特定异常时停止。否则，当处理任何 Ada 异常时，调试器会停止执行。

当在用户定义的异常上插入处理器 catchpoint 时，如果该异常的名称与语言定义的异常之一相同，则必须使用完全限定名称作为异常名称。否则，GDB 会假定它应该停止预定义的异常，而不是用户定义的异常。例如，假设在包 `Pck` 中定义了一个名为 `Constraint_Error` 的异常，那么要捕获此类异常处理的命令是 `catch handlers Pck.Constraint_Error`。

方便变量 `$_ada_exception` 与 `catch exception` 类似地设置。

assert A 失败的 Ada 断言。请注意，此 catchpoint 不会设置方便变量 `$_ada_exception`。

exec A 调用 `exec`。

系统调用
系统调用 [*name* | *number* | *组:groupname* | *g:groupname*] ...
对系统调用（syscall）的调用或返回。系统调用是一种应用程序请求操作系统（OS）或OS系统服务的一种机制。

operating system (OS) or one of the OS system services. GDB can catch some or all of the syscalls issued by the debuggee, and show the related information for each syscall. If no argument is specified, calls to and returns from all system calls will be caught.

name can be any system call name that is valid for the underlying OS. Just what syscalls are valid depends on the OS. On GNU and Unix systems, you can find the full list of valid syscall names on `/usr/include/asm/unistd.h`.

Normally, GDB knows in advance which syscalls are valid for each OS, so you can use the GDB command-line completion facilities (see Section 3.3 [command completion], page 24) to list the available choices.

You may also specify the system call numerically. A syscall’s number is the value passed to the OS’s syscall dispatcher to identify the requested service. When you specify the syscall by its name, GDB uses its database of syscalls to convert the name into the corresponding numeric code, but using the number directly may be useful if GDB’s database does not have the complete list of syscalls on your system (e.g., because GDB lags behind the OS upgrades).

You may specify a group of related syscalls to be caught at once using the **group:** syntax (**g:** is a shorter equivalent). For instance, on some platforms GDB allows you to catch all network related syscalls, by passing the argument **group:network** to **catch syscall**. Note that not all syscall groups are available in every system. You can use the command completion facilities (see Section 3.3 [command completion], page 24) to list the syscall groups available on your environment.

The example below illustrates how this command works if you don’t provide arguments to it:

```
(gdb) catch syscall
Catchpoint 1 (syscall)
(gdb) r
Starting program: /tmp/catch-syscall

Catchpoint 1 (call to syscall 'close'), \
0xffffe424 in __kernel_vsyscall ()
(gdb) c
Continuing.

Catchpoint 1 (returned from syscall 'close'), \
0xffffe424 in __kernel_vsyscall ()
(gdb)
```

Here is an example of catching a system call by name:

```
(gdb) catch syscall chroot
Catchpoint 1 (syscall 'chroot' [61])
(gdb) r
Starting program: /tmp/catch-syscall

Catchpoint 1 (call to syscall 'chroot'), \
```

GDB 可以捕获调试目标发出的部分或全部系统调用，并显示每个系统调用相关的信息。如果没有指定参数，将捕获所有系统调用的调用和返回。

名称可以是任何对底层操作系统有效的系统调用名称。哪些系统调用有效取决于操作系统。在GNU和Unix系统上，您可以在 `/usr/include/asm/unistd.h` 中找到所有有效的系统调用名称列表。

通常情况下，GDB 会提前知道每个操作系统上哪些系统调用是有效的，因此您可以使用 GDB 命令行补全功能（参见第 3.3 [命令补全]，页 24）来列出可用选项。

您也可以通过数字指定系统调用。系统调用的编号是传递给操作系统调用调度器的值，用于标识请求的服务。当您通过名称指定系统调用时，GDB 会使用其系统调用数据库将名称转换为相应的数字代码，但如果 GDB 的数据库没有包含您系统上的完整系统调用列表（例如，因为 GDB 落后于操作系统升级），直接使用数字可能更有用。

您可以使用 **group:** 语法（**g:** 是更简洁的等效写法）一次性指定一组相关的系统调用进行捕获。例如，在某些平台上 GDB 允许您通过传递参数 **group:network** 来捕获所有与网络相关的系统调用。请注意，并非所有系统调用组在所有系统中都可用。您可以使用命令补全功能（参见第 3.3 [命令补全]，页 24）来列出您环境中可用的系统调用组。

下面的示例说明了如果向该命令不提供参数，它是如何工作的：

```
(gdb) catch syscall
Catchpoint 1 (syscall)
(gdb) r
Starting program: /tmp/catch-syscall

Catchpoint 1 (call to syscall 'close'), \
0xffffe424 in __kernel_vsyscall ()
(gdb) c
Continuing.

Catchpoint 1 (returned from syscall 'close'), \
0xffffe424 in __kernel_vsyscall ()
(gdb)
```

Here is an example of catching a system call by name:

```
(gdb) catch syscall chroot
Catchpoint 1 (syscall 'chroot' [61])
(gdb) r
Starting program: /tmp/catch-syscall

捕获点 1 (调用系统调用 'chroot') , \
```

```

0xffffe424 in __kernel_vsyscall ()
(gdb) c
Continuing.

Catchpoint 1 (returned from syscall 'chroot'), \
0xffffe424 in __kernel_vsyscall ()
(gdb)

```

An example of specifying a system call numerically. In the case below, the syscall number has a corresponding entry in the XML file, so GDB finds its name and prints it:

```

(gdb) catch syscall 252
Catchpoint 1 (syscall(s) 'exit_group')
(gdb) r
Starting program: /tmp/catch-syscall

Catchpoint 1 (call to syscall 'exit_group'), \
0xffffe424 in __kernel_vsyscall ()
(gdb) c
Continuing.

Program exited normally.
(gdb)

```

Here is an example of catching a syscall group:

```

(gdb) catch syscall group:process
Catchpoint 1 (syscalls 'exit' [1] 'fork' [2] 'waitpid' [7]
'execve' [11] 'wait4' [114] 'clone' [120] 'vfork' [190]
'exit_group' [252] 'waitid' [284] 'unshare' [310])
(gdb) r
Starting program: /tmp/catch-syscall

Catchpoint 1 (call to syscall fork), 0x00007ffff7df4e27 in open64 ()
from /lib64/ld-linux-x86-64.so.2

(gdb) c
Continuing.

```

However, there can be situations when there is no corresponding name in XML file for that syscall number. In this case, GDB prints a warning message saying that it was not able to find the syscall name, but the catchpoint will be set anyway. See the example below:

```

(gdb) catch syscall 764
warning: The number '764' does not represent a known syscall.
Catchpoint 2 (syscall 764)
(gdb)

```

If you configure GDB using the '--without-expat' option, it will not be able to display syscall names. Also, if your architecture does not have an XML file describing its system calls, you will not be able to see the syscall names. It is important to notice that these two features are used for accessing the syscall name database. In either case, you will see a warning like this:

```

(gdb) catch syscall
warning: Could not open "syscalls/i386-linux.xml"
warning: Could not load the syscall XML file 'syscalls/i386-linux.xml'.

```

```

0xffffe424 in __kernel_vsyscall ()
(gdb) c
Continuing.

Catchpoint 1 (returned from syscall 'chroot'), \
0xffffe424 in __kernel_vsyscall ()
(gdb)

```

指定系统调用编号的示例。在下面的情况下，系统调用编号在 XML 文件中有一个对应的条目，因此 GDB 找到它的名称并打印它：

```

(gdb) catch syscall 252
Catchpoint 1 (syscall(s) 'exit_group')
(gdb) r
Starting program: /tmp/catch-syscall

Catchpoint 1 (call to syscall 'exit_group'), \
0xffffe424 in __kernel_vsyscall ()
(gdb) c
Continuing.

程序正常退出。
(gdb)

```

Here is an example of catching a syscall group:

```

(gdb) catch syscall group:process
Catchpoint 1 (syscalls 'exit' [1] 'fork' [2] 'waitpid' [7]
'execve' [11] 'wait4' [114] 'clone' [120] 'vfork' [190]
'exit_group' [252] 'waitid' [284] 'unshare' [310])
(gdb) r
Starting program: /tmp/catch-syscall

Catchpoint 1 (call to syscall fork), 0x00007ffff7df4e27 in open64 ()
from /lib64/ld-linux-x86-64.so.2

(gdb) c
Continuing.

```

然而，有时在XML文件中找不到与该系统调用号对应的名称。在这种情况下，GDB 会打印一条警告消息，说明它无法找到系统调用名称，但仍然会设置捕获点。请参考以下示例：

```

(gdb) catch syscall 764 警告：数字“764”不代表一个已知的系
统调用。捕获点 2 (系统调用 764) (gdb)

```

如果你使用 '--without-expat' 选项配置 GDB，它将无法显示系统调用名称。此外，如果你的架构没有描述其系统调用的 XML 文件，你将无法看到系统调用名称。需要注意的是，这两个功能用于访问系统调用名称数据库。在任一情况下，你都会看到类似以下的警告：

```

(gdb) catch syscall
warning: Could not open "syscalls/i386-linux.xml"
warning: Could not load the syscall XML file 'syscalls/i386-linux.xml'.

```

```
GDB will not be able to display syscall names.
Catchpoint 1 (syscall)
(gdb)
```

Of course, the file name will change depending on your architecture and system.

Still using the example above, you can also try to catch a syscall by its number. In this case, you would see something like:

```
(gdb) catch syscall 252
Catchpoint 1 (syscall(s) 252)
```

Again, in this case GDB would not be able to display syscall’s names.

fork A call to **fork**.

vfork A call to **vfork**.

load [regex]

unload [regex]

The loading or unloading of a shared library. If *regex* is given, then the catchpoint will stop only if the regular expression matches one of the affected libraries.

signal [signal... | ‘all’]

The delivery of a signal.

With no arguments, this catchpoint will catch any signal that is not used internally by GDB, specifically, all signals except ‘SIGTRAP’ and ‘SIGINT’.

With the argument ‘all’, all signals, including those used by GDB, will be caught. This argument cannot be used with other signal names.

Otherwise, the arguments are a list of signal names as given to **handle** (see Section 5.4 [Signals], page 92). Only signals specified in this list will be caught.

One reason that **catch signal** can be more useful than **handle** is that you can attach commands and conditions to the catchpoint.

When a signal is caught by a catchpoint, the signal’s **stop** and **print** settings, as specified by **handle**, are ignored. However, whether the signal is still delivered to the inferior depends on the **pass** setting; this can be changed in the catchpoint’s commands.

tcatch event

Set a catchpoint that is enabled only for one stop. The catchpoint is automatically deleted after the first time the event is caught.

Use the **info break** command to list the current catchpoints.

5.1.4 Deleting Breakpoints

It is often necessary to eliminate a breakpoint, watchpoint, or catchpoint once it has done its job and you no longer want your program to stop there. This is called *deleting* the breakpoint. A breakpoint that has been deleted no longer exists; it is forgotten.

```
GDB will not be able to display syscall names.
Catchpoint 1 (syscall)
(gdb)
```

当然，文件名会根据你的架构和系统而变化。

仍然使用上面的示例，你也可以尝试通过系统调用的编号来捕获它。在这种情况下，你会看到类似的内容：

```
(gdb) catch syscall 252
Catchpoint 1 (syscall(s) 252)
```

同样，在这种情况下，GDB 将无法显示系统调用的名称。

fork Acall来调用fork。

vfork Acall来调用vfork。

load [regex]

unload [regex] 加载或卸载共享库。如果给出了正则表达式，那么只有在正则表达式匹配受影响的库之一时，捕获点才会停止。

signal [signal... | ‘all’]

发送信号。

如果没有参数，此捕捉点将捕捉由 GDB内部未使用的任何信号，具体来说，除了 ‘SIGTRAP’ 和 ‘SIGINT’ 之外的所有信号。

使用参数 ‘all’，所有信号（包括由 GDB使用的信号）都将被捕捉。此参数不能与其他信号名称一起使用。

否则，参数是作为handle提供的信号名称列表（参见第5.4 [信号], 页92）。只有在此列表中指定的信号才会被捕捉。

捕捉信号比handle更有用的一个原因是，你可以将命令和条件附加到捕捉点上。

当信号被捕捉点捕捉时，信号的中断和打印设置（由handle指定）将被忽略。但是，信号是否仍然传递给子进程取决于传递设置；这可以在捕捉点的命令中更改。

捕捉 event

设置一个仅对一次停止生效的捕捉点。该捕捉点在事件被捕捉后自动删除。

使用 **info break** 命令列出当前的捕捉点。

5.1.4 删除断点

当断点、观察点或捕获点完成其任务后，通常需要将其删除，不再希望程序在此停止。这称为删除断点。已被删除的断点不再存在；它被遗忘了。

With the `clear` command you can delete breakpoints according to where they are in your program. With the `delete` command you can delete individual breakpoints, watchpoints, or catchpoints by specifying their breakpoint numbers.

It is not necessary to delete a breakpoint to proceed past it. GDB automatically ignores breakpoints on the first instruction to be executed when you continue execution without changing the execution address.

clear Delete any breakpoints at the next instruction to be executed in the selected stack frame (see Section 8.3 [Selecting a Frame], page 119). When the innermost frame is selected, this is a good way to delete a breakpoint where your program just stopped.

clear locspec Delete any breakpoint with a code location that corresponds to *locspec*. See Section 9.2 [Location Specifications], page 128, for the various forms of *locspec*. Which code locations correspond to *locspec* depends on the form used in the location specification *locspec*:

```
linenum
filename:linenum
-line linenum
-source filename -line linenum
```

If *locspec* specifies a line number, with or without a file name, the command deletes any breakpoint with a code location that is at or within the specified line *linenum* in files that match the specified *filename*. If *filename* is omitted, it defaults to the current source file.

***address** If *locspec* specifies an address, the command deletes any breakpoint with a code location that is at the given *address*.

```
function
-function function
```

If *locspec* specifies a function, the command deletes any breakpoint with a code location that is at the entry to any function whose name matches *function*.

Ambiguity in names of files and functions can be resolved as described in Section 9.2 [Location Specifications], page 128.

delete [breakpoints] [list...] Delete the breakpoints, watchpoints, tracepoints, or catchpoints of the breakpoint list specified as argument. If no argument is specified, delete all breakpoints, watchpoints, tracepoints, and catchpoints (GDB asks confirmation, unless you have `set confirm off`). You can abbreviate this command as `d`.

5.1.5 Disabling Breakpoints

Rather than deleting a breakpoint, watchpoint, or catchpoint, you might prefer to *disable* it. This makes the breakpoint inoperative as if it had been deleted, but remembers the information on the breakpoint so that you can *enable* it again later.

使用 `clear` 命令可以根据断点在程序中的位置删除断点。使用 `delete` 命令可以通过指定断点编号来删除单个断点、观察点或捕获点。

继续执行时无需删除断点即可跳过它。GDB 在您不更改执行地址的情况下继续执行时，会自动忽略第一个要执行的指令上的断点。

清除 选择选定栈帧中下一个要执行的断点（参见第8.3 [选择帧], 页119）。当选择最内层帧时，这是删除您程序刚刚停止处的断点的好方法。

清除 *locspec*

删除与 *locspec* 对应的代码位置的任何断点。参见第9.2 [位置规范], 页128, 了解 *locspec* 的各种形式。哪些代码位置对应于 *locspec* 取决于位置规范 *locspec* 中使用的形式:

```
linenum
filename:linenum
-line linenum
-source filename -line linenum
```

如果 *locspec* 指定了行号，无论是否指定了文件名，该命令都会删除任何位于指定文件 *filename* 中指定行 *linenum* 上或其内的代码位置的断点。如果省略了 *filename*，则默认为当前源文件。

***address** 如果 *locspec* 指定了一个地址，该命令会删除所有位于给定地址的断点。

```
function
-function function
```

如果 *locspec* 指定了一个函数，该命令会删除所有位于任何匹配函数名的函数入口处的断点。

文件和函数名称的歧义可以按照第9节.2 [位置规范], 页128中的描述进行解决。

删除 [断点] [list...] 删除作为参数指定的断点列表中的断点、观察点、跟踪点或捕获点。如果没有指定参数，则删除所有断点、观察点、跟踪点和捕获点（GDB 询问确认，除非您已设置 `confirm off`）。您可以缩写此命令为 `d`。

5.1.5 禁用断点

与其删除断点、观察点或捕获点，您可能更喜欢禁用它。这会使断点失效，就像已删除一样，但会保留断点的信息，以便您稍后重新启用它。

You disable and enable breakpoints, watchpoints, tracepoints, and catchpoints with the **enable** and **disable** commands, optionally specifying one or more breakpoint numbers as arguments. Use **info break** to print a list of all breakpoints, watchpoints, tracepoints, and catchpoints if you do not know which numbers to use.

Disabling and enabling a breakpoint that has multiple locations affects all of its locations.

A breakpoint, watchpoint, or catchpoint can have any of several different states of enablement:

- Enabled. The breakpoint stops your program. A breakpoint set with the **break** command starts out in this state.
- Disabled. The breakpoint has no effect on your program.
- Enabled once. The breakpoint stops your program, but then becomes disabled.
- Enabled for a count. The breakpoint stops your program for the next N times, then becomes disabled.
- Enabled for deletion. The breakpoint stops your program, but immediately after it does so it is deleted permanently. A breakpoint set with the **tbreak** command starts out in this state.

You can use the following commands to enable or disable breakpoints, watchpoints, tracepoints, and catchpoints:

disable [breakpoints] [list...]

Disable the specified breakpoints—or all breakpoints, if none are listed. A disabled breakpoint has no effect but is not forgotten. All options such as ignore-counts, conditions and commands are remembered in case the breakpoint is enabled again later. You may abbreviate **disable** as **dis**.

enable [breakpoints] [list...]

Enable the specified breakpoints (or all defined breakpoints). They become effective once again in stopping your program.

enable [breakpoints] once list...

Enable the specified breakpoints temporarily. GDB disables any of these breakpoints immediately after stopping your program.

enable [breakpoints] count count list...

Enable the specified breakpoints temporarily. GDB records *count* with each of the specified breakpoints, and decrements a breakpoint's count when it is hit. When any count reaches 0, GDB disables that breakpoint. If a breakpoint has an ignore count (see Section 5.1.6 [Break Conditions], page 78), that will be decremented to 0 before *count* is affected.

enable [breakpoints] delete list...

Enable the specified breakpoints to work once, then die. GDB deletes any of these breakpoints as soon as your program stops there. Breakpoints set by the **tbreak** command start out in this state.

Except for a breakpoint set with **tbreak** (see Section 5.1.1 [Setting Breakpoints], page 60), breakpoints that you set are initially enabled; subsequently, they become disabled or enabled only when you use one of the commands above. (The command **until**

您使用 **enable** 和 **disable** 命令禁用和启用断点、观察点、跟踪点和捕获点，可选择指定一个或多个断点编号作为参数。如果您不知道要使用哪些编号，可以使用 **info break** 命令打印所有断点、观察点、跟踪点和捕获点的列表。

禁用或启用一个具有多个位置的中断点会影响其所有位置。

一个中断点、观察点或捕获点可以具有多种不同的启用状态：

- 已启用。中断点会停止你的程序。使用 **break** 命令设置的中断点初始处于此状态。
- 已禁用。中断点对你的程序没有任何影响。
- 仅启用一次。中断点会停止你的程序，但随后会变为禁用状态。
- 用于计数。断点会暂停你的程序 N 次，然后自动禁用。
- 用于删除。断点会暂停程序，但立即被永久删除。使用 **tbreak** 命令设置的断点初始处于此状态。

你可以使用以下命令来启用或禁用断点、观察点、跟踪点和捕获点：

disable [断点] [list...]

禁用指定的断点——如果没有指定，则禁用所有断点。已禁用的断点无效但不会被遗忘。所有选项（如忽略计数、条件和命令）都会被保留，以便以后重新启用断点。你可以将 **disable** 缩写为 **dis**。

enable [断点] [list...]

启用指定的断点使其再次生效并继续运行你的程序。

启用 [断点] 一次 list...

暂时启用指定的断点。GDB 在停止您的程序后立即禁用这些断点中的任何断点。

启用 [断点] 计数 count list...

暂时启用指定的断点。GDB 记录每个指定断点的计数，并在命中断点时递减断点的计数。当任何计数达到 0 时，GDB 禁用该断点。如果断点有忽略计数（参见第 5.1.6 [断点条件]，页 78），则该忽略计数将在影响计数之前递减至 0。

启用 [断点] 删除 list...

启用指定的断点使其一次性工作，然后消失。GDB 当你的程序停止时，会立即删除这些断点。使用 **tbreak** 命令设置的断点初始处于此状态。

除使用 **tbreak** 设置的断点（参见第 5.1.1 [设置断点]，页 60）外，您设置的断点最初是启用的；随后，它们仅在您使用上述命令之一时才会被禁用或启用。（命令 **until**

can set and delete a breakpoint of its own, but it does not change the state of your other breakpoints; see Section 5.2 [Continuing and Stepping], page 85.)

5.1.6 Break Conditions

The simplest sort of breakpoint breaks every time your program reaches a specified place. You can also specify a *condition* for a breakpoint. A condition is just a Boolean expression in your programming language (see Section 10.1 [Expressions], page 147). A breakpoint with a condition evaluates the expression each time your program reaches it, and your program stops only if the condition is *true*.

This is the converse of using assertions for program validation; in that situation, you want to stop when the assertion is violated—that is, when the condition is false. In C, if you want to test an assertion expressed by the condition `assert`, you should set the condition ‘`! assert`’ on the appropriate breakpoint.

Conditions are also accepted for watchpoints; you may not need them, since a watchpoint is inspecting the value of an expression anyhow—but it might be simpler, say, to just set a watchpoint on a variable name, and specify a condition that tests whether the new value is an interesting one.

Break conditions can have side effects, and may even call functions in your program. This can be useful, for example, to activate functions that log program progress, or to use your own print functions to format special data structures. The effects are completely predictable unless there is another enabled breakpoint at the same address. (In that case, GDB might see the other breakpoint first and stop your program without checking the condition of this one.) Note that breakpoint commands are usually more convenient and flexible than break conditions for the purpose of performing side effects when a breakpoint is reached (see Section 5.1.7 [Breakpoint Command Lists], page 79).

Breakpoint conditions can also be evaluated on the target’s side if the target supports it. Instead of evaluating the conditions locally, GDB encodes the expression into an agent expression (see Appendix F [Agent Expressions], page 871) suitable for execution on the target, independently of GDB. Global variables become raw memory locations, locals become stack accesses, and so forth.

In this case, GDB will only be notified of a breakpoint trigger when its condition evaluates to true. This mechanism may provide faster response times depending on the performance characteristics of the target since it does not need to keep GDB informed about every breakpoint trigger, even those with false conditions.

Break conditions can be specified when a breakpoint is set, by using ‘`if`’ in the arguments to the `break` command. See Section 5.1.1 [Setting Breakpoints], page 60. They can also be changed at any time with the `condition` command.

You can also use the `if` keyword with the `watch` command. The `catch` command does not recognize the `if` keyword; `condition` is the only way to impose a further condition on a catchpoint.

`condition bnum expression`

Specify *expression* as the break condition for breakpoint, watchpoint, or catchpoint number *bnum*. After you set a condition, breakpoint *bnum* stops your program only if the value of *expression* is true (nonzero, in C). When you use `condition`, GDB checks *expression* immediately for syntactic correctness,

可以设置和删除自己的断点，但它不会改变您其他断点的状态；参见第 5.2 [继续和单步执行], 页 85。)

5.1.6 断点条件

最简单的断点会在你的程序到达指定位置时每次触发。你也可以为断点指定一个条件。条件只是你编程语言中的一个布尔表达式（参见第 10.1 [表达式], 页 147）。带有条件的断点会在每次程序到达该断点时评估表达式，并且只有当条件为真时，程序才会停止。

这相当于使用断言进行程序验证的情况的逆操作；在这种情况下，你希望在断言被违反时停止——也就是说，当条件为假时。在 C 语言中，如果你想要测试由条件 `assert` 表达的断言，你应该在相应的断点上设置条件 ‘`! assert`’。

条件也适用于监视点；你可能不需要它们，因为监视点无论如何都在检查表达式的值——但也许更简单，比如，直接在变量名上设置一个监视点，并指定一个条件来测试新值是否有趣。

中断条件可能产生副作用，甚至可能调用程序中的函数。这很有用，例如，可以激活记录程序进度的函数，或使用自己的打印函数来格式化特殊的数据结构。只要在相同地址上没有其他启用的断点，这些效果是完全可预测的。（在这种情况下，GDB 可能会先看到其他断点并停止程序，而不会检查此断点的条件。）请注意，当断点被触发时，执行副作用的目的（见第 5.1.7 [断点命令列表], 页 79），断点命令通常比中断条件更方便和灵活。

如果目标支持，断点条件也可以在目标端进行评估。而不是在本地评估条件，GDB 将表达式编码为适合在目标上执行的代理表达式（见附录 F [代理表达式], 页 871），而与 GDB 无关。全局变量变成原始内存位置，局部变量变成栈访问，等等。

在这种情况下，GDB 只有在其条件评估为真时才会收到断点触发的通知。由于它不需要告知 GDB 每个断点触发（即使那些条件为假的情况），因此这种机制可能根据目标性能特征提供更快响应时间。

断点条件可以在设置断点时指定，通过在 `break` 命令的参数中使用 ‘`if`’。参见第 5.1.1 [设置断点], 页 60。它们也可以随时使用 `condition` 命令更改。

您也可以在 `watch` 命令中使用 `if` 关键字。catch 命令不识别 `if` 关键字；`condition` 是唯一可以进一步对 catchpoint 施加条件的方式。

`condition bnum expression`

将表达式指定为断点、观察点或捕获点编号 *bnum* 的中断条件。设置条件后，断点 *bnum* 仅当表达式的值为真（在 C 中为非零）时才会停止您的程序。使用条件时，GDB 会立即检查表达式的语法正确性，

and to determine whether symbols in it have referents in the context of your breakpoint. If *expression* uses symbols not referenced in the context of the breakpoint, GDB prints an error message:

No symbol "foo" in current context.

GDB does not actually evaluate *expression* at the time the `condition` command (or a command that sets a breakpoint with a condition, like `break if ...`) is given, however. See Section 10.1 [Expressions], page 147.

condition -force *bnum expression*

When the `-force` flag is used, define the condition even if *expression* is invalid at all the current locations of breakpoint *bnum*. This is similar to the `-force-condition` option of the `break` command.

condition *bnum*

Remove the condition from breakpoint number *bnum*. It becomes an ordinary unconditional breakpoint.

A special case of a breakpoint condition is to stop only when the breakpoint has been reached a certain number of times. This is so useful that there is a special way to do it, using the *ignore count* of the breakpoint. Every breakpoint has an ignore count, which is an integer. Most of the time, the ignore count is zero, and therefore has no effect. But if your program reaches a breakpoint whose ignore count is positive, then instead of stopping, it just decrements the ignore count by one and continues. As a result, if the ignore count value is *n*, the breakpoint does not stop the next *n* times your program reaches it.

ignore *bnum count*

Set the ignore count of breakpoint number *bnum* to *count*. The next *count* times the breakpoint is reached, your program's execution does not stop; other than to decrement the ignore count, GDB takes no action.

To make the breakpoint stop the next time it is reached, specify a count of zero.

When you use `continue` to resume execution of your program from a breakpoint, you can specify an ignore count directly as an argument to `continue`, rather than using `ignore`. See Section 5.2 [Continuing and Stepping], page 85.

If a breakpoint has a positive ignore count and a condition, the condition is not checked. Once the ignore count reaches zero, GDB resumes checking the condition.

You could achieve the effect of the ignore count with a condition such as `'$foo-- <= 0'` using a debugger convenience variable that is decremented each time. See Section 10.12 [Convenience Variables], page 173.

Ignore counts apply to breakpoints, watchpoints, tracepoints, and catchpoints.

5.1.7 Breakpoint Command Lists

You can give any breakpoint (or watchpoint or catchpoint) a series of commands to execute when your program stops due to that breakpoint. For example, you might want to print the values of certain expressions, or enable other breakpoints.

并确定其中符号在您的断点上下文中是否有引用。如果表达式使用了在断点上下文中未引用的符号，GDB 会打印错误消息：

当前上下文中没有“foo”符号。

GDB 在条件命令（或使用条件设置断点的命令，如 `break if ...`）给出时，实际上并不评估表达式，不过。参见第10.1 [表达式], 页147。

condition -force *bnum expression*

当使用 `-force` 标志时，即使表达式在断点 *bnum* 的所有当前位置都无效，也会定义条件。这与 `break` 命令的 `-force` 条件选项类似。

condition *bnum*

从断点编号 *bnum* 中移除条件。它将变成一个普通的无条件断点。

断点条件的一种特殊情况是，仅在断点被达到一定次数时才停止。这非常实用，因此有一种特殊的方法来处理，使用断点的忽略计数。每个断点都有一个忽略计数，它是一个整数。大多数情况下，忽略计数为零，因此没有效果。但如果您的程序达到一个忽略计数为正的断点，那么它不会停止，而是将忽略计数减一然后继续。结果，如果忽略计数值为 *n*，则断点在您的程序接下来 *n* 次达到它时不会停止。

忽略 *bnum count*

将断点编号 *bnum* 的忽略计数设置为 *count*。在接下来的 *count* 次断点被达到时，您的程序的执行不会停止；除了减去忽略计数，GDB 不执行任何操作。

要使断点在下一一次被达到时停止，请指定计数为零。

当您使用 `continue` 从断点处恢复程序执行时，可以直接将忽略计数作为参数传递给 `continue`，而不是使用 `ignore`。参见第 5.2 [继续和步进], 页 85。

如果一个断点具有正的忽略计数并且有条件，则不会检查该条件。一旦忽略计数变为零，GDB 将重新检查该条件。

您可以使用类似 `'$foo-- <= 0'` 的条件，通过一个每次都会递减的调试器便利变量来实现忽略计数的功能。参见第 10.12 [便利变量], 页 173。

忽略计数适用于断点、观察点、跟踪点和捕获点。

5.1.7 断点命令列表

你可以为任何断点（或观察点或捕获点）设置一系列命令，当程序因该断点停止时执行这些命令。例如，你可能希望打印某些表达式的值，或启用其他断点。

```
commands [list...]
... command-list ...
```

end Specify a list of commands for the given breakpoints. The commands themselves appear on the following lines. Type a line containing just **end** to terminate the commands.

To remove all commands from a breakpoint, type **commands** and follow it immediately with **end**; that is, give no commands.

With no argument, **commands** refers to the last breakpoint, watchpoint, or catchpoint set (not to the breakpoint most recently encountered). If the most recent breakpoints were set with a single command, then the **commands** will apply to all the breakpoints set by that command. This applies to breakpoints set by **rbreak**, and also applies when a single **break** command creates multiple breakpoints (see Section 10.2 [Ambiguous Expressions], page 148).

Pressing **RET** as a means of repeating the last GDB command is disabled within a *command-list*.

Inside a command list, you can use the command *disable \$hit_bnum* to disable the encountered breakpoint.

If your breakpoint has several code locations, the command *disable \$hit_bnum.\$hit_locno* will disable the specific breakpoint code location encountered. If the breakpoint has only one location, this command will disable the encountered breakpoint.

You can use breakpoint commands to start your program up again. Simply use the **continue** command, or **step**, or any other command that resumes execution.

Any other commands in the command list, after a command that resumes execution, are ignored. This is because any time you resume execution (even with a simple **next** or **step**), you may encounter another breakpoint—which could have its own command list, leading to ambiguities about which list to execute.

If the first command you specify in a command list is **silent**, the usual message about stopping at a breakpoint is not printed. This may be desirable for breakpoints that are to print a specific message and then continue. If none of the remaining commands print anything, you see no sign that the breakpoint was reached. **silent** is meaningful only at the beginning of a breakpoint command list.

The commands **echo**, **output**, and **printf** allow you to print precisely controlled output, and are often useful in silent breakpoints. See Section 23.1.4 [Commands for Controlled Output], page 403.

For example, here is how you could use breakpoint commands to print the value of *x* at entry to *foo* whenever *x* is positive.

```
break foo if x>0
commands
silent
printf "x is %d\n",x
cont
end
```

One application for breakpoint commands is to compensate for one bug so you can test for another. Put a breakpoint just after the erroneous line of code, give it a condition

```
commands [list...]
... command-list ...
```

end 为给定的断点指定命令列表。命令本身出现在下一行。输入仅包含 **end** 的行以终止命令。

要从断点中删除所有命令，请输入 **commands**，并立即在其后跟 **end**；也就是说，不要输入任何命令。

如果没有参数，**commands** 指的是最后一次设置的最后一个断点、观察点或捕获点（而不是最近遇到的断点）。如果最近的断点是用单个命令设置的，那么 **commands** 将适用于由该命令设置的所有断点。这适用于 **rbreak** 设置的断点，也适用于单个 **break** 命令创建多个断点的情况（见第 10.2 [歧义表达式], 页 148）。

在命令列表中，按回车键重复最后一个 GDB 命令的功能被禁用。

在命令列表中，您可以使用命令 *disable \$hit_bnum* 来禁用遇到的断点。

If your 断点 has 几个 code 位置, the 指令 *disable \$hit_bnum.\$hit_locno* 将禁用特定的 断点代码位置 *en-* 已处理。如果断点只有一个位置，这条命令将禁用该遇到的断点。

您可以使用断点命令重新启动程序。只需使用继续命令，或单步执行，或任何其他恢复执行的命令。

在恢复执行命令之后，命令列表中的任何其他命令都将被忽略。这是因为每次恢复执行时（即使使用简单的下一步或单步执行），您可能会遇到另一个断点——该断点可能有自己的命令列表，从而导致对要执行哪个列表产生歧义。

如果在命令列表中指定的第一个命令是静默的，则不会打印通常在断点处停止的消息。这对于要打印特定消息然后继续的断点可能是可取的。如果其余命令都不打印任何内容，您将看不到断点被触达的迹象。**silent** 仅在断点命令列表的开头才有意义。

echo、**output** 和 **printf** 命令允许你精确控制输出内容，在静默断点中经常很有用。参见第 23.1.4 [受控输出], 页 403。

例如，这里展示了如何使用断点命令在每次 *x* 为正时，在进入 *foo* 的入口处打印 *x* 的值。

```
break foo if x>0命令
silent printf "x is %
d\n",x cont end
```

断点命令的一个应用是补偿一个 bug，以便你可以测试另一个。在出错的代码行之后放置一个断点，并为其设置一个条件

to detect the case in which something erroneous has been done, and give it commands to assign correct values to any variables that need them. End with the `continue` command so that your program does not stop, and start with the `silent` command so that no output is produced. Here is an example:

```
break 403
commands
silent
set x = y + 4
cont
end
```

5.1.8 Dynamic Printf

The dynamic printf command `dprintf` combines a breakpoint with formatted printing of your program’s data to give you the effect of inserting `printf` calls into your program on-the-fly, without having to recompile it.

In its most basic form, the output goes to the GDB console. However, you can set the variable `dprintf-style` for alternate handling. For instance, you can ask to format the output by calling your program’s `printf` function. This has the advantage that the characters go to the program’s output device, so they can be recorded in redirects to files and so forth.

If you are doing remote debugging with a stub or agent, you can also ask to have the printf handled by the remote agent. In addition to ensuring that the output goes to the remote program’s device along with any other output the program might produce, you can also ask that the `dprintf` remain active even after disconnecting from the remote target. Using the stub/agent is also more efficient, as it can do everything without needing to communicate with GDB.

`dprintf locspec,template,expression[,expression...]`
Whenever execution reaches a code location that results from resolving *locspec*, print the values of one or more *expressions* under the control of the string *template*. To print several values, separate them with commas.

`set dprintf-style style`
Set the `dprintf` output to be handled in one of several different styles enumerated below. A change of style affects all existing dynamic printf’s immediately. (If you need individual control over the print commands, simply define normal breakpoints with explicitly-supplied command lists.)

- `gdb` Handle the output using the GDB `printf` command. When using this style, it is possible to use the “%V” format specifier (see [%V Format Specifier], page 404).
- `call` Handle the output by calling a function in your program (normally `printf`). When using this style the supported format specifiers depend entirely on the function being called.

Most of GDB’s format specifiers align with those supported by the `printf` function, however, GDB’s “%V” format specifier extension is not supported by `printf`. When using ‘call’ style `dprintf`, care should be taken to ensure that only format specifiers supported by

以检测是否已执行了错误操作，并为其分配命令，为需要它们的任何变量赋正确值。以继续命令结束，以防止程序停止，并以静默命令开始，以避免产生任何输出。以下是一个示例：

```
break 403
commands
silent
set x = y + 4
cont
end
```

5.1.8 动态printf

动态printf命令dprintf结合了断点与程序数据格式化打印，让你能够动态地在程序中插入printf调用，而无需重新编译程序。

最基本的形式下，输出会发送到GDB控制台。不过，你可以设置变量dprintf-style以进行替代处理。例如，你可以通过调用程序的printf函数来格式化输出。这样做的好处是字符会发送到程序的输出设备，因此它们可以被重定向到文件等。

如果你使用stub或代理进行远程调试，也可以要求将printf处理交给远程代理。除了确保输出会发送到远程程序的设备，以及程序可能产生的任何其他输出之外，你还可以要求dprintf在断开与远程目标的连接后仍然保持激活状态。使用stub/代理也更高效，因为它可以完成所有操作而无需与 GDB通信。

`dprintf locspec,template,expression[,expression...]`
当执行到达由解析 locspec 得到的代码位置时，打印由字符串模板控制的一个或多个表达式的值。要打印多个值，用逗号分隔。

设置 dprintf 样式 `style`
将 dprintf 输出设置为以下列出的几种不同样式之一。样式的更改会立即影响所有现有的动态打印。(如果你需要对打印命令进行单独控制，只需定义带有显式命令列表的普通断点即可。)

- `gdb` 使用 GDB printf 命令处理输出。使用此样式时，可以使用 “%V” 格式说明符（参见 [%V 格式说明符], 页 404）。
- `call` 通过调用程序中的函数（通常是 printf）处理输出。使用此样式时，支持的格式说明符完全取决于所调用的函数。

大部分 GDB 的格式说明符与 printf 函数支持的那些一致，但是，GDB 的 “%V” 格式说明符扩展不被 printf 支持。使用 “调用” 样式 dprintf 时，应注意确保仅使用由

the output function are used, otherwise the results will be undefined.

agent Have the remote debugging agent (such as `gdbserver`) handle the output itself. This style is only available for agents that support running commands on the target. This style does not support the ‘%V’ format specifier.

set dprintf-function *function*

Set the function to call if the dprintf style is `call`. By default its value is `printf`. You may set it to any expression that GDB can evaluate to a function, as per the `call` command.

set dprintf-channel *channel*

Set a “channel” for dprintf. If set to a non-empty value, GDB will evaluate it as an expression and pass the result as a first argument to the `dprintf-function`, in the manner of `fprintf` and similar functions. Otherwise, the dprintf format string will be the first argument, in the manner of `printf`.

As an example, if you wanted dprintf output to go to a logfile that is a standard I/O stream assigned to the variable `mylog`, you could do the following:

```
(gdb) set dprintf-style call
(gdb) set dprintf-function fprintf
(gdb) set dprintf-channel mylog
(gdb) dprintf 25,"at line 25, glob=%d\n",glob
Dprintf 1 at 0x123456: file main.c, line 25.
(gdb) info break
1      dprintf      keep y  0x00123456 in main at main.c:25
      call (void) fprintf (mylog,"at line 25, glob=%d\n",glob)
      continue
(gdb)
```

Note that the `info break` displays the dynamic printf commands as normal breakpoint commands; you can thus easily see the effect of the variable settings.

set disconnected-dprintf on

set disconnected-dprintf off

Choose whether dprintf commands should continue to run if GDB has disconnected from the target. This only applies if the `dprintf-style` is `agent`.

show disconnected-dprintf off

Show the current choice for disconnected dprintf.

GDB does not check the validity of function and channel, relying on you to supply values that are meaningful for the contexts in which they are being used. For instance, the function and channel may be the values of local variables, but if that is the case, then all enabled dynamic prints must be at locations within the scope of those locals. If evaluation fails, GDB will report an error.

5.1.9 How to save breakpoints to a file

To save breakpoint definitions to a file use the `save breakpoints` command.

如果使用输出函数，否则结果将未定义。

代理 让远程调试代理（如gdbserver）自行处理输出。这种模式仅适用于支持在目标上执行命令的代理。这种模式不支持 ‘%V’ 格式说明符。

设置dprintf-function *function*

如果使用dprintf模式，则设置要调用的函数。默认值为printf。您可以将其设置为任何 GDB 可以计算为函数的表达式，具体取决于调用命令。

设置dprintf-channel *channel*

为dprintf设置一个“通道”。如果设置为非空值，GDB 会将其计算为表达式，并将结果作为dprintf-function的第一个参数传递，类似于fprintf和类似函数的方式。否则，dprintf格式字符串将是printf的第一个参数。

例如，如果您希望 dprintf 的输出重定向到变量 `mylog` 指向的标准 I/O 流文件中，您可以执行以下操作：

```
(gdb) set dprintf-style call
(gdb) set dprintf-function fprintf
(gdb) set dprintf-channel mylog
(gdb) dprintf 25,"at line 25, glob=%d\n",glob
Dprintf 1 at 0x123456: 文件 main.c, 第 25 行。
(gdb) info break
1 dprintf 保持 y 0x00123456 在 main 中 main.c:25 调用(void) fprintf
(mylog,"在第25行, glob=%d\n",glob) 继续
(gdb)
```

注意，info break 会将动态 printf 命令显示为普通断点命令；因此您可以轻松看到变量设置的生效效果。

set disconnected-dprintf on

set disconnected-dprintf off

选择是否在 GDB 与目标断开连接后继续运行 dprintf 命令。这仅适用于 agent 模式下的 dprintf 风格。

显示已断开连接的 dprintf 状态

显示当前已断开连接的 dprintf 的选择。

GDB 不会检查函数和通道的有效性，依赖你提供对使用上下文有意义的值。例如，函数和通道可能是局部变量的值，但如果是这种情况，则所有启用的动态打印都必须位于这些局部变量的作用域内。如果求值失败，GDB 将报告错误。

5.1.9 如何将断点保存到文件

将断点定义保存到文件，请使用保存断点命令。

save breakpoints [filename]

This command saves all current breakpoint definitions together with their commands and ignore counts, into a file *filename* suitable for use in a later debugging session. This includes all types of breakpoints (breakpoints, watchpoints, catchpoints, tracepoints). To read the saved breakpoint definitions, use the **source** command (see Section 23.1.3 [Command Files], page 401). Note that watchpoints with expressions involving local variables may fail to be recreated because it may not be possible to access the context where the watchpoint is valid anymore. Because the saved breakpoint definitions are simply a sequence of GDB commands that recreate the breakpoints, you can edit the file in your favorite editing program, and remove the breakpoint definitions you're not interested in, or that can no longer be recreated.

5.1.10 Static Probe Points

GDB supports *SDT* probes in the code. SDT stands for Statically Defined Tracing, and the probes are designed to have a tiny runtime code and data footprint, and no dynamic relocations.

Currently, the following types of probes are supported on ELF-compatible systems:

- **SystemTap** (<http://sourceware.org/systemtap/>) SDT probes¹. SystemTap probes are usable from assembly, C and C++ languages².
- **DTrace** (<http://oss.oracle.com/projects/DTrace>) USDT probes. DTrace probes are usable from C and C++ languages.

Some SystemTap probes have an associated semaphore variable; for instance, this happens automatically if you defined your probe using a DTrace-style *.d* file. If your probe has a semaphore, GDB will automatically enable it when you specify a breakpoint using the **~probe-stop** notation. But, if you put a breakpoint at a probe's location by some other method (e.g., **break file:line**), then GDB will not automatically set the semaphore. DTrace probes do not support semaphores.

You can examine the available static probes using **info probes**, with optional arguments:

info probes [type] [provider [name [objfile]]]

If given, *type* is either **stap** for listing SystemTap probes or **dtrace** for listing DTrace probes. If omitted all probes are listed regardless of their types.

If given, *provider* is a regular expression used to match against provider names when selecting which probes to list. If omitted, probes by all providers are listed.

If given, *name* is a regular expression to match against probe names when selecting which probes to list. If omitted, probe names are not considered when deciding whether to display them.

¹ See <http://sourceware.org/systemtap/wiki/AddingUserSpaceProbingToApps> for more information on how to add SystemTap SDT probes in your applications.

² See <http://sourceware.org/systemtap/wiki/UserSpaceProbeImplementation> for a good reference on how the SDT probes are implemented.

保存断点 [filename]

此命令将所有当前断点定义及其命令和忽略计数保存到文件 *filename*，以便在后续调试会话中使用。这包括所有类型的断点（断点、观察点、捕获点、跟踪点）。要读取保存的断点定义，请使用 **source** 命令（参见第 23.1.3 [命令文件]，页 401）。请注意，涉及局部变量的表达式观察点可能无法重新创建，因为可能无法访问观察点有效的上下文。由于保存的断点定义仅仅是重新创建断点的命令序列，因此您可以在最喜欢的编辑程序中编辑该文件，并删除您不感兴趣的断点定义，或无法重新创建的断点定义。

5.1.10 静态探测点

GDB 支持代码中的 SDT 探测。SDT 代表静态定义跟踪，探测点设计为具有极小的运行时代码和数据占用，且无需动态重定位。

目前，在 ELF 兼容系统上支持以下类型的探测点：

- **SystemTap** (<http://sourceware.org/systemtap/>) SDT 探测点¹。SystemTap 探测点可在汇编、C 和 C++ 语言²中使用。
- **DTrace** (<http://oss.oracle.com/projects/DTrace>) USDT 探针。DTrace 探针可用于 C 和 C++ 语言。

一些 SystemTap 探针有一个关联的信号量变量；例如，如果你使用 DTrace 风格的 *.d* 文件定义探针，这种情况会自动发生。如果你的探针有信号量，GDB 在你使用 **~probe-stop** 语法指定断点时将自动启用它。但是，如果你通过其他方法（例如，**break file:line**）在探针位置放置断点，那么 GDB 将不会自动设置信号量。DTrace 探针不支持信号量。

你可以使用 **info probes** 查看可用的静态静态探针，并可选地使用参数：

info probes [type] [provider [name [objfile]]]

如果提供，*type* 是 **stap**（用于列出 SystemTap 探针）或 **dtrace**（用于列出 DTrace 探针）。如果省略，将列出所有类型的探针。

如果提供，*provider* 是一个正则表达式，用于匹配 *provider* 名称时选择要列出的探针。如果省略，则会列出所有 *provider* 的所有探针。

如果提供，*name* 是一个正则表达式，用于匹配探针名称时选择要列出的探针。如果省略，则在决定是否显示它们时不会考虑探针名称。

¹ 有关如何在您的应用程序中添加 SystemTap SDT 探针的更多信息，请参阅

<http://sourceware.org/systemtap/wiki/AddingUserSpaceProbingToApps>。² 有关 SDT 探针实现的良好参考，请参阅 <http://sourceware.org/systemtap/wiki/UserSpaceProbeImplementation>。

If given, *objfile* is a regular expression used to select which object files (executable or shared libraries) to examine. If not given, all object files are considered.

info probes all

List the available static probes, from all types.

Some probe points can be enabled and/or disabled. The effect of enabling or disabling a probe depends on the type of probe being handled. Some DTrace probes can be enabled or disabled, but SystemTap probes cannot be disabled.

You can enable (or disable) one or more probes using the following commands, with optional arguments:

enable probes [*provider* [*name* [*objfile*]]]

If given, *provider* is a regular expression used to match against provider names when selecting which probes to enable. If omitted, all probes from all providers are enabled.

If given, *name* is a regular expression to match against probe names when selecting which probes to enable. If omitted, probe names are not considered when deciding whether to enable them.

If given, *objfile* is a regular expression used to select which object files (executable or shared libraries) to examine. If not given, all object files are considered.

disable probes [*provider* [*name* [*objfile*]]]

See the **enable probes** command above for a description of the optional arguments accepted by this command.

A probe may specify up to twelve arguments. These are available at the point at which the probe is defined—that is, when the current PC is at the probe’s location. The arguments are available using the convenience variables (see Section 10.12 [Convenience Vars], page 173) `$_probe_arg0...$_probe_arg11`. In SystemTap probes each probe argument is an integer of the appropriate size; types are not preserved. In DTrace probes types are preserved provided that they are recognized as such by GDB; otherwise the value of the probe argument will be a long integer. The convenience variable `$_probe_argc` holds the number of arguments at the current probe point.

These variables are always available, but attempts to access them at any location other than a probe point will cause GDB to give an error message.

5.1.11 “Cannot insert breakpoints”

If you request too many active hardware-assisted breakpoints and watchpoints, you will see this error message:

```
Stopped; cannot insert breakpoints.
You may have requested too many hardware breakpoints and watchpoints.
```

This message is printed when you attempt to resume the program, since only then GDB knows exactly how many hardware breakpoints and watchpoints it needs to insert.

When this message is printed, you need to disable or remove some of the hardware-assisted breakpoints and watchpoints, and then continue.

如果提供, *objfile* 是一个正则表达式, 用于选择要检查哪些对象文件 (可执行文件或共享库)。如果未提供, 则考虑所有对象文件。

信息探测器全部

列出所有可用的静态探测器, 涵盖所有类型。

某些探测器点可以启用和/或禁用。启用或禁用探测器的效果取决于所处理的探测器类型。某些DTrace探测器可以启用或禁用, 但SystemTap探测器无法禁用。

您可以使用以下命令启用 (或禁用) 一个或多个探测器, 并可选提供参数:

启用探测器 [*provider* [*name* [*objfile*]]]

如果提供, *provider* 是一个正则表达式, 用于匹配 *provider* 名称, 以便在选择要启用的探针时使用。如果省略, 则启用所有 *provider* 的所有探针。

如果提供, *name* 是一个正则表达式, 用于匹配探针名称, 以便在选择要启用的探针时使用。如果省略, 则在决定是否启用它们时不考虑探针名称。

如果提供, *objfile* 是一个正则表达式, 用于选择要检查的对象文件 (可执行文件或共享库)。如果未提供, 则考虑所有对象文件。

禁用探针 [*provider* [*name* [*objfile*]]]

请参阅上文的 **enable probes** 命令, 了解该命令接受的可选参数的描述。

一个探针可以指定最多十二个参数。这些参数在探针定义时可用——也就是说, 当当前 PC 位于探针位置时。这些参数可以使用便利变量 (参见第 10.12 [便利变量], 页 173) `$_probe_arg0...$_probe_arg11`。在 SystemTap 探针中, 每个探针参数都是适当大小的整数; 类型不会被保留。在 DTrace 探针中, 只要 GDB 将其识别为类型, 类型就会被保留; 否则, 探针参数的值将是一个长整数。便利变量 `$_probe_argc` 持有当前探针点的参数数量。

这些变量始终可用, 但在除探测点以外的任何位置尝试访问它们都会导致 GDB 报错。

5.1.11 “无法插入断点”

如果你请求的主动硬件辅助断点和观察点过多, 你会看到这个错误消息:

```
停止; 无法插入断点。
你可能请求的硬件断点和观察点过多。
```

当你尝试恢复程序时, 会打印此消息, 因为只有那时 GDB 才确切知道需要插入多少硬件断点和观察点。

当这条消息打印时, 你需要禁用或移除一些硬件辅助的断点和观察点, 然后继续。

5.1.12 “Breakpoint address adjusted...”

Some processor architectures place constraints on the addresses at which breakpoints may be placed. For architectures thus constrained, GDB will attempt to adjust the breakpoint’s address to comply with the constraints dictated by the architecture.

One example of such an architecture is the Fujitsu FR-V. The FR-V is a VLIW architecture in which a number of RISC-like instructions may be bundled together for parallel execution. The FR-V architecture constrains the location of a breakpoint instruction within such a bundle to the instruction with the lowest address. GDB honors this constraint by adjusting a breakpoint’s address to the first in the bundle.

It is not uncommon for optimized code to have bundles which contain instructions from different source statements, thus it may happen that a breakpoint’s address will be adjusted from one source statement to another. Since this adjustment may significantly alter GDB’s breakpoint related behavior from what the user expects, a warning is printed when the breakpoint is first set and also when the breakpoint is hit.

A warning like the one below is printed when setting a breakpoint that’s been subject to address adjustment:

```
warning: Breakpoint address adjusted from 0x00010414 to 0x00010410.
```

Such warnings are printed both for user settable and GDB’s internal breakpoints. If you see one of these warnings, you should verify that a breakpoint set at the adjusted address will have the desired affect. If not, the breakpoint in question may be removed and other breakpoints may be set which will have the desired behavior. E.g., it may be sufficient to place the breakpoint at a later instruction. A conditional breakpoint may also be useful in some cases to prevent the breakpoint from triggering too often.

GDB will also issue a warning when stopping at one of these adjusted breakpoints:

```
warning: Breakpoint 1 address previously adjusted from 0x00010414
to 0x00010410.
```

When this warning is encountered, it may be too late to take remedial action except in cases where the breakpoint is hit earlier or more frequently than expected.

5.2 Continuing and Stepping

Continuing means resuming program execution until your program completes normally. In contrast, *stepping* means executing just one more “step” of your program, where “step” may mean either one line of source code, or one machine instruction (depending on what particular command you use). Either when continuing or when stepping, your program may stop even sooner, due to a breakpoint or a signal. (If it stops due to a signal, you may want to use `handle`, or use ‘`signal 0`’ to resume execution (see Section 5.4 [Signals], page 92), or you may step into the signal’s handler (see [stepping and signal handlers], page 94).)

```
continue [ignore-count]
c [ignore-count]
fg [ignore-count]
```

Resume program execution, at the address where your program last stopped; any breakpoints set at that address are bypassed. The optional argument *ignore-count* allows you to specify a further number of times to ignore a breakpoint at this location; its effect is like that of `ignore` (see Section 5.1.6 [Break Conditions], page 78).

5.1.12 “断点地址调整...”

某些处理器架构对断点可以放置的地址施加限制。对于这些受限的架构，GDB 将尝试调整断点的地址以符合架构规定的限制。

这类架构的一个例子是富士通FR-V。FR-V是一种VLIW架构，其中多个类似RISC的指令可以捆绑在一起并行执行。FR-V架构将断点指令的位置约束在这样捆绑中的最低地址指令上。GDB 通过将断点的地址调整为捆绑中的第一个指令来遵守这一约束。

优化代码中包含来自不同源语句的指令捆绑的情况并不少见，因此断点的地址可能会从一个源语句调整到另一个源语句。由于这种调整可能会显著改变 GDB 的断点相关行为，与用户预期不符，因此当断点首次设置时以及当断点被命中时都会打印警告。

设置已进行地址调整的断点时，会打印如下警告：

```
警告：断点地址从0x00010414调整到0x00010410。
```

这些警告既会为用户设置的断点，也会为 GDB 的内部断点打印。如果你看到这类警告，应该验证在调整后的地址设置的断点是否会产生预期效果。如果不会，则可以将该断点移除，并设置其他断点以产生预期行为。例如，将断点放在稍后的指令处可能就足够了。在某些情况下，条件断点也可能有用，以防止断点过于频繁地触发。

GDB 在停止于这些调整后的断点时，也会发出警告：

```
警告：断点 1 地址先前已从 0x00010414 调整为 0x00010410。
```

当遇到此警告时，除非断点比预期更早或更频繁地命中，否则可能已经太晚采取补救措施了。

5.2 持续执行和单步执行

继续执行意味着恢复程序运行，直到程序正常完成。相比之下，单步执行意味着执行程序的一小步，这里的“步”可能指一行源代码，也可能指一条机器指令（具体取决于你使用的命令）。无论是继续执行还是单步执行，你的程序都可能因为断点或信号而提前停止。（如果程序因信号停止，你可能需要使用 `handle`，或者使用 ‘`signal 0`’ 来恢复执行（见第 5.4 [信号], 页 92），或者你可能需要单步进入信号的处理程序（见 [单步执行和信号处理器], 页 94）。）

```
continue [ignore-count]
c [ignore-count]
fg [ignore-count]
```

在程序上次停止的地址处恢复程序执行；在该地址设置的任何断点都会被跳过。可选参数 *ignore-count* 允许您指定在当前位置忽略断点的额外次数；其效果与 `ignore` 类似（参见第 5.1.6 [断点条件], 页 78）。

The argument *ignore-count* is meaningful only when your program stopped due to a breakpoint. At other times, the argument to **continue** is ignored.

The synonyms **c** and **fg** (for *foreground*, as the debugged program is deemed to be the foreground program) are provided purely for convenience, and have exactly the same behavior as **continue**.

To resume execution at a different place, you can use **return** (see Section 17.4 [Returning from a Function], page 282) to go back to the calling function; or **jump** (see Section 17.2 [Continuing at a Different Address], page 280) to go to an arbitrary location in your program.

A typical technique for using stepping is to set a breakpoint (see Section 5.1 [Breakpoints; Watchpoints; and Catchpoints], page 59) at the beginning of the function or the section of your program where a problem is believed to lie, run your program until it stops at that breakpoint, and then step through the suspect area, examining the variables that are interesting, until you see the problem happen.

step Continue running your program until control reaches a different source line, then stop it and return control to GDB. This command is abbreviated **s**.

Warning: If you use the **step** command while control is within a function that was compiled without debugging information, execution proceeds until control reaches a function that does have debugging information. Likewise, it will not step into a function which is compiled without debugging information. To step through functions without debugging information, use the **stepi** command, described below.

The **step** command only stops at the first instruction of a source line. This prevents the multiple stops that could otherwise occur in **switch** statements, **for** loops, etc. **step** continues to stop if a function that has debugging information is called within the line. In other words, **step** *steps inside* any functions called within the line.

Also, the **step** command only enters a function if there is line number information for the function. Otherwise it acts like the **next** command. This avoids problems when using **cc -g1** on MIPS machines. Previously, **step** entered sub-routines if there was any debugging information about the routine.

step count Continue running as in **step**, but do so *count* times. If a breakpoint is reached, or a signal not related to stepping occurs before *count* steps, stepping stops right away.

next [count] Continue to the next source line in the current (innermost) stack frame. This is similar to **step**, but function calls that appear within the line of code are executed without stopping. Execution stops when control reaches a different line of code at the original stack level that was executing when you gave the **next** command. This command is abbreviated **n**.

An argument *count* is a repeat count, as for **step**.

参数 *ignore-count* 只有在程序因断点停止时才有意义。在其他情况下, 继续的参数会被忽略。

同义词 **c** 和 **fg** (用于前景, 因为调试的程序被认为是前景程序) 纯粹是为了方便, 它们的行为与 **continue** 完全相同。

要在不同的地方恢复执行, 您可以使用 **return** (参见第 17.4 [从函数返回], 页 282) 回到调用函数; 或者使用 **jump** (参见第 17.2 [在不同地址继续], 页 280) 跳转到程序中的任意位置。

使用单步执行的一个典型技术是在函数或您程序中可能存在问题的部分设置断点 (参见第 5.1 [断点; 观察点; 和捕获点], 页 59), 运行您的程序直到它停在断点处, 然后通过可疑区域进行单步执行, 检查感兴趣的变量, 直到您看到问题发生。

step 继续运行您的程序, 直到控制到达不同的源行, 然后停止它并将控制权返回给 GDB。此命令的缩写为 **s**。

警告: 如果你在控制流位于未编译调试信息的函数内时使用 **step** 命令, 执行会继续直到控制流到达具有调试信息的函数。同样, 它不会进入未编译调试信息的函数。要逐步执行未编译调试信息的函数, 请使用下面描述的 **stepi** 命令。

step 命令仅在源代码行的第一条指令处停止。这防止了 **switch** 语句、**for** 循环等可能发生的多次停止。如果源代码行内调用了具有调试信息的函数, **step** 会继续停止。换句话说, **step** 会进入源代码行内调用的任何函数。

此外, **step** 命令只有在函数具有行号信息时才会进入该函数。否则, 它表现得像下一个命令。这避免了在 MIPS 机器上使用 **cc -g1** 时的問題。以前, 如果任何关于例程的调试信息存在, **step** 会进入子例程。

step count 像 **step** 一样继续运行, 但执行 *count* 次。如果在 *count* 步之前达到断点, 或发生与单步无关的信号, 单步会立即停止。

next [count] 继续到当前 (最内层) 栈帧中的下一条源代码行。这与 **step** 类似, 但函数调用会执行, 而不会停止。当控制到达原始栈级别上不同的代码行时, 执行会停止——该原始栈级别是在你给出 **next** 命令时正在执行的。此命令缩写为 **n**。

参数计数是重复计数, 与步骤相同。

The `next` command only stops at the first instruction of a source line. This prevents multiple stops that could otherwise occur in `switch` statements, `for` loops, etc.

`set step-mode`
`set step-mode on`

The `set step-mode on` command causes the `step` command to stop at the first instruction of a function which contains no debug line information rather than stepping over it.

This is useful in cases where you may be interested in inspecting the machine instructions of a function which has no symbolic info and do not want GDB to automatically skip over this function.

`set step-mode off`

Causes the `step` command to step over any functions which contains no debug information. This is the default.

`show step-mode`

Show whether GDB will stop in or step over functions without source line debug information.

`finish`

Continue running until just after function in the selected stack frame returns. Print the returned value (if any). This command can be abbreviated as `fin`. Contrast this with the `return` command (see Section 17.4 [Returning from a Function], page 282).

`set print finish [on|off]`

`show print finish`

By default the `finish` command will show the value that is returned by the function. This can be disabled using `set print finish off`. When disabled, the value is still entered into the value history (see Section 10.11 [Value History], page 172), but not displayed.

`until`

`u`

Continue running until a source line past the current line, in the current stack frame, is reached. This command is used to avoid single stepping through a loop more than once. It is like the `next` command, except that when `until` encounters a jump, it automatically continues execution until the program counter is greater than the address of the jump.

This means that when you reach the end of a loop after single stepping though it, `until` makes your program continue execution until it exits the loop. In contrast, a `next` command at the end of a loop simply steps back to the beginning of the loop, which forces you to step through the next iteration.

`until` always stops your program if it attempts to exit the current stack frame. `until` may produce somewhat counterintuitive results if the order of machine code does not match the order of the source lines. For example, in the following excerpt from a debugging session, the `f (frame)` command shows that execution is stopped at line 206; yet when we use `until`, we get to line 195:

(gdb) f

下一个命令仅在源行的第一条指令处停止。这防止了在 `switch` 语句、`for` 循环等情况下可能发生的多次停止。

设置步进模式
开启步进模式

设置开启步进模式的命令会导致步进命令在包含无调试行信息的函数的第一条指令处停止，而不是跳过它。

这在您可能对没有符号信息的函数的机器指令感兴趣且不希望自动跳过此函数的情况下很有用。

关闭步进模式

使步进命令跳过任何不包含调试信息的函数。这是默认行为。

显示步进模式

显示 GDB 是否会在不包含源代码行调试信息的函数中停止或跳过。

完成 继续运行直到选定栈帧中的函数返回之后。打印返回值（如果有）。此命令可以缩写为 `fin`。

与此对比一下返回命令（参见第17.4 [从函数返回], 页282）。

设置打印完成 [开|关]

显示打印效果

默认情况下，`finish` 命令将显示函数返回的值。可以使用 `set print finish off` 来禁用此功能。当禁用时，输入的值仍然会进入值历史记录（参见第 10.11 [值历史], 页 172），但不会显示。

直到

`u`

继续执行直到当前栈帧中当前行之后的源代码行被遇到。此命令用于避免对循环进行多次单步执行。它与 `next` 命令类似，但当 `until` 遇到跳转时，它会自动继续执行直到程序计数器大于跳转地址。

这意味着，当您在单步执行后到达循环的末尾时，`until` 会使您的程序继续执行直到退出循环。相比之下，在循环末尾的 `next` 命令只是单步返回到循环的开始，这会迫使您单步执行下一次迭代。

直到始终会在它尝试退出当前栈帧时停止你的程序。如果机器代码的顺序与源代码行的顺序不匹配，直到可能会产生一些反直觉的结果。例如，在以下调试会话的摘录中，`f (帧)` 命令显示执行停在行206；然而当我们使用直到时，我们会到达行195：

(gdb) f

```
#0 main (argc=4, argv=0xf7fffae8) at m4.c:206
206      expand_input();
(gdb) until
195      for ( ; argc > 0; NEXTARG) {
```

This happened because, for execution efficiency, the compiler had generated code for the loop closure test at the end, rather than the start, of the loop—even though the test in a C `for`-loop is written before the body of the loop. The `until` command appeared to step back to the beginning of the loop when it advanced to this expression; however, it has not really gone to an earlier statement—not in terms of the actual machine code.

`until` with no argument works by means of single instruction stepping, and hence is slower than `until` with an argument.

`until locspec`

`u locspec` Continue running your program until either it reaches a code location that results from resolving *locspec*, or the current stack frame returns. *locspec* is any of the forms described in Section 9.2 [Location Specifications], page 128. This form of the command uses temporary breakpoints, and hence is quicker than `until` without an argument. The specified location is actually reached only if it is in the current frame. This implies that `until` can be used to skip over recursive function invocations. For instance in the code below, if the current location is line 96, issuing `until 99` will execute the program up to line 99 in the same invocation of `factorial`, i.e., after the inner invocations have returned.

```
94 int factorial (int value)
95 {
96     if (value > 1) {
97         value *= factorial (value - 1);
98     }
99     return (value);
100 }
```

`advance locspec`

Continue running your program until either it reaches a code location that results from resolving *locspec*, or the current stack frame returns. *locspec* is any of the forms described in Section 9.2 [Location Specifications], page 128. This command is similar to `until`, but `advance` will not skip over recursive function calls, and the target code location doesn't have to be in the same frame as the current one.

`stepi`

`stepi arg`

`si` Execute one machine instruction, then stop and return to the debugger.

It is often useful to do `'display/i $pc'` when stepping by machine instructions. This makes GDB automatically display the next instruction to be executed, each time your program stops. See Section 10.8 [Automatic Display], page 158.

An argument is a repeat count, as in `step`.

```
#0 main (argc=4, argv=0xf7fffae8) at m4.c:206
206 expand_input(); (gdb) until 195 for ( ; argc > 0;
NEXTARG) {
```

这发生的原因是，为了执行效率，编译器在循环的末尾而不是开头生成了循环闭包测试的代码——尽管在C `for`循环中，测试写在循环体的前面。当它推进到这个表达式时，`until`命令似乎回退到循环的开头；然而，它并没有真正到达更早的语句——在实际的机器代码方面。

直到不带参数通过单指令逐步执行，因此比带参数的直到更慢。

直到 *locspec*

`u locspec` 继续运行您的程序，直到它到达由解析 *locspec* 得到的代码位置，或者当前栈帧返回。*locspec* 是第 9.2 [位置规范], 页 128 中描述的任何形式。这种命令形式使用临时断点，因此比没有参数的 `until` 快。指定的位置实际上只有当它在当前帧中时才会到达。这意味着 `until` 可以用来跳过递归函数调用。例如，在下面的代码中，如果当前位置是第 96 行，发出 `until 99` 将在 `factorial` 的同一调用中执行程序到第 99 行，即在内层调用返回之后。

```
94 int factorial(int value) 95 { 96 if (value > 1) {
97 value *=factorial (value - 1); 98 } 99 return
(value); 100 }
```

`advance locspec`

继续运行您的程序，直到它到达由解析 *locspec* 得到的代码位置，或者当前栈帧返回。*locspec* 是第 9.2 [位置规范], 页 128 中描述的任何形式。此命令与 `until` 类似，但 `advance` 不会跳过递归函数调用，目标代码位置不必与当前帧在同一帧中。

`stepi`

`stepi arg`

`si` 执行一条机器指令，然后停止并返回到调试器。

在单步执行机器指令时，经常使用 `'display/i $pc'` 会很有用。这会使得 GDB 在程序每次停止时自动显示将要执行的下一个指令。参见第10.8 [自动显示], 页158。

一个参数是一个重复计数，就像步骤中那样。

nexti
nexti arg
ni Execute one machine instruction, but if it is a function call, proceed until the function returns.

 An argument is a repeat count, as in **next**.

By default, and if available, GDB makes use of target-assisted *range stepping*. In other words, whenever you use a stepping command (e.g., **step**, **next**), GDB tells the target to step the corresponding range of instruction addresses instead of issuing multiple single-steps. This speeds up line stepping, particularly for remote targets. Ideally, there should be no reason you would want to turn range stepping off. However, it's possible that a bug in the debug info, a bug in the remote stub (for remote targets), or even a bug in GDB could make line stepping behave incorrectly when target-assisted range stepping is enabled. You can use the following command to turn off range stepping if necessary:

set range-stepping
show range-stepping
 Control whether range stepping is enabled.

 If **on**, and the target supports it, GDB tells the target to step a range of addresses itself, instead of issuing multiple single-steps. If **off**, GDB always issues single-steps, even if range stepping is supported by the target. The default is **on**.

5.3 Skipping Over Functions and Files

The program you are debugging may contain some functions which are uninteresting to debug. The **skip** command lets you tell GDB to skip a function, all functions in a file or a particular function in a particular file when stepping.

For example, consider the following C function:

```
101     int func()  
102     {  
103         foo(boring());  
104         bar(boring());  
105     }
```

Suppose you wish to step into the functions **foo** and **bar**, but you are not interested in stepping through **boring**. If you run **step** at line 103, you'll enter **boring()**, but if you run **next**, you'll step over both **foo** and **boring**!

One solution is to **step** into **boring** and use the **finish** command to immediately exit it. But this can become tedious if **boring** is called from many places.

A more flexible solution is to execute **skip boring**. This instructs GDB never to step into **boring**. Now when you execute **step** at line 103, you'll step over **boring** and directly into **foo**.

Functions may be skipped by providing either a function name, linespec (see Section 9.2 [Location Specifications], page 128), regular expression that matches the function's name, file name or a **glob**-style pattern that matches the file name.

On Posix systems the form of the regular expression is "Extended Regular Expressions". See for example 'man 7 regex' on GNU/Linux systems. On non-Posix systems the form of the regular expression is whatever is provided by the **regcomp** function of the underlying

nexti
nexti arg
ni 执行一条机器指令，但如果它是函数调用，则继续执行直到函数返回。

参数是一个重复计数，就像next一样。

默认情况下，如果可用，GDB 会使用目标辅助的范围步进。换句话说，每当你使用步进命令（例如，step、next）时，GDB 会告诉目标步进相应的指令地址范围，而不是发出多个单步指令。这会加快行步进速度，特别是对于远程目标。理想情况下，你没有任何理由要关闭范围步进。然而，调试信息中的错误、远程存根（对于远程目标）中的错误，甚至 GDB 中的错误可能会导致在启用目标辅助范围步进时行步进行为不正确。如果需要，你可以使用以下命令关闭范围步进：

set range-stepping
显示范围步进
 控制是否启用范围步进。

 如果开启，并且目标支持，GDB 会告诉目标自行步进地址范围，而不是发出多个单步指令。如果关闭，GDB 始终发出单步指令，即使目标支持范围步进。默认为开启。

5.3 跳过函数和文件

你正在调试的程序可能包含一些对调试不感兴趣的函数。跳过命令允许你告诉 GDB 在步进时跳过某个函数、某个文件中的所有函数或某个特定文件中的特定函数。

例如，考虑以下C函数：

```
101 int func() 102 { 103  
foo(boring()); 104  
bar(boring()); 105 }
```

假设你想进入 **foo** 和 **bar** 函数，但你对 **boring** 不感兴趣。如果你在 103 行运行 **step**，你会进入 **boring()**，但如果运行 **next**，你会跳过 **foo** 和 **boring**！

一种解决方案是进入 **boring** 并使用 **finish** 命令立即退出它。但如果 **boring** 被从许多地方调用，这可能会变得繁琐。

更灵活的解决方案是执行 **skip boring**。这会指示 GDB 永不进入 **boring**。现在当你执行 103 行的 **step** 时，你会跳过 **boring** 并直接进入 **foo**。

可以通过提供函数名、行号（参见第 9.2 节[位置规范], 页 128）、匹配函数名的正则表达式、文件名或匹配文件名的通配符样式模式来跳过函数。

在 Posix 系统中，正则表达式的形式是“扩展正则表达式”。例如，在 GNU/Linux 系统上可以查看 'man 7 regex'。在非 Posix 系统中，正则表达式的形式由底层的 **regcomp** 函数提供

system. See for example ‘man 7 glob’ on GNU/Linux systems for a description of glob-style patterns.

skip [options]

The basic form of the **skip** command takes zero or more options that specify what to skip. The *options* argument is any useful combination of the following:

-file *file*

Functions in *file* will be skipped over when stepping.

-gfile *file-glob-pattern*

Functions in files matching *file-glob-pattern* will be skipped over when stepping.

```
(gdb) skip -gfile utils/*.c
```

If *file-glob-pattern* is an absolute filename, for example, on Unix systems, if *file-glob-pattern* starts with /, then the glob will only match full filenames.

```
(gdb) skip -gfile /include/*.h
```

In this case `/include/foo.h` would match, but `/usr/include/foo.h` would not. In contrast

```
(gdb) skip -gfile include/*.h
```

This would match both `/include/foo.h` and `/usr/include/foo.h` as the glob pattern is not an absolute filename.

In addition to the standard glob special characters, GDB supports ‘**’. The ‘**’ pattern, called *globstar*, can be used to match against 0 or more filename components. For example, given the files `/include/a.h`, `/include/xxx/b.h`, `/include/yyy/zzz/c.h`, then all of these could be skipped using

```
(gdb) skip -gfile /include/**/*.*h
```

The ‘**’ pattern must appear on its own within the pattern, it cannot be combined with other characters, so ‘`include/**/*.*h`’ will provide the globstar behaviour, but ‘`include/x**/*.*h`’ will not as ‘`x**`’ combines the ‘**’ with the ‘x’ character. The ‘`x**`’ will be treated as ‘x’ followed by two ‘*’ characters, matching zero or more characters within that one directory component.

-function *linespec*

Functions named by *linespec* or the function containing the line named by *linespec* will be skipped over when stepping. See Section 9.2 [Location Specifications], page 128.

-rfunction *regex*

Functions whose name matches *regex* will be skipped over when stepping.

This form is useful for complex function names. For example, there is generally no need to step into C++ `std::string` constructors or destructors. Plus with C++ templates it can be hard to write out the full name of the function, and often it doesn’t matter what the

系统。例如，在 GNU/Linux 系统上查看 “man 7 glob” 以了解 glob 风格模式的描述。

跳过 [options]

跳过命令的基本形式包含零个或多个选项，用于指定要跳过的内容。选项参数可以是以下任意组合的有用组合：

-file *file*

在单步执行时，文件中的函数将被跳过。

-gfile *file-glob-pattern*

匹配文件模式的文件中的函数在单步执行时将被跳过。

```
(gdb) skip -gfile utils/*.c
```

如果文件模式是一个绝对路径名，例如在 Unix 系统上，如果文件模式以 / 开头，那么匹配将仅限于完整路径名。

```
(gdb) 跳过 -gfile /include/*.h
```

在这种情况下 `/include/foo.h` 会匹配，但 `/usr/include/foo.h` 不会。相比之下

```
(gdb) skip -gfile include/*.h
```

这会匹配 `both/include/foo.h` 和 `/usr/include/foo.h`，因为通配符模式不是绝对路径名称。

除了标准的通配符特殊字符外，GDB 还支持 ‘**’。‘**’ 模式，称为 *globstar*，可用于匹配 0 个或多个文件名组件。例如，给定文件 `/include/a.h`、`/include/xxx/b.h`、`/include/yyy/zzz/c.h`，则可以使用以下方式跳过所有这些文件：

```
(gdb) skip -gfile /include/**/*.*h
```

‘**’ 模式必须在模式中单独出现，不能与其他字符组合，因此 ‘`include/**/*.*h`’ 会提供 globstar 行为，但 ‘`include/x**/*.*h`’ 不会，因为 ‘`x**`’ 将 ‘**’ 与 ‘x’ 字符组合在一起。‘`x**`’ 将被视为 ‘x’ 后跟两个 ‘*’ 字符，匹配该目录组件中的零个或多个字符。

-function *linespec*

以行号命名的函数或包含以行号命名的行的函数在单步执行时会被跳过。参见第 9.2 [位置规范]，页 128。

-r函数 *regex*

当逐步执行时，名称与正则表达式匹配的函数将被跳过。

此表单适用于复杂的函数名称。例如，通常无需深入 C++ `std::string` 的构造函数或析构函数。此外，在 C++ 模板中，写出函数的完整名称可能很困难，而且通常函数的模板参数是什么并不重要。将要跳过的函数指定为正则表达式可以简化这一操作。

template arguments are. Specifying the function to be skipped as a regular expression makes this easier.

```
(gdb) skip -rfunction ^std::(allocator|basic_string)<.*>::~~?\\1 *\\(
```

If you want to skip every templated C++ constructor and destructor in the `std` namespace you can do:

```
(gdb) skip -rfunction ^std::([a-zA-z0-9_]+)<.*>::~~?\\1 *\\(
```

If no options are specified, the function you’re currently debugging will be skipped.

skip function [*linespec*]

After running this command, the function named by *linespec* or the function containing the line named by *linespec* will be skipped over when stepping. See Section 9.2 [Location Specifications], page 128.

If you do not specify *linespec*, the function you’re currently debugging will be skipped.

(If you have a function called `file` that you want to skip, use *skip function file*.)

skip file [*filename*]

After running this command, any function whose source lives in *filename* will be skipped over when stepping.

```
(gdb) skip file boring.c
File boring.c will be skipped when stepping.
```

If you do not specify *filename*, functions whose source lives in the file you’re currently debugging will be skipped.

Skips can be listed, deleted, disabled, and enabled, much like breakpoints. These are the commands for managing your list of skips:

info skip [*range*]

Print details about the specified skip(s). If *range* is not specified, print a table with details about all functions and files marked for skipping. *info skip* prints the following information about each skip:

<i>Identifier</i>	A number identifying this skip.
<i>Enabled or Disabled</i>	Enabled skips are marked with ‘y’. Disabled skips are marked with ‘n’.
<i>Glob</i>	If the file name is a ‘glob’ pattern this is ‘y’. Otherwise it is ‘n’.
<i>File</i>	The name or ‘glob’ pattern of the file to be skipped. If no file is specified this is ‘<none>’.
<i>RE</i>	If the function name is a ‘regular expression’ this is ‘y’. Otherwise it is ‘n’.
<i>Function</i>	The name or regular expression of the function to skip. If no function is specified this is ‘<none>’.

将要跳过的函数指定为正则表达式可以简化这一操作。

```
(gdb) skip -rfunction ^std::(allocator|basic_string)<.*>::~~?\\1 *\\(
```

如果你想要跳过std命名空间中每个模板化的C++ 构造函数和析构函数，你可以这样做：

```
(gdb) skip -rfunction ^std::([a-zA-z0-9_]+)<.*>::~~?\\1 *\\(
```

如果没有指定选项，你当前正在调试的函数将被跳过。

skip function [*linespec*]

执行此命令后，在单步执行时将跳过由*linespec*指定命名的函数，或包含*linespec*指定行的函数。参见第9.2 [位置规范], 页128。

如果你没有指定*linespec*，你当前正在调试的函数将被跳过。

(如果你有一个名为 `file` 的函数想要跳过，请使用 *skip functionfile*。)

跳过 file [*filename*]

执行此命令后，任何源代码位于 *filename* 中的函数在单步执行时都会被跳过。

```
(gdb) 跳过文件 boring.c
单步执行时将跳过文件 boring.c。
```

如果你未指定 *filename*，当前正在调试的文件中源代码所在的函数将被跳过。

跳过项可以列出、删除、禁用和启用，与断点类似。这些是管理您的跳过项列表的命令：

info skip [*range*]

打印指定跳过项的详细信息。如果未指定范围，则打印包含所有标记为跳过的函数和文件详细信息的表格。*info skip* 会为每个跳过项打印以下信息：

<i>Identifier</i>	用于标识此跳过项的编号。
启用或禁用	启用的跳过项会标记为 ‘y’ 。禁用的跳过项会标记为 ‘n’ 。
<i>Glob</i>	如果文件名是 ‘glob’ 模式，则为 ‘y’ 。否则为 ‘n’ 。
<i>File</i>	要跳过的文件名称或 ‘glob’ 模式。如果没有指定文件，则为<无>。
<i>RE</i>	如果函数名是 ‘正则表达式’ ，则为 ‘y’ 。其他情况为 ‘n’ 。
功能	要跳过的函数的名称或正则表达式。如果没有指定函数，则为 “<无>” 。

```
skip delete [range]
delete skip [range]
    Delete the specified skip(s). If range is not specified, delete all skips.

skip enable [range]
enable skip [range]
    Enable the specified skip(s). If range is not specified, enable all skips.

skip disable [range]
disable skip [range]
    Disable the specified skip(s). If range is not specified, disable all skips.

save skip filename
    Save all skips to the file filename. This command writes out the skips as a script
    that can be re-read into GDB using the source command.

set debug skip [on|off]
    Set whether to print the debug output about skipping files and functions.

show debug skip
    Show whether the debug output about skipping files and functions is printed.
```

5.4 Signals

A signal is an asynchronous event that can happen in a program. The operating system defines the possible kinds of signals, and gives each kind a name and a number. For example, in Unix SIGINT is the signal a program gets when you type an interrupt character (often *Ctrl-c*); SIGSEGV is the signal a program gets from referencing a place in memory far away from all the areas in use; SIGALRM occurs when the alarm clock timer goes off (which happens only if your program has requested an alarm).

Some signals, including SIGALRM, are a normal part of the functioning of your program. Others, such as SIGSEGV, indicate errors; these signals are *fatal* (they kill your program immediately) if the program has not specified in advance some other way to handle the signal. SIGINT does not indicate an error in your program, but it is normally fatal so it can carry out the purpose of the interrupt: to kill the program.

GDB has the ability to detect any occurrence of a signal in your program. You can tell GDB in advance what to do for each kind of signal, apart from SIGKILL, which has its usual effect regardless.

When specifying a signal by number, GDB translates the number to the target platform according to the corresponding signal name. For example, GDB always treats signal 1 as SIGHUP. So, when specifying ‘1’ as a signal, GDB will translate this to the target’s SIGHUP, whatever that might be.

Numbers may only be used for signals 1 through 15. GDB uses this mapping:

Number	Name
1	SIGHUP
2	SIGINT
3	SIGQUIT
4	SIGILL

```
skip delete [range]
delete skip [range]
    删除指定的跳过项。如果未指定范围，则删除所有跳过项。

skip enable [range]
enable skip [range]
    启用指定的跳过项。如果未指定范围，则启用所有跳过项。

skip disable [range]
disable ski  r p n e g 禁用指定的跳过项。如果未指定范围，则禁用所有跳过项。

保存跳过项 filename
    将所有跳过项保存到文件filename。此命令将跳过项写入脚本，该脚本可以使用
    source命令重新读取到 GDB 。

set debug skip [on|off]
    Set whether to print the debug output about skipping files and functions.

显示调试跳过项
    显示是否打印了跳过文件和函数的调试输出。
```

5.4 信号

信号是一种程序中可以发生的异步事件。操作系统定义了可能的信号类型，并为每种类型命名和编号。例如，在 Unix 中，SIGINT 是当你输入中断字符（通常是*Ctrl-c*）时程序收到的信号；SIGSEGV 是程序从引用远离所有使用区域的内存位置时收到的信号；SIGALRM 是当闹钟定时器响起时发生的信号（只有当你的程序请求了闹钟才会发生）。

某些信号，包括 SIGALRM，是程序正常运行的一部分。而另一些信号，如 SIGSEGV，则表示错误；如果程序没有预先指定其他处理信号的方式，这些信号是致命的（它们会立即终止程序）。SIGINT 并不表示你的程序出错，但它通常是致命的，以便能够执行中断的目的：终止程序。

GDB 具有检测程序中任何信号出现的能力。除了SIGKILL外，你可以提前告诉GDB 如何处理每种信号，SIGKILL则按其常规效果处理。

当通过数字指定信号时， GDB 会根据相应的信号名称将数字转换为目标平台的信号。例如， GDB 始终将信号1视为SIGHUP。因此，当指定 ‘1’ 作为信号时， GDB 会将其转换为目标平台的SIGHUP，无论具体是什么。

数字仅可用于信号 1 到 15。 GDB 使用此映射：

数字	Name
1	SIGHUP
2	SIGINT
3	SIGQUIT
4	SIGILL

- 5 SIGTRAP
- 6 SIGABRT
- 7 SIGEMT
- 8 SIGFPE
- 9 SIGKILL
- 10 SIGBUS
- 11 SIGSEGV
- 12 SIGSYS
- 13 SIGPIPE
- 14 SIGALRM
- 15 SIGTERM

Normally, GDB is set up to let the non-erroneous signals like **SIGALRM** be silently passed to your program (so as not to interfere with their role in the program’s functioning) but to stop your program immediately whenever an error signal happens. You can change these settings with the **handle** command.

```
info signals
info handle
    Print a table of all the kinds of signals and how GDB has been told to handle each one. You can use this to see the signal numbers of all the defined types of signals.

info signals sig
    Similar, but print information only about the specified signal number.
    info handle is an alias for info signals.

catch signal [signal... | 'all']
    Set a catchpoint for the indicated signals. See Section 5.1.3 [Set Catchpoints], page 71, for details about this command.

handle signal [ signal ... ] [keywords...]
    Change the way GDB handles each signal. Each signal can be the number of a signal or its name (with or without the ‘SIG’ at the beginning); a list of signal numbers of the form ‘low-high’; or the word ‘all’, meaning all the known signals, except SIGINT and SIGTRAP, which are used by GDB. Optional argument keywords, described below, say what changes to make to all of the specified signals.
```

The keywords allowed by the **handle** command can be abbreviated. Their full names are:

- nostop** GDB should not stop your program when this signal happens. It may still print a message telling you that the signal has come in.
- stop** GDB should stop your program when this signal happens. This implies the **print** keyword as well.
- print** GDB should print a message when this signal happens.
- noprint** GDB should not mention the occurrence of the signal at all. This implies the **nostop** keyword as well.

- 5 SIGTRAP
- 6 SIGABRT
- 7 SIGEMT
- 8 SIGFPE
- 9 SIGKILL
- 10 SIGBUS
- 11 SIGSEGV
- 12 SIGSYS
- 13 SIGPIPE
- 14 SIGALRM
- 15 SIGTERM

通常， GDB 设置为让非错误信号（如 **SIGALRM**）静默地传递给您的程序（以免干扰它们在程序运行中的作用），但在发生错误信号时立即停止您的程序。您可以使用 **handle** 命令更改这些设置。

信息信号
信息处理
打印所有信号类型及其如何被 GDB 告知如何处理。您可以使用此功能查看所有定义信号类型的信号编号。

信息信号 **sig**
类似，但仅打印指定信号编号的信息。
info handle是**info signals**的别名。

catch signal [signal... | ‘all’]
为指定的信号设置一个捕获点。有关此命令的详细信息，请参阅第 5.1.3 [设置捕获点],页 71。

handle signal [signal ...] [keywords...]
更改 GDB 处理每个信号的方式。每个信号可以是信号的编号或其名称（可以或不可以以“SIG”开头）；一个信号编号的列表“low-high”形式；或单词“all”，表示所有已知的信号，除了由 GDB 使用的 SIGINT 和 SIGTRAP。可选参数关键字，如

handle 命令允许的关键字可以缩写。它们的完整名称是：

- nostop** GDB 当此信号发生时，不应停止您的程序。它仍然可以打印一条消息告诉你信号已经到达。
- stop** GDB 这意味着需要立即停止您的程序。
- print** GDB 当这个信号发生时，应打印一条消息。
- noprint** GDB 完全不应提及信号的发生。这也意味着需要 **nostop** 关键字。

pass
noignore GDB should allow your program to see this signal; your program can handle the signal, or else it may terminate if the signal is fatal and not handled. **pass** and **noignore** are synonyms.

nopass
ignore GDB should not allow your program to see this signal. **nopass** and **ignore** are synonyms.

When a signal stops your program, the signal is not visible to the program until you continue. Your program sees the signal then, if **pass** is in effect for the signal in question *at that time*. In other words, after GDB reports a signal, you can use the **handle** command with **pass** or **nopass** to control whether your program sees that signal when you continue.

The default is set to **nostop**, **noprint**, **pass** for non-erroneous signals such as **SIGALRM**, **SIGWINCH** and **SIGCHLD**, and to **stop**, **print**, **pass** for the erroneous signals.

You can also use the **signal** command to prevent your program from seeing a signal, or cause it to see a signal it normally would not see, or to give it any signal at any time. For example, if your program stopped due to some sort of memory reference error, you might store correct values into the erroneous variables and continue, hoping to see more execution; but your program would probably terminate immediately as a result of the fatal signal once it saw the signal. To prevent this, you can continue with **'signal 0'**. See Section 17.3 [Giving your Program a Signal], page 281.

GDB optimizes for stepping the mainline code. If a signal that has **handle nostop** and **handle pass** set arrives while a stepping command (e.g., **stepi**, **step**, **next**) is in progress, GDB lets the signal handler run and then resumes stepping the mainline code once the signal handler returns. In other words, GDB steps over the signal handler. This prevents signals that you've specified as not interesting (with **handle nostop**) from changing the focus of debugging unexpectedly. Note that the signal handler itself may still hit a breakpoint, stop for another signal that has **handle stop** in effect, or for any other event that normally results in stopping the stepping command sooner. Also note that GDB still informs you that the program received a signal if **handle print** is set.

If you set **handle pass** for a signal, and your program sets up a handler for it, then issuing a stepping command, such as **step** or **stepi**, when your program is stopped due to the signal will step *into* the signal handler (if the target supports that).

Likewise, if you use the **queue-signal** command to queue a signal to be delivered to the current thread when execution of the thread resumes (see Section 17.3 [Giving your Program a Signal], page 281), then a stepping command will step into the signal handler.

Here's an example, using **stepi** to step to the first instruction of **SIGUSR1**'s handler:

```
(gdb) handle SIGUSR1
Signal      Stop      Print    Pass to program Description
SIGUSR1     Yes       Yes      Yes      User defined signal 1
(gdb) c
Continuing.

Program received signal SIGUSR1, User defined signal 1.
main () sigusr1.c:28
28      p = 0;
(gdb) si
sigusr1_handler () at sigusr1.c:9
```

pass
noignore GDB 应允许您的程序看到此信号；您的程序可以处理该信号，否则如果该信号是致命的且未被处理，程序可能会终止。**pass**和**noignore**是同义词。

nopass
忽略 GDB 不应允许您的程序看到此信号。**nopass** 和 **ignore** 是同义词。

当信号停止您的程序时，直到您继续执行，信号对程序不可见。如果此时针对该信号启用了**pass**，您的程序会看到该信号。换句话说，在 GDB 报告信号后，您可以使用带有**pass**或**nopass**的**handle**命令，在继续执行时控制您的程序是否看到该信号。

默认情况下，对于非错误信号（如**SIGALRM**、**SIGWINCH**和**SIGCHLD**），设置为**nostop**、**noprint**、**pass**；对于错误信号，设置为**stop**、**print**、**pass**。

您还可以使用**signal**命令来阻止您的程序看到某个信号，或使其看到通常不会看到的信号，或在任何时间给程序发送任何信号。例如，如果由于某种内存引用错误导致您的程序停止，您可以将正确值存储到有问题的变量中并继续执行，希望看到更多执行；但一旦程序看到信号，很可能会立即因致命信号而终止。为防止这种情况，您可以使用**'signal 0'**继续执行。参见第17.3节[给程序发送信号]，页281。

GDB 优化主程序代码的执行。如果在执行单步命令（如 **stepi**、**step**、**next**）时，收到一个设置了 **handle nostop** 和 **handle pass** 的信号，GDB 将允许信号处理器运行，并在信号处理器返回后继续执行主程序代码。换句话说，GDB 会跳过信号处理器。这可以防止那些你指定为不感兴趣的信号（通过 **handle nostop**）意外改变调试焦点。请注意，信号处理器本身仍可能触发断点、因另一个设置了 **handle stop** 的信号而停止，或因任何其他通常会导致提前停止单步命令的事件而停止。另外请注意，如果设置了 **handle print**，GDB 仍然会通知你程序收到了信号。

如果你为某个信号设置了 **handle pass**，并且你的程序为其设置了处理器，那么当程序因该信号而停止时，执行单步命令（如 **step** 或 **stepi**）将会进入信号处理器（如果目标支持该功能）。

同样地，如果你使用 **queue-signal** 命令将信号排队，以便在线程执行恢复时（见第 17.3 [给你的程序发送信号]，页 281）将其传递给当前线程，那么一个单步执行命令将会单步进入信号处理程序。

这里有一个示例，使用 **stepi** 单步执行到 **SIGUSR1** 处理程序的第一条指令：

```
(gdb) handle SIGUSR1 signal
Signal      Stop      Print    Pass to program Description
SIGUSR1     Yes       Yes      Yes      User defined signal 1
(gdb) c
Continuing.

Program received signal SIGUSR1, User defined signal 1.
main () sigusr1.c:28
28      p = 0;
(gdb) si
sigusr1_handler () at sigusr1.c:9
```

```
9      {
```

The same, but using `queue-signal` instead of waiting for the program to receive the signal first:

```
(gdb) n
28      p = 0;
(gdb) queue-signal SIGUSR1
(gdb) si
sigusr1_handler () at sigusr1.c:9
9      {
(gdb)
```

On some targets, GDB can inspect extra signal information associated with the intercepted signal, before it is actually delivered to the program being debugged. This information is exported by the convenience variable `$_siginfo`, and consists of data that is passed by the kernel to the signal handler at the time of the receipt of a signal. The data type of the information itself is target dependent. You can see the data type using the `ptype $_siginfo` command. On Unix systems, it typically corresponds to the standard `siginfo_t` type, as defined in the `signal.h` system header.

Here's an example, on a GNU/Linux system, printing the stray referenced address that raised a segmentation fault.

```
(gdb) continue
Program received signal SIGSEGV, Segmentation fault.
0x0000000000400766 in main ()
69      *(int *)p = 0;
(gdb) ptype $_siginfo
type = struct {
  int si_signo;
  int si_errno;
  int si_code;
  union {
    int _pad[28];
    struct {...} _kill;
    struct {...} _timer;
    struct {...} _rt;
    struct {...} _sigchld;
    struct {...} _sigfault;
    struct {...} _sigpoll;
  } _sifields;
}
(gdb) ptype $_siginfo._sifields._sigfault
type = struct {
  void *si_addr;
}
(gdb) p $_siginfo._sifields._sigfault.si_addr
$1 = (void *) 0x7ffff7ff7000
```

Depending on target support, `$_siginfo` may also be writable.

5.5 Stopping and Starting Multi-thread Programs

GDB supports debugging programs with multiple threads (see Section 4.10 [Debugging Programs with Multiple Threads], page 48). There are two modes of controlling execution of your program within the debugger. In the default mode, referred to as *all-stop mode*, when any thread in your program stops (for example, at a breakpoint or while being stepped), all

```
9      {
```

相同，但使用 `queue-signal` 而不是先等待程序接收信号：

```
(gdb) n
28      p = 0;
(gdb) queue-signal SIGUSR1
(gdb) si
sigusr1_handler () at sigusr1.c:9
9      {
(gdb)
```

在某些目标上，GDB 可以在被调试的程序实际接收中断信号之前，检查与该中断信号关联的额外信号信息。这些信息通过便利变量 `$_siginfo` 导出，包含内核在接收信号时传递给信号处理程序的数据。信息的类型本身取决于目标。您可以使用 `ptype $_siginfo` 命令查看数据类型。在 Unix 系统上，它通常对应于 `signal.h` 系统头文件中定义的标准 `siginfo_t` 类型。

这里有一个示例，在 GNU/Linux 系统上，打印引发段错误（segmentation fault）的逸出引用地址。

```
(gdb) continue
Program received signal SIGSEGV, Segmentation fault.
0x0000000000400766 in main ()
69      *(int *)p = 0;
(gdb) ptype $_siginfo
type = struct {
  int si_signo;
  int si_errno;
  int si_code;
  union {
    int _pad[28];
    struct {...} _kill;
    struct {...} _timer;
    struct {...} _rt;
    struct {...} _sigchld;
    struct {...} _sigfault;
    struct {...} _sigpoll;
  } _sifields;
}
(gdb) ptype $_siginfo._sifields._sigfault
type = struct {
  void *si_addr;
}
(gdb) p $_siginfo._sifields._sigfault.si_addr
$1 = (void *) 0x7ffff7ff7000
```

根据目标支持情况，`$_siginfo` 也可能可写。

5.5 停止与启动多线程程序

GDB 支持调试多线程程序（参见第 4.10 [多线程调试程序]，页 48）。在调试器中，您的程序执行控制有两种模式。在默认模式，称为全停止模式中，当您的程序中任何线程停止（例如，在断点处或执行单步时），所有其他线程也由 GDB 停止。在某些目标上，10 [也支持非停止模式，在这种模式下，当您在调试器中检查已停止的线程时，其他线程可以自由继续运行。

other threads in the program are also stopped by GDB. On some targets, GDB also supports *non-stop mode*, in which other threads can continue to run freely while you examine the stopped thread in the debugger.

5.5.1 All-Stop Mode

In all-stop mode, whenever your program stops under GDB for any reason, *all* threads of execution stop, not just the current thread. This allows you to examine the overall state of the program, including switching between threads, without worrying that things may change underfoot.

Conversely, whenever you restart the program, *all* threads start executing. *This is true even when single-stepping* with commands like `step` or `next`.

In particular, GDB cannot single-step all threads in lockstep. Since thread scheduling is up to your debugging target’s operating system (not controlled by GDB), other threads may execute more than one statement while the current thread completes a single step. Moreover, in general other threads stop in the middle of a statement, rather than at a clean statement boundary, when the program stops.

You might even find your program stopped in another thread after continuing or even single-stepping. This happens whenever some other thread runs into a breakpoint, a signal, or an exception before the first thread completes whatever you requested.

Whenever GDB stops your program, due to a breakpoint or a signal, it automatically selects the thread where that breakpoint or signal happened. GDB alerts you to the context switch with a message such as ‘[Switching to Thread n]’ to identify the thread.

On some OSes, you can modify GDB’s default behavior by locking the OS scheduler to allow only a single thread to run.

set scheduler-locking mode

Set the scheduler locking mode. It applies to normal execution, record mode, and replay mode. *mode* can be one of the following:

- | | |
|---------------|--|
| off | There is no locking and any thread may run at any time. |
| on | Only the current thread may run when the inferior is resumed. New threads created by the resumed thread are held stopped at their entry point, before they execute any instruction. |
| step | Behaves like on when stepping, and off otherwise. Threads other than the current never get a chance to run when you step, and they are completely free to run when you use commands like ‘ <code>continue</code> ’, ‘ <code>until</code> ’, or ‘ <code>finish</code> ’.

This mode optimizes for single-stepping; it prevents other threads from preempting the current thread while you are stepping, so that the focus of debugging does not change unexpectedly. However, unless another thread hits a breakpoint during its timeslice, GDB does not change the current thread away from the thread that you are debugging. |
| replay | Behaves like on in replay mode, and off in either record mode or during normal execution. This is the default mode. |

在调试器中，您的程序执行控制有两种模式。在默认模式，称为全停止模式中，当您的程序中任何线程停止（例如，在断点处或执行单步时），所有其他线程也由 GDB 停止。在某些目标上，GDB 也支持非停止模式，在这种模式下，当您在调试器中检查已停止的线程时，其他线程可以自由继续运行。

5.5.1 全停止模式

在全停止模式下，无论你的程序因任何原因在 GDB 下停止，所有执行线程都会停止，而不仅仅是当前线程。这允许你检查程序的整体状态，包括在线程之间切换，而不用担心事情可能在脚下发生变化。

相反，每次你重新启动程序时，所有线程都会开始执行。即使使用 `step` 或 `next` 等命令进行单步执行时也是如此。

特别是，GDB 无法让所有线程同步进行单步执行。由于线程调度取决于你的调试目标操作系统的线程调度（而不是由 GDB 控制），其他线程可能在当前线程完成单步执行时执行多个语句。此外，通常情况下，当程序停止时，其他线程会在语句中间停止，而不是在干净的语句边界处停止。

你甚至可能发现你的程序在继续或甚至单步执行后停止在另一个线程中。这发生在其他线程在第一个线程完成你请求的操作之前遇到断点、信号或异常时。

当 GDB 由于断点或信号停止您的程序时，它会自动选择发生断点或信号的线程。GDB 通过类似 “[切换到线程 n]” 的消息提醒您上下文切换，以识别该线程。

在某些操作系统上，您可以通过锁定操作系统调度器来修改 GDB 的默认行为，以允许仅单个线程运行。

设置调度器锁定 mode

设置调度器锁定模式。它适用于正常执行、记录模式和重放模式。模式可以是以下之一：

- | | |
|-------------|--|
| 关闭 | 无锁定，任何线程随时都可以运行。 |
| on | 当被调试进程恢复执行时，只有当前线程可以运行。由恢复的线程创建的新线程在执行任何指令之前，会在它们的入口点处停止。 |
| step | 单步执行时启用，其他情况则禁用。单步执行时，当前线程外的其他线程永远不会有机会运行，而使用“继续”、“直到”或“完成”等命令时，它们可以完全自由地运行。

此模式针对单步执行进行优化；它会在你单步执行时阻止其他线程抢占当前线程，从而确保调试焦点不会意外改变。然而，除非另一个线程在其时间片内触发断点，GDB 不会将当前线程切换到正在调试的线程之外。 |
| | 在回放模式下表现与回放模式一致，而在录制模式或正常执行期间则关闭。这是默认模式。 |

show scheduler-locking

Display the current scheduler locking mode.

By default, when you issue one of the execution commands such as `continue`, `next` or `step`, GDB allows only threads of the current inferior to run. For example, if GDB is attached to two inferiors, each with two threads, the `continue` command resumes only the two threads of the current inferior. This is useful, for example, when you debug a program that forks and you want to hold the parent stopped (so that, for instance, it doesn't run to exit), while you debug the child. In other situations, you may not be interested in inspecting the current state of any of the processes GDB is attached to, and you may want to resume them all until some breakpoint is hit. In the latter case, you can instruct GDB to allow all threads of all the inferiors to run with the `set schedule-multiple` command.

set schedule-multiple

Set the mode for allowing threads of multiple processes to be resumed when an execution command is issued. When `on`, all threads of all processes are allowed to run. When `off`, only the threads of the current process are resumed. The default is `off`. The `scheduler-locking` mode takes precedence when set to `on`, or while you are stepping and set to `step`.

show schedule-multiple

Display the current mode for resuming the execution of threads of multiple processes.

5.5.2 Non-Stop Mode

By default, when a thread stops to report a debugging event, GDB stops all other threads as well. This is called *all-stop* mode.

For some multi-threaded targets, GDB supports an optional mode of operation in which you can examine stopped program threads in the debugger while other threads continue to execute freely. This minimizes intrusion when debugging live systems, such as programs where some threads have real-time constraints or must continue to respond to external events. This is referred to as *non-stop* mode.

In non-stop mode, when a thread stops to report a debugging event, *only* that thread is stopped; GDB does not stop other threads as well, in contrast to the all-stop mode behavior. Additionally, execution commands such as `continue` and `step` apply by default only to the current thread in non-stop mode, rather than all threads as in all-stop mode. This allows you to control threads explicitly in ways that are not possible in all-stop mode — for example, stepping one thread while allowing others to run freely, stepping one thread while holding all others stopped, or stepping several threads independently and simultaneously.

You can use these commands to manipulate the non-stop mode setting:

set non-stop on

Enable non-stop mode.

set non-stop off

Disable non-stop mode. Also known as enabling all-stop mode. This is the default.

show non-stop

Show the current non-stop enablement setting.

显示调度锁定

显示当前的调度锁定模式。

默认情况下，当你发出 `continue`、`next` 或 `step` 等执行命令时，GDB 仅允许当前子进程的线程运行。例如，如果 GDB 附加到两个子进程，每个子进程有两个线程，则 `continue` 命令仅恢复当前子进程的两个线程。这在调试一个会分叉的程序时很有用，例如，当你希望暂停父进程（以便它不会运行到退出），同时调试子进程时。在其他情况下，你可能不关心检查附加到 GDB 的任何进程的当前状态，并且可能希望恢复它们全部，直到遇到某个断点。在后一种情况下，你可以指示 GDB 使用 `set schedule-multiple` 命令允许所有子进程的所有线程运行。

设置 schedule-multiple

设置在发出执行命令时允许多个进程的线程恢复的模式。开启时，允许所有进程的所有线程运行。关闭时，仅恢复当前进程的线程。默认为关闭。当 `set` 为开启，或在 `step` 模式下设置 `step` 时，调度锁定模式优先。

显示多进程调度

显示当前模式，用于恢复多个进程的线程执行。

5.5.2 非停止模式

默认情况下，当线程停止报告调试事件时，GDB 会停止所有其他线程。这称为全停止模式。

对于某些多线程目标，GDB 支持一种可选的操作模式，您可以在调试器中检查已停止的程序线程，同时其他线程可以自由执行。这最大限度地减少了在调试实时系统时的干扰，例如某些线程具有实时约束或必须继续响应外部事件的程序。这称为非停止模式。

在非停止模式下，当线程停止报告调试事件时，仅该线程停止；GDB 不会停止其他线程，与全停止模式的行为相反。此外，在非停止模式下，继续和单步等执行命令默认仅适用于当前线程，而不是全停止模式中的所有线程。这允许您以全停止模式中不可能的方式显式控制线程——例如，单步执行一个线程，同时允许其他线程自由运行，单步执行一个线程，同时暂停所有其他线程，或独立并同时单步执行多个线程。

您可以使用这些命令来操作非连续模式设置：

set non-stop on

启用非连续模式。

set non-stop off

禁用非连续模式。也称为启用全停止模式。这是默认设置。

连续显示

显示当前的连续调试启用设置。

Note the `set non-stop` preference is only consulted when GDB starts or connects to the target program, and it is not possible to switch modes once debugging has started. Furthermore, since not all targets support non-stop mode, even when you have enabled non-stop mode, GDB may still fall back to all-stop operation by default.

In non-stop mode, all execution commands apply only to the current thread by default. That is, `continue` only continues one thread. To continue all threads, issue `continue -a` or `c -a`.

You can use GDB's background execution commands (see Section 5.5.3 [Background Execution], page 98) to run some threads in the background while you continue to examine or step others from GDB. The MI execution commands (see Section 27.15 [GDB/MI Program Execution], page 647) are always executed asynchronously in non-stop mode.

Suspending execution is done with the `interrupt` command when running in the background, or `Ctrl-c` during foreground execution. In all-stop mode, this stops the whole process; but in non-stop mode the interrupt applies only to the current thread. To stop the whole program, use `interrupt -a`.

Other execution commands do not currently support the `-a` option.

In non-stop mode, when a thread stops, GDB doesn't automatically make that thread current, as it does in all-stop mode. This is because the thread stop notifications are asynchronous with respect to GDB's command interpreter, and it would be confusing if GDB unexpectedly changed to a different thread just as you entered a command to operate on the previously current thread.

5.5.3 Background Execution

GDB's execution commands have two variants: the normal foreground (synchronous) behavior, and a background (asynchronous) behavior. In foreground execution, GDB waits for the program to report that some thread has stopped before prompting for another command. In background execution, GDB immediately gives a command prompt so that you can issue other commands while your program runs.

If the target doesn't support async mode, GDB issues an error message if you attempt to use the background execution commands.

To specify background execution, add a `&` to the command. For example, the background form of the `continue` command is `continue&`, or just `c&`. The execution commands that accept background execution are:

<code>run</code>	See Section 4.2 [Starting your Program], page 34.
<code>attach</code>	See Section 4.7 [Debugging an Already-running Process], page 41.
<code>step</code>	See Section 5.2 [Continuing and Stepping], page 85.
<code>stepi</code>	See Section 5.2 [Continuing and Stepping], page 85.
<code>next</code>	See Section 5.2 [Continuing and Stepping], page 85.
<code>nexti</code>	See Section 5.2 [Continuing and Stepping], page 85.
<code>continue</code>	See Section 5.2 [Continuing and Stepping], page 85.
<code>finish</code>	See Section 5.2 [Continuing and Stepping], page 85.

请注意, 设置的连续调试偏好仅在 GDB 启动或连接到目标程序时才会被参考, 一旦开始调试就无法切换模式。此外, 由于并非所有目标都支持连续调试模式, 即使您已启用连续调试模式, GDB 默认仍可能回退到全步操作。

在非停止模式下, 所有执行命令默认仅适用于当前线程。也就是说, `continue`命令仅继续一个线程。要继续所有线程, 请使用`continue -a`或`c -a`命令。

您可以使用 GDB的后台执行命令(参见第5.5.3 [后台执行], 页98)来在后台运行某些线程, 同时您继续检查或单步执行其他线程。MI执行命令(参见第27.15 [GDB/MI程序执行], 页647)在非停止模式下始终异步执行。

在后台运行时, 使用中断命令暂停执行; 在前景运行时, 使用 `Ctrl-c` 暂停执行。在全部停止模式下, 这会停止整个进程; 但在非停止模式下, 中断命令仅适用于当前线程。要停止整个程序, 请使用`interrupt -a`命令。

其他执行命令目前不支持-a选项。

在非停止模式下, 当线程停止时, GDB 不会自动将其设置为当前线程, 就像在全部停止模式下那样。这是因为线程停止通知相对于 GDB的命令解释器是异步的, 如果 GDB在您输入命令操作先前当前线程时意外切换到其他线程, 将会造成混淆。

5.5.3 背景执行

GDB的执行命令有两种变体: 正常的前台(同步)行为和后台(异步)行为。在前台执行时, GDB 会等待程序报告某个线程已停止后才会提示下一个命令。在后台执行时, GDB 会立即给出命令提示, 以便您在程序运行时可以发出其他命令。

如果目标不支持异步模式, 当您尝试使用后台执行命令时, GDB 会发出错误消息。

要指定后台执行, 请在命令后添加一个 `&`。例如, `continue`命令的后台形式是 `continue&`, 或者直接是`c&`。接受后台执行的执行命令有:

<code>run</code>	参见第4.2 [启动您的程序], 页34。
<code>attach</code>	参见第4.7 [调试已运行进程], 页41。
<code>step</code>	参见第5.2 [继续和单步执行], 页85。
<code>stepi</code>	参见第5.2 [继续和单步执行], 页85。
<code>next</code>	参见第5.2 [继续和单步执行], 页85。
<code>nexti</code>	参见第5.2 [继续和步进], 页85。
继续	参见第5.2 [继续和步进], 页85。
完成	参见第5.2 [继续和步进], 页85。

until See Section 5.2 [Continuing and Stepping], page 85.

Background execution is especially useful in conjunction with non-stop mode for debugging programs with multiple threads; see Section 5.5.2 [Non-Stop Mode], page 97. However, you can also use these commands in the normal all-stop mode with the restriction that you cannot issue another execution command until the previous one finishes. Examples of commands that are valid in all-stop mode while the program is running include **help** and **info break**.

You can interrupt your program while it is running in the background by using the **interrupt** command.

interrupt

interrupt -a

Suspend execution of the running program. In all-stop mode, **interrupt** stops the whole process, but in non-stop mode, it stops only the current thread. To stop the whole program in non-stop mode, use **interrupt -a**.

5.5.4 Thread-Specific Breakpoints

When your program has multiple threads (see Section 4.10 [Debugging Programs with Multiple Threads], page 48), you can choose whether to set breakpoints on all threads, or on a particular thread.

break locspec thread thread-id

break locspec thread thread-id if ...

locspec specifies a code location or locations in your program. See Section 9.2 [Location Specifications], page 128, for details.

Use the qualifier ‘**thread thread-id**’ with a breakpoint command to specify that you only want GDB to stop the program when a particular thread reaches this breakpoint. The *thread-id* specifier is one of the thread identifiers assigned by GDB, shown in the first column of the ‘**info threads**’ display.

If you do not specify ‘**thread thread-id**’ when you set a breakpoint, the breakpoint applies to *all* threads of your program.

You can use the **thread** qualifier on conditional breakpoints as well; in this case, place ‘**thread thread-id**’ before or after the breakpoint condition, like this:

```
(gdb) break frik.c:13 thread 28 if bartab > lim
```

Thread-specific breakpoints are automatically deleted when GDB detects the corresponding thread is no longer in the thread list. For example:

```
(gdb) c
```

```
Thread-specific breakpoint 3 deleted - thread 28 no longer in the thread list.
```

There are several ways for a thread to disappear, such as a regular thread exit, but also when you detach from the process with the **detach** command (see Section 4.7 [Debugging an Already-running Process], page 41), or if GDB loses the remote connection (see Chapter 20 [Remote Debugging], page 317), etc. Note that with some targets, GDB is only able to detect a thread has exited when the user explicitly asks for the thread list with the **info threads** command.

参见第5.2 [继续与单步], 页85。

后台执行在与非停止模式结合时特别有用，用于调试多线程程序；参见第5.5节.2 [非停止模式], 页97。但是，你也可以在正常全停止模式下使用这些命令，限制是你必须等到前一个执行命令完成才能发出另一个执行命令。在程序运行时，全停止模式下有效的命令示例包括help和info break。

您可以使用中断命令在程序在后台运行时中断它。

中断

中断 -a

挂起正在运行程序的执行。在全停止模式下，中断会停止整个进程，但在非停止模式下，它只停止当前线程。要在非停止模式下停止整个程序，请使用中断 -a。

5.5.4 线程特定断点

当您的程序有多个线程（参见第 4.10 [使用多个线程调试程序], 页 48）时，您可以选择在所有线程上设置断点，或在特定线程上设置断点。

断 *locspec* 线程 *thread-id*

中断 *locspec* 线程 *thread-id* 如果 ...

locspec 指定程序中的代码位置或多个位置。参见第 9.2 节 [位置规范], 页 128, 了解详情。

使用 ‘**thread thread-id**’ 限定符与断点命令，指定您只想在特定线程到达此断点时让程序停止。线程ID限定符是 GDB分配的线程标识符之一，显示在 ‘**info threads**’ 显示的第一列中。

如果您在设置断点时未指定 ‘**thread thread-id**’，则断点将适用于您的程序的所有线程。

您也可以在条件断点上使用线程限定符；在这种情况下，将 ‘**thread thread-id**’ 放在断点条件之前或之后，如下所示：

```
(gdb) break frik.c:13 thread 28 if bartab > lim
```

当 GDB 检测到相应线程不再在线程列表中时，线程特定的断点会自动删除。例如：

```
(gdb) c
线程特定断点 3 已删除 - 线程 28 不再在线程列表中。
```

线程消失有多种方式，例如正常退出，但也可能在您使用detach命令从进程中分离时（参见第4.7 [调试已运行进程], 页41），或者当 GDB 失去远程连接时（参见第20.[远程调试], 页317），等等。请注意，对于某些目标，GDB只有在用户使用info threads命令明确请求线程列表时，才能检测到线程已退出。

A breakpoint can't be both thread-specific and inferior-specific (see Section 4.9.1 [Inferior-Specific Breakpoints], page 47), or task-specific (see Section 15.4.10.7 [Ada Tasks], page 256); using more than one of the `thread`, `inferior`, or `task` keywords when creating a breakpoint will give an error.

5.5.5 Interrupted System Calls

There is an unfortunate side effect when using GDB to debug multi-threaded programs. If one thread stops for a breakpoint, or for some other reason, and another thread is blocked in a system call, then the system call may return prematurely. This is a consequence of the interaction between multiple threads and the signals that GDB uses to implement breakpoints and other events that stop execution.

To handle this problem, your program should check the return value of each system call and react appropriately. This is good programming style anyways.

For example, do not write code like this:

```
sleep (10);
```

The call to `sleep` will return early if a different thread stops at a breakpoint or for some other reason.

Instead, write this:

```
int unslept = 10;
while (unslept > 0)
    unslept = sleep (unslept);
```

A system call is allowed to return early, so the system is still conforming to its specification. But GDB does cause your multi-threaded program to behave differently than it would without GDB.

Also, GDB uses internal breakpoints in the thread library to monitor certain events such as thread creation and thread destruction. When such an event happens, a system call in another thread may return prematurely, even though your program does not appear to stop.

5.5.6 Observer Mode

If you want to build on non-stop mode and observe program behavior without any chance of disruption by GDB, you can set variables to disable all of the debugger's attempts to modify state, whether by writing memory, inserting breakpoints, etc. These operate at a low level, intercepting operations from all commands.

When all of these are set to `off`, then GDB is said to be *observer mode*. As a convenience, the variable `observer` can be set to disable these, plus enable non-stop mode.

Note that GDB will not prevent you from making nonsensical combinations of these settings. For instance, if you have enabled `may-insert-breakpoints` but disabled `may-write-memory`, then breakpoints that work by writing trap instructions into the code stream will still not be able to be placed.

```
set observer on
set observer off
```

When set to `on`, this disables all the permission variables below (except for `insert-fast-tracepoints`), plus enables non-stop debugging. Setting this to `off` switches back to normal debugging, though remaining in non-stop mode.

断点不能同时是线程特定的和从属特定的（参见第4.9.1.[从属特定断点], 页47），或者任务特定的（参见第15.4.10.7 [Ada任务], 页256）；在创建断点时使用多个线程、从属或任务关键字将导致错误。

5.5.5 中断系统调用

使用 GDB 调试多线程程序时存在一个不幸的副作用。如果一个线程因断点而停止，或因其他原因停止，而另一个线程被阻塞在系统调用中，那么系统调用可能会提前返回。这是多个线程与 GDB 使用信号实现断点和其他停止执行事件之间的交互的结果。

要处理这个问题，你的程序应该检查每个系统调用的返回值并做出适当的反应。这本身也是良好的编程风格。

例如，不要编写这样的代码：

```
sleep (10);
```

如果其他线程在断点处停止或出于其他原因，则请求休眠的调用会提前返回。

相反，请这样写：

```
int unslept = 10;
while (unslept > 0)
    未睡觉 =睡眠 (未睡觉);
```

系统调用允许提前返回，因此系统仍然符合其规范。但 GDB 确实会导致您的多线程程序的行为与没有 GDB 时不同。

此外，GDB 在threadlibrary中使用了内部断点来监控某些事件，例如线程创建和线程销毁。当发生此类事件时，另一个线程中的系统调用可能会提前返回，即使您的程序看起来并未停止。

5.5.6 观察者模式

如果您想在非停止模式下构建并观察程序行为，而没有任何机会被 GDB中断，您可以将变量设置为禁用调试器尝试修改所有状态，无论通过写入内存、插入断点等。这些操作在低级别运行，拦截所有命令的操作。

当所有这些都设置为关闭时，GDB 就被称为观察者模式。为了方便起见，可以将变量 `observer` 设置为禁用这些，并启用非停止模式。

请注意，GDB 不会阻止您对这些设置进行无意义的组合。例如，如果您已启用 `may-insert-breakpoints` 但禁用了 `may-write-memory`，那么通过向代码流写入陷阱指令来起作用的断点仍然无法放置。

设置观察者

```
set observer off
```

设置为开启时，此选项禁用所有下方的权限变量（除 `insert-fast-tracepoints` 外），同时启用非停止调试。将其设置为关闭会切换回正常调试，但将保持非停止模式。

`show observer`

Show whether observer mode is on or off.

`set may-write-registers on`

`set may-write-registers off`

This controls whether GDB will attempt to alter the values of registers, such as with assignment expressions in `print`, or the `jump` command. It defaults to `on`.

`show may-write-registers`

Show the current permission to write registers.

`set may-write-memory on`

`set may-write-memory off`

This controls whether GDB will attempt to alter the contents of memory, such as with assignment expressions in `print`. It defaults to `on`.

`show may-write-memory`

Show the current permission to write memory.

`set may-insert-breakpoints on`

`set may-insert-breakpoints off`

This controls whether GDB will attempt to insert breakpoints. This affects all breakpoints, including internal breakpoints defined by GDB. It defaults to `on`.

`show may-insert-breakpoints`

Show the current permission to insert breakpoints.

`set may-insert-tracepoints on`

`set may-insert-tracepoints off`

This controls whether GDB will attempt to insert (regular) tracepoints at the beginning of a tracing experiment. It affects only non-fast tracepoints, fast tracepoints being under the control of `may-insert-fast-tracepoints`. It defaults to `on`.

`show may-insert-tracepoints`

Show the current permission to insert tracepoints.

`set may-insert-fast-tracepoints on`

`set may-insert-fast-tracepoints off`

This controls whether GDB will attempt to insert fast tracepoints at the beginning of a tracing experiment. It affects only fast tracepoints, regular (non-fast) tracepoints being under the control of `may-insert-tracepoints`. It defaults to `on`.

`show may-insert-fast-tracepoints`

Show the current permission to insert fast tracepoints.

`set may-interrupt on`

`set may-interrupt off`

This controls whether GDB will attempt to interrupt or stop program execution. When this variable is `off`, the `interrupt` command will have no effect, nor will `Ctrl-c`. It defaults to `on`.

`show may-interrupt`

Show the current permission to interrupt or stop the program.

显示观察者

显示观察者模式是否开启或关闭。

设置允许写入寄存器为开启

设置允许写入寄存器为关闭

此选项控制 GDB 是否尝试修改寄存器的值，例如在 `print` 中的赋值表达式或跳转命令中。默认为开启。

显示允许写入寄存器

显示当前写入寄存器的权限。

设置允许写入内存

`set may-write-memory off`

此设置控制 GDB 是否尝试修改内存内容，例如在 `print` 语句中使用赋值表达式。默认为允许。

显示允许写入内存的权限

显示当前的内存写入权限。

设置允许插入断点

设置禁止插入断点

这控制 GDB 是否尝试插入断点。这会影响所有断点，包括由 GDB 定义的内部断点。默认为开启。

`show may-insert-breakpoints`

显示当前插入断点的权限。

`set may-insert-tracepoints on`

`set may-insert-tracepoints off`

这控制 GDB 是否尝试在跟踪实验开始时插入（常规）跟踪点。这仅影响非快速跟踪点，快速跟踪点受 `may-insert-fast-tracepoints` 控制。默认为开启。

显示插入跟踪点权限

显示当前插入跟踪点的权限。

设置 `may-insert-fast-tracepoints` 开启

设置 `may-insert-fast-tracepoints` 关闭

此选项控制 GDB 是否会在跟踪实验开始时尝试插入快速跟踪点。它仅影响快速跟踪点，常规（非快速）跟踪点受 `may-insert-tracepoints` 控制。默认为开启。

显示 `may-insert-fast-tracepoints`

显示插入快速断点的当前权限。

`set may-interrupt on`

`set may-interrupt off`

此设置控制 GDB 是否会尝试中断或停止程序执行。当此变量关闭时，中断命令将无效，`Ctrl-c` 也将无效。默认为开启。

`show may-interrupt`

显示中断或停止程序的当前权限。

6 Running programs backward

When you are debugging a program, it is not unusual to realize that you have gone too far, and some event of interest has already happened. If the target environment supports it, GDB can allow you to “rewind” the program by running it backward.

A target environment that supports reverse execution should be able to “undo” the changes in machine state that have taken place as the program was executing normally. Variables, registers etc. should revert to their previous values. Obviously this requires a great deal of sophistication on the part of the target environment; not all target environments can support reverse execution.

When a program is executed in reverse, the instructions that have most recently been executed are “un-executed”, in reverse order. The program counter runs backward, following the previous thread of execution in reverse. As each instruction is “un-executed”, the values of memory and/or registers that were changed by that instruction are reverted to their previous states. After executing a piece of source code in reverse, all side effects of that code should be “undone”, and all variables should be returned to their prior values¹.

On some platforms, GDB has built-in support for reverse execution, activated with the `record` or `record btrace` commands. See Chapter 7 [Process Record and Replay], page 105. Some remote targets, typically full system emulators, support reverse execution directly without requiring any special command.

If you are debugging in a target environment that supports reverse execution, GDB provides the following commands.

`reverse-continue` [*ignore-count*]
`rc` [*ignore-count*]

Beginning at the point where your program last stopped, start executing in reverse. Reverse execution will stop for breakpoints and synchronous exceptions (signals), just like normal execution. Behavior of asynchronous signals depends on the target environment.

`reverse-step` [*count*]

Run the program backward until control reaches the start of a different source line; then stop it, and return control to GDB.

Like the `step` command, `reverse-step` will only stop at the beginning of a source line. It “un-executes” the previously executed source line. If the previous source line included calls to debuggable functions, `reverse-step` will step (backward) into the called function, stopping at the beginning of the *last* statement in the called function (typically a return statement).

Also, as with the `step` command, if non-debuggable functions are called, `reverse-step` will run through them backward without stopping.

¹ Note that some side effects are easier to undo than others. For instance, memory and registers are relatively easy, but device I/O is hard. Some targets may be able undo things like device I/O, and some may not.

The contract between GDB and the reverse executing target requires only that the target do something reasonable when GDB tells it to execute backwards, and then report the results back to GDB. Whatever the target reports back to GDB, GDB will report back to the user. GDB assumes that the memory and registers that the target reports are in a consistent state, but GDB accepts whatever it is given.

6 向后运行程序

当你调试程序时，发现自己已经走得太远，某个感兴趣的事件已经发生是很常见的情况。如果目标环境支持，GDB 可以让你通过向后运行程序来“倒带”程序。

支持反向执行的目标环境应该能够“撤销”程序正常运行时发生的机器状态变化。变量、寄存器等应该恢复到之前的状态。显然，这对目标环境的要求非常高；并非所有目标环境都能支持反向执行。

当程序反向执行时，最近执行的指令会被“撤销”，且顺序相反。程序计数器会向后运行，遵循之前执行的线程。随着每条指令被“撤销”，该指令改变的内存和/或寄存器的值会恢复到之前的状态。反向执行一段源代码后，该代码的所有副作用应该被“撤销”，所有变量都应该恢复到之前的值¹。

在某些平台上，GDB 内置了反向执行支持，可通过 `record` 或 `record btrace` 命令激活。参见第 7 [进程记录与回放], 页 105。一些远程目标（通常是全系统模拟器）可以直接支持反向执行，无需任何特殊命令。

如果你正在支持反向执行的目标环境中进行调试，GDB 提供了以下命令。

`reverse-continue` [*ignore-count*]
`rc` [*ignore-count*]

从程序上次停止的位置开始，反向执行。反向执行会在断点和同步异常（信号）处停止，就像正常执行一样。异步信号的行为取决于目标环境。

`reverse-step` [*count*]

向后运行程序，直到控制到达不同源行的起始位置；然后停止它，并将控制权返回给 GDB。

与步进命令类似，反向步进仅会在源代码行的开头停止。

它会“撤销”先前执行的源代码行。如果先前执行的源代码行包含对可调试函数的调用，反向步进将（向后）步入被调用的函数，并在被调用函数中最后一行语句的开头停止（通常是返回语句）。

与步进命令类似，反向步进仅会在源代码行的开头停止。

被调用函数中的语句（通常是返回语句）。

此外，与单步执行命令类似，如果调用了不可调试的函数，反向单步执行会通过它们向后执行而不会停止。

¹ 请注意，某些副作用比其他副作用更容易撤销。例如，内存和寄存器相对容易处理，但设备 I/O 则比较困难。某些目标可能能够撤销设备 I/O 等操作，而其他目标则可能无法做到。GDB 与反向执行目标之间的协议仅要求目标在 GDB 指示其反向执行时采取合理的操作，然后向 GDB 报告结果。无论目标向 GDB 报告了什么，GDB 都会向用户报告。GDB 假设目标报告的内存和寄存器处于一致状态，但 GDB 接受任何它所得到的内容。

reverse-stepi [*count*]

Reverse-execute one machine instruction. Note that the instruction to be reverse-executed is *not* the one pointed to by the program counter, but the instruction executed prior to that one. For instance, if the last instruction was a jump, **reverse-stepi** will take you back from the destination of the jump to the jump instruction itself.

reverse-next [*count*]

Run backward to the beginning of the previous line executed in the current (innermost) stack frame. If the line contains function calls, they will be “un-executed” without stopping. Starting from the first line of a function, **reverse-next** will take you back to the caller of that function, *before* the function was called, just as the normal **next** command would take you from the last line of a function back to its return to its caller².

reverse-nexti [*count*]

Like **nexti**, **reverse-nexti** executes a single instruction in reverse, except that called functions are “un-executed” atomically. That is, if the previously executed instruction was a return from another function, **reverse-nexti** will continue to execute in reverse until the call to that function (from the current stack frame) is reached.

reverse-finish

Just as the **finish** command takes you to the point where the current function returns, **reverse-finish** takes you to the point where it was called. Instead of ending up at the end of the current function invocation, you end up at the beginning.

set exec-direction

Set the direction of target execution.

set exec-direction reverse

GDB will perform all execution commands in reverse, until the exec-direction mode is changed to “forward”. Affected commands include **step**, **stepi**, **next**, **nexti**, **continue**, and **finish**. The **return** command cannot be used in reverse mode.

set exec-direction forward

GDB will perform all execution commands in the normal fashion. This is the default.

² Unless the code is too heavily optimized.

反向单步执行 [*count*]

反向执行一条机器指令。请注意，要反向执行的指令不是程序计数器指向的那条，而是之前执行的那条指令。例如，如果最后一条指令是跳转，反向单步执行会带你从跳转目标回到跳转指令本身。

反向下一步执行 [*count*]

向后运行到当前（最内层）栈帧中先前执行的那一行。如果该行包含函数调用，它们将被“撤销执行”，而不会停止。从函数的第一行开始，反向下一步执行将带你回到调用该函数的调用者，在函数被调用之前，就像正常的下一步命令会带你从函数的最后一行返回到其调用者一样²。

反向下一步执行i [*count*]

与nexti类似，reverse-nexti反向执行单条指令，但调用的函数将“原子性撤销”。也就是说，如果先前执行的指令是来自另一个函数的返回，reverse-nexti将继续反向执行，直到到达当前栈帧对该函数的调用。

反向完成

就像完成命令将你带到当前函数返回的位置一样，反向完成将你带到它被调用的位置。你不是结束在当前函数调用的末尾，而是结束在开头。

设置执行方向

设置目标执行的方向。

设置执行方向反向

GDB 将执行所有命令并反向操作，直到执行方向模式更改为“正向”。受影响的命令包括 **step**、**stepi**、**next**、**nexti**、**continue** 和 **finish**。在反向模式下不能使用 **return** 命令。

设置执行方向正向

GDB 将按正常方式执行所有执行命令。这是默认设置。

² 除非代码经过过度优化。

7 Recording Inferior’s Execution and Replaying It

On some platforms, GDB provides a special *process record and replay* target that can record a log of the process execution, and replay it later with both forward and reverse execution commands.

When this target is in use, if the execution log includes the record for the next instruction, GDB will debug in *replay mode*. In the replay mode, the inferior does not really execute code instructions. Instead, all the events that normally happen during code execution are taken from the execution log. While code is not really executed in replay mode, the values of registers (including the program counter register) and the memory of the inferior are still changed as they normally would. Their contents are taken from the execution log.

If the record for the next instruction is not in the execution log, GDB will debug in *record mode*. In this mode, the inferior executes normally, and GDB records the execution log for future replay.

The process record and replay target supports reverse execution (see Chapter 6 [Reverse Execution], page 103), even if the platform on which the inferior runs does not. However, the reverse execution is limited in this case by the range of the instructions recorded in the execution log. In other words, reverse execution on platforms that don’t support it directly can only be done in the replay mode.

When debugging in the reverse direction, GDB will work in replay mode as long as the execution log includes the record for the previous instruction; otherwise, it will work in record mode, if the platform supports reverse execution, or stop if not.

Currently, process record and replay is supported on ARM, Aarch64, LoongArch, Moxie, PowerPC, PowerPC64, S/390, RISC-V and x86 (i386/amd64) running GNU/Linux. Process record and replay can be used both when native debugging, and when remote debugging via `gdbserver`.

When recording an inferior, GDB may print auxiliary information during stepping commands and commands displaying the execution history.

For architecture environments that support process record and replay, GDB provides the following commands:

record method

This command starts the process record and replay target. The recording method can be specified as parameter. Without a parameter the command uses the `full` recording method. The following recording methods are available:

full Full record/replay recording using GDB’s software record and replay implementation. This method allows replaying and reverse execution.

btrace format

Hardware-supported instruction recording, supported on Intel processors. This method does not record data. Further, the data is collected in a ring buffer so old data will be overwritten when the buffer is full. It allows limited reverse execution. Variables and

7 录制下级执行并回放

在某些平台上，GDB 提供了一种特殊的进程记录和回放目标，可以记录进程执行的日志，稍后使用正向和反向执行命令进行回放。

当此目标被使用时，如果执行日志包含下一条指令的记录，GDB 将在回放模式下进行调试。在回放模式下，下级并不会真正执行代码指令。相反，所有在代码执行期间通常发生的事件都会从执行日志中获取。虽然回放模式下代码并未真正执行，但寄存器（包括程序计数器寄存器）的值以及下级的内存仍然会像正常情况下一样发生变化。它们的内容会从执行日志中获取。

如果执行日志中不存在下一条指令的记录，GDB 将在记录模式下进行调试。在这种模式下，下级会正常执行，并 GDB 记录执行日志以供未来回放。

进程记录和回放目标支持反向执行（参见第 6 [反向执行], 页 103），即使下级运行的平台不支持。然而，在这种情况下，反向执行受限于执行日志中记录的指令范围。换句话说，在不直接支持反向执行的平台上的反向执行只能通过回放模式进行。

在反向调试时，GDB 只要执行日志包含前一条指令的记录，就会在重放模式下工作；否则，如果平台支持反向执行，就会在记录模式下工作，否则将停止。

目前，在运行GNU/Linux的ARM、Aarch64、LoongArch、Moxie、PowerPC、PowerPC64、S/390、RISC-V和x86（i386/amd64）平台上支持进程记录和重放。进程记录和重放既可用于本地调试，也可用于通过gdbserver进行的远程调试。

在记录目标进程时，GDB 在单步命令和显示执行历史的命令期间可能会打印辅助信息。

对于支持进程记录和重放架构环境，GDB 提供以下命令：

record method

此命令启动进程记录和重放目标。记录方法可以作为参数指定。如果没有参数，命令将使用完整录制方法。以下录制方法是可用的：

:

full 完整录制/回放录制，使用 GDB 的软件录制和回放实现。此方法允许回放和反向执行。

btrace format

硬件支持的指令录制，支持在 Intel 处理器上。此方法不录制数据。此外，数据会收集到环形缓冲区中，当缓冲区满时，旧数据将被覆盖。它允许有限的逆向执行。变量和寄存器在逆向执行期间不可用。在远程调试时，断开连接后录制会继续。重新连接后可以检查录制的的数据。可以使用 `record stop` 命令停止录制。

registers are not available during reverse execution. In remote debugging, recording continues on disconnect. Recorded data can be inspected after reconnecting. The recording may be stopped using **record stop**.

The recording format can be specified as parameter. Without a parameter the command chooses the recording format. The following recording formats are available:

bts	Use the <i>Branch Trace Store</i> (BTS) recording format. In this format, the processor stores a from/to record for each executed branch in the btrace ring buffer.
pt	Use the <i>Intel Processor Trace</i> recording format. In this format, the processor stores the execution trace in a compressed form that is afterwards decoded by GDB. The trace can be recorded with very low overhead. The compressed trace format also allows small trace buffers to already contain a big number of instructions compared to BTS. Decoding the recorded execution trace, on the other hand, is more expensive than decoding BTS trace. This is mostly due to the increased number of instructions to process. You should increase the buffer-size with care.

Not all recording formats may be available on all processors.

The process record and replay target can only debug a process that is already running. Therefore, you need first to start the process with the **run** or **start** commands, and then start the recording with the **record method** command.

Displaced stepping (see Appendix D [displaced stepping], page 777) will be automatically disabled when process record and replay target is started. That's because the process record and replay target doesn't support displaced stepping.

If the inferior is in the non-stop mode (see Section 5.5.2 [Non-Stop Mode], page 97) or in the asynchronous execution mode (see Section 5.5.3 [Background Execution], page 98), not all recording methods are available. The **full** recording method does not support these two modes.

record stop

Stop the process record and replay target. When process record and replay target stops, the entire execution log will be deleted and the inferior will either be terminated, or will remain in its final state.

When you stop the process record and replay target in record mode (at the end of the execution log), the inferior will be stopped at the next instruction that would have been recorded. In other words, if you record for a while and then stop recording, the inferior process will be left in the same state as if the recording never happened.

On the other hand, if the process record and replay target is stopped while in replay mode (that is, not at the end of the execution log, but at some earlier

在远程调试时，断开连接后录制会继续。重新连接后可以检查录制的数 据。可以使用 **record stop** 命令停止录制。

录制格式可以作为参数指定。如果没有参数，命令将选择录制格式。以下可用录制格式：

bts	使用分支跟踪存储 (BTS) 录制格式。在这种格式中，处理器在 btrace 环形缓冲区中为每个执行的分支存储一个从/到记录。
pt	使用英特尔处理器跟踪录制格式。在这种格式中，处理器以压缩形式存储执行跟踪，之后由 GDB 解码。跟踪可以以非常低的开销录制。压缩跟踪格式还允许较小的跟踪缓冲区 与 BTS 相比，已经包含大量指令。

另一方面，解码录制的执行跟踪比解码 BTS 跟踪更昂贵。这主要是由于要处理的指令数量增加。你应该谨慎增加缓冲区大小。

并非所有录制格式在所有处理器上都可用。

进程记录和回放目标只能调试已运行的进程。因此，您要先用 **run** 或 **start** 命令启动进程，然后才能用 **record method** 命令开始录制。

偏移步进（参见附录 D [偏移步进]，第 777 页）将在进程记录和回放目标启动时自动禁用。这是因为进程记录和回放目标不支持偏移步进。

如果从属进程处于非停止模式（参见第 5.5.2 [非停止模式]，第 97 页）或在异步执行模式（参见第 5.5.3 [后台执行]，第 98 页）下，并非所有录制方法都可用。完整录制方法不支持这两种模式。

录制停止

停止进程记录和回放目标。当进程记录和回放目标停止时，整个执行日志将被删除，从属进程将终止，或保持其最终状态。

当你在记录模式下停止进程记录和重放目标（在执行日志的末尾）时，目标进程将在下一个本应被记录的指令处停止。换句话说，如果你记录一段时间然后停止记录，目标进程将保持与记录从未发生时的相同状态。

另一方面，如果在回放模式下停止进程记录和回放目标（也就是说，不在执行日志的末尾，而是在某个更早的阶段），那么从那个更早的阶段开始，从属进程将变为“活动”状态，然后就可以从该状态继续进行常规的“活动”调试。

point), the inferior process will become “live” at that earlier state, and it will then be possible to continue the usual “live” debugging of the process from that state.

When the inferior process exits, or GDB detaches from it, process record and replay target will automatically stop itself.

record goto

Go to a specific location in the execution log. There are several ways to specify the location to go to:

record goto begin

record goto start

Go to the beginning of the execution log.

record goto end

Go to the end of the execution log.

record goto *n*

Go to instruction number *n* in the execution log.

record save *filename*

Save the execution log to a file *filename*. Default filename is `gdb_record.process_id`, where *process_id* is the process ID of the inferior.

This command may not be available for all recording methods.

record restore *filename*

Restore the execution log from a file *filename*. File must have been created with **record save**.

set record full insn-number-max *limit*

set record full insn-number-max unlimited

Set the limit of instructions to be recorded for the **full** recording method. Default value is 200000.

If *limit* is a positive number, then GDB will start deleting instructions from the log once the number of the record instructions becomes greater than *limit*. For every new recorded instruction, GDB will delete the earliest recorded instruction to keep the number of recorded instructions at the limit. (Since deleting recorded instructions loses information, GDB lets you control what happens when the limit is reached, by means of the **stop-at-limit** option, described below.)

If *limit* is **unlimited** or zero, GDB will never delete recorded instructions from the execution log. The number of recorded instructions is limited only by the available memory.

show record full insn-number-max

Show the limit of instructions to be recorded with the **full** recording method.

set record full stop-at-limit

Control the behavior of the **full** recording method when the number of recorded instructions reaches the limit. If **ON** (the default), GDB will stop when the limit is reached for the first time and ask you whether you want to stop the inferior or

在该更早的阶段，从属进程将变为“活动”状态，然后就可以从该状态继续进行常规的“活动”调试。

当下级进程退出，或 GDB 与其分离时，进程记录和重放目标将自动停止自身。

record goto

前往执行日志中的特定位置。有多种方法可以指定要前往的位置：

record goto begin

record goto start

前往执行日志的开头。

记录 跳至末尾

前往执行日志的末尾。

记录 跳至 *n*

前往执行日志中的第 *n* 条指令。

记录 保存 *filename*

将执行日志保存到文件 *filename*。默认文件名是 `gdb_record.process_id`，其中进程 ID 是子进程的进程 ID。

此命令可能并非所有录制方法都可用。

record restore *filename*

从文件恢复执行日志 *filename*。文件必须使用 **record save** 创建。

set record full insn-number-max *limit*

set record full insn-number-max unlimited

设置全记录方法下要记录的指令数量上限。默认值为200000。

如果上限是一个正数，那么 GDB 一旦记录的指令数量超过上限，就会开始从日志中删除指令。对于每条新记录的指令，GDB 会删除最早记录的指令，以保持记录的指令数量不超过上限。（由于删除记录的指令会丢失信息，GDB 通过下文描述的 **stop-at-limit** 选项，让你控制达到上限时会发生什么。）

如果上限是无限或零，GDB 将永远不会从执行日志中删除记录的指令。记录的指令数量仅受可用内存的限制。

show record full insn-number-max

显示全记录方法下要记录的指令数量上限。

set record full stop-at-limit

当录制的指令数量达到限制时，控制完整录制方法的行为。如果为 **ON**（默认值），GDB 会在第一次达到限制时停止，并询问您是否要停止较差的或

continue running it and recording the execution log. If you decide to continue recording, each new recorded instruction will cause the oldest one to be deleted. If this option is OFF, GDB will automatically delete the oldest record to make room for each new one, without asking.

show record full stop-at-limit
Show the current setting of **stop-at-limit**.

set record full memory-query
Control the behavior when GDB is unable to record memory changes caused by an instruction for the **full** recording method. If ON, GDB will query whether to stop the inferior in that case.
If this option is OFF (the default), GDB will automatically ignore the effect of such instructions on memory. Later, when GDB replays this execution log, it will mark the log of this instruction as not accessible, and it will not affect the replay results.

show record full memory-query
Show the current setting of **memory-query**.

The **btrace** record target does not trace data. As a convenience, when replaying, GDB reads read-only memory off the live program directly, assuming that the addresses of the read-only areas don't change. This for example makes it possible to disassemble code while replaying, but not to print variables. In some cases, being able to inspect variables might be useful. You can use the following command for that:

set record btrace replay-memory-access
Control the behavior of the **btrace** recording method when accessing memory during replay. If **read-only** (the default), GDB will only allow accesses to read-only memory. If **read-write**, GDB will allow accesses to read-only and to read-write memory. Beware that the accessed memory corresponds to the live target and not necessarily to the current replay position.

set record btrace cpu identifier
Set the processor to be used for enabling workarounds for processor errata when decoding the trace.

Processor errata are defects in processor operation, caused by its design or manufacture. They can cause a trace not to match the specification. This, in turn, may cause trace decode to fail. GDB can detect erroneous trace packets and correct them, thus avoiding the decoding failures. These corrections are known as *errata workarounds*, and are enabled based on the processor on which the trace was recorded.

By default, GDB attempts to detect the processor automatically, and apply the necessary workarounds for it. However, you may need to specify the processor if GDB does not yet support it. This command allows you to do that, and also allows to disable the workarounds.

The argument *identifier* identifies the CPU and is of the form: **vendor:processor identifier**. In addition, there are two special identifiers, **none** and **auto** (default).

继续运行它并记录执行日志。如果您决定继续录制，每个新的录制指令都会导致最旧的指令被删除。如果此选项为 OFF，GDB 会自动删除最旧的记录以腾出空间给每个新记录，而无需询问。

显示记录-限制停止
显示停止-限制的当前设置。

设置记录-内存查询
控制 GDB 在完整录制方法无法记录指令引起的内存变化时的行为。如果开启，GDB 将查询是否停止下级程序。

如果此选项关闭（默认），GDB 将自动忽略此类指令对内存的影响。稍后，当 GDB 回放此执行日志时，它将标记此指令的日志为不可访问，并且不会影响回放结果。

显示完整内存查询记录
显示当前内存查询的设置。
btrace记录目标不会跟踪数据。为了方便起见，在重放时，GDB 会直接从运行程序中读取只读内存，假设只读区域的地址没有变化。例如，这可以使其在重放时反汇编代码，但不能打印变量。在某些情况下，能够检查变量可能很有用。您可以使用以下命令来实现：

设置记录 **btrace** 重放内存访问
控制**btrace**记录方法在重放期间访问内存的行为。如果设置为只读（默认值），GDB 将只允许访问只读内存。如果设置为读写，GDB 将允许访问只读和读写内存。请注意，访问的内存对应于实时目标，而不一定是当前重放位置。

设置记录 **btrace** *cpu identifier*
将处理器设置为在解码跟踪时启用处理器勘误的工作方式。

处理器勘误是处理器操作中的缺陷，由其设计或制造引起。它们可能导致跟踪与规范不匹配。这反过来又可能导致跟踪解码失败。GDB 可以检测错误的跟踪数据包并纠正它们，从而避免解码失败。这些纠正被称为勘误工作方式，并根据记录跟踪的处理器启用。

默认情况下，GDB 尝试自动检测处理器，并为其应用必要的工作方式。但是，如果 GDB 尚未支持它，您可能需要指定处理器。此命令允许您这样做，并允许禁用工作方式。

参数标识符标识了 CPU，其形式为：**vendor:processor identifier**。此外，还有两个特殊标识符：**none**和**auto**（默认）。

The following vendor identifiers and corresponding processor identifiers are currently supported:

intel family/model[/stepping]

On GNU/Linux systems, the processor *family*, *model*, and *stepping* can be obtained from `/proc/cpuinfo`.

If *identifier* is `auto`, enable errata workarounds for the processor on which the trace was recorded. If *identifier* is `none`, errata workarounds are disabled.

For example, when using an old GDB on a new system, decode may fail because GDB does not support the new processor. It often suffices to specify an older processor that GDB supports.

```
(gdb) info record
Active record target: record-btrace
Recording format: Intel Processor Trace.
Buffer size: 16kB.
Failed to configure the Intel Processor Trace decoder: unknown cpu.
(gdb) set record btrace cpu intel:6/158
(gdb) info record
Active record target: record-btrace
Recording format: Intel Processor Trace.
Buffer size: 16kB.
Recorded 84872 instructions in 3189 functions (0 gaps) for thread 1 (...).
```

show record btrace replay-memory-access

Show the current setting of `replay-memory-access`.

show record btrace cpu

Show the processor to be used for enabling trace decode errata workarounds.

set record btrace bts buffer-size size

set record btrace bts buffer-size unlimited

Set the requested ring buffer size for branch tracing in BTS format. Default is 64KB.

If *size* is a positive number, then GDB will try to allocate a buffer of at least *size* bytes for each new thread that uses the btrace recording method and the BTS format. The actually obtained buffer size may differ from the requested *size*. Use the `info record` command to see the actual buffer size for each thread that uses the btrace recording method and the BTS format.

If *limit* is `unlimited` or zero, GDB will try to allocate a buffer of 4MB.

Bigger buffers mean longer traces. On the other hand, GDB will also need longer to process the branch trace data before it can be used.

show record btrace bts buffer-size size

Show the current setting of the requested ring buffer size for branch tracing in BTS format.

set record btrace pt buffer-size size

set record btrace pt buffer-size unlimited

Set the requested ring buffer size for branch tracing in Intel Processor Trace format. Default is 16KB.

目前支持的以下供应商标识符及其对应的处理器标识符：

intel 家族/型号[/步进]

在GNU/Linux系统上，处理器家族、型号和步进信息可以从`/proc/cpuinfo`中获取。

如果`identifier`是`auto`，将为录制跟踪的处理器启用错误更正工作绕过。如果`identifier`是`none`，则禁用错误更正工作绕过。

例如，当在旧系统上使用一个老的 GDB 时，`decode`可能会失败，因为GDB 不支持新的处理器。通常指定一个 GDB 支持的较旧处理器就足够了。

```
(gdb) info record
Active record target: record-btrace
录制格式：Intel处理器跟踪。
Buffer size: 16kB.
配置 Intel 处理器跟踪解码器失败：未知 CPU。
(gdb) set record btrace cpu intel:6/158
(gdb) info record
Active record target: record-btrace
Recording format: Intel Processor Trace.
Buffer size: 16kB.
为线程 1 (...) 记录了 84872 条指令，涉及 3189 个函数 (0 个间隙)。
```

show record btrace replay-memory-access

显示 `replay-memory-access` 的当前设置。

show record btrace cpu

显示用于启用跟踪解码错误更正工作的处理器。

设置记录 btrace bts 缓冲区大小 *size*

set record btrace bts buffer-size unlimited

设置 BTS 格式下分支跟踪的请求环形缓冲区大小。默认为 64KB。

如果大小是一个正数，那么 GDB 会尝试为每个使用 btrace 记录方法和 BTS 格式的新线程分配至少大小字节的缓冲区。实际获得的缓冲区大小可能与请求的大小不同。使用 `info` 记录命令查看使用 btrace 记录方法和 BTS 格式的每个线程的实际缓冲区大小。

如果限制是 `unlimited` 或零，那么 GDB 会尝试分配 4MB 的缓冲区。更大的缓冲区意味着更长的跟踪记录。另一方面，GDB 也需要更长时间来处理分支跟踪数据，才能使用。

显示记录 btrace bts 缓冲区大小 *size*

显示请求的分支跟踪环形缓冲区大小在 BTS 格式下的当前设置。

设置记录 btrace pt 缓冲区大小 *size*

设置记录 btrace pt 缓冲区大小 无限

为 Intel Processor Trace 格式的分支跟踪设置请求的环形缓冲区大小。默认为 16KB。

If *size* is a positive number, then GDB will try to allocate a buffer of at least *size* bytes for each new thread that uses the btrace recording method and the Intel Processor Trace format. The actually obtained buffer size may differ from the requested *size*. Use the `info record` command to see the actual buffer size for each thread.

If *limit* is `unlimited` or zero, GDB will try to allocate a buffer of 4MB.

Bigger buffers mean longer traces. On the other hand, GDB will also need longer to process the branch trace data before it can be used.

`show record btrace pt buffer-size size`

Show the current setting of the requested ring buffer size for branch tracing in Intel Processor Trace format.

`set record btrace pt event-tracing`

Enable or disable event tracing for branch tracing in Intel Processor Trace format. When enabled, events are recorded during execution as auxiliary information and will be printed during stepping commands and commands displaying the execution history. Changing this setting has no effect on an active recording. The default is off.

`show record btrace pt event-tracing`

Show the current setting of Intel Processor Trace event tracing.

`info record`

Show various statistics about the recording depending on the recording method:

full For the **full** recording method, it shows the state of process record and its in-memory execution log buffer, including:

- Whether in record mode or replay mode.
- Lowest recorded instruction number (counting from when the current execution log started recording instructions).
- Highest recorded instruction number.
- Current instruction about to be replayed (if in replay mode).
- Number of instructions contained in the execution log.
- Maximum number of instructions that may be contained in the execution log.

btrace For the **btrace** recording method, it shows:

- Recording format.
- Number of instructions that have been recorded.
- Number of blocks of sequential control-flow formed by the recorded instructions.
- Whether in record mode or replay mode.

For the **bts** recording format, it also shows:

- Size of the perf ring buffer.

For the **pt** recording format, it also shows:

- Size of the perf ring buffer.

如果大小是一个正数，那么 GDB 会尝试为每个使用 btrace 录制方法和 Intel Processor Trace 格式的新线程分配至少为 *size* 字节大小的缓冲区。实际获得的缓冲区大小可能与请求的大小不同。使用 `info` 记录命令查看每个线程的实际缓冲区大小。

如果限制为无限或零，GDB 将尝试分配4MB的缓冲区。

更大的缓冲区意味着更长的跟踪。另一方面，GDB 也需要更长时间来处理分支跟踪数据，才能使用。

显示记录btrace pt缓冲区大小 *size*

显示Intel处理器跟踪格式中请求的环形缓冲区大小的当前设置。

设置记录btrace pt事件跟踪

启用或禁用 Intel 处理器跟踪格式中的分支跟踪的事件跟踪。启用时，事件将在执行过程中作为辅助信息记录，并在单步执行命令和显示执行历史的命令中打印。更改此设置对活动录制没有影响。默认为禁用。

显示 btrace pt 事件跟踪记录

显示 Intel 处理器跟踪事件跟踪的当前设置。

信息记录

根据录制方法显示关于录制的各种统计信息：

full 对于完整录制方法，它显示进程记录的状态及其内存中的执行日志缓冲区，包括：

- 无论是记录模式还是回放模式。
- 最低记录的指令编号（从当前执行日志开始记录指令时计数）。
- 最高记录的指令编号。
- 当前即将回放的指令（如果在回放模式）。
- 执行日志中包含的指令数量。
- 执行日志可能包含的最大指令数量。

btrace 对于 btrace 录制方法，它显示：

- 录制格式。
- 已录制的指令数量。
- 由已录制指令形成的顺序控制流块数量。

- 是否处于记录模式或重放模式。

对于bts录制格式，它还显示：

- 性能环缓冲区的大小。

对于 pt 录制格式，它还显示：

- 性能环缓冲区的大小。

record delete

When record target runs in replay mode (“in the past”), delete the subsequent execution log and begin to record a new execution log starting from the current address. This means you will abandon the previously recorded “future” and begin recording a new “future”.

record instruction-history

Disassembles instructions from the recorded execution log. By default, ten instructions are disassembled. This can be changed using the `set record instruction-history-size` command. Instructions are printed in execution order.

It can also print mixed source+disassembly if you specify the `/m` or `/s` modifier, and print the raw instructions in hex as well as in symbolic form by specifying the `/r` or `/b` modifier. The behavior of the `/m`, `/s`, `/r`, and `/b` modifiers are the same as for the `disassemble` command (see [disassemble], page 138).

The current position marker is printed for the instruction at the current program counter value. This instruction can appear multiple times in the trace and the current position marker will be printed every time. To omit the current position marker, specify the `/p` modifier.

To better align the printed instructions when the trace contains instructions from more than one function, the function name may be omitted by specifying the `/f` modifier.

Printing auxiliary information is enabled by default and can be omitted with the `/a` modifier.

Speculatively executed instructions are prefixed with ‘?’ . This feature is not available for all recording formats.

There are several ways to specify what part of the execution log to disassemble:

record instruction-history *insn*

Disassembles ten instructions starting from instruction number *insn*.

record instruction-history *insn*, +/-*n*

Disassembles *n* instructions around instruction number *insn*. If *n* is preceded with +, disassembles *n* instructions after instruction number *insn*. If *n* is preceded with -, disassembles *n* instructions before instruction number *insn*.

record instruction-history

Disassembles ten more instructions after the last disassembly.

record instruction-history -

Disassembles ten more instructions before the last disassembly.

record instruction-history *begin*, *end*

Disassembles instructions beginning with instruction number *begin* until instruction number *end*. The instruction number *end* is included.

This command may not be available for all recording methods.

记录删除

当录制目标以重放模式运行（“在过去”）时，删除后续的执行日志并从当前地址开始记录新的执行日志。这意味着您将放弃之前录制的“未来”，并开始录制一个新的“未来”。

记录指令历史

从录制的执行日志中反汇编指令。默认情况下，十条指令被反汇编。这可以使用 `set record instruction-history-size` 命令。指令在执行时打印

按执行顺序。

如果指定 `/m` 或 `/s` 修饰符，它还可以打印混合源+反汇编，并通过指定 `/r` 或 `/b` 修饰符以十六进制和符号形式打印原始指令。`/m`、`/s`、`/r` 和 `/b` 修饰符的行为与 `disassemble` 命令相同（参见[disassemble]，第 138 页）。

当前程序计数器值处的指令的当前位置标记将被打印。此指令可以在跟踪中多次出现，并且每次都会打印当前位置标记。要省略当前位置标记，请指定 `/p` 修饰符。

当跟踪包含来自多个函数的指令时，为了更好地对齐打印的指令，可以通过指定 `/f` 修饰符来省略函数名。

默认情况下启用打印辅助信息，并可通过修饰符（with the/a modifier）省略。

推测执行（speculatively executed）的指令以 ‘?’ 前缀标识。此功能并非所有录制格式都支持。

指定要反汇编（disassemble）执行日志（execution log）的部分有多种方法：

记录指令历史 `record instruction-history insn`

从指定指令编号开始反汇编十条指令
insn.

记录指令历史 `insn, +/-n`

在指令编号 *insn* 附近反汇编 *n* 条指令。如果 *n* 前面带有 +，则反汇编指令编号 *insn* 之后的 *n* 条指令。如果 *n* 前面带有 -，则反汇编指令编号 *insn* 之前的 *n* 条指令。

记录指令历史

在最后一次反汇编之后再反汇编十条指令。

记录指令历史 -

在最后一次反汇编之前再反汇编十条指令。

记录指令历史 `begin, end`

从指令编号 *begin* 开始，直到指令编号 *end* 结束进行反汇编。指令编号 *end* 包含在内。

此命令可能不适用于所有录制方法。

set record instruction-history-size size

set record instruction-history-size unlimited

Define how many instructions to disassemble in the **record instruction-history** command. The default value is 10. A *size* of **unlimited** means unlimited instructions.

show record instruction-history-size

Show how many instructions to disassemble in the **record instruction-history** command.

record function-call-history

Prints the execution history at function granularity. For each sequence of instructions that belong to the same function, it prints the name of that function, the source lines for this instruction sequence (if the */l* modifier is specified), and the instructions numbers that form the sequence (if the */i* modifier is specified). The function names are indented to reflect the call stack depth if the */c* modifier is specified. Printing auxiliary information is enabled by default and can be omitted with the */a* modifier. The */l*, */i*, */a*, and */c* modifiers can be given together.

```
(gdb) list 1, 10
1 void foo (void)
2 {
3 }
4
5 void bar (void)
6 {
7 ...
8 foo ();
9 ...
10 }
(gdb) record function-call-history /ilc
1 bar inst 1,4 at foo.c:6,8
2 foo inst 5,10 at foo.c:2,3
3 bar inst 11,13 at foo.c:9,10
```

By default, ten functions are printed. This can be changed using the **set record function-call-history-size** command. Functions are printed in execution order. There are several ways to specify what to print:

record function-call-history func

Prints ten functions starting from function number *func*.

record function-call-history func, +/-n

Prints *n* functions around function number *func*. If *n* is preceded with *+*, prints *n* functions after function number *func*. If *n* is preceded with *-*, prints *n* functions before function number *func*.

record function-call-history

Prints ten more functions after the last ten-function print.

record function-call-history -

Prints ten more functions before the last ten-function print.

设置记录指令历史大小 *size*

设置记录指令历史大小为无限

定义 how many 指令 to 反汇编 in the 记录指令历史命令。默认值为10。大小为无限表示无限指令。

显示记录指令历史大小

Show how many 说明 在录制中拆卸指令历史命令。

记录函数调用历史

在函数粒度上打印执行历史。对于属于同一函数的指令序列，它会打印该函数的名称、此指令序列的源代码行（如果指定了*/l*修饰符），以及构成该序列的指令编号（如果指定了*/i*修饰符）。如果指定了*/c*修饰符，函数名称会缩进以反映调用栈深度。默认情况下会打印辅助信息，可以使用*/a*修饰符省略。*/l*、*/i*、*/a*和*/c*修饰符可以一起使用。

```
(gdb) 列表 1, 10
1 void foo (void)
2 {
3 }
4
5 void bar (void)
6 {
7 ...
8 foo ();
9 ...
10 }
(gdb) 记录 函数调用历史 /ilc
1 bar 实例化 1,4 at foo.c:6,8
2 foo 实例化 5,10 at foo.c:2,3
3 bar 实例化 11,13 at foo.c:9,10
```

默认情况下，会打印十个函数。可以使用 ``set record function-call-history-size`` 命令来更改此设置。函数按执行顺序打印。有几种方法可以指定要打印的内容：

记录函数调用历史 *func*

从 *func* 号函数开始打印十个函数。

record function-call-history func, +/-n

打印 *func* 号函数周围的 *n* 个函数。如果 *n* 前面有 *+*，则打印 *func* 号函数之后的 *n* 个函数。如果 *n* 前面有 *-*，则打印 *func* 号函数之前的 *n* 个函数。

记录函数调用历史

在最后十个函数打印之后，再打印十个函数。

记录函数调用历史 -

在最后十个函数打印之前，再打印十个更多函数。

record function-call-history *begin, end*

Prints functions beginning with function number *begin* until function number *end*. The function number *end* is included.

This command may not be available for all recording methods.

set record function-call-history-size *size*

set record function-call-history-size unlimited

Define how many functions to print in the **record function-call-history** command. The default value is 10. A size of **unlimited** means unlimited functions.

show record function-call-history-size

Show how many functions to print in the **record function-call-history** command.

记录函数调用历史 *begin, end*

打印从函数编号 *begin* 开始到函数编号 *end* 结束的所有函数。函数编号 *end* 将被包含在内。

此命令可能并非所有录制方法都可用。

set record function-call-history-size *size*

set record function-call-history-size unlimited

定义 **recordfunction-call-history** 命令中要打印的函数数量。默认值为 10。

unlimited 大小表示无限数量的函数。

show record function-call-history-size

显示记录函数调用历史命令中要打印多少个函数。

8 Examining the Stack

When your program has stopped, the first thing you need to know is where it stopped and how it got there.

Each time your program performs a function call, information about the call is generated. That information includes the location of the call in your program, the arguments of the call, and the local variables of the function being called. The information is saved in a block of data called a *stack frame*. The stack frames are allocated in a region of memory called the *call stack*.

When your program stops, the GDB commands for examining the stack allow you to see all of this information.

One of the stack frames is *selected* by GDB and many GDB commands refer implicitly to the selected frame. In particular, whenever you ask GDB for the value of a variable in your program, the value is found in the selected frame. There are special GDB commands to select whichever frame you are interested in. See Section 8.3 [Selecting a Frame], page 119.

When your program stops, GDB automatically selects the currently executing frame and describes it briefly, similar to the `frame` command (see Section 8.4 [Information about a Frame], page 121).

8.1 Stack Frames

The call stack is divided up into contiguous pieces called *stack frames*, or *frames* for short; each frame is the data associated with one call to one function. The frame contains the arguments given to the function, the function's local variables, and the address at which the function is executing.

When your program is started, the stack has only one frame, that of the function `main`. This is called the *initial* frame or the *outermost* frame. Each time a function is called, a new frame is made. Each time a function returns, the frame for that function invocation is eliminated. If a function is recursive, there can be many frames for the same function. The frame for the function in which execution is actually occurring is called the *innermost* frame. This is the most recently created of all the stack frames that still exist.

Inside your program, stack frames are identified by their addresses. A stack frame consists of many bytes, each of which has its own address; each kind of computer has a convention for choosing one byte whose address serves as the address of the frame. Usually this address is kept in a register called the *frame pointer register* (see Section 10.14 [Registers], page 181) while execution is going on in that frame.

GDB labels each existing stack frame with a *level*, a number that is zero for the innermost frame, one for the frame that called it, and so on upward. These level numbers give you a way of designating stack frames in GDB commands. The terms *frame number* and *frame level* can be used interchangeably to describe this number.

Some compilers provide a way to compile functions so that they operate without stack frames. (For example, the GCC option

```
'-fomit-frame-pointer'
```

generates functions without a frame.) This is occasionally done with heavily used library functions to save the frame setup time. GDB has limited facilities for dealing with

8 检查堆栈

当你的程序停止时，你需要知道的第一件事是它停止的位置以及它是如何到达那里的。

每次你的程序执行函数调用时，都会生成关于调用的信息。这些信息包括调用在程序中的位置、调用的参数以及被调用函数的局部变量。这些信息保存在一个称为堆栈帧的数据块中。堆栈帧在称为调用堆栈的内存区域中分配。

当你的程序停止时，GDB 检查堆栈的命令允许你查看所有这些信息。

其中一个堆栈帧由 GDB 选择，许多 GDB 命令隐式地引用选定的帧。特别是，每当你要求 GDB 获取程序中变量的值时，该值就在选定的帧中。有特殊的 GDB 命令来选择你感兴趣的任何帧。参见第 8.3 [选择帧], 页 119。

当您的程序停止时，GDB 会自动选择当前正在执行的栈帧并简要描述它，类似于栈帧命令（参见第8.4 [节“栈帧信息”，第121页）。

8.1 栈帧

调用栈被划分为连续的片段，称为栈帧，或简称为帧；每个帧是与对单个函数的一次调用相关的数据。帧包含传递给函数的参数、函数的局部变量以及函数正在执行的地址。

当您的程序启动时，栈只有一个帧，即函数main的帧。这称为初始帧或最外层帧。每次调用函数时，都会创建一个新的帧。每次函数返回时，该函数调用的帧就会被消除。如果一个函数是递归的，那么同一个函数可能会有多个帧。实际正在执行的函数的帧称为最内层帧。这是所有仍然存在的栈帧中最最近创建的帧。

在你的程序中，栈帧通过它们的地址来标识。一个栈帧由许多字节组成，每个字节都有自己的地址；每种计算机都有一个约定，用于选择一个字节，其地址作为栈帧的地址。通常，这个地址在执行该栈帧时保存在一个称为栈帧指针寄存器（参见第10.14 [寄存器], 页 181）的寄存器中。

GDB 为每个现有的栈帧分配一个级别，这是一个数字，最内层栈帧为0，调用它的栈帧为1，等等向上。这些级别数字为你提供了一种在 GDB 命令中指定栈帧的方法。可以使用帧编号和帧级别这两个术语来互换地描述这个数字。

一些编译器提供了一种编译函数的方法，以便它们在没有栈帧的情况下运行。（例如，GCC 选项

```
'-fomit-frame-pointer'
```

生成无需栈帧的函数。）这有时用于处理频繁使用的库函数，以节省栈帧设置时间。GDB 处理这些函数调用的功能有限，

these function invocations. If the innermost function invocation has no stack frame, GDB nevertheless regards it as though it had a separate frame, which is numbered zero as usual, allowing correct tracing of the function call chain. However, GDB has no provision for frameless functions elsewhere in the stack.

8.2 Backtraces

A backtrace is a summary of how your program got where it is. It shows one line per frame, for many frames, starting with the currently executing frame (frame zero), followed by its caller (frame one), and on up the stack.

To print a backtrace of the entire stack, use the **backtrace** command, or its alias **bt**. This command will print one line per frame for frames in the stack. By default, all stack frames are printed. You can stop the backtrace at any time by typing the system interrupt character, normally **Ctrl-c**.

backtrace [*option*]... [*qualifier*]... [*count*]

bt [*option*]... [*qualifier*]... [*count*]

Print the backtrace of the entire stack.

The optional *count* can be one of the following:

n

n Print only the innermost *n* frames, where *n* is a positive number.

-n

-n Print only the outermost *n* frames, where *n* is a positive number.

Options:

-full Print the values of the local variables also. This can be combined with the optional *count* to limit the number of frames shown.

-no-filters

Do not run Python frame filters on this backtrace. See Section 23.3.2.10 [Frame Filter API], page 436, for more information. Additionally use [disable frame-filter all], page 124, to turn off all frame filters. This is only relevant when GDB has been configured with Python support.

-hide A Python frame filter might decide to “elide” some frames. Normally such elided frames are still printed, but they are indented relative to the filtered frames that cause them to be elided. The **-hide** option causes elided frames to not be printed at all.

The **backtrace** command also supports a number of options that allow overriding relevant global print settings as set by **set backtrace** and **set print** subcommands:

-past-main [on|off]

Set whether backtraces should continue past **main**. Related setting: [set backtrace past-main], page 118.

-past-entry [on|off]

Set whether backtraces should continue past the entry point of a program. Related setting: [set backtrace past-entry], page 118.

如果最内层的函数调用没有栈帧，GDB 仍然将其视为具有独立的栈帧，通常编号为零，从而允许正确追踪函数调用链。然而，GDB 在栈的其他位置没有处理无栈帧函数的机制。

8.2 调试回溯

调试回溯是一个总结，说明你的程序是如何到达当前状态的。它为每个栈帧显示一行，包含多个栈帧，从当前正在执行的栈帧（栈帧零）开始，然后是其调用者（栈帧一），并一直向上追溯到栈顶。

要打印整个栈的调试回溯，请使用 **backtrace** 命令，或其别名 **bt**。该命令将为栈中的每个栈帧打印一行。默认情况下，将打印所有栈帧。你可以通过输入系统中断字符（通常是 **Ctrl-c**）随时停止调试回溯。

backtrace [*option*]... [*qualifier*]... [*count*]

bt [*option*]... [*qualifier*]... [*count*]

打印整个栈的调试回溯。

可选的 *count* 可以是以下值之一：

n

n 仅打印最内层 *n* 帧，其中 *n* 是一个正数。

-n

-n 仅打印最外层 *n* 帧，其中 *n* 是一个正数。

选项：

-full 同时打印局部变量的值。这可以与可选的计数选项结合使用，以限制显示的帧数。

-no-filters

不要在这个回溯上运行 Python 帧过滤器。参见第 23.3.2.10 [帧过滤器 API], 页 436, 获取更多信息。此外使用 [disable frame-filter all], 页 124, 以关闭所有帧过滤器。这仅在与 GDB 配置了 Python 支持时相关。已配置 Python 支持。

-hide

Python 帧过滤器可能会决定“省略”某些帧。通常情况下，被省略的帧仍然会打印，但它们相对于导致它们被省略的已过滤帧进行缩进。**-hide** 选项会导致被省略的帧完全不被打印。

回溯命令还支持多个选项，允许覆盖由 **set backtrace** 和 **set print** 子命令设置的相应全局打印设置：

-past-main [on|off]

设置是否应继续回溯至 **main**。相关设置: [设置回溯 past-main], 第 118 页。

-past-entry [on|off]

设置是否应让回溯继续执行到程序入口点之后。相关设置: [设置回溯越过入口], 页码 118。

-entry-values no|only|preferred|if-needed|both|compact|default
Set printing of function arguments at function entry. Related setting: [set print entry-values], page 163.

-frame-arguments all|scalars|none
Set printing of non-scalar frame arguments. Related setting: [set print frame-arguments], page 162.

-raw-frame-arguments [on|off]
Set whether to print frame arguments in raw form. Related setting: [set print raw-frame-arguments], page 163.

-frame-info auto|source-line|location|source-and-location|location-and-address|short-location
Set printing of frame information. Related setting: [set print frame-info], page 165.

The optional *qualifier* is maintained for backward compatibility. It can be one of the following:

full Equivalent to the **-full** option.

no-filters
Equivalent to the **-no-filters** option.

hide Equivalent to the **-hide** option.

The names **where** and **info stack** (abbreviated **info s**) are additional aliases for **backtrace**.

In a multi-threaded program, GDB by default shows the backtrace only for the current thread. To display the backtrace for several or all of the threads, use the command **thread apply** (see Section 4.10 [Threads], page 48). For example, if you type **thread apply all backtrace**, GDB will display the backtrace for all the threads; this is handy when you debug a core dump of a multi-threaded program.

Each line in the backtrace shows the frame number and the function name. The program counter value is also shown—unless you use **set print address off**. The backtrace also shows the source file name and line number, as well as the arguments to the function. The program counter value is omitted if it is at the beginning of the code for that line number.

Here is an example of a backtrace. It was made with the command **'bt 3'**, so it shows the innermost three frames.

```
#0 m4_traceon (obs=0x24eb0, argc=1, argv=0x2b8c8)
   at builtin.c:993
#1 0x6e38 in expand_macro (sym=0x2b600, data=...) at macro.c:242
#2 0x6840 in expand_token (obs=0x0, t=177664, td=0xf7fffb08)
   at macro.c:71
(More stack frames follow...)
```

The display for frame zero does not begin with a program counter value, indicating that your program has stopped at the beginning of the code for line 993 of **builtin.c**.

The value of parameter **data** in frame 1 has been replaced by **....** By default, GDB prints the value of a parameter only if it is a scalar (integer, pointer, enumeration, etc). See command **set print frame-arguments** in Section 10.9 [Print Settings], page 159, for more

-entry-values no|only|preferred|if-needed|both|compact|default 设置在函数入口处打印函数参数。相关设置: [set print entry-values], 页163。

-frame-arguments all|scalars|none
设置是否应打印非标量帧参数。相关设置: [设置打印帧参数], 页码162。

-raw-frame-arguments [on|off]
设置是否以原始形式打印帧参数。相关设置: [set print raw-frame-arguments], 页163。

-frame-info auto|source-line|location|source-and-location|location-and-address|short-location
设置帧信息的打印。相关设置: [set print frame-info], 页165。

可选限定符为向后兼容而保留。它可以是一个以下值之一:

full 等同于 **-full** 选项。

无过滤 相当于 **-no-filters** 选项。

隐藏 Equivalent to the **-hide** 选项。

where 和 **info stack** (缩写 **info s**) 是 **backtrace** 的额外别名。

在多线程程序中, GDB 默认仅显示当前线程的 backtrace。要显示多个或所有线程的 backtrace, 请使用命令 **thread apply** (参见第 4.10 [线程], 页 48)。例如, 如果你输入 **thread apply all backtrace**, GDB 将显示所有线程的 backtrace; 这在调试多线程程序的 core dump 时很有用。

backtrace 中的每一行显示帧号和函数名。还会显示程序计数器值——除非你使用 **set print address off**。backtrace 还会显示源文件名和行号, 以及函数的参数。如果程序计数器值是该行代码的开头, 则会省略该值。

这是一个回溯示例。它是使用命令 **'bt 3'** 生成的, 因此显示了最内层的三个帧。

```
#0 m4_traceon (obs=0x24eb0, argc=1, argv=0x2b8c8)
   at builtin.c:993
#1 0x6e38 in expand_macro (sym=0x2b600, data=...) at macro.c:242
#2 0x6840 in expand_token (obs=0x0, t=177664, td=0xf7fffb08)
   at macro.c:71
(More stack frames follow...)
```

帧零的显示内容不以程序计数器值开头, 表明您的程序在builtin.c文件的第993行代码开头处停止。

参数 **data** 在第 1 帧中的值已被替换为 **....** 默认情况下, GDB 仅在参数是标量 (整数、指针、枚举等) 时才打印其值。有关更多信息, 请参阅第 10.9 [打印设置], 页面中的命令

set print frame-arguments。

details on how to configure the way function parameter values are printed. The command `set print frame-info` (see Section 10.9 [Print Settings], page 159) controls what frame information is printed.

If your program was compiled with optimizations, some compilers will optimize away arguments passed to functions if those arguments are never used after the call. Such optimizations generate code that passes arguments through registers, but doesn't store those arguments in the stack frame. GDB has no way of displaying such arguments in stack frames other than the innermost one. Here's what such a backtrace might look like:

```
#0  m4_traceon (obs=0x24eb0, argc=1, argv=0x2b8c8)
    at builtin.c:993
#1  0x6e38 in expand_macro (sym=<optimized out>) at macro.c:242
#2  0x6840 in expand_token (obs=0x0, t=<optimized out>, td=0xf7ffb08)
    at macro.c:71
(More stack frames follow...)
```

The values of arguments that were not saved in their stack frames are shown as '`<optimized out>`'.

If you need to display the values of such optimized-out arguments, either deduce that from other variables whose values depend on the one you are interested in, or recompile without optimizations.

Most programs have a standard user entry point—a place where system libraries and startup code transition into user code. For C this is `main`¹. When GDB finds the entry function in a backtrace it will terminate the backtrace, to avoid tracing into highly system-specific (and generally uninteresting) code.

If you need to examine the startup code, or limit the number of levels in a backtrace, you can change this behavior:

```
set backtrace past-main
set backtrace past-main on
    Backtraces will continue past the user entry point.

set backtrace past-main off
    Backtraces will stop when they encounter the user entry point. This is the
    default.

show backtrace past-main
    Display the current user entry point backtrace policy.

set backtrace past-entry
set backtrace past-entry on
    Backtraces will continue past the internal entry point of an application. This
    entry point is encoded by the linker when the application is built, and is likely
    before the user entry point main (or equivalent) is called.

set backtrace past-entry off
    Backtraces will stop when they encounter the internal entry point of an appli-
    cation. This is the default.
```

¹ Note that embedded programs (the so-called “free-standing” environment) are not required to have a `main` function as the entry point. They could even have multiple entry points.

关于如何配置函数参数值打印方式的详细信息。命令 `set print frame-info` (参见第10.9 [打印设置], 页159) 控制打印哪些帧信息。

如果你的程序经过优化编译, 某些编译器会优化掉在函数调用后未被使用的参数。这种优化会生成通过寄存器传递参数但不在栈帧中存储这些参数的代码。GDB 无法在除最内层栈帧以外的栈帧中显示此类参数。以下是此类回溯可能的样子:

```
#0  m4_traceon (obs=0x24eb0, argc=1, argv=0x2b8c8)
    at builtin.c:993
#1  0x6e38 in expand_macro (sym=<optimized out>) at macro.c:242
#2  0x6840 in expand_token (obs=0x0, t=<optimized out>, td=0xf7ffb08)
    at macro.c:71
(More stack frames follow...)
```

未在栈帧中保存的参数值会显示为 '`<优化掉>`'。

如果你需要显示此类优化掉参数的值, 可以从其他依赖你感兴趣值的变量中推断, 或者重新编译不进行优化。

大多数程序都有一个标准用户入口点——系统库和启动代码过渡到用户代码的地方。对于C语言, 这是`main`¹。当 GDB 在回溯中找到入口函数时, 它会终止回溯, 以避免追踪到高度系统特定 (通常不感兴趣) 的代码。

如果您需要检查启动代码, 或限制回溯的层级数量, 您可以更改此行为:

```
设置回溯 past-main
设置回溯 past-main 开启
    回溯将继续经过用户入口点。

设置回溯 past-main 关闭
    当回溯遇到用户入口点时会停止。这是默认行为。

显示回溯 past-main
    显示当前用户入口点回溯策略。

设置回溯 past-entry
设置回溯 past-entry 开启
    回溯将越过应用程序的内部入口点。该入口点由链接器在应用程序构建时编码,
    通常在用户入口点 main (或等效) 被调用之前。

设置回溯 past-entry 关闭
    回溯在遇到应用程序的内部入口点时将停止。
    功能。这是默认设置。
```

¹ 请注意, 嵌入式程序 (所谓的 “独立式” 环境) 不需要将 `main` 函数作为入口点。它们甚至可以有多个入口点。

`show backtrace past-entry`
Display the current internal entry point backtrace policy.

`set backtrace limit n`
`set backtrace limit 0`
`set backtrace limit unlimited`
Limit the backtrace to *n* levels. A value of `unlimited` or zero means unlimited levels.

`show backtrace limit`
Display the current limit on backtrace levels.

You can control how file names are displayed.

`set filename-display`
`set filename-display relative`
Display file names relative to the compilation directory. This is the default.

`set filename-display basename`
Display only basename of a filename.

`set filename-display absolute`
Display an absolute filename.

`show filename-display`
Show the current way to display filenames.

8.3 Selecting a Frame

Most commands for examining the stack and other data in your program work on whichever stack frame is selected at the moment. Here are the commands for selecting a stack frame; all of them finish by printing a brief description of the stack frame just selected.

`frame [ frame-selection-spec ]`
`f [ frame-selection-spec ]`
The `frame` command allows different stack frames to be selected. The `frame-selection-spec` can be any of the following:

num
`level num` Select frame level *num*. Recall that frame zero is the innermost (currently executing) frame, frame one is the frame that called the innermost one, and so on. The highest level frame is usually the one for `main`.
As this is the most common method of navigating the frame stack, the string `level` can be omitted. For example, the following two commands are equivalent:
`(gdb) frame 3`
`(gdb) frame level 3`

`address stack-address`
Select the frame with stack address *stack-address*. The *stack-address* for a frame can be seen in the output of `info frame`, for example:
`(gdb) info frame`

显示回溯 `past-entry`
显示当前内部入口点回溯策略。

设置回溯限制 *n*
设置回溯限制 0
设置回溯限制 `unlimited`
将回溯限制为 *n* 级。 `unlimited` 或零值表示无限制级别。

显示回溯限制
显示当前回溯级别的限制。

您可以控制文件名如何显示。

`set filename-display`
设置 `filename-display` 相对
相对于编译目录显示文件名。这是默认设置。

设置 `filename-display` 基名
仅显示文件名的基本名。

设置 `filename-display` 为绝对值
显示绝对文件名。

显示 `filename-display`
显示当前文件名显示方式。

8.3 选择框架

用于检查程序中堆栈和其他数据的命令，都作用于当前选中的堆栈帧。以下是用于选择堆栈帧的命令；所有这些命令最后都会打印出刚刚选中的堆栈帧的简要描述。

`frame [ frame-selection-spec ]`
`f [ frame-selection-spec ]`
frame命令允许选择不同的堆栈帧。堆栈帧选择规范可以是以下任何一种：

num
`level num` 选择堆栈帧的层级*num*。回想一下，堆栈帧零是最内层的（当前正在执行的帧），堆栈帧一调用的是最内层的帧，以此类推。最高的层级帧通常是`main`的帧。
由于这是导航堆栈帧最常用的方法，字符串级别可以省略。例如，以下两个命令是等效的：
`(gdb) frame 3`
`(gdb) frame level 3`

`address stack-address`
选择具有栈地址 *stack-address* 的帧。帧的栈地址 *stack-address* 可以在 `info frame` 的输出中看到，例如：
`(gdb) info frame`

```
Stack level 1, frame at 0x7fffffffda30:
rip = 0x40066d in b (amd64-entry-value.cc:59); saved rip 0x4004c5
tail call frame, caller of frame at 0x7fffffffda30
source language c++.
Arglist at unknown address.
Locals at unknown address, Previous frame's sp is 0x7fffffffda30
```

The *stack-address* for this frame is 0x7fffffffda30 as indicated by the line:

```
Stack level 1, frame at 0x7fffffffda30:
```

function *function-name*

Select the stack frame for function *function-name*. If there are multiple stack frames for function *function-name* then the inner most stack frame is selected.

view *stack-address* [*pc-addr*]

View a frame that is not part of GDB's backtrace. The frame viewed has stack address *stack-addr*, and optionally, a program counter address of *pc-addr*.

This is useful mainly if the chaining of stack frames has been damaged by a bug, making it impossible for GDB to assign numbers properly to all frames. In addition, this can be useful when your program has multiple stacks and switches between them.

When viewing a frame outside the current backtrace using **frame view** then you can always return to the original stack using one of the previous stack frame selection instructions, for example **frame level 0**.

- up *n*** Move *n* frames up the stack; *n* defaults to 1. For positive numbers *n*, this advances toward the outermost frame, to higher frame numbers, to frames that have existed longer.
- down *n*** Move *n* frames down the stack; *n* defaults to 1. For positive numbers *n*, this advances toward the innermost frame, to lower frame numbers, to frames that were created more recently. You may abbreviate **down** as **do**.

All of these commands end by printing two lines of output describing the frame. The first line shows the frame number, the function name, the arguments, and the source file and line number of execution in that frame. The second line shows the text of that source line.

For example:

```
(gdb) up
#1 0x22f0 in main (argc=1, argv=0xf7ffbf4, env=0xf7ffbf4)
    at env.c:10
10      read_input_file (argv[i]);
```

After such a printout, the **list** command with no arguments prints ten lines centered on the point of execution in the frame. You can also edit the program at the point of execution with your favorite editing program by typing **edit**. See Section 9.1 [Printing Source Lines], page 127, for details.

```
栈层级 1, 位于 0x7fffffffda30 的帧:
rip = 0x40066d in b (amd64-entry-value.cc:59); 保存的 rip 0x4004c5 尾调用帧,
调用帧位于 0x7fffffffda30
source language c++.
参数列表位于未知地址。
Locals at unknown address, Previous frame's sp is 0x7fffffffda30
```

此帧的栈地址为 0x7fffffffda30, 如该行所示:

栈层级 1, 位于 0x7fffffffda30 的帧:

函数 *function-name*

选择函数 *function-name* 的栈帧。如果函数 *function-name* 有多个栈帧, 则选择最内层的栈帧。

查看 *stack-address* [*pc-addr*]

查看不属于 GDB 调用栈的一部分的栈帧。所查看的栈帧具有栈地址 *stack-addr*, 并且可选地, 具有程序计数器地址 *pc-addr*。

这主要在栈帧链因错误而损坏时有用, 导致 GDB 无法为所有栈帧正确分配编号。此外, 当您的程序有多个栈并在它们之间切换时, 这也很有用。

使用 **frame view** 查看当前调用栈之外的栈帧时, 您始终可以使用先前的栈帧选择指令之一返回原始栈, 例如 **frame level 0**。

up *n* 向上移动 *n* 帧, *n* 默认为 1。对于正数 *n*, 这将向最外层帧、更高帧号、存在时间更长的帧推进。

down *n* 向下移动 *n* 帧, *n* 默认为 1。对于正数 *n*, 这将向最内层帧、更低帧号、创建时间更近的帧推进。你可以将 **down** 缩写为 **do**。

所有这些命令都以打印两条描述该帧的输出结束。第一行显示帧号、函数名、参数、以及该帧中执行的源文件和行号。第二行显示该源行的文本。

例如:

```
(gdb) up #1 0x22f0 in main (argc=1, argv=0xf7ffbf4, env=0xf7ffbf4)
    at env.c:10 10 读取_输入_文件 (argv[i]);
```

在打印输出之后, 不带参数的列表命令会在执行点所在的框架中将十行内容居中显示。您还可以通过输入 **edit** 来使用您喜欢的编辑程序在执行点处编辑程序。有关详细信息, 请参阅第 9.1 [打印源代码行], 页 127。

`select-frame [ frame-selection-spec ]`

The `select-frame` command is a variant of `frame` that does not display the new frame after selecting it. This command is intended primarily for use in GDB command scripts, where the output might be unnecessary and distracting. The *frame-selection-spec* is as for the `frame` command described in Section 8.3 [Selecting a Frame], page 119.

`up-silently n`

`down-silently n`

These two commands are variants of `up` and `down`, respectively; they differ in that they do their work silently, without causing display of the new frame. They are intended primarily for use in GDB command scripts, where the output might be unnecessary and distracting.

8.4 Information About a Frame

There are several other commands to print information about the selected stack frame.

`frame`

`f`

When used without any argument, this command does not change which frame is selected, but prints a brief description of the currently selected stack frame. It can be abbreviated `f`. With an argument, this command is used to select a stack frame. See Section 8.3 [Selecting a Frame], page 119.

`info frame`

`info f`

This command prints a verbose description of the selected stack frame, including:

- the address of the frame
- the address of the next frame down (called by this frame)
- the address of the next frame up (caller of this frame)
- the language in which the source code corresponding to this frame is written
- the address of the frame’s arguments
- the address of the frame’s local variables
- the program counter saved in it (the address of execution in the caller frame)
- which registers were saved in the frame

The verbose description is useful when something has gone wrong that has made the stack format fail to fit the usual conventions.

`info frame [ frame-selection-spec ]`

`info f [ frame-selection-spec ]`

Print a verbose description of the frame selected by *frame-selection-spec*. The *frame-selection-spec* is the same as for the `frame` command (see Section 8.3 [Selecting a Frame], page 119). The selected frame remains unchanged by this command.

`info args [-q]`

Print the arguments of the selected frame, each on a separate line.

`select-frame [ frame-selection-spec ]`

选择框架命令是框架的一种变体，在选择后不会显示新框架。此命令主要用于在GDB 命令脚本中使用，在这些脚本中，输出可能是多余的并且会分散注意力。框架选择规范与第8.3节[选择框架], 第119页中描述的框架命令相同。

无声上移 *n*

无声下移 *n*

这两个命令分别是 `up` 和 `down` 的变体；它们的不同之处在于会静默执行，不会显示新帧。它们主要用于在 GDB 命令脚本中使用，因为输出可能是不必要的且会分散注意力。

8.4 帧信息

还有几个命令可以打印关于选定堆栈帧的信息。

框架

`f`

当不使用任何参数时，此命令不会更改选定的框架，但会打印当前选定栈框架的简要描述。它可以缩写为 `f`。带参数时，此命令用于选择栈框架。参见第 8.3 [选择框架],页 119。

`info 框架`

`info f` 此命令打印选定栈框架的详细描述，包括：

- 框架的地址
- 此框架调用的下一个框架的地址（称为该框架）
- 上一层帧的地址（当前帧的调用者）
- 该帧对应的源代码所写的语言
- 该帧参数的地址
- 该帧局部变量的地址
- 其中保存的程序计数器（调用者帧中的执行地址）

- 该帧中保存了哪些寄存器

当出现导致栈格式不符合常规约定的问题时，冗长的描述很有用。

`info frame [ frame-selection-spec ]`

`info f [ frame-selection-spec ]`

打印由frame选择规范指定的帧的冗长描述。frame选择规范与frame命令相同（参见第8.3节[选择帧], 页119）。此命令不会更改选定的帧。

`info 参数 [-q]`

打印选中框架的参数，每个参数占一行。

The optional flag ‘-q’, which stands for ‘quiet’, disables printing header information and messages explaining why no argument have been printed.

info args [-q] [-t *type_regex*] [*regex*]

Like *info args*, but only print the arguments selected with the provided *regex*(s).

If *regex* is provided, print only the arguments whose names match the regular expression *regex*.

If *type_regex* is provided, print only the arguments whose types, as printed by the *what is* command, match the regular expression *type_regex*. If *type_regex* contains space(s), it should be enclosed in quote characters. If needed, use backslash to escape the meaning of special characters or quotes.

If both *regex* and *type_regex* are provided, an argument is printed only if its name matches *regex* and its type matches *type_regex*.

info locals [-q]

Print the local variables of the selected frame, each on a separate line. These are all variables (declared either static or automatic) accessible at the point of execution of the selected frame.

The optional flag ‘-q’, which stands for ‘quiet’, disables printing header information and messages explaining why no local variables have been printed.

info locals [-q] [-t *type_regex*] [*regex*]

Like *info locals*, but only print the local variables selected with the provided *regex*(s).

If *regex* is provided, print only the local variables whose names match the regular expression *regex*.

If *type_regex* is provided, print only the local variables whose types, as printed by the *what is* command, match the regular expression *type_regex*. If *type_regex* contains space(s), it should be enclosed in quote characters. If needed, use backslash to escape the meaning of special characters or quotes.

If both *regex* and *type_regex* are provided, a local variable is printed only if its name matches *regex* and its type matches *type_regex*.

The command *info locals -q -t type_regex* can usefully be combined with the commands *frame apply* and *thread apply*. For example, your program might use Resource Acquisition Is Initialization types (RAII) such as *lock_something_t*: each local variable of type *lock_something_t* automatically places a lock that is destroyed when the variable goes out of scope. You can then list all acquired locks in your program by doing

```
thread apply all -s frame apply all -s info locals -q -t lock_something_t
or the equivalent shorter form
tfaas i lo -q -t lock_something_t
```

8.5 Applying a Command to Several Frames.

frame apply [all | count | -count | level level...] [option]... command

The *frame apply* command allows you to apply the named *command* to one or more frames.

可选标志 ‘-q’, 代表 ‘安静’, 禁用打印头部信息和解释为什么没有打印参数的消息。

信息参数 [-q] [-t *type_regex*] [*regex*]

类似于 *info args*, 但仅打印使用提供的正则表达式(s)选择的参数。

如果提供了正则表达式, 则仅打印名称与正则表达式匹配的参数。

如果提供了type正则表达式, 则仅打印其类型与what is命令打印的类型匹配的参数。如果type正则表达式包含空格, 则应将其用引号字符括起来。如有必要, 使用反斜杠转义特殊字符或引号的意义。

如果同时提供了正则表达式和type正则表达式, 则只有当参数名称与正则表达式匹配且其类型与type正则表达式匹配时, 才会打印该参数。

info locals [-q]

打印选定框架的局部变量, 每个变量占一行。这些都是在选定框架执行点可访问的变量 (无论声明为静态还是自动)。

可选标志 ‘-q’, 代表 ‘安静’, 禁用打印头部信息和解释为什么没有打印局部变量的消息。

info locals [-q] [-t *type_regex*] [*regex*]

类似于 *info locals*, 但仅打印使用提供的正则表达式 (regex) 选择的局部变量。

如果提供了正则表达式, 则仅打印名称与正则表达式匹配的局部变量。

如果提供了 type regex 类型, 则仅打印那些其类型与 what is 命令打印的类型匹配的局部变量。如果 type regex 包含空格, 则应将其括在引号字符中。如有必要, 使用反斜杠来转义特殊字符或引号的含义。

如果同时提供了regex和type regex, 只有当局部变量的名称匹配regex且类型匹配type regex时, 才会打印该局部变量。

命令 *info locals -q -t type_regex* 可以有效地与命令 *frame apply* 和 *thread apply* 结合使用。例如, 您的程序可能使用资源获取即初始化类型 (RAII) 的锁, 例如 *lock_something_t*: 每个类型为 *lock_something_t* 的局部变量都会自动获取一个锁, 当变量超出作用域时该锁会被销毁。然后您可以通过执行来列出程序中所有已获取的锁。

线程应用所有 -s 帧应用所有 -s 信息局部变量 -q -t 锁_something_t 或等效的更短形式

```
tfaas i lo -q -t lock_something_t
```

8.5 将命令应用于多个帧。

frame apply [全部 | count | -count | 级别 level...] [option]... commandThe fr

ame apply 命令允许您将命名命令应用于一个或多个框架。

- all** Specify **all** to apply *command* to all frames.
- count** Use *count* to apply *command* to the innermost *count* frames, where *count* is a positive number.
- count** Use *-count* to apply *command* to the outermost *count* frames, where *count* is a positive number.
- level** Use **level** to apply *command* to the set of frames identified by the *level* list. *level* is a frame level or a range of frame levels as *level1-level2*. The frame level is the number shown in the first field of the ‘**backtrace**’ command output. E.g., ‘2-4 6-8 3’ indicates to apply *command* for the frames at levels 2, 3, 4, 6, 7, 8, and then again on frame at level 3.

Note that the frames on which **frame apply** applies a command are also influenced by the **set backtrace** settings such as **set backtrace past-main** and **set backtrace limit N**. See Section 8.2 [Backtraces], page 116.

The **frame apply** command also supports a number of options that allow overriding relevant **set backtrace** settings:

- past-main [on|off]**
Whether backtraces should continue past **main**. Related setting: **[set backtrace past-main]**, page 118.
- past-entry [on|off]**
Whether backtraces should continue past the entry point of a program. Related setting: **[set backtrace past-entry]**, page 118.

By default, GDB displays some frame information before the output produced by *command*, and an error raised during the execution of a *command* will abort **frame apply**. The following options can be used to fine-tune these behaviors:

- c** The flag **-c**, which stands for ‘**continue**’, causes any errors in *command* to be displayed, and the execution of **frame apply** then continues.
- s** The flag **-s**, which stands for ‘**silent**’, causes any errors or empty output produced by a *command* to be silently ignored. That is, the execution continues, but the frame information and errors are not printed.
- q** The flag **-q** (‘**quiet**’) disables printing the frame information.

The following example shows how the flags **-c** and **-s** are working when applying the command **p j** to all frames, where variable **j** can only be successfully printed in the outermost **#1 main** frame.

- all** 指定 **all** 将应用命令到所有框架。
- count** 使用计数来将命令应用于最内层的count帧，其中count是一个正数。
- count** 使用-count来将命令应用于最外层的count帧，其中count是一个正数。
- 级别** 使用级别来将命令应用于由级别列表标识的帧集。级别是一个帧级别或一个帧级别范围，如 level1-level2。帧级别是在 ‘backtrace’ 命令输出的第一字段中显示的数字。例如， ‘2-4 6-8 3’ 表示要对级别为2、3、4、6、7、8的帧应用命令，然后再次对级别为3的帧应用命令。
‘backtrace’ 命令输出。例如， ‘2-4 6-8 3’ 表示要对级别为2、3、4、6、7、8的帧应用命令，然后再次对级别为3的帧应用命令。

请注意，应用命令的帧也受 **set backtrace** 设置的影响，例如 **set backtrace past-main** 和 **set backtrace limit N**。参见第 8.2 [回溯],页 116。

该框架应用命令也支持多种选项，允许覆盖相关的设置回溯：

- past-main [on|off]**
是否应该让回溯继续到 **main** 之前。相关设置:[设置回溯到 **main** 之前],第 118 页。
- past-entry [on|off]**
是否应该让回溯继续通过程序的入口点。相关设置： [设置回溯通过入口], 页码118。

默认情况下， GDB 会在命令生成的输出之前显示一些框架信息，并且命令执行期间出现的错误会中止框架应用。可以使用以下选项来微调这些行为：

- c** 标志 -c（代表 ‘继续’ ）会导致命令中的任何错误被显示，然后继续框架应用执行。
- s** 标志 -s（代表 ‘静默’ ）会导致命令产生的任何错误或空输出被静默忽略。也就是说，执行会继续，但框架信息和错误不会被打印。
- q** 标志 -q（ ‘安静’ ）会禁用打印框架信息。

以下示例展示了在将命令 ‘**p j**’ 应用于所有帧时，标志 ‘-c’ 和 ‘-s’ 的作用，其中变量 ‘**j**’ 只能在最外层的 **#1** 主帧中成功打印。

```
(gdb) frame apply all p j
#0 some_function (i=5) at fun.c:4
No symbol "j" in current context.
(gdb) frame apply all -c p j
#0 some_function (i=5) at fun.c:4
No symbol "j" in current context.
#1 0x565555fb in main (argc=1, argv=0xffffd2c4) at fun.c:11
$1 = 5
(gdb) frame apply all -s p j
#1 0x565555fb in main (argc=1, argv=0xffffd2c4) at fun.c:11
$2 = 5
(gdb)
```

By default, ‘frame apply’, prints the frame location information before the command output:

```
(gdb) frame apply all p $sp
#0 some_function (i=5) at fun.c:4
$4 = (void *) 0xffffd1e0
#1 0x565555fb in main (argc=1, argv=0xffffd2c4) at fun.c:11
$5 = (void *) 0xffffd1f0
(gdb)
```

If the flag -q is given, no frame information is printed:

```
(gdb) frame apply all -q p $sp
$12 = (void *) 0xffffd1e0
$13 = (void *) 0xffffd1f0
(gdb)
```

faas command

Shortcut for `frame apply all -s command`. Applies *command* on all frames, ignoring errors and empty output.

It can for example be used to print a local variable or a function argument without knowing the frame where this variable or argument is, using:

```
(gdb) faas p some_local_var_i_do_not_remember_where_it_is
```

The `faas` command accepts the same options as the `frame apply` command. See Section 8.5 [frame apply], page 122.

Note that the command `tfaas command` applies *command* on all frames of all threads. See Section 4.10 [Threads], page 48.

8.6 Management of Frame Filters.

Frame filters are Python based utilities to manage and decorate the output of frames. See Section 23.3.2.10 [Frame Filter API], page 436, for further information.

Managing frame filters is performed by several commands available within GDB, detailed here.

info frame-filter

Print a list of installed frame filters from all dictionaries, showing their name, priority and enabled status.

disable frame-filter filter-dictionary filter-name

Disable a frame filter in the dictionary matching *filter-dictionary* and *filter-name*. The *filter-dictionary* may be `all`, `global`, `progspace`, or the name of

```
(gdb) frame apply all p j
#0 some_function (i=5) at fun.c:4
No symbol "j" in current context.
(gdb) frame apply all -c p j
#0 some_function (i=5) at fun.c:4
No symbol "j" in current context.
#1 0x565555fb in main (argc=1, argv=0xffffd2c4) at fun.c:11
$1 = 5
(gdb) frame apply all -s p j
#1 0x565555fb in main (argc=1, argv=0xffffd2c4) at fun.c:11
$2 = 5
(gdb)
```

默认情况下，‘frame apply’，会在命令输出之前打印帧位置信息：

```
(gdb) frame apply all p $sp
#0 some_function (i=5) at fun.c:4
$4 = (void *) 0xffffd1e0
#1 0x565555fb in main (argc=1, argv=0xffffd2c4) at fun.c:11
$5 = (void *) 0xffffd1f0
(gdb)
```

If the flag -q is given, no frame information is printed:

```
(gdb) frame apply all -q p $sp
$12 = (void *) 0xffffd1e0
$13 = (void *) 0xffffd1f0
(gdb)
```

faas command

框架应用全部的快捷方式 `-s command`。在所有框架上应用命令，忽略错误和空输出。

例如，它可以用来打印一个本地变量或函数参数，而不知道该变量或参数所在的框架，使用：

```
(gdb) faas p some_local_var_i_do_not_remember_where_it_is
```

`faas` 命令与 `frameapply` 命令接受相同的选项。参见第 8.5 [frameapply], 页第 122 页。

请注意，命令 `tfaas command` 会应用于所有线程的所有帧。参见第 4.10 [章], 页。

8.6 帧过滤器管理。

帧过滤器是基于 Python 的工具，用于管理和装饰帧的输出。有关更多信息，请参阅第 23.3.2.10 [帧过滤器 API], 页。

管理帧过滤器是通过 GDB 中提供的多个命令来完成的，此处将详细介绍。

info frame-filter

打印所有字典中已安装的帧过滤器列表，显示它们的名称、优先级和启用状态。

禁用帧过滤器 filter-dictionary filter-name

在匹配 *filter-dictionary* 和 *filter-name* 的字典中禁用帧过滤器。*filter-dictionary* 可能是 `all`、`global`、`progspace` 或其名称。

the object file where the frame filter dictionary resides. When `all` is specified, all frame filters across all dictionaries are disabled. The `filter-name` is the name of the frame filter and is used when `all` is not the option for `filter-dictionary`. A disabled frame-filter is not deleted, it may be enabled again later.

`enable frame-filter filter-dictionary filter-name`

Enable a frame filter in the dictionary matching `filter-dictionary` and `filter-name`. The `filter-dictionary` may be `all`, `global`, `progspace` or the name of the object file where the frame filter dictionary resides. When `all` is specified, all frame filters across all dictionaries are enabled. The `filter-name` is the name of the frame filter and is used when `all` is not the option for `filter-dictionary`.

Example:

```
(gdb) info frame-filter

global frame-filters:
Priority  Enabled  Name
1000     No       PrimaryFunctionFilter
100      Yes       Reverse

progspace /build/test frame-filters:
Priority  Enabled  Name
100      Yes       ProgspaceFilter

objfile /build/test frame-filters:
Priority  Enabled  Name
999      Yes       BuildProgramFilter

(gdb) disable frame-filter /build/test BuildProgramFilter
(gdb) info frame-filter

global frame-filters:
Priority  Enabled  Name
1000     No       PrimaryFunctionFilter
100      Yes       Reverse

progspace /build/test frame-filters:
Priority  Enabled  Name
100      Yes       ProgspaceFilter

objfile /build/test frame-filters:
Priority  Enabled  Name
999      No       BuildProgramFilter

(gdb) enable frame-filter global PrimaryFunctionFilter
(gdb) info frame-filter

global frame-filters:
Priority  Enabled  Name
1000     Yes       PrimaryFunctionFilter
100      Yes       Reverse

progspace /build/test frame-filters:
Priority  Enabled  Name
100      Yes       ProgspaceFilter

objfile /build/test frame-filters:
```

当指定 `all` 时，所有字典中的所有帧过滤器都将被禁用。`filter-name` 是帧过滤器的名称，当 `filter-dictionary` 不是 `all` 选项时使用。被禁用的帧过滤器不会被删除，稍后可以再次启用。

启用 frame-filter `filter-dictionary filter-name`

在匹配 `filter-dictionary` 和 `filter-name` 的字典中启用一个帧过滤器。`filter-dictionary` 可以是 `all`、`global`、`progspace` 或帧过滤器字典所在的对象文件名称。当指定 `all` 时，所有字典中的所有帧过滤器都将被启用。`filter-name` 是帧过滤器的名称，当 `filter-dictionary` 不是 `all` 选项时使用。

示例：

```
(gdb) info frame-filter

全局 frame-filters:
优先级  启用名称
1000    No       主要功能筛选
100     Yes       反向

progspace /build/test frame-filters:
优先级  启用名称
100     Yes       ProgspaceFilter

objfile /build/test frame-filters:
优先级  启用名称
999     Yes       BuildProgramFilter

(gdb) 禁用 frame-filter /build/test BuildProgramFilter
(gdb) 查询 frame-filter

全局框架过滤器:
优先级  启用名称
1000    No       主要功能过滤器
100     Yes       反向

progspace /build/test 框架过滤器:
优先级  启用名称
100     Yes       ProgspaceFilter

objfile /build/test frame-filters:
优先级  启用名称
999     No       构建程序过滤器

(gdb) 启用 frame-filter 全局 PrimaryFunctionFilter
(gdb) info frame-filter

全局 frame-filters:
优先级  启用名称
1000    Yes       主要功能过滤器
100     Yes       反向

progspace /build/test frame-filters:
优先级  启用名称
100     Yes       Progspace过滤器

objfile /build/test frame-filters:
```


PriorityEnabledName999NoBuildProgramFilter

set frame-filter priority filter-dictionary filter-name priority

Set the priority of a frame filter in the dictionary matching filter-dictionary, and the frame filter name matching filter-name. The filter-dictionary may be global, progspace or the name of the object file where the frame filter dictionary resides. The priority is an integer.

show frame-filter priority filter-dictionary filter-name

Show the priority of a frame filter in the dictionary matching filter-dictionary, and the frame filter name matching filter-name. The filter-dictionary may be global, progspace or the name of the object file where the frame filter dictionary resides.

Example:

(gdb) info frame-filter

global frame-filters:

PriorityEnabledName1000YesPrimaryFunctionFilter100YesReverse

progspace /build/test frame-filters:

PriorityEnabledName100YesProgspaceFilter

objfile /build/test frame-filters:

PriorityEnabledName999NoBuildProgramFilter

(gdb) set frame-filter priority global Reverse 50

(gdb) info frame-filter

global frame-filters:

PriorityEnabledName1000YesPrimaryFunctionFilter50YesReverse

progspace /build/test frame-filters:

PriorityEnabledName100YesProgspaceFilter

objfile /build/test frame-filters:

PriorityEnabledName999NoBuildProgramFilter

优先级 启用名称 999 不 构建程序过滤器

set frame-filter priority filter-dictionary filter-name priority在字典匹配过滤器 filter-dictionary中设置帧过滤器的优先级，并匹配帧过滤器名称filter-name。 filter-dictionary可以是全局的、progspace，或帧过滤器字典所在的目标文件名称。优先级是一个整数。

显示frame-filter priority filter-dictionary filter-name

显示在字典匹配过滤器filter-dictionary中，并匹配帧过滤器名称 filter-name的帧过滤器的优先级。filter-dictionary可以是全局的、progspace，或帧过滤器字典所在的目标文件名称。

示例:

(gdb) info frame-filter

全局框架过滤器: 优先级启用 名称 1000 是 主要函数过滤器 100 是 反向

程序空间 /build/test 框架过滤器: 优先级启用 名称 100 是 程序空间过滤器

目标文件 /build/testframe-filters: 优先级启用 名称 999 否 构建程序过滤器

(gdb) 设置框架过滤器优先级 全局 反向 50 (gdb) 信息 框架过滤器

全局框架过滤器: 优先级启用 名称 1000 是 主要函数过滤器 50 是 反向

程序空间 /build/test 框架过滤器: 优先级启用 名称 100 是 程序空间过滤器

objfile /build/testframe-filters: 优先级启用名称 999 无 BuildProgramFilter

9 Examining Source Files

GDB can print parts of your program's source, since the debugging information recorded in the program tells GDB what source files were used to build it. When your program stops, GDB spontaneously prints the line where it stopped. Likewise, when you select a stack frame (see Section 8.3 [Selecting a Frame], page 119), GDB prints the line where execution in that frame has stopped. You can print other portions of source files by explicit command.

If you use GDB through its GNU Emacs interface, you may prefer to use Emacs facilities to view source; see Chapter 26 [Using GDB under GNU Emacs], page 607.

9.1 Printing Source Lines

To print lines from a source file, use the `list` command (abbreviated `l`). By default, ten lines are printed. There are several ways to specify what part of the file you want to print; see Section 9.2 [Location Specifications], page 128, for the full list.

Here are the forms of the `list` command most commonly used:

<code>list <i>linenum</i></code>	Print lines centered around line number <i>linenum</i> in the current source file.
<code>list <i>function</i></code>	Print lines centered around the beginning of function <i>function</i> .
<code>list</code>	Print more lines. If the last lines printed were printed with a <code>list</code> command, this prints lines following the last lines printed; however, if the last line printed was a solitary line printed as part of displaying a stack frame (see Chapter 8 [Examining the Stack], page 115), this prints lines centered around that line. If no <code>list</code> command has been used and no solitary line was printed, it prints the lines around the function <code>main</code> .
<code>list +</code>	Same as using with no arguments.
<code>list -</code>	Print lines just before the lines last printed.
<code>list .</code>	Print the lines surrounding the point of execution within the currently selected frame. If the inferior is not running, print lines around the start of the main function instead.

By default, GDB prints ten source lines with any of these forms of the `list` command. You can change this using `set listsize`:

<code>set listsize <i>count</i></code>	
<code>set listsize unlimited</code>	Make the <code>list</code> command display <i>count</i> source lines (unless the <code>list</code> argument explicitly specifies some other number). Setting <i>count</i> to <code>unlimited</code> or 0 means there's no limit.
<code>show listsize</code>	Display the number of lines that <code>list</code> prints.

Repeating a `list` command with `RET` discards the argument, so it is equivalent to typing just `list`. This is more useful than listing the same lines again. An exception is made for

9 检查源文件

GDB 可以打印您程序源代码的部分内容，因为程序中记录的调试信息告诉 GDB 它是由哪些源文件构建的。当您的程序停止时，GDB 会自动打印停止的行。同样地，当您选择一个栈帧（参见第8.3 [选择一个帧], 页119）时，GDB 会打印该帧执行停止的行。您可以通过显式命令打印源文件的其他部分。

如果你使用 GDB 通过其 GNU Emacs 界面，你可能会倾向于使用 Emacs 功能来查看源文件；参见第 26 [使用 GDB 章节在 GNU Emacs], 页面 607。

9.1 打印源代码行

要打印源文件中的行，请使用列表命令（缩写为 `l`）。默认情况下，会打印十行。有几种方法可以指定你想打印文件的部分；参见第 9.2 [位置规范], 页面 128，以获取完整列表。

以下是列表命令最常用的形式：

<code>list <i>linenum</i></code>	在当前源文件中，围绕行号 <i>linenum</i> 打印行。
列表 <code><i>function</i></code>	围绕函数 <i>function</i> 的开始位置打印行。
<code>list</code>	打印更多行。如果最后打印的行是使用列表命令打印的，则此命令打印最后打印行的后续行；然而，如果最后打印的行是一个单独的行，作为显示栈帧的一部分打印（参见第 8 章[检查栈], 第 115 页），则此命令围绕该行打印行。如果没有使用列表命令且没有打印单独的行，则打印围绕函数 <code>main</code> 的行。

列表 `+` 与不带参数使用相同。

`list -` 打印最后打印行的前几行。

`list .` 打印当前选定框架内执行点周围的行。如果从属进程未运行，则打印主函数开头的行。

默认情况下，GDB 使用 `list` 命令的这些形式打印十行源代码。您可以使用 `set listsize` 更改此设置：

<code>set listsize <i>count</i></code>	
<code>set listsize unlimited</code>	使 <code>list</code> 命令显示源代码行数（除非 <code>list</code> 参数明确指定其他数字）。将 <i>count</i> 设置为 <code>unlimited</code> 或 0 表示没有限制。

显示列表大小
显示列表打印的行数。

使用 `RET` 重复列表命令会丢弃参数，因此它相当于只输入 `list`。这比再次列出相同行更有用。但 ‘`-`’ 作为参数时是个例外；该参数在重复时会被保留，以便每次重复都在源文件中向上移动。

an argument of ‘-’; that argument is preserved in repetition so that each repetition moves up in the source file.

In general, the `list` command expects you to supply zero, one or two location specs. These location specs are interpreted to resolve to source code lines; there are several ways of writing them (see Section 9.2 [Location Specifications], page 128), but the effect is always to resolve to some source lines to display.

Here is a complete description of the possible arguments for `list`:

`list locspec`

Print lines centered around the line or lines of all the code locations that result from resolving *locspec*.

`list first,last`

Print lines from *first* to *last*. Both arguments are location specs. When a `list` command has two location specs, and the source file of the second location spec is omitted, this refers to the same source file as the first location spec. If either *first* or *last* resolve to more than one source line in the program, then the list command shows the list of resolved source lines and does not proceed with the source code listing.

`list ,last`

Print lines ending with *last*.

Likewise, if *last* resolves to more than one source line in the program, then the list command prints the list of resolved source lines and does not proceed with the source code listing.

`list first,`

Print lines starting with *first*.

`list +` Print lines just after the lines last printed.

`list -` Print lines just before the lines last printed.

`list` As described in the preceding table.

9.2 Location Specifications

Several GDB commands accept arguments that specify a location or locations of your program’s code. Many times locations are specified using a source line number, but they can also be specified by a function name, an address, a label, etc. The different forms of specifying a location that GDB recognizes are collectively known as forms of *location specification*, or *location spec*. This section documents the forms of specifying locations that GDB recognizes.

When you specify a location, GDB needs to find the place in your program, known as *code location*, that corresponds to the given location spec. We call this process of finding actual code locations corresponding to a location spec *location resolution*.

A concrete code location in your program is uniquely identifiable by a set of several attributes: its source line number, the name of its source file, the fully-qualified and proto-typed function in which it is defined, and an instruction address. Because each inferior has its own address space, the inferior number is also a necessary part of these attributes.

该参数在重复时会被保留，以便每次重复都在源文件中向上移动。

通常情况下，`list`命令期望你提供零个、一个或两个位置规范。这些位置规范会被解释为源代码行；有几种不同的写法方式（参见第9.2 [位置规范], 页128），但效果始终是解析出某些源代码行以供显示。

以下是`list`命令可能参数的完整描述

`list locspec`

打印围绕`locspec`解析结果的所有代码位置对应的行，并使这些行居中显示

`list first,last`

从第一行到最后一行打印。两个参数都是位置规范。当一个列表命令有两个位置规范时，如果省略第二个位置规范的源文件，则它指的是与第一个位置规范相同的源文件。如果第一个或最后一个解析为程序中的多行源代码，则列表命令显示解析的源代码行列表，并且不会继续进行源代码列表。

命令显示解析的源代码行列表，并且不会继续进行源代码列表。

列表，`last`

打印以最后行为结束的行。

同样地，如果最后一个解析为程序中的多行源代码，则列表命令打印解析的源代码行列表，并且不会继续进行源代码列表。

列表 `first,`

打印以 `first` 开头的行。

列表 `+` 打印在最后打印的行之后。

列表 `-` 打印在最后打印的行之前。

列表 如前表所述。

9.2 位置规范

一些 GDB 命令接受参数来指定程序代码的位置。很多时候位置是通过源代码行号来指定的，但也可以通过函数名、地址、标签等来指定。GDB 识别的位置指定形式统称为位置指定形式，或简称位置指定。本节记录了GDB 识别的位置指定形式。

当你指定一个位置时，GDB 需要找到程序中对应于给定位置指定的代码位置，即代码位置。我们称这个过程为位置解析，即根据位置指定找到实际的代码位置。

程序中一个具体的代码位置可以通过一组属性唯一标识：其源代码行号、源文件名称、定义它的完全限定和原型化的函数，以及指令地址。由于每个从属进程（inferior）都有自己的地址空间，从属进程编号也是这些属性中必要的一部分。

By contrast, location specs you type will many times omit some of these attributes. For example, it is customary to specify just the source line number to mean a line in the current source file, or specify just the basename of the file, omitting its directories. In other words, a location spec is usually incomplete, a kind of blueprint, and GDB needs to complete the missing attributes by using the implied defaults, and by considering the source code and the debug information available to it. This is what location resolution is about.

The resolution of an incomplete location spec can produce more than a single code location, if the spec doesn't allow distinguishing between them. Here are some examples of situations that result in a location spec matching multiple code locations in your program:

- The location spec specifies a function name, and there are several functions in the program which have that name. (To distinguish between them, you can specify a fully-qualified and prototyped function name, such as `A::func(int)` instead of just `func`.)
- The location spec specifies a source file name, and there are several source files in the program that share the same name, for example several files with the same base-name in different subdirectories. (To distinguish between them, specify enough leading directories with the file name.)
- For a C++ constructor, the GCC compiler generates several instances of the function body, used in different cases, but their source-level names are identical.
- For a C++ template function, a given line in the function can correspond to any number of instantiations.
- For an inlined function, a given source line can correspond to several actual code locations with that function's inlined code.

Resolution of a location spec can also fail to produce a complete code location, or even fail to produce any code location. Here are some examples of such situations:

- Some parts of the program lack detailed enough debug info, so the resolved code location lacks some attributes, like source file name and line number, leaving just the instruction address and perhaps also a function name. Such an incomplete code location is only usable in contexts that work with addresses and/or function names. Some commands can only work with complete code locations.
- The location spec specifies a function name, and there are no functions in the program by that name, or they only exist in a yet-unloaded shared library.
- The location spec specifies a source file name, and there are no source files in the program by that name, or they only exist in a yet-unloaded shared library.
- The location spec specifies both a source file name and a source line number, and even though there are source files in the program that match the file name, none of those files has the specified line number.

Locations may be specified using three different formats: linespec locations, explicit locations, or address locations. The following subsections describe these formats.

9.2.1 Linespec Locations

A *linespec* is a colon-separated list of source location parameters such as file name, function name, etc. Here are all the different ways of specifying a linespec:

linenum Specifies the line number *linenum* of the current source file.

相比之下，你输入的位置规范（location specs）经常会省略其中一些属性。例如，通常只需指定源代码行号来表示当前源文件中的一行，或者只指定文件的 *basename*，而省略其目录。换句话说，位置规范通常是 *incomplete*（不完整的），一种 *kind of blueprint*（蓝图），需要通过使用隐含的默认值、并考虑可用的源代码和调试信息来补全缺失的属性。这就是位置解析（location resolution）要解决的问题。

一个不完整的定位规范解析可能产生多个代码位置，如果该规范无法区分它们的话。以下是一些导致定位规范在程序中匹配多个代码位置的情况示例：

- 定位规范指定了一个函数名，而程序中有多个函数具有该名称。（要区分它们，可以指定一个完全限定和声明过的函数名，例如 `A::func(int)` 而不是仅仅 `func`。）
完全限定和声明过的函数名，例如 `A::func(int)` 而不是仅仅 `func`。）
- 定位规范指定了一个源文件名，而程序中有多个源文件共享相同名称，例如不同子目录中具有相同基本名称的多个文件。（要区分它们，使用文件名指定足够的目录前缀。）

- 对于 C++ 构造函数，GCC 编译器会生成该函数的多个实例 body，在不同的场景中使用，但它们的源级名称是相同的。
- 对于 C++ 模板函数，函数中的某一行可以对应任意数量的实例化。

- 对于内联函数，给定的源代码行可以对应多个实际代码位置。
该函数的内联代码的位置。

位置规范解析也可能失败，无法生成完整的代码位置，甚至可能无法生成任何代码位置。

以下是一些此类情况的示例：

- 程序的一些部分缺乏足够详细的调试信息，因此解析的代码位置缺少某些属性，如源文件名和行号，只留下指令地址，也许还有函数名。这种不完整的代码位置只能在处理地址和/或函数名的上下文中使用。某些命令只能处理完整的代码位置。

- 位置规范指定了函数名称，而程序中没有该名称的函数，或者它们仅存在于尚未加载的共享库中。
- 位置规范指定了源文件名称，而程序中没有该名称的源文件，或者它们仅存在于尚未加载的共享库中。
- 位置规范同时指定了源文件名称和源代码行号，尽管程序中有与文件名匹配的源文件，但其中没有任何文件具有指定的行号。

位置可以使用三种不同的格式指定：行号位置、显式位置或地址位置。以下小节将描述这些格式。

9.2.1 行号位置

行号是一个由冒号分隔的源位置参数列表，例如文件名、函数名等。以下是指定行号的所有不同方法：

linenum 指定当前源文件的行号 *linenum*。

-offset

+offset Specifies the line *offset* lines before or after the *current line*. For the **list** command, the current line is the last one printed; for the breakpoint commands, this is the line at which execution stopped in the currently selected *stack frame* (see Section 8.1 [Frames], page 115, for a description of stack frames.) When used as the second of the two linespecs in a **list** command, this specifies the line *offset* lines up or down from the first linespec.

filename:linenum

Specifies the line *linenum* in the source file *filename*. If *filename* is a relative file name, then it will match any source file name with the same trailing components. For example, if *filename* is 'gcc/expr.c', then it will match source file name of /build/trunk/gcc/expr.c, but not /build/trunk/libcpp/expr.c or /build/trunk/gcc/x-expr.c.

function Specifies the line that begins the body of the function *function*. For example, in C, this is the line with the open brace.

By default, in C++ and Ada, *function* is interpreted as specifying all functions named *function* in all scopes. For C++, this means in all namespaces and classes. For Ada, this means in all packages.

For example, assuming a program with C++ symbols named **A::B::func** and **B::func**, both commands **break func** and **break B::func** set a breakpoint on both symbols.

Commands that accept a linespec let you override this with the **-qualified** option. For example, **break -qualified func** sets a breakpoint on a free-function named **func** ignoring any C++ class methods and namespace functions called **func**.

See Section 9.2.2 [Explicit Locations], page 131.

function:label

Specifies the line where *label* appears in *function*.

filename:function

Specifies the line that begins the body of the function *function* in the file *filename*. You only need the file name with a function name to avoid ambiguity when there are identically named functions in different source files.

label Specifies the line at which the label named *label* appears in the function corresponding to the currently selected stack frame. If there is no current selected stack frame (for instance, if the inferior is not running), then GDB will not search for a label.

-pstap|-probe-stap [objfile:[provider:]]name

The GNU/Linux tool **SystemTap** provides a way for applications to embed static probes. See Section 5.1.10 [Static Probe Points], page 83, for more information on finding and using static probes. This form of linespec specifies the location of such a static probe.

If *objfile* is given, only probes coming from that shared library or executable matching *objfile* as a regular expression are considered. If *provider* is given,

-offset

+offset

指定当前行之前或之后的行偏移量。对于list命令，当前行是最后打印的一行；对于断点命令，这是当前选定的栈帧中执行停止的行（有关栈帧的描述，请参阅第8.1 [帧]，页）。当用作list命令中的两条行规范中的第二条时，这指定了从第一条行规范向上或向下偏移的行数。

filename:linenum

指定源文件filename中的行linenum。如果filename是相对文件名，则它将匹配任何具有相同尾部的源文件名。例如，如果filename是 'gcc/expr.c'，则它将匹配源文件名/build/trunk/gcc/expr.c，但不会匹配/build/trunk/libcpp/expr.c或/build/trunk/gcc/x-expr.c。

function

指定函数function的体开始的行。例如，在C语言中，这是带有开括号的行。

默认情况下，在 C++ 和 Ada 中，函数被视为指定所有作用域中名为 function 的函数。对于 C++，这意味着在所有命名空间和类中。对于 Ada，这意味着在所有包中。

例如，假设一个程序中有 C++ 符号名为 **A::B::func** 和 **B::func**，bothcommands **break func** 和 **break B::func** 都在两个符号上设置断点。

接受行号 spec 的命令允许您使用 **-qualified** 选项覆盖此设置。例如，**break -qualified func** 在名为 func 的自由函数上设置断点，忽略任何 C++ 类方法和命名空间函数中名为 func 的函数。

参见第 9.2.2 [显式位置]，页 131。

function:label

指定标签在函数中出现的行。

filename:function

指定文件 filename 中函数 function 的主体开始的行。当不同源文件中存在同名的函数时，您只需要文件名和函数名即可避免歧义。

label 指定当前选中的栈帧对应的函数中，名为 label 的标签出现的行。如果没有当前选中的栈帧（例如，如果目标程序未运行），则 GDB 不会搜索标签。

-pstap|-probe-stap [objfile:[provider:]]name

The GNU/Linux 工具 **SystemTap** 为应用程序嵌入静态探针提供了一种方式。有关查找和使用静态探针的更多信息，请参阅第 5.1.10 [静态探针点]，页 83。这种行规范形式指定了此类静态探针的位置。

如果指定了objfile，则仅考虑来自该共享库或与objfile作为正则表达式匹配的可执行文件的探针。如果指定了provider，

then only probes from that provider are considered. If several probes match the spec, GDB will insert a breakpoint at each one of those probes.

9.2.2 Explicit Locations

Explicit locations allow the user to directly specify the source location’s parameters using option-value pairs.

Explicit locations are useful when several functions, labels, or file names have the same name (base name for files) in the program’s sources. In these cases, explicit locations point to the source line you meant more accurately and unambiguously. Also, using explicit locations might be faster in large programs.

For example, the linespec ‘foo:bar’ may refer to a function bar defined in the file named foo or the label bar in a function named foo. GDB must search either the file system or the symbol table to know.

The list of valid explicit location options is summarized in the following table:

-source filename	The value specifies the source file name. To differentiate between files with the same base name, prepend as many directories as is necessary to uniquely identify the desired file, e.g., foo/bar/baz.c. Otherwise GDB will use the first file it finds with the given base name. This option requires the use of either -function or -line.
-function function	The value specifies the name of a function. Operations on function locations unmodified by other options (such as -label or -line) refer to the line that begins the body of the function. In C, for example, this is the line with the open brace. By default, in C++ and Ada, function is interpreted as specifying all functions named function in all scopes. For C++, this means in all namespaces and classes. For Ada, this means in all packages. For example, assuming a program with C++ symbols named A::B::func and B::func, both commands break -function func and break -function B::func set a breakpoint on both symbols. You can use the -qualified flag to override this (see below).
-qualified	This flag makes GDB interpret a function name specified with -function as a complete fully-qualified name. For example, assuming a C++ program with symbols named A::B::func and B::func, the break -qualified -function B::func command sets a breakpoint on B::func, only. (Note: the -qualified option can precede a linespec as well (see Section 9.2.1 [Linespec Locations], page 129), so the particular example above could be simplified as break -qualified B::func.)
-label label	The value specifies the name of a label. When the function name is not specified, the label is searched in the function of the currently selected stack frame.

则仅考虑来自该provider的探针。如果有多个探针匹配spec， GDB 将在这些探针中的每一个处插入断点。

9.2.2 显式位置

显式位置允许用户使用选项-值对直接指定源位置的参数。

当程序源代码中多个函数、标签或文件名具有相同名称（文件的基本名称）时，显式位置很有用。在这些情况下，显式位置可以更准确、无歧义地指向您想要的那条源代码行。此外，在大型程序中使用显式位置可能更快。

例如，行规范符 “foo:bar” 可能引用在名为foo的文件中定义的函数bar，或者函数名为foo中的标签bar。 GDB 必须搜索文件系统或符号表才能知道。

有效的显式位置选项列表总结在下表中：

-source filename	该值指定源文件名。若要区分具有相同基本名称的文件，请添加必要数量的目录以唯一标识目标文件，例如 foo/bar/baz.c。否则 GDB将使用具有给定基本名称的第一个找到的文件。此选项需要使用 -function 或 -line 之一。
-function function	该值指定函数的名称。未受其他选项（如 -label 或 -line）修改的函数位置操作指的是函数体开始的行。例如在 C 语言中，这是带有开大括号的行。 默认情况下，在 C++ 和 Ada 语言中，函数被解释为指定所有名为 function 的函数，无论其作用域如何。对于 C++，这意味着在所有命名空间和类中。对于 Ada，这意味着在所有包中。 例如，假设一个程序中有名为 A::B::func 和 B::func 的 C++ 符号，则命令 break -function func和 break -function B::func都会两个符号上设置断点。 您可以使用 -qualified flag 来覆盖此设置（见下文）。
-qualified	此 flag 使 GDB 将使用 -function 指定的函数名解释为完全限定名称。 例如，假设一个 C++ 程序中有名为 A::B::func 和 B::func 的符号，则 break -qualified -function B::func 命令仅在 B::func 上设置断点。 (注意： -qualified 选项也可以作为行规范符之前（见第 9.2.1[行规范符位置]，页 129），因此上述特定示例可以简化为 break -qualified B::func。)
-label label	该值指定标签的名称。当未指定函数名时，标签会在当前选中的栈帧函数中搜索。

-line number

The value specifies a line offset for the location. The offset may either be absolute (**-line 3**) or relative (**-line +3**), depending on the command. When specified without any other options, the line offset is relative to the current line.

Explicit location options may be abbreviated by omitting any non-unique trailing characters from the option name, e.g., **break -s main.c -li 3**.

9.2.3 Address Locations

Address locations indicate a specific program address. They have the generalized form **address*.

For line-oriented commands, such as **list** and **edit**, this specifies a source line that contains *address*. For **break** and other breakpoint-oriented commands, this can be used to set breakpoints in parts of your program which do not have debugging information or source files.

Here *address* may be any expression valid in the current working language (see Chapter 15 [Languages], page 229) that specifies a code address. In addition, as a convenience, GDB extends the semantics of expressions used in locations to cover several situations that frequently occur during debugging. Here are the various forms of *address*:

expression

Any expression valid in the current working language.

funcaddr An address of a function or procedure derived from its name. In C, C++, Objective-C, Fortran, Pascal, Modula-2, minimal, and assembly, this is simply the function's name *function* (and actually a special case of a valid expression). In Ada, this is *function*'Address (although **&function** also works).

This form specifies the address of the function's first instruction, before the stack frame and arguments have been set up.

'filename'::funcaddr

Like **funcaddr** above, but also specifies the name of the source file explicitly. This is useful if the name of the function does not specify the function unambiguously, e.g., if there are several functions with identical names in different source files, see [variable name conflict], page 149. This may not be supported for all languages, see Section 10.1 [Expressions], page 147.

9.3 Editing Source Files

To edit the lines in a source file, use the **edit** command. The editing program of your choice is invoked with the current line set to the active line in the program. Alternatively, there are several ways to specify what part of the file you want to print if you want to see other parts of the program:

edit locspec

Edit the source file of the code location that results from resolving **locspec**. Editing starts at the source file and source line **locspec** resolves to. See Section 9.2 [Location Specifications], page 128, for all the possible forms of the *locspec* argument.

-行 number

该值指定位置行的偏移量。偏移量可以是绝对值（-行3）或相对值（-行 +3），具体取决于命令。当未指定其他选项时，行偏移量相对于当前行。

显式位置选项可以通过省略选项名称中任何非唯一的尾随字符来缩写，例如

break -s main.c -li 3。

9.2.3 地址位置

地址位置指示一个特定的程序地址。它们具有通用的形式*地址。

对于行导向命令，例如**list**和**edit**，这指定了包含地址的源行。对于**break**和其他断点导向命令，这可用于在您的程序中没有调试信息或源文件的部分设置断点。

地址可以是当前工作语言中任何有效的表达式（参见第 15 [语言]，页 229），该表达式指定一个代码地址。此外，为了方便起见，GDB 扩展了用于位置的表达式的语义，以涵盖调试过程中经常发生的情况。以下是各种地址形式：

expression

当前工作语言中任何有效的表达式。

funcaddr 从其名称派生出的函数或过程的地址。在 C、C++、Objective-C、Fortran、Pascal、Modula-2、minimal 和汇编语言中，这仅仅是函数的名称（实际上是一个有效表达式的特殊情况）。在 Ada 中，这是 *function*' 地址（尽管 **&function** 也有效）。

这种形式指定函数第一条指令的地址，在堆栈帧和参数设置之前。

' filename'::funcaddr

与 **funcaddr** 类似，但还明确指定了源文件名称。如果函数名称不能明确指定函数，例如，如果不同源文件中有多个具有相同名称的函数，则这很有用，参见 [变量名冲突]，页 149。这可能不受所有语言支持，参见第 10.1 [表达式]，页 147。

9.3 编辑源文件

要编辑源文件中的行，请使用编辑命令。编辑程序会根据当前行被设置为程序中的活动行。或者，如果你想查看程序的其他部分，有几种方法可以指定要打印的文件部分：

edit locspec

编辑由解析 **locspec** 得到的代码位置的源文件。编辑从 **locspec** 解析到的源文件和源行开始。参见第 9.2 [位置规范]，页 128，了解 **locspec** 参数的所有可能形式。

If `locspect` resolves to more than one source line in your program, then the command prints the list of resolved source lines and does not proceed with the editing.

Here are the forms of the `edit` command most commonly used:

```
edit number
    Edit the current source file with number as the active line number.

edit function
    Edit the file containing function at the beginning of its definition.
```

9.3.1 Choosing your Editor

You can customize GDB to use any editor you want¹. By default, it is `/bin/ex`, but you can change this by setting the environment variable `EDITOR` before using GDB. For example, to configure GDB to use the `vi` editor, you could use these commands with the `sh` shell:

```
EDITOR=/usr/bin/vi
export EDITOR
gdb ...

or in the csh shell,
setenv EDITOR /usr/bin/vi
gdb ...
```

9.4 Searching Source Files

There are two commands for searching through the current source file for a regular expression.

forward-search *regexp*
search *regexp*

The command ‘**forward-search *regexp***’ checks each line, starting with the one following the last line listed, for a match for *regexp*. It lists the line that is found and saves it in the convenience variable `$_` (see Section 10.12 [Convenience Variables], page 173). You can use the synonym ‘**search *regexp***’ or abbreviate the command name as `fo`.

reverse-search *regexp*

The command ‘**reverse-search *regexp***’ checks each line, starting with the one before the last line listed and going backward, for a match for *regexp*. It lists the line that is found and saves it in the convenience variable `$_` (see Section 10.12 [Convenience Variables], page 173). You can abbreviate this command as `rev`.

9.5 Specifying Source Directories

Executable programs sometimes do not record the directories of the source files from which they were compiled, just the names. Even when they do, the directories could be moved between the compilation and your debugging session. GDB has a list of directories to search

¹ The only restriction is that your editor (say `ex`), recognizes the following command-line syntax:

```
ex +number file
```

The optional numeric value *+number* specifies the number of the line in the file where to start editing.

如果 `locspect` 在您的程序中解析到多行源代码，则命令会打印解析到的源代码行列表，并且不会继续进行编辑。

以下是编辑命令最常用的形式：

```
edit number
    使用编号作为活动行号编辑当前源文件。

edit function
    编辑包含函数定义开头处函数的文件。
```

9.3.1 选择编辑器

你可以自定义 GDB 以使用任何你想要的编辑器¹。默认情况下，它是 `/bin/ex`，但你可以通过在使用 GDB 之前设置环境变量 `EDITOR` 来更改它。例如，要配置 GDB 以使用 `vi` 编辑器，你可以使用以下命令与 `sh` shell 一起使用：

```
EDITOR=/usr/bin/vi
export EDITOR
gdb ...

或在 csh shell 中，
setenv EDITOR /usr/bin/vi
gdb ...
```

9.4 搜索源文件

有两个命令用于在当前源文件中搜索正则表达式。

正向搜索 *regexp*
search *regexp*

命令 ‘**forward-search *regexp***’ 会检查每一行，从列出的最后一行之后的那一行开始，查找与正则表达式匹配的内容。它会列出找到的行，并将其保存在便利变量 `$_` 中（参见第 10.12 [节 便利变量], 页 173）。您可以使用同义词 ‘**search *regexp***’ 或将命令名缩写为 `fo`。

reverse-search *regexp*

命令 ‘**reverse-search *regexp***’ 会检查每一行，从列出的最后一行之前的那一行开始向后查找，查找与正则表达式匹配的内容。它会列出找到的行，并将其保存在便利变量 `$_` 中（参见第 10.12 [节 便利变量], 页 173）。您可以将此命令缩写为 `rev`。

9.5 指定源目录

可执行程序有时不会记录编译所用的源文件目录，只记录文件名。即便记录了，编译时和调试时目录也可能已移动。GDB 有一个要搜索的目录列表

¹ 唯一限制是您的编辑器（例如 `ex`）需要识别以下命令行语法：`ex +number文件` 可选的数值 *+number* 指定在文件中的行号，从此处开始编辑。

for source files; this is called the *source path*. Each time GDB wants a source file, it tries all the directories in the list, in the order they are present in the list, until it finds a file with the desired name.

For example, suppose an executable references the file `/usr/src/foo-1.0/lib/foo.c`, does not record a compilation directory, and the *source path* is `/mnt/cross`. GDB would look for the source file in the following locations:

1. `/usr/src/foo-1.0/lib/foo.c`
2. `/mnt/cross/usr/src/foo-1.0/lib/foo.c`
3. `/mnt/cross/foo.c`

If the source file is not present at any of the above locations then an error is printed. GDB does not look up the parts of the source file name, such as `/mnt/cross/src/foo-1.0/lib/foo.c`. Likewise, the subdirectories of the source path are not searched: if the source path is `/mnt/cross`, and the binary refers to `foo.c`, GDB would not find it under `/mnt/cross/usr/src/foo-1.0/lib`.

Plain file names, relative file names with leading directories, file names containing dots, etc. are all treated as described above, except that non-absolute file names are not looked up literally. If the *source path* is `/mnt/cross`, the source file is recorded as `../lib/foo.c`, and no compilation directory is recorded, then GDB will search in the following locations:

1. `/mnt/cross/../lib/foo.c`
2. `/mnt/cross/foo.c`

The *source path* will always include two special entries ‘`$cdir`’ and ‘`$cwd`’, these refer to the compilation directory (if one is recorded) and the current working directory respectively.

‘`$cdir`’ causes GDB to search within the compilation directory, if one is recorded in the debug information. If no compilation directory is recorded in the debug information then ‘`$cdir`’ is ignored.

‘`$cwd`’ is not the same as ‘`.`’—the former tracks the current working directory as it changes during your GDB session, while the latter is immediately expanded to the current directory at the time you add an entry to the source path.

If a compilation directory is recorded in the debug information, and GDB has not found the source file after the first search using *source path*, then GDB will combine the compilation directory and the filename, and then search for the source file again using the *source path*.

For example, if the executable records the source file as `/usr/src/foo-1.0/lib/foo.c`, the compilation directory is recorded as `/project/build`, and the *source path* is `/mnt/cross:$cdir:$cwd` while the current working directory of the GDB session is `/home/user`, then GDB will search for the source file in the following locations:

1. `/usr/src/foo-1.0/lib/foo.c`
2. `/mnt/cross/usr/src/foo-1.0/lib/foo.c`
3. `/project/build/usr/src/foo-1.0/lib/foo.c`
4. `/home/user/usr/src/foo-1.0/lib/foo.c`
5. `/mnt/cross/project/build/usr/src/foo-1.0/lib/foo.c`
6. `/project/build/project/build/usr/src/foo-1.0/lib/foo.c`
7. `/home/user/project/build/usr/src/foo-1.0/lib/foo.c`

用于查找源文件；这称为源路径。每次 GDB 需要源文件时，它会按列表中出现的顺序依次尝试所有目录，直到找到所需名称的文件。

例如，假设可执行文件引用了文件 `/usr/src/foo-1.0/lib/foo.c`，未记录编译目录，且源路径为 `/mnt/cross`。GDB 会查找源文件在以下位置：

1. `/usr/src/foo-1.0/lib/foo.c`
2. `/mnt/cross/usr/src/foo-1.0/lib/foo.c`
3. `/mnt/cross/foo.c`

如果源文件不在上述任何位置，则会打印错误。GDB 不会查找源文件名的一部分，例如 `/mnt/cross/src/foo-1.0/lib/foo.c`。同样，也不会搜索源路径的子目录：如果源路径为 `/mnt/cross`，且二进制文件引用 `foo.c`，GDB 不会在 `/mnt/cross/usr/src/foo-1.0/lib` 下找到它。

普通文件名、带路径前缀的相对文件名、包含点的文件名等均按上述方式处理，只是非绝对文件名不会被字面查找。如果源路径是 `/mnt/cross`，源文件记录为 `../lib/foo.c`，且未记录编译目录，则 GDB 将搜索以下位置：

1. `/mnt/cross/../lib/foo.c`
2. `/mnt/cross/foo.c`

源路径始终包含两个特殊条目 ‘`$cdir`’ 和 ‘`$cwd`’，分别指向编译目录（如果记录了）和当前工作目录。

‘`$cdir`’ 会使 GDB 在编译目录内搜索，前提是调试信息中记录了编译目录。如果调试信息中未记录编译目录，则忽略 ‘`$cdir`’。

‘`$cwd`’ 与 ‘`.`’ 不同——前者会跟踪 GDB 会话中当前工作目录的变化，而后者在添加条目到源路径时立即展开为当时的当前目录。

如果在调试信息中记录了编译目录，并且 GDB 在使用源路径进行首次搜索后未找到源文件，那么 GDB 将会结合编译目录和文件名，然后再次使用源路径搜索源文件。

例如，如果可执行文件将源文件记录为 `/usr/src/foo-1.0/lib/foo.c`，将编译目录记录为 `/project/build`，而源路径为 `/mnt/cross:$cdir:$cwd`，同时 GDB 会话的当前工作目录为 `/home/user`，那么 GDB 将在以下位置搜索源文件：

1. `/usr/src/foo-1.0/lib/foo.c`
2. `/mnt/cross/usr/src/foo-1.0/lib/foo.c`
3. `/project/build/usr/src/foo-1.0/lib/foo.c`
4. `/home/user/usr/src/foo-1.0/lib/foo.c`
5. `/mnt/cross/project/build/usr/src/foo-1.0/lib/foo.c`
6. `/project/build/project/build/usr/src/foo-1.0/lib/foo.c`
7. `/home/user/project/build/usr/src/foo-1.0/lib/foo.c`

8. `/mnt/cross/foo.c`
9. `/project/build/foo.c`
10. `/home/user/foo.c`

If the file name in the previous example had been recorded in the executable as a relative path rather than an absolute path, then the first look up would not have occurred, but all of the remaining steps would be similar.

When searching for source files on MS-DOS and MS-Windows, where absolute paths start with a drive letter (e.g. `C:/project/foo.c`), GDB will remove the drive letter from the file name before appending it to a search directory from *source path*; for instance if the executable references the source file `C:/project/foo.c` and *source path* is set to `D:/mnt/cross`, then GDB will search in the following locations for the source file:

1. `C:/project/foo.c`
2. `D:/mnt/cross/project/foo.c`
3. `D:/mnt/cross/foo.c`

Note that the executable search path is *not* used to locate the source files.

Whenever you reset or rearrange the source path, GDB clears out any information it has cached about where source files are found and where each line is in the file.

When you start GDB, its source path includes only ‘`$cdir`’ and ‘`$cwd`’, in that order. To add other directories, use the `directory` command.

The search path is used to find both program source files and GDB script files (read using the ‘`-command`’ option and ‘`source`’ command).

In addition to the source path, GDB provides a set of commands that manage the list of source path substitution rules. A *substitution rule* specifies how to rewrite source directories stored in the program’s debug information in case the sources were moved to a different directory between compilation and debugging. A rule is made of two strings, the first specifying what needs to be rewritten in the path, and the second specifying how it should be rewritten. In [set substitute-path], page 136, we name these two parts *from* and *to* respectively. GDB does a simple string replacement of *from* with *to* at the start of the directory part of the source file name, and uses that result instead of the original file name to look up the sources.

Using the previous example, suppose the `foo-1.0` tree has been moved from `/usr/src` to `/mnt/cross`, then you can tell GDB to replace `/usr/src` in all source path names with `/mnt/cross`. The first lookup will then be `/mnt/cross/foo-1.0/lib/foo.c` in place of the original location of `/usr/src/foo-1.0/lib/foo.c`. To define a source path substitution rule, use the `set substitute-path` command (see [set substitute-path], page 136).

To avoid unexpected substitution results, a rule is applied only if the *from* part of the directory name ends at a directory separator. For instance, a rule substituting `/usr/source` into `/mnt/cross` will be applied to `/usr/source/foo-1.0` but not to `/usr/ware/foo-2.0`. And because the substitution is applied only at the beginning of the directory name, this rule will not be applied to `/root/usr/source/baz.c` either.

In many cases, you can achieve the same result using the `directory` command. However, `set substitute-path` can be more efficient in the case where the sources are organized in a complex tree with multiple subdirectories. With the `directory` command, you need to

8. `/mnt/cross/foo.c`
9. `/project/build/foo.c`
10. `/home/user/foo.c`

如果前一个示例中的文件名在可执行文件中记录为相对路径而不是绝对路径，那么第一次查找就不会发生，但所有剩余步骤将类似。

在MS-DOS和MS-Windows上搜索源文件时，绝对路径以驱动器字母开头（例如 `C:/project/foo.c`），GDB 会在将文件名附加到从源路径指定的搜索目录之前移除驱动器字母；例如，如果可执行文件引用源文件 `C:/project/foo.c`，而源路径设置为 `D:/mnt/cross`，那么 GDB 将在以下位置搜索源文件：

1. `C:/project/foo.c`
2. `D:/mnt/cross/project/foo.c`
3. `D:/mnt/cross/foo.c`

请注意，可执行文件搜索路径不会用于定位源文件。

每次您重置或重新排列源路径时，GDB 都会清除其缓存的关于源文件位置以及文件中每行位置的信息。

启动 GDB 时，其源路径仅包含 ‘`$cdir`’ 和 ‘`$cwd`’，按此顺序排列。要添加其他目录，请使用目录命令。

搜索路径用于查找程序源文件和 GDB 脚本文件（使用 ‘`-command`’ 选项和 ‘`source`’ 命令读取）。

除了源路径外，GDB 还提供了一套管理源路径替换规则的命令。替换规则指定了如何在程序调试信息中存储的源目录被移动到不同目录后，重新编写源目录路径。一条规则由两个字符串组成，第一个字符串指定路径中需要被替换的内容，第二个字符串指定替换方式。在 [set substitute-path]，页面的第136页，我们将这两个部分分别命名为 *from* 和 *to*。

GDB 会在源文件名目录部分的开始处，将 *from* 简单替换为 *to*，并使用替换后的结果代替原始文件名来查找源文件。

以之前的示例为例，假设 `foo-1.0` 树从 `/usr/src` 移动到了 `/mnt/cross`，那么你可以让 GDB 将所有源路径名称中的 `/usr/src` 替换为 `/mnt/cross`。第一次查找将是 `/mnt/cross/foo-1.0/lib/foo.c`，而不是原始的 `/usr/src/foo-1.0/lib/foo.c` 的位置。要定义源路径替换规则，请使用 `set substitute-path` 命令（参见 [set substitute-path]，页面的第136页）。

为避免意外的替换结果，只有当目录名的 *from* 部分以目录分隔符结尾时才会应用规则。例如，将 `/usr/source` 替换为 `/mnt/cross` 的规则将应用于 `/usr/source/foo-1.0`，但不会应用于 `/usr/ware/foo-2.0`。并且因为替换仅在目录名的开头应用，所以这个规则也不会应用于 `/root/usr/source/baz.c`。

在许多情况下，您可以使用目录命令实现相同的结果。但是，当源代码以包含多个子目录的复杂树形结构组织时，`set substitute-path` 可以更高效。使用目录命令时，您需要

add each subdirectory of your project. If you moved the entire tree while preserving its internal organization, then `set substitute-path` allows you to direct the debugger to all the sources with one single command.

`set substitute-path` is also more than just a shortcut command. The source path is only used if the file at the original location no longer exists. On the other hand, `set substitute-path` modifies the debugger behavior to look at the rewritten location instead. So, if for any reason a source file that is not relevant to your executable is located at the original location, a substitution rule is the only method available to point GDB at the new location.

You can configure a default source path substitution rule by configuring GDB with the `--with-relocated-sources=dir` option. The *dir* should be the name of a directory under GDB's configured prefix (set with `--prefix` or `--exec-prefix`), and directory names in debug information under *dir* will be adjusted automatically if the installed GDB is moved to a new location. This is useful if GDB, libraries or executables with debug information and corresponding source code are being moved together.

directory *dirname* ...

dir *dirname* ...

Add directory *dirname* to the front of the source path. Several directory names may be given to this command, separated by `:` (`;` on MS-DOS and MS-Windows, where `:` usually appears as part of absolute file names) or white-space. You may specify a directory that is already in the source path; this moves it forward, so GDB searches it sooner.

The special strings `$cdir` (to refer to the compilation directory, if one is recorded), and `$cwd` (to refer to the current working directory) can also be included in the list of directories *dirname*. Though these will already be in the source path they will be moved forward in the list so GDB searches them sooner.

directory

Reset the source path to its default value (`$cdir:$cwd` on Unix systems). This requires confirmation.

set directories *path-list*

Set the source path to *path-list*. `$cdir:$cwd` are added if missing.

show directories

Print the source path: show which directories it contains.

set substitute-path *from to*

Define a source path substitution rule, and add it at the end of the current list of existing substitution rules. If a rule with the same *from* was already defined, then the old rule is also deleted.

For example, if the file `/foo/bar/baz.c` was moved to `/mnt/cross/baz.c`, then the command

```
(gdb) set substitute-path /foo/bar /mnt/cross
```

will tell GDB to replace `/foo/bar` with `/mnt/cross`, which will allow GDB to find the file `baz.c` even though it was moved.

添加项目中的每个子目录。如果您在保留其内部组织的情况下移动了整个树形结构，那么 `set substitute-path` 允许您通过一条命令直接引导调试器访问所有源代码。

设置替换路径不仅仅是快捷命令。如果原始位置上的文件不再存在，源路径才会被使用。另一方面，设置替换路径会修改调试器行为，使其查看重写后的位置。因此，如果由于任何原因，与可执行文件无关的源文件位于原始位置，替换规则是唯一的方法来指示 GDB 指向新位置。

您可以通过使用 `--with-relocated-sources=dir` 选项配置 GDB 来配置默认的源路径替换规则。*dir* 应该是 GDB 配置的前缀（使用 `--prefix` 或 `--exec-prefix` 设置）下的一个目录名称，如果安装的 GDB 被移动到新位置，*dir* 下的调试信息中的目录名称将自动调整。如果 GDB、库或带有调试信息的可执行文件及其对应的源代码一起移动，这会很有用。

目录 *dirname* ...

dir *dirname*...

将目录 *dirname* 添加到源路径的前面。该命令可以给出多个目录名，用 `:`（在 MS-DOS 和 MS-Windows 上是 `;`，因为 `:` 通常作为绝对文件名的一部分出现）或空格分隔。您可以指定源路径中已经存在的目录；这将把它移到前面，以便 GDB 更早地搜索它。

特殊字符串 `$cdir`（用于引用编译目录，如果已记录）和 `$cwd`（用于引用当前工作目录）也可以包含在目录 *dirname* 列表中。尽管这些字符串可能已经存在于源路径中，但它们会被移到列表的前面，以便 GDB 更早地搜索它们。

目录

将源路径重置为其默认值（在 Unix 系统上是 `$cdir:$cwd`）。此操作需要确认。

设置目录 *path-list*

将源路径设置为路径列表。如果缺失，会添加 `$cdir:$cwd`。

显示目录

打印源路径：显示其包含的目录。

设置替换路径 *from to*

定义一个源路径替换规则，并将其添加到当前现有替换规则列表的末尾。如果已定义具有相同源（*from*）的规则，则旧规则也会被删除。

例如，如果文件 `/foo/bar/baz.c` 被移动到 `/mnt/cross/baz.c`，那么命令

```
(gdb) set substitute-path /foo/bar /mnt/cross
```

将告诉 GDB 将 `/foo/bar` 替换为 `/mnt/cross`，这将允许 GDB 找到即使已被移动的文件 `baz.c`。

In the case when more than one substitution rule have been defined, the rules are evaluated one by one in the order where they have been defined. The first one matching, if any, is selected to perform the substitution.

For instance, if we had entered the following commands:

```
(gdb) set substitute-path /usr/src/include /mnt/include
(gdb) set substitute-path /usr/src /mnt/src
```

GDB would then rewrite `/usr/src/include/defs.h` into `/mnt/include/defs.h` by using the first rule. However, it would use the second rule to rewrite `/usr/src/lib/foo.c` into `/mnt/src/lib/foo.c`.

unset substitute-path [path]

If a path is specified, search the current list of substitution rules for a rule that would rewrite that path. Delete that rule if found. A warning is emitted by the debugger if no rule could be found.

If no path is specified, then all substitution rules are deleted.

show substitute-path [path]

If a path is specified, then print the source path substitution rule which would rewrite that path, if any.

If no path is specified, then print all existing source path substitution rules.

If your source path is cluttered with directories that are no longer of interest, GDB may sometimes cause confusion by finding the wrong versions of source. You can correct the situation as follows:

1. Use `directory` with no argument to reset the source path to its default value.
2. Use `directory` with suitable arguments to reinstall the directories you want in the source path. You can add all the directories in one command.

9.6 Source and Machine Code

You can use the command `info line` to map source lines to program addresses (and vice versa), and the command `disassemble` to display a range of addresses as machine instructions. You can use the command `set disassemble-next-line` to set whether to disassemble next source line when execution stops. When run under GNU Emacs mode, the `info line` command causes the arrow to point to the line specified. Also, `info line` prints addresses in symbolic form as well as hex.

info line

info line locspec

Print the starting and ending addresses of the compiled code for the source lines of the code locations that result from resolving `locspec`. See Section 9.2 [Location Specifications], page 128, for the various forms of `locspec`.

With no `locspec`, information is printed about either:

- the last line listed (see Section 9.1 [List], page 127)
- the line after the line shown by the previous `info line` command (see [info line, repeated calls], page 138)
- the current source line

当定义了多个替换规则时，这些规则将按照定义的顺序逐个进行评估。如果有匹配的，将选择第一个匹配的规则来执行替换。

例如，如果我们输入了以下命令：

```
(gdb) set substitute-path /usr/src/include /mnt/include
(gdb) set substitute-path /usr/src /mnt/src
```

GDB 会使用第一条规则将 `/usr/src/include/defs.h` 重写为 `/mnt/include/defs.h`。然而，它会使用第二条规则将 `/usr/src/lib/foo.c` 重写为 `/mnt/src/lib/foo.c`。

unset substitute-path [路径]

如果指定了路径，则搜索当前替换规则列表，查找能够重写该路径的规则。如果找到该规则，则将其删除。如果找不到规则，则调试器会发出警告。

如果没有指定路径，则所有替换规则都将被删除。

显示替换路径 [路径]

如果指定了路径，则打印出源路径替换规则，该规则会重写该路径（如果存在）。

如果没有指定路径，则打印所有现有的源路径替换规则。

如果你的源路径中混杂了不再感兴趣的目录，GDB 有时会因为找到错误的版本而造成混淆。你可以按以下方式纠正这种情况：

1. 使用不带参数的目录命令将源路径重置为其默认值。
2. 使用合适的参数的目录命令重新安装你希望在源路径中出现的目录。你可以在一个命令中添加所有目录。

9.6 源代码和机器代码

您可以使用 `info` 命令行将源代码行映射到程序地址（反之亦然），以及使用 `disassemble` 命令以机器指令的形式显示指定地址范围。您可以使用 `set disassemble-next-line` 命令设置当执行停止时是否反汇编下一个源代码行。在 GNU Emacs 模式下运行时，`info` 命令行会使箭头指向指定的行。此外，`info` 命令行还会以符号形式和十六进制形式打印地址。

info 命令行

info 命令行 locspec

显示由解析 `locspec` 得到的代码位置（即源代码行）的编译代码的起始和结束地址。参见第 9.2[位置规范],页 128，了解 `locspec` 的各种形式。

如果没有 `locspec`，将打印关于以下内容的信息：

- 最后列出的行（参见第9.1 [List],页127）
- 在先前信息行命令显示的行之后的那一行（参见 [info 行, 重复调用], 页138）
- 当前源代码行

For example, we can use `info line` to discover the location of the object code for the first line of function `m4_changequote`:

```
(gdb) info line m4_changequote
Line 895 of "builtin.c" starts at pc 0x634c <m4_changequote> and \
ends at 0x6350 <m4_changequote+4>.
```

We can also inquire, using `*addr` as the form for *locs**pec*, what source line covers a particular address *addr*:

```
(gdb) info line *0x63ff
Line 926 of "builtin.c" starts at pc 0x63e4 <m4_changequote+152> and \
ends at 0x6404 <m4_changequote+184>.
```

After `info line`, the default address for the `x` command is changed to the starting address of the line, so that `'x/i'` is sufficient to begin examining the machine code (see Section 10.6 [Examining Memory], page 154). Also, this address is saved as the value of the convenience variable `$_` (see Section 10.12 [Convenience Variables], page 173).

After `info line`, using `info line` again without specifying a location will display information about the next source line.

```
disassemble
disassemble /m
disassemble /s
disassemble /r
disassemble /b
```

This specialized command dumps a range of memory as machine instructions. It can also print mixed source+disassembly by specifying the `/m` or `/s` modifier and print the raw instructions in hex as well as in symbolic form by specifying the `/r` or `/b` modifier.

Only one of `/m` and `/s` can be used, attempting to use both flag will give an error.

Only one of `/r` and `/b` can be used, attempting to use both flag will give an error.

The default memory range is the function surrounding the program counter of the selected frame. A single argument to this command is a program counter value; GDB dumps the function surrounding this value. When two arguments are given, they should be separated by a comma, possibly surrounded by white-space. The arguments specify a range of addresses to dump, in one of two forms:

```
start,end
    the addresses from start (inclusive) to end (exclusive)

start,+length
    the addresses from start (inclusive) to start+length (exclusive).
```

When 2 arguments are specified, the name of the function is also printed (since there could be several functions in the given range).

The argument(s) can be any expression yielding a numeric value, such as `'0x32c4'`, `'&main+10'` or `'$pc - 8'`.

If the range of memory being disassembled contains current program counter, the instruction at that location is shown with a `=>` marker.

例如，我们可以使用`info line`来发现函数`m4_changequote`:的第一行对象代码的位置

```
(gdb) info line m4_changequote
Line 895 of "builtin.c" starts at pc 0x634c <m4_changequote> and \
ends at 0x6350 <m4_changequote+4>.
```

我们还可以使用 `*addr`作为`locs``pec`的格式来查询，某个地址`addr`对应的源代码行：

```
(gdb) info line *0x63ff
Line 926 of "builtin.c" starts at pc 0x63e4 <m4_changequote+152> and \
ends at 0x6404 <m4_changequote+184>.
```

执行`info line`后，`x`命令的默认地址会改为该行的起始地址，因此 `'x/i'` 就足够开始检查机器代码（参见第10.6 [内存检查], 页154）。此外，此地址还会被保存为便利变量`$_`的值（参见第10.12 [便利变量],页173）。

执行`info line`后，再次使用`info line`而不指定位置将会显示关于下一行源代码的信息。

```
反汇编
反汇编 /m
反汇编 /s
反汇编 /r
反汇编 /b
```

此专用命令将内存范围转储为机器指令。它还可以通过指定 `/m` 或 `/s` 修饰符来打印混合源+反汇编，并通过指定 `/r` 或 `/b` 修饰符来以十六进制形式以及符号形式打印原始指令。

`/m` 和 `/s` 只能使用其中一个，尝试同时使用这两个标志将导致错误。

`/r` 和 `/b` 只能使用其中一个，尝试同时使用这两个标志将导致错误。

默认的内存范围是所选帧的程序计数器周围的函数。此命令的一个参数是程序计数器值； GDB将此值周围的函数转储。当给出两个参数时，它们应该用逗号分隔，可能被空格包围。这些参数指定要转储的地址范围，有两种形式：

```
start,end
    从开始（包含）到结束（不包含）的地址

start,+length
    从开始（包含）到 start+length （不包含）。
```

当指定了2个参数时，也会打印函数的名称（因为给定范围内可能有多个函数）。

参数可以是任何能产生数值的表达式，例如 `'0x32c4'`、`'&main+10'` 或 `'$pc - 8'`。如果正在反汇编的内存范围包含当前程序计数器，则该位置上的指令会显示一个 `=>` 标记。

The following example shows the disassembly of a range of addresses of HP PA-RISC 2.0 code:

```
(gdb) disas 0x32c4, 0x32e4
Dump of assembler code from 0x32c4 to 0x32e4:
0x32c4 <main+204>:      addil 0,dp
0x32c8 <main+208>:      ldw 0x22c(sr0,r1),r26
0x32cc <main+212>:      ldil 0x3000,r31
0x32d0 <main+216>:      ble 0x3f8(sr4,r31)
0x32d4 <main+220>:      ldo 0(r31),rp
0x32d8 <main+224>:      addil -0x800,dp
0x32dc <main+228>:      ldo 0x588(r1),r26
0x32e0 <main+232>:      ldil 0x3000,r31
End of assembler dump.
```

The following two examples are for RISC-V, and demonstrates the difference between the /r and /b modifiers. First with /b, the bytes of the instruction are printed, in hex, in memory order:

```
(gdb) disassemble /b 0x00010150,0x0001015c
Dump of assembler code from 0x10150 to 0x1015c:
0x00010150 <call_me+4>:      22 dc          sw s0,56(sp)
0x00010152 <call_me+6>:      80 00          addi s0,sp,64
0x00010154 <call_me+8>:      23 26 a4 fe      sw a0,-20(s0)
0x00010158 <call_me+12>:     23 24 b4 fe      sw a1,-24(s0)
End of assembler dump.
```

In contrast, with /r the bytes of the instruction are displayed in the instruction order, for RISC-V this means that the bytes have been swapped to little-endian order:

```
(gdb) disassemble /r 0x00010150,0x0001015c
Dump of assembler code from 0x10150 to 0x1015c:
0x00010150 <call_me+4>:      dc22          sw s0,56(sp)
0x00010152 <call_me+6>:      0080          addi s0,sp,64
0x00010154 <call_me+8>:      fea42623      sw a0,-20(s0)
0x00010158 <call_me+12>:     feb42423      sw a1,-24(s0)
End of assembler dump.
```

Here is an example showing mixed source+assembly for Intel x86 with /m or /s, when the program is stopped just after function prologue in a non-optimized function with no inline code.

```
(gdb) disas /m main
Dump of assembler code for function main:
5      {
    0x08048330 <+0>:      push    %ebp
    0x08048331 <+1>:      mov     %esp,%ebp
    0x08048333 <+3>:      sub     $0x8,%esp
    0x08048336 <+6>:      and     $0xffffffff0,%esp
    0x08048339 <+9>:      sub     $0x10,%esp

6      printf ("Hello.\n");
=> 0x0804833c <+12>:     movl    $0x8048440,(%esp)
    0x08048343 <+19>:     call   0x8048284 <puts@plt>

7      return 0;
8      }
    0x08048348 <+24>:     mov     $0x0,%eax
    0x0804834d <+29>:     leave   %eax
    0x0804834e <+30>:     ret
```

以下示例展示了 HP PA-RISC 2.0 代码地址范围的反汇编：

```
(gdb) disas 0x32c4,0x32e4 报错汇编代码从
0x32c4到0x32e4: 0x32c4 <main+204>: addil 0,dp
0x32c8 <main+208>: ldw 0x22c(sr0,r1),r26 0x32cc <
main+212>: ldil 0x3000,r31 0x32d0 <main+216>:
ble 0x3f8(sr4,r31) 0x32d4 <main+220>: ldo
0(r31),rp 0x32d8 <main+224>: addil -0x800,dp
0x32dc <main+228>: ldo 0x588(r1),r26 0x32e0 <
main+232>: ldil 0x3000,r31 汇编代码报错结束。
```

以下两个示例适用于 RISC-V，并展示了 /r 和 /b 修饰符之间的区别。首先使用 /b, 指令的字节将按内存顺序以十六进制形式打印：

```
(gdb) disassemble /b 0x00010150,0x0001015c
Dump of assembler code from 0x10150 to 0x1015c:
0x00010150 <call_me+4>:      22 dc          sw s0,56(sp)
0x00010152 <call_me+6>:      80 00          addi s0,sp,64
0x00010154 <call_me+8>:      23 26 a4 fe      sw a0,-20(s0)
0x00010158 <call_me+12>:     23 24 b4 fe      sw a1,-24(s0)
End of assembler dump.
```

相比之下，使用 /r 时，指令的字节将按指令顺序显示，对于 RISC-V 来说这意味着字节需要被交换到小端序：

```
(gdb) disassemble /r 0x00010150,0x0001015c
Dump of assembler code from 0x10150 to 0x1015c:
0x00010150 <call_me+4>:      dc22          sw s0,56(sp)
0x00010152 <call_me+6>:      0080          addi s0,sp,64
0x00010154 <call_me+8>:      fea42623      sw a0,-20(s0)
0x00010158 <call_me+12>:     feb42423      sw a1,-24(s0)
End of assembler dump.
```

这里有一个示例，展示了在 Intel x86 汇编中使用 /m 或 /s 的混合源 + 汇编，当程序在非优化函数中的函数序言之后停止时。

```
(gdb) disas /m main
Dump of assembler code for function main:
5      {
    0x08048330 <+0>:      push    %ebp
    0x08048331 <+1>:      mov     %esp,%ebp
    0x08048333 <+3>:      sub     $0x8,%esp
    0x08048336 <+6>:      and     $0xffffffff0,%esp
    0x08048339 <+9>:      sub     $0x10,%esp

6      printf ("Hello.\n");
=> 0x0804833c <+12>:     movl    $0x8048440,(%esp)
    0x08048343 <+19>:     call   0x8048284 <puts@plt>

7      return 0;
8      }
    0x08048348 <+24>:     mov     $0x0,%eax
    0x0804834d <+29>:     leave   %eax
    0x0804834e <+30>:     ret
```


End of assembler dump.

The `/m` option is deprecated as its output is not useful when there is either inlined code or re-ordered code. The `/s` option is the preferred choice. Here is an example for AMD x86-64 showing the difference between `/m` output and `/s` output. This example has one inline function defined in a header file, and the code is compiled with `-O2` optimization. Note how the `/m` output is missing the disassembly of several instructions that are present in the `/s` output.

```
foo.h:
int
foo (int a)
{
    if (a < 0)
        return a * 2;
    if (a == 0)
        return 1;
    return a + 10;
}

foo.c:
#include "foo.h"
volatile int x, y;
int
main ()
{
    x = foo (y);
    return 0;
}

(gdb) disas /m main
Dump of assembler code for function main:
5 {

6   x = foo (y);
   0x0000000000400400 <+0>: mov     0x200c2e(%rip),%eax # 0x601034 <y>
   0x0000000000400417 <+23>: mov     %eax,0x200c13(%rip) # 0x601030 <x>

7   return 0;
8 }
   0x000000000040041d <+29>: xor     %eax,%eax
   0x000000000040041f <+31>: retq
   0x0000000000400420 <+32>: add     %eax,%eax
   0x0000000000400422 <+34>: jmp     0x400417 <main+23>

End of assembler dump.
(gdb) disas /s main
Dump of assembler code for function main:
foo.c:
5 {
6   x = foo (y);
   0x0000000000400400 <+0>: mov     0x200c2e(%rip),%eax # 0x601034 <y>

foo.h:
4   if (a < 0)
   0x0000000000400406 <+6>: test    %eax,%eax
   0x0000000000400408 <+8>: js      0x400420 <main+32>

6   if (a == 0)
```

End of assembler dump.

`/m` 选项已弃用，因为当存在内联代码或代码重新排序时，其输出没有用。`/s` 选项是首选选择。这里有一个 AMD x86-64 的示例，展示了 `/m` 输出和 `/s` 输出的区别。这个示例中有一个在头文件中定义的内联函数，代码使用 `-O2` 优化编译。注意 `/m` 输出缺少了 `/s` 输出中存在的几条指令的汇编代码。

```
foo.h:
int
foo (int a)
{
    if (a < 0)
        return a * 2;
    if (a == 0)
        return 1;
    return a + 10;
}

foo.c:
#include "foo.h"
volatile int x, y;
int
main ()
{
    x = foo (y);
    return 0;
}

(gdb) disas /m main
Dump of assembler code for function main:
5 {

6   x = foo (y);
   0x0000000000400400 <+0>: mov     0x200c2e(%rip),%eax # 0x601034 <y>
   0x0000000000400417 <+23>: mov     %eax,0x200c13(%rip) # 0x601030 <x>

7   return 0;
8 }

6   x = foo (y);
   0x0000000000400400 <+0>: mov     0x200c2e(%rip),%eax # 0x601034 <y>
   0x0000000000400417 <+23>: mov     %eax,0x200c13(%rip) # 0x601030 <x>

7   return 0;
8 }
   0x000000000040041d <+29>: xor     %eax,%eax
   0x000000000040041f <+31>: retq
   0x0000000000400420 <+32>: add     %eax,%eax
   0x0000000000400422 <+34>: jmp     0x400417 <main+23>

End of assembler dump.
(gdb) disas /s main
Dump of assembler code for function main:
foo.c:
5 {
6   x = foo (y);
   0x0000000000400400 <+0>: mov     0x200c2e(%rip),%eax # 0x601034 <y>

foo.h:
4   if (a < 0)
   0x0000000000400406 <+6>: test    %eax,%eax
   0x0000000000400408 <+8>: js      0x400420 <main+32>

6   if (a == 0)
```

```
7     return 1;
8     return a + 10;
0x000000000040040a <+10>: lea    0xa(%rax),%edx
0x000000000040040d <+13>: test   %eax,%eax
0x000000000040040f <+15>: mov    $0x1,%eax
0x0000000000400414 <+20>: cmovne %edx,%eax

foo.c:
6     x = foo (y);
0x0000000000400417 <+23>: mov    %eax,0x200c13(%rip) # 0x601030 <x>

7     return 0;
8 }
0x000000000040041d <+29>: xor     %eax,%eax
0x000000000040041f <+31>: retq

foo.h:
5     return a * 2;
0x0000000000400420 <+32>: add     %eax,%eax
0x0000000000400422 <+34>: jmp     0x400417 <main+23>
End of assembler dump.
```

Here is another example showing raw instructions in hex for AMD x86-64,

```
(gdb) disas /r 0x400281,+10
Dump of assembler code from 0x400281 to 0x40028b:
0x0000000000400281: 38 36 cmp    %dh,(%rsi)
0x0000000000400283: 2d 36 34 2e 73 sub    $0x732e3436,%eax
0x0000000000400288: 6f      outsl  %ds:(%rsi),(%dx)
0x0000000000400289: 2e 32 00      xor    %cs:(%rax),%al
End of assembler dump.
```

Note that the ‘disassemble’ command’s address arguments are specified using expressions in your programming language (see Section 10.1 [Expressions], page 147), not location specs (see Section 9.2 [Location Specifications], page 128). So, for example, if you want to disassemble function `bar` in file `foo.c`, you must type ‘disassemble ‘foo.c’::bar’ and not ‘disassemble foo.c:bar’.

Some architectures have more than one commonly-used set of instruction mnemonics or other syntax.

For programs that were dynamically linked and use shared libraries, instructions that call functions or branch to locations in the shared libraries might show a seemingly bogus location—it’s actually a location of the relocation table. On some architectures, GDB might be able to resolve these to actual function names.

```
set disassembler-options option1[,option2...]
```

This command controls the passing of target specific information to the disassembler. For a list of valid options, please refer to the `-M/--disassembler-options` section of the ‘objdump’ manual and/or the output of `objdump --help` (see Section “objdump” in *The GNU Binary Utilities*). The default value is the empty string.

If it is necessary to specify more than one disassembler option, then multiple options can be placed together into a comma separated list. Currently this command is only supported on targets ARC, ARM, MIPS, PowerPC and S/390.

```
show disassembler-options
```

Show the current setting of the disassembler options.

```
7     return 1;
8     return a + 10;
0x000000000040040a <+10>: lea    0xa(%rax),%edx
0x000000000040040d <+13>: test   %eax,%eax
0x000000000040040f <+15>: mov    $0x1,%eax
0x0000000000400414 <+20>: cmovne %edx,%eax

foo.c:
6     x = foo (y);
0x0000000000400417 <+23>: mov    %eax,0x200c13(%rip) # 0x601030 <x>

7     return 0;
8 }
0x000000000040041d <+29>: xor     %eax,%eax
0x000000000040041f <+31>: retq

foo.h:
5     return a * 2;
0x0000000000400420 <+32>: add     %eax,%eax
0x0000000000400422 <+34>: jmp     0x400417 <main+23>
End of assembler dump.
```

这里有一个例子，展示了AMD x86-64的原始十六进制指令，

```
(gdb) disas /r 0x400281,+10
Dump of assembler code from 0x400281 to 0x40028b:
0x0000000000400281: 38 36 cmp    %dh,(%rsi)
0x0000000000400283: 2d 36 34 2e 73 sub    $0x732e3436,%eax
0x0000000000400288: 6f      outsl  %ds:(%rsi),(%dx)
0x0000000000400289: 2e 32 00      xor    %cs:(%rax),%al
End of assembler dump.
```

请注意，‘disassemble’命令的地址参数是使用您的编程语言中的表达式指定的（参见第10.1 [表达式], 页147），而不是位置规范（参见第9.2 [位置规范], 页128）。因此，例如，如果您想要反汇编文件`foo.c`中的函数`bar`，您必须输入 ‘disassemble ‘foo.c’::bar’，而不是 ‘disassemble foo.c:bar’。

有些架构有多个常用的指令助记符或其他语法集。

对于动态链接并使用共享库的程序，调用共享库中函数或跳转到共享库位置的指令可能会显示一个看似无效的位置——实际上是重定位表的位置。在某些架构上，GDB 可能能够解析这些为实际函数名。

```
set disassembler-options option1[,option2...]
```

此命令控制将目标特定信息传递给反汇编器。有关有效选项的列表，请参阅“objdump”手册的-M/--disassembler-选项部分以及或 `objdump --help` 的输出（参见GNU二进制工具中的“objdump”部分）。默认值为空字符串。

如果需要指定多个反汇编器选项，则可以将多个选项放在一起，用逗号分隔的列表。目前该命令仅支持目标 ARC、ARM、MIPS、PowerPC 和 S/390。

```
show disassembler-options
```

显示反汇编选项的当前设置。

set disassembly-flavor *instruction-set*

Select the instruction set to use when disassembling the program via the **disassemble** or **x/i** commands.

Currently this command is only defined for the Intel x86 family. You can set *instruction-set* to either **intel** or **att**. The default is **att**, the AT&T flavor used by default by Unix assemblers for x86-based targets.

show disassembly-flavor

Show the current setting of the disassembly flavor.

set disassemble-next-line

show disassemble-next-line

Control whether or not GDB will disassemble the next source line or instruction when execution stops. If **ON**, GDB will display disassembly of the next source line when execution of the program being debugged stops. This is *in addition* to displaying the source line itself, which GDB always does if possible. If the next source line cannot be displayed for some reason (e.g., if GDB cannot find the source file, or there's no line info in the debug info), GDB will display disassembly of the next *instruction* instead of showing the next source line. If **AUTO**, GDB will display disassembly of next instruction only if the source line cannot be displayed. This setting causes GDB to display some feedback when you step through a function with no line info or whose source file is unavailable. The default is **OFF**, which means never display the disassembly of the next line or instruction.

9.7 Disable Reading Source Code

In some cases it can be desirable to prevent GDB from accessing source code files. One case where this might be desirable is if the source code files are located over a slow network connection.

The following command can be used to control whether GDB should access source code files or not:

set source open [on|off]

show source open

When this option is **on**, which is the default, GDB will access source code files when needed, for example to print source lines when GDB stops, or in response to the **list** command.

When this option is **off**, GDB will not access source code files.

设置反汇编风格 *instruction-set*

通过反汇编器 **x/i** 命令反汇编程序时，选择要使用的指令集。

目前此命令仅适用于Intel x86系列。您可以设置指令集为**intel**或**att**。默认为**att**，即Unix汇编器在x86目标上默认使用的AT&T风格。

显示反汇编风格

显示当前反汇编风格的设置。

设置反汇编下一行

显示反汇编下一行

控制当执行停止时，是否反汇编下一行源代码或指令。如果开启，GDB 将在调试程序执行停止时显示下一行源代码的反汇编。这除了显示源代码本身外，GDB 如果可能，始终会显示源代码行。如果由于某种原因（例如，GDB 找不到源文件，或调试信息中没有行信息），GDB 将显示下一条指令的反汇编，而不是显示下一行源代码。如果为自动，GDB 只有在无法显示源代码行时才会显示下一条指令的反汇编。此设置导致 GDB在单步执行没有行信息或源文件不可用的函数时显示一些反馈。默认为关闭，这意味着从不显示下一行或指令的反汇编。

9.7 禁用读取源代码

在某些情况下，可能希望阻止 GDB 访问源代码文件。一个可能希望这样做的情况是，如果源代码文件位于慢速网络连接上。

以下命令可用于控制是否允许 GDB 访问源代码文件：

设置源打开 [关闭|关闭]

show source open

当此选项开启时（默认设置），GDB 会在需要时访问源代码文件，例如在 GDB 停止时打印源代码行，或在响应列表命令时。

当此选项关闭时，GDB 将不会访问源代码文件。

10 Examining Data

The usual way to examine data in your program is with the `print` command (abbreviated `p`), or its synonym `inspect`. It evaluates and prints the value of an expression of the language your program is written in (see Chapter 15 [Using GDB with Different Languages], page 229). It may also print the expression using a Python-based pretty-printer (see Section 10.10 [Pretty Printing], page 170).

```
print [[options] --] expr
print [[options] --] /f expr
```

`expr` is an expression (in the source language). By default the value of `expr` is printed in a format appropriate to its data type; you can choose a different format by specifying `/f`, where `f` is a letter specifying the format; see Section 10.5 [Output Formats], page 152.

The `print` command supports a number of options that allow overriding relevant global print settings as set by `set print` subcommands:

- `-address [on|off]`
Set printing of addresses. Related setting: `[set print address]`, page 159.
- `-array [on|off]`
Pretty formatting of arrays. Related setting: `[set print array]`, page 161.
- `-array-indexes [on|off]`
Set printing of array indexes. Related setting: `[set print array-indexes]`, page 161.
- `-characters number-of-characters|elements|unlimited`
Set limit on string characters to print. The value `elements` causes the limit on array elements to print to be used. The value `unlimited` causes there to be no limit. Related setting: `[set print characters]`, page 161.
- `-elements number-of-elements|unlimited`
Set limit on array elements and optionally string characters to print. See `[set print characters]`, page 161, and the `-characters` option above for when this option applies to strings. The value `unlimited` causes there to be no limit. See `[set print elements]`, page 162, for a related CLI command.
- `-max-depth depth|unlimited`
Set the threshold after which nested structures are replaced with ellipsis. Related setting: `[set print max-depth]`, page 166.
- `-nibbles [on|off]`
Set whether to print binary values in groups of four bits, known as “nibbles”. See `[set print nibbles]`, page 161.
- `-memory-tag-violations [on|off]`
Set printing of additional information about memory tag violations. See `[set print memory-tag-violations]`, page 166.

10 检查数据

检查程序中数据的一般方法是使用 `print` 命令（缩写为 `p`）或其同义词 `inspect`。它会计算并打印用程序所写语言编写的表达式的值（参见第 15 [使用 GDB 与不同语言], 页 229）。它还可以使用基于 Python 的格式化打印机打印表达式（参见第 10.10[格式化打印], 页 170）。

```
print [[options] --] expr
print [[options] --] /f expr
```

`expr` 是一个表达式（在源语言中）。默认情况下，`expr` 的值是以适合其数据类型的格式打印；你可以通过指定 `/f` 选择不同的格式，其中 `f` 是指定格式的字母；参见第 10.5[输出格式], 页 152。

`print` 命令支持多个选项，允许覆盖 `setprint` 子命令设置的相应全局打印设置：

- `-address [on|off]`
设置地址的打印。相关设置： `[set print address]`, 页 159。
- `-array [on|off]`
数组的漂亮格式化。相关设置： `[设置打印数组]`, 第161页。
- `-数组索引 [开关]`
设置打印数组索引。相关设置： `[设置打印数组索引]`, 第161页。
- `-characters number-of-characters|元素|无限制`
设置打印字符串字符数的限制。值 `elements` 会使用打印数组元素的限制。值 `unlimited` 表示无限制。相关设置： `[设置打印字符]`, 页 161。
- `-elements number-of-elements|无限制`
设置打印数组元素和可选的字符串字符数。参见 `[设置打印字符]`, 页 161, 以及上面 `-characters` 选项，了解此选项何时适用于字符串。值 `unlimited` 表示无限制。参见 `[设置打印元素]`, 页 162, 了解相关的 CLI 命令。
- `-max-depth depth|无限制`
设置嵌套结构的替换为省略号的阈值。相关设置： `[设置打印最大深度]`, 页 166。
- `-nibbles [开关]`
设置是否以四位二进制值为一组打印，称为“半字节”。参见 `[设置打印半字节]`, 页161。
- `-内存标签违规 [开关]`
设置打印内存标签违规的附加信息。参见 `[设置打印内存标签违规]`, 页 166。

- null-stop** [on|off]
Set printing of char arrays to stop at first null char. Related setting: [set print null-stop], page 167.
- object** [on|off]
Set printing C++ virtual function tables. Related setting: [set print object], page 169.
- pretty** [on|off]
Set pretty formatting of structures. Related setting: [set print pretty], page 167.
- raw-values** [on|off]
Set whether to print values in raw form, bypassing any pretty-printers for that value. Related setting: [set print raw-values], page 167.
- repeats** *number-of-repeats*|unlimited
Set threshold for repeated print elements. **unlimited** causes all elements to be individually printed. Related setting: [set print repeats], page 165.
- static-members** [on|off]
Set printing C++ static members. Related setting: [set print static-members], page 169.
- symbol** [on|off]
Set printing of symbol names when printing pointers. Related setting: [set print symbol], page 161.
- union** [on|off]
Set printing of unions interior to structures. Related setting: [set print union], page 168.
- vtbl** [on|off]
Set printing of C++ virtual function tables. Related setting: [set print vtbl], page 169.

Because the **print** command accepts arbitrary expressions which may look like options (including abbreviations), if you specify any command option, then you must use a double dash (--) to mark the end of option processing.

For example, this prints the value of the **-p** expression:

```
(gdb) print -p
```

While this repeats the last value in the value history (see below) with the **-pretty** option in effect:

```
(gdb) print -p --
```

Here is an example including both on option and an expression:

- null**停止 [开|关]
设置字符数组打印在第一个空字符处停止。相关设置: [设置打印空停止], 第167页。
- 对象** [开|关]
设置打印 C++ 虚拟函数表。相关设置: [设置打印对象],第169页。
- 美化** [开|关]
设置结构的美化格式。相关设置: [设置打印美化],第167页。
- raw-values** [on|off]
设置是否以原始格式打印值, 绕过该值的任何美化打印器。相关设置: [set print raw-values],页 167。
- repeats** *number-of-repeats*|unlimited
设置重复打印元素的阈值。**unlimited** 会导致所有元素被重复打印。[set print repeats],页 165。
- 静态成员** [开|关]
设置打印 C++ 静态成员。相关设置: [设置打印静态成员],第 169 页。
- 符号** [开|关]
设置打印指针时打印符号名称。相关设置: [设置打印符号],第 161 页。
- 联合** [开|关]
设置打印结构内部联合的内容。相关设置: [设置打印联合], 第 168 页。
- vtbl** [on|off]
设置 C++ 虚拟函数表的打印。相关设置: [设置打印 vtbl], 页 169。

因为打印命令接受任意表达式, 这些表达式可能看起来像选项 (包括缩写), 如果你指定任何命令选项, 那么你必须使用双破折号 (--) 来标记选项处理的结束。

例如, 这会打印 **-p** 表达式的值:

```
(gdb) print -p
```

而这是用 **-pretty** 选项无效地重复了值历史中的最后一个值 (见下文):

```
(gdb) print -p --
```

这里有一个包含选项和表达式的示例:

```
(gdb) print -pretty -- *myptr
$1 = {
  next = 0x0,
  flags = {
    sweet = 1,
    sour = 1
  },
  meat = 0x54 "Pork"
}
```

```
print [options]
print [options] /f
```

If you omit *expr*, GDB displays the last value again (from the *value history*; see Section 10.11 [Value History], page 172). This allows you to conveniently inspect the same value in an alternative format.

If the architecture supports memory tagging, the **print** command will display pointer/memory tag mismatches if what is being printed is a pointer or reference type. See Section 10.7 [Memory Tagging], page 157.

A more low-level way of examining data is with the **x** command. It examines data in memory at a specified address and prints it in a specified format. See Section 10.6 [Examining Memory], page 154.

If you are interested in information about types, or about how the fields of a struct or a class are declared, use the **ptype *expr*** command rather than **print**. See Chapter 16 [Examining the Symbol Table], page 265.

Another way of examining values of expressions and type information is through the Python extension command **explore** (available only if the GDB build is configured with **--with-python**). It offers an interactive way to start at the highest level (or, the most abstract level) of the data type of an expression (or, the data type itself) and explore all the way down to leaf scalar values/fields embedded in the higher level data types.

explore *arg*

arg is either an expression (in the source language), or a type visible in the current context of the program being debugged.

The working of the **explore** command can be illustrated with an example. If a data type **struct ComplexStruct** is defined in your C program as

```
struct SimpleStruct
{
  int i;
  double d;
};

struct ComplexStruct
{
  struct SimpleStruct *ss_p;
  int arr[10];
};
```

followed by variable declarations as

```
struct SimpleStruct ss = { 10, 1.11 };
struct ComplexStruct cs = { &ss, { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 } };
```

```
(gdb) print -pretty -- *myptr
$1 = {
  next = 0x0,
  flags = {
    sweet = 1,
    sour = 1
  },
  meat = 0x54 "Pork"
}
```

```
print [options]
print [options] /f
```

如果你省略 *expr*, GDB 将再次显示最后一个值 (来自值历史记录; 参见第 10.11 [值历史], 页 172)。这允许你以替代格式方便地检查相同的值。

如果架构支持内存标记, 打印命令将显示指针/内存标记不匹配, 如果打印的内容是指针或引用类型。参见第10.7 [内存标记], 页157。

检查数据的一种更底层的做法是使用 **x** 命令。它检查指定地址处的内存数据, 并以指定格式打印。参见第10.6 [检查内存], 页154。

如果您对类型信息感兴趣, 或想了解结构或类的字段如何声明, 请使用 **ptype *expr*** 命令而不是 **print**。参见第16 [检查符号表], 页265。

另一种检查表达式值和类型信息的方法是通过 Python 扩展命令 **explore** (仅在 GDB 构建配置为 **--with-python** 时可用)。它提供了一种交互式方法, 可以从最高级别 (或最抽象级别) 开始, 逐步深入到表达式数据类型 (或数据类型本身) 的所有层次, 直到嵌入在较高级别数据类型中的叶标量值/字段。

探索 *arg*

arg 要么是源语言中的表达式, 要么是当前调试程序上下文中可见的类型。

探索命令的工作原理可以通过一个例子来说明。如果您的 C 程序中定义了数据类型

struct ComplexStruct, 则

```
struct SimpleStruct
{
  int i;
  double d;
};

struct ComplexStruct
{
  struct SimpleStruct *ss_p;
  int arr[10];
};
```

followed by variable declarations as

```
struct SimpleStruct ss = { 10, 1.11 };
struct ComplexStruct cs = { &ss, { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 } };
```

then, the value of the variable `cs` can be explored using the `explore` command as follows.

```
(gdb) explore cs
The value of 'cs' is a struct/class of type 'struct ComplexStruct' with
the following fields:
```

```
ss_p = <Enter 0 to explore this field of type 'struct SimpleStruct *'>
arr = <Enter 1 to explore this field of type 'int [10]'>
```

Enter the field number of choice:

Since the fields of `cs` are not scalar values, you are being prompted to chose the field you want to explore. Let's say you choose the field `ss_p` by entering 0. Then, since this field is a pointer, you will be asked if it is pointing to a single value. From the declaration of `cs` above, it is indeed pointing to a single value, hence you enter `y`. If you enter `n`, then you will be asked if it were pointing to an array of values, in which case this field will be explored as if it were an array.

```
'cs.ss_p' is a pointer to a value of type 'struct SimpleStruct'
Continue exploring it as a pointer to a single value [y/n]: y
The value of '*(cs.ss_p)' is a struct/class of type 'struct
SimpleStruct' with the following fields:
```

```
i = 10 .. (Value of type 'int')
d = 1.1100000000000001 .. (Value of type 'double')
```

Press enter to return to parent value:

If the field `arr` of `cs` was chosen for exploration by entering 1 earlier, then since it is as array, you will be prompted to enter the index of the element in the array that you want to explore.

```
'cs.arr' is an array of 'int'.
Enter the index of the element you want to explore in 'cs.arr': 5
```

```
'(cs.arr)[5]' is a scalar value of type 'int'.
```

```
(cs.arr)[5] = 4
```

Press enter to return to parent value:

In general, at any stage of exploration, you can go deeper towards the leaf values by responding to the prompts appropriately, or hit the return key to return to the enclosing data structure (the *higher* level data structure).

Similar to exploring values, you can use the `explore` command to explore types. Instead of specifying a value (which is typically a variable name or an expression valid in the current context of the program being debugged), you specify a type name. If you consider the same example as above, your can explore the type `struct ComplexStruct` by passing the argument `struct ComplexStruct` to the `explore` command.

```
(gdb) explore struct ComplexStruct
```

By responding to the prompts appropriately in the subsequent interactive session, you can explore the type `struct ComplexStruct` in a manner similar to how the value `cs` was explored in the above example.

The `explore` command also has two sub-commands, `explore value` and `explore type`. The former sub-command is a way to explicitly specify that value exploration of the argu-

那么, 可以使用探索命令以如下方式查看变量 `cs` 的值。

(gdb) 探索 `cs` ‘`cs`’ 的值是一个类型为 ‘`struct ComplexStruct`’ 的结构/类的实例, 包含以下字段:

```
ss_p = <输入0以探索类型为 'struct SimpleStruct *' 的字段>
arr = <输入1以探索类型为 'int [10]' 的字段>
```

输入您要探索的字段编号:

由于`cs`的字段不是标量值, 系统提示您选择要探索的字段。假设您选择输入0以探索字段`ss_p`。然后, 因为这是一个指针, 系统会询问它是否指向单个值。根据`cs`的声明, 它确实指向单个值, 因此您输入`y`。如果您输入`n`, 系统会询问它是否指向值的数组, 在这种情况下, 该字段将作为数组进行探索。

```
'cs.ss_p' 是指向类型为 'struct SimpleStruct' 的值的指针, 继续
将其作为指向单个值的指针进行探索 [y/n]: y
'*(cs.ss_p)' 的值是一个类型为 'struct SimpleStruct' 的结构/类的实例, 包含以下字段:
```

```
i = 10 .. (值为 'int' 类型)
d = 1.1100000000000001 .. (值为 'double' 类型)
```

按回车键返回父值:

如果之前输入1选择了`cs`的field `arr`进行探索, 那么由于它是数组, 系统会提示你输入要探索的数组元素的索引。

```
'cs.arr'是一个'int'类型的数组。
输入要探索的'cs.arr'中的元素索引: 5
```

```
'(cs.arr)[5]'是一个'int'类型的标量值。
```

```
(cs.arr)[5] = 4
```

按回车键返回父值:

通常情况下, 在任何探索阶段, 您可以通过适当响应提示来深入到叶值, 或者按回车键返回到包含数据结构 (高级数据结构)。

与探索值类似, 您可以使用 `explore` 命令来探索类型。与指定值 (通常是程序当前调试上下文中有用的变量名或表达式) 不同, 您需要指定类型名。如果您考虑上述相同示例, 可以通过向 `explore` 命令传递参数 `struct ComplexStruct` 来探索类型 `struct ComplexStruct`。

```
(gdb) explore struct ComplexStruct
```

在后续的交互会话中适当响应提示, 您可以以类似于上述示例中探索值 `cs` 的方式来探索类型 `struct ComplexStruct`。

探索命令还有两个子命令, 探索值和探索类型。前者子命令是一种明确指定正在调用参数值探索的方式, 而后者则是一种明确指定正在调用参数类型探索的方式。

ment is being invoked, while the latter is a way to explicitly specify that type exploration of the argument is being invoked.

explore value *expr*

This sub-command of **explore** explores the value of the expression *expr* (if *expr* is an expression valid in the current context of the program being debugged). The behavior of this command is identical to that of the behavior of the **explore** command being passed the argument *expr*.

explore type *arg*

This sub-command of **explore** explores the type of *arg* (if *arg* is a type visible in the current context of program being debugged), or the type of the value/expression *arg* (if *arg* is an expression valid in the current context of the program being debugged). If *arg* is a type, then the behavior of this command is identical to that of the **explore** command being passed the argument *arg*. If *arg* is an expression, then the behavior of this command will be identical to that of the **explore** command being passed the type of *arg* as the argument.

10.1 Expressions

print and many other GDB commands accept an expression and compute its value. Any kind of constant, variable or operator defined by the programming language you are using is valid in an expression in GDB. This includes conditional expressions, function calls, casts, and string constants. It also includes preprocessor macros, if you compiled your program to include this information; see Section 4.1 [Compilation], page 33.

GDB supports array constants in expressions input by the user. The syntax is {*element, element...*}. For example, you can use the command **print {1, 2, 3}** to create an array of three integers. If you pass an array to a function or assign it to a program variable, GDB copies the array to memory that is **malloced** in the target program.

Because C is so widespread, most of the expressions shown in examples in this manual are in C. See Chapter 15 [Using GDB with Different Languages], page 229, for information on how to use expressions in other languages.

In this section, we discuss operators that may be available in GDB expressions in addition to those of your programming language. However, they are not necessarily available in all working languages.

Casts are supported in all languages, not just in C, because it is so useful to cast a number into a pointer in order to examine a structure at that address in memory.

GDB supports these operators, in addition to those common to programming languages:

- @ '@' is a binary operator for treating parts of memory as arrays. See Section 10.4 [Artificial Arrays], page 152, for more information.
- :: '::' allows you to specify a variable in terms of the file or function where it is defined. See Section 10.3 [Program Variables], page 149. This is currently not supported in Ada, Rust, Pascal, Modula-2 and D.

{*type*} *addr*

Refers to an object of type *type* stored at address *addr* in memory. The address *addr* may be any expression whose value is an integer or pointer (but parenthe-

前者子命令是一种明确指定正在调用参数值探索的方式，而后者则是一种明确指定正在调用参数类型探索的方式。

探索值 *expr*

explore 的这个子命令用于探索表达式 *expr* 的值（如果 *expr* 是当前调试程序上下文中有效的表达式）。该命令的行为与传递参数 *expr* 给 **explore** 命令的行为相同。

explore 类型 *arg*

explore 的这个子命令用于探索 *arg* 的类型（如果 *arg* 是当前调试程序上下文中可见的类型），或值/表达式的类型（如果 *arg* 是当前调试程序上下文中有效的表达式）。如果 *arg* 是类型，则该命令的行为与传递参数 *arg* 给 **explore** 命令的行为相同。如果 *arg* 是表达式，则该命令的行为将与传递 *arg* 类型的类型作为参数给 **explore** 命令的行为相同。

10.1 表达式

打印和许多其他 GDB 命令接受一个表达式并计算其值。你正在使用的编程语言定义的任何类型的常量、变量或运算符在 GDB 的表达式中都是有效的。这包括条件表达式、函数调用、类型转换和字符串常量。如果你编译程序以包含此信息，还包括预处理器宏；参见第 4.1 [编译]，页 33。

GDB 支持在用户输入的表达式中使用数组常量。语法是 {元素, 元素...}。例如，你可以使用命令 **print {1, 2, 3}** 来创建一个包含三个整数的数组。如果你将数组传递给函数或将其赋值给程序变量，GDB 会将数组复制到目标程序中 **malloc** 分配的内存中。

由于C语言非常普遍，本手册中示例中显示的大多数表达式都是用C语言编写的。有关如何在其他语言中使用表达式的信息，请参阅第 15 [使用 GDB 与不同语言]，页229。

在本节中，我们讨论除了编程语言之外，可能可在 GDB 表达式中使用的操作符。然而，它们并不一定在所有工作语言中都可用。

所有语言都支持强制转换，而不仅仅是 C 语言，因为将数字强制转换为指针以检查内存中该地址处的结构是非常有用的。

GDB 支持这些操作符，除了编程语言中常见的那些：

- @ '@' 是一个二元操作符，用于将内存的一部分视为数组。有关更多信息，请参阅第 10.4 [人工数组]，页 152。
- :: '::' 允许您根据其定义的文件或函数来指定一个变量。请参阅第 10.3 [程序变量]，页 149。目前这在 Ada、Rust、Pascal、Modula-2 和 D 中都不支持。

{*type*} *addr*

指在内存地址 *addr* 处存储的 *type* 类型对象。地址 *addr* 可以是任何值是整数或指针的表达式（但在二元运算符周围需要括号，就像在类型转换中一样）。无论 *addr* 通常预期存储何种类型的数据，这种结构都是允许的。

ses are required around binary operators, just as in a cast). This construct is allowed regardless of what kind of data is normally supposed to reside at *addr*.

10.2 Ambiguous Expressions

Expressions can sometimes contain some ambiguous elements. For instance, some programming languages (notably Ada, C++ and Objective-C) permit a single function name to be defined several times, for application in different contexts. This is called *overloading*. Another example involving Ada is generics. A *generic package* is similar to C++ templates and is typically instantiated several times, resulting in the same function name being defined in different contexts.

In some cases and depending on the language, it is possible to adjust the expression to remove the ambiguity. For instance in C++, you can specify the signature of the function you want to break on, as in *break function(types)*. In Ada, using the fully qualified name of your function often makes the expression unambiguous as well.

When an ambiguity that needs to be resolved is detected, the debugger has the capability to display a menu of numbered choices for each possibility, and then waits for the selection with the prompt '>'. The first option is always '[0] cancel', and typing *0 RET* aborts the current command. If the command in which the expression was used allows more than one choice to be selected, the next option in the menu is '[1] all', and typing *1 RET* selects all possible choices.

For example, the following session excerpt shows an attempt to set a breakpoint at the overloaded symbol `String::after`. We choose three particular definitions of that function name:

```
(gdb) b String::after
[0] cancel
[1] all
[2] file:String.cc; line number:867
[3] file:String.cc; line number:860
[4] file:String.cc; line number:875
[5] file:String.cc; line number:853
[6] file:String.cc; line number:846
[7] file:String.cc; line number:735
> 2 4 6
Breakpoint 1 at 0xb26c: file String.cc, line 867.
Breakpoint 2 at 0xb344: file String.cc, line 875.
Breakpoint 3 at 0xafcc: file String.cc, line 846.
Multiple breakpoints were set.
Use the "delete" command to delete unwanted
breakpoints.
(gdb)
```

`set multiple-symbols mode`

This option allows you to adjust the debugger behavior when an expression is ambiguous.

By default, *mode* is set to *all*. If the command with which the expression is used allows more than one choice, then GDB automatically selects all possible choices. For instance, inserting a breakpoint on a function using an ambiguous name results in a breakpoint inserted on each possible match. However, if a unique choice must be made, then GDB uses the menu to help you disambiguate

括号是必需的，就像在类型转换中一样）。无论 *addr* 通常预期存储何种类型的数据，这种结构都是允许的。

10.2 模糊表达式

表达式有时可能包含一些模糊元素。例如，某些编程语言（尤其是Ada、C++ 和 Objective-C）允许单个函数名被多次定义，用于不同的上下文。这称为重载。另一个涉及Ada的例子是泛型。一个泛型包类似于C++ 模板，通常会被多次实例化，导致相同的函数名在不同的上下文中被定义。

在某些情况下，根据语言的不同，可以调整表达式以消除模糊性。例如，在C++中，你可以指定你想要中断的函数的签名，就像 *break function(types)*那样。在Ada中，使用函数的完全限定名通常也能使表达式变得明确。

当检测到需要解决的模糊性时，调试器能够为每个可能性显示一个编号选项菜单，并等待使用提示 '>' 的选择。第一个选项总是 '[0]取消'，输入 *0 RET* 会中止当前命令。如果表达式所使用的命令允许多个选项被选中，菜单中的下一个选项是 '[1] 全部'，输入 *1 RET* 会选中所有可能的选项。

例如，以下会话摘录显示了一次尝试在重载符号 `String::after` 处设置断点。我们选择了该函数名的三个特定定义：

```
(gdb) b String::after
[0] cancel
[1] all
[2] file:String.cc; line number:867
[3] file:String.cc; line number:860
[4] file:String.cc; line number:875
[5] file:String.cc; line number:853
[6] file:String.cc; line number:846
[7] file:String.cc; line number:735
> 2 4 6
Breakpoint 1 at 0xb26c: file String.cc, line 867.
Breakpoint 2 at 0xb344: file String.cc, line 875.
Breakpoint 3 at 0xafcc: file String.cc, line 846.
Multiple breakpoints were set.
Use the "delete" command to delete unwanted
breakpoints.
(gdb)
```

设置多符号 *mode*

此选项允许您在表达式模糊时调整调试器行为。

默认情况下，模式设置为全部。如果用于表达式的命令允许多于一个选项，则

GDB 会自动选择所有可能的选项。例如，在名称模糊的函数上插入断点会导致在每个可能的匹配上插入断点。但是，如果必须选择一个唯一选项，则

GDB 会使用菜单来帮助您消除歧义

the expression. For instance, printing the address of an overloaded function will result in the use of the menu.

When *mode* is set to `ask`, the debugger always uses the menu when an ambiguity is detected.

Finally, when *mode* is set to `cancel`, the debugger reports an error due to the ambiguity and the command is aborted.

```
show multiple-symbols
Show the current value of the multiple-symbols setting.
```

10.3 Program Variables

The most common kind of expression to use is the name of a variable in your program.

Variables in expressions are understood in the selected stack frame (see Section 8.3 [Selecting a Frame], page 119); they must be either:

- global (or file-static)

or

- visible according to the scope rules of the programming language from the point of execution in that frame

This means that in the function

```
foo (a)
    int a;
{
    bar (a);
    {
        int b = test ();
        bar (b);
    }
}
```

you can examine and use the variable `a` whenever your program is executing within the function `foo`, but you can only use or examine the variable `b` while your program is executing inside the block where `b` is declared.

There is an exception: you can refer to a variable or function whose scope is a single source file even if the current execution point is not in this file. But it is possible to have more than one such variable or function with the same name (in different source files). If that happens, referring to that name has unpredictable effects. If you wish, you can specify a static variable in a particular function or file by using the colon-colon (`::`) notation (this may not be supported for all languages, see Section 10.1 [Expressions], page 147):

```
file::variable
function::variable
```

Here *file* or *function* is the name of the context for the static *variable*. In the case of file names, you can use quotes to make sure GDB parses the file name as a single word—for example, to print a global value of `x` defined in `f2.c`:

```
(gdb) p 'f2.c'::x
```

The `::` notation is normally used for referring to static variables, since you typically disambiguate uses of local variables in functions by selecting the appropriate frame and

对表达式。例如，打印重载函数的地址将导致使用菜单

当模式设置为“询问”时，调试器在检测到歧义时始终使用菜单。

最后，当模式设置为“取消”时，调试器由于歧义报告错误，并且命令被中止。歧义，并且命令被中止。

```
显示多个符号
显示多个符号设置的当前值。
```

10.3 程序变量

最常用的表达式类型是程序中变量的名称。

表达式中使用的变量是在选定的栈帧中理解的（参见第8.3节[选择一个帧], 第119页）；它们必须是：

- 全局（或文件静态）
- or
- 根据该帧中执行点的编程语言作用域规则可见

这意味着在函数

```
foo (a)
    int a;
{
    bar (a);
    {
        int b = test ();
        bar (b);
    }
}
```

你可以在程序在函数 `foo` 中执行时检查和使用变量 `a`，但在程序在声明 `b` 的代码块中执行时，你只能使用或检查变量 `b`。

有一个例外：即使当前执行点不在该文件中，你也可以引用作用域为单个源文件的变量或函数。但可能存在多个具有相同名称的此类变量或函数（位于不同的源文件中）。如果发生这种情况，引用该名称会产生不可预测的效果。如果你希望，可以通过使用双冒号 (`::`) 表示法在特定函数或文件中指定静态变量（这可能不适用于所有语言，请参阅第 10.1 [表达式], 页 147）：

```
file::variable
function::variable
```

这里，文件或函数是静态变量的上下文名称。对于文件名，你可以使用引号来确保 GDB 将文件名解析为单个单词——例如，以打印在 `f2.c` 中定义的全局 `x` 值：

```
(gdb) p 'f2.c'::x
```

`::` 语法通常用于引用静态变量，因为您通常通过选择合适的栈帧来消除函数中局部变量的歧义，并使用变量的简单名称。不过，您也可以使用这种语法来引用所选栈帧外围栈帧中的局部变量：

using the simple name of the variable. However, you may also use this notation to refer to local variables in frames enclosing the selected frame:

```
void
foo (int a)
{
    if (a < 10)
        bar (a);
    else
        process (a);    /* Stop here */
}

int
bar (int a)
{
    foo (a + 5);
}
```

For example, if there is a breakpoint at the commented line, here is what you might see when the program stops after executing the call `bar(0)`:

```
(gdb) p a
$1 = 10
(gdb) p bar::a
$2 = 5
(gdb) up 2
#2 0x080483d0 in foo (a=5) at foobar.c:12
(gdb) p a
$3 = 5
(gdb) p bar::a
$4 = 0
```

These uses of `::` are very rarely in conflict with the very similar use of the same notation in C++. When they are in conflict, the C++ meaning takes precedence; however, this can be overridden by quoting the file or function name with single quotes.

For example, suppose the program is stopped in a method of a class that has a field named `includefile`, and there is also an include file named `includefile` that defines a variable, `some_global`.

```
(gdb) p includefile
$1 = 23
(gdb) p includefile::some_global
A syntax error in expression, near ``.
(gdb) p 'includefile'::some_global
$2 = 27
```

Warning: Occasionally, a local variable may appear to have the wrong value at certain points in a function—just after entry to a new scope, and just before exit.

You may see this problem when you are stepping by machine instructions. This is because, on most machines, it takes more than one instruction to set up a stack frame (including local variable definitions); if you are stepping by machine instructions, variables may appear to have the wrong values until the stack frame is completely built. On exit, it usually also takes more than one machine instruction to destroy a stack frame; after you begin stepping through that group of instructions, local variable definitions may be gone.

This may also happen when the compiler does significant optimizations. To be sure of always seeing accurate values, turn off all optimization when compiling.

不过，您也可以使用这种语法来引用所选栈帧外围栈帧中的局部变量：

```
void
foo (int a)
{
    if (a < 10)
        bar (a);
    else
        process (a);    /* Stop here */
}

int
bar (int a)
{
    foo (a + 5);
}
```

例如，如果注释行处有一个断点，当程序执行完调用 `bar(0)` 后停止时，你可能会看到以下内容：

```
(gdb) p a
$1 = 10
(gdb) p bar::a
$2 = 5
(gdb) up 2
#2 0x080483d0 in foo (a=5) at foobar.c:12
(gdb) p a
$3 = 5
(gdb) p bar::a
$4 = 0
```

这些使用 `::` 的情况很少与 C++ 中非常相似的相同符号用法冲突。当它们冲突时，C++ 的含义优先；但是，这可以通过用单引号引用文件或函数名来覆盖。

例如，假设程序在一个名为 `includefile` 的字段的方法中停止，同时还有一个名为 `includefile` 的包含文件定义了一个变量，一些_全局变量。

```
(gdb) p includefile
$1 = 23
(gdb) p includefile::some_global
A syntax error in expression, near ``.
(gdb) p 'includefile'::some_global
$2 = 27
```

警告：偶尔，局部变量在函数的某些点可能显示为错误的值——刚刚进入新作用域时，以及刚刚退出时。

当您单步执行机器指令时，可能会遇到这个问题。这是因为，在大多数机器上，建立栈帧（包括局部变量定义）需要多条指令；如果您单步执行机器指令，变量可能会在栈帧完全建立之前显示错误的值。在退出时，销毁栈帧通常也需要多条机器指令；在您开始单步执行那组指令后，局部变量定义可能会消失。

这也可能发生在编译器进行重大优化时。为确保始终看到准确值，请在编译时关闭所有优化。

Another possible effect of compiler optimizations is to optimize unused variables out of existence, or assign variables to registers (as opposed to memory addresses). Depending on the support for such cases offered by the debug info format used by the compiler, GDB might not be able to display values for such local variables. If that happens, GDB will print a message like this:

```
No symbol "foo" in current context.
```

To solve such problems, either recompile without optimizations, or use a different debug info format, if the compiler supports several such formats. See Section 4.1 [Compilation], page 33, for more information on choosing compiler options. See Section 15.4.1 [C and C++], page 233, for more information about debug info formats that are best suited to C++ programs.

If you ask to print an object whose contents are unknown to GDB, e.g., because its data type is not completely specified by the debug information, GDB will say ‘<incomplete type>’. See Chapter 16 [Symbols], page 265, for more about this.

If you try to examine or use the value of a (global) variable for which GDB has no type information, e.g., because the program includes no debug information, GDB displays an error message. See Chapter 16 [Symbols], page 265, for more about unknown types. If you cast the variable to its declared type, GDB gets the variable’s value using the cast-to type as the variable’s type. For example, in a C program:

```
(gdb) p var
'var' has unknown type; cast it to its declared type
(gdb) p (float) var
$1 = 3.14
```

If you append `@entry` string to a function parameter name you get its value at the time the function got called. If the value is not available an error message is printed. Entry values are available only with some compilers. Entry values are normally also printed at the function parameter list according to [set print entry-values], page 163.

```
Breakpoint 1, d (i=30) at gdb.base/entry-value.c:29
29  i++;
(gdb) next
30  e (i);
(gdb) print i
$1 = 31
(gdb) print i@entry
$2 = 30
```

Strings are identified as arrays of `char` values without specified signedness. Arrays of either `signed char` or `unsigned char` get printed as arrays of 1 byte sized integers. `-fsigned-char` or `-funsigned-char` GCC options have no effect as GDB defines literal string type `"char"` as `char` without a sign. For program code

```
char var0[] = "A";
signed char var1[] = "A";
```

You get during debugging

```
(gdb) print var0
$1 = "A"
(gdb) print var1
$2 = {65 'A', 0 '\0'}
```

编译器优化的另一个可能效果是优化掉未使用的变量，或将变量分配给寄存器（而不是内存地址）。根据编译器使用的调试信息格式对这类情况的支持情况，GDB可能无法显示此类局部变量的值。如果发生这种情况，GDB 将打印类似以下的消息：

```
No symbol "foo" in current context.
```

要解决这类问题，可以重新编译不进行优化，或者使用不同的调试信息格式（如果编译器支持多种此类格式）。关于选择编译器选项的更多信息，请参见第4.1 [编译],页33。关于最适合C++程序的调试信息格式的更多信息，请参见第15.4.1 [C和C++],页233。

如果你要求打印一个其内容对 GDB未知（例如，因为其数据类型未完全由调试信息指定）的对象，GDB 会报告 ‘<不完整类型>’。关于这方面的更多信息，请参见第 16 [符号],页265。

如果你尝试检查或使用一个（全局）变量的值，而 GDB 没有该变量的类型信息（例如，因为程序不包含调试信息），GDB 会显示错误消息。关于未知类型的更多信息，请参见第 16 [符号], 页265。如果你将变量强制转换为它的声明类型，GDB 会使用强制转换的类型作为变量的类型来获取变量的值。例如，在一个C程序中：

```
(gdb) p var
'var' has unknown type; cast it to its declared type
(gdb) p (float) var
$1 = 3.14
```

如果你将 `@entry` string 追加到函数参数名中，你将获取该函数被调用时的值。如果值不可用，则会打印错误消息。入口值仅在特定编译器中可用。根据 [设置打印入口值], 页面 163，入口值通常也会在函数参数列表中打印。

```
Breakpoint 1, d (i=30) at gdb.base/entry-value.c:29
29  i++;
(gdb) next
30  e (i);
(gdb) print i
$1 = 31
(gdb) print i@entry
$2 = 30
```

字符串被视为未指定符号的无符号 `char` 值数组。有符号 `char` 或无符号 `char` 数组被打印为 1 字节大小的整数数组。`-fsigned-char` 或 `-funsigned-char` gcc 选项无效，因为 GDB 将字面量字符串类型 `"char"` 定义为无符号 `char`。对于程序代码

```
char var0[] = "A";
signed char var1[] = "A";
```

You get during debugging

```
(gdb) print var0
$1 = "A"
(gdb) print var1
$2 = {65 'A', 0 '\0' }
```

10.4 Artificial Arrays

It is often useful to print out several successive objects of the same type in memory; a section of an array, or an array of dynamically determined size for which only a pointer exists in the program.

You can do this by referring to a contiguous span of memory as an *artificial array*, using the binary operator ‘@’. The left operand of ‘@’ should be the first element of the desired array and be an individual object. The right operand should be the desired length of the array. The result is an array value whose elements are all of the type of the left argument. The first element is actually the left argument; the second element comes from bytes of memory immediately following those that hold the first element, and so on. Here is an example. If a program says

```
int *array = (int *) malloc (len * sizeof (int));
```

you can print the contents of array with

```
p *array@len
```

The left operand of ‘@’ must reside in memory. Array values made with ‘@’ in this way behave just like other arrays in terms of subscripting, and are coerced to pointers when used in expressions. Artificial arrays most often appear in expressions via the value history (see Section 10.11 [Value History], page 172), after printing one out.

Another way to create an artificial array is to use a cast. This re-interprets a value as if it were an array. The value need not be in memory:

```
(gdb) p/x (short[2])0x12345678
$1 = {0x1234, 0x5678}
```

As a convenience, if you leave the array length out (as in ‘(type[])value’) GDB calculates the size to fill the value (as ‘sizeof(value)/sizeof(type)’):

```
(gdb) p/x (short[])0x12345678
$2 = {0x1234, 0x5678}
```

Sometimes the artificial array mechanism is not quite enough; in moderately complex data structures, the elements of interest may not actually be adjacent—for example, if you are interested in the values of pointers in an array. One useful work-around in this situation is to use a convenience variable (see Section 10.12 [Convenience Variables], page 173) as a counter in an expression that prints the first interesting value, and then repeat that expression via RET. For instance, suppose you have an array dtab of pointers to structures, and you are interested in the values of a field fv in each structure. Here is an example of what you might type:

```
set $i = 0
p dtab[$i++]>fv
RET
RET
...
```

10.5 Output Formats

By default, GDB prints a value according to its data type. Sometimes this is not what you want. For example, you might want to print a number in hex, or a pointer in decimal. Or you might want to view data in memory at a certain address as a character string or as an instruction. To do these things, specify an *output format* when you print a value.

10.4 人工数组

在内存中打印出多个连续的同类型对象通常很有用；数组的一部分，或仅程序中存在一个指向动态确定大小数组的指针。

你可以通过将一个连续的内存空间当作一个人工数组来做到这一点，使用二进制运算符“@”。“@”的左操作数应该是所需数组的第一个元素，并且是一个单独的对象。右操作数应该是所需数组的长度。结果是元素类型都为左操作数类型的数组值。第一个元素实际上是左操作数；第二个元素来自紧随存放第一个元素的内存字节之后，以此类推。这里有一个例子。如果一个程序说

```
int *array = (int *) malloc (len * sizeof (int));
```

你可以打印数组的内容

```
p *数组@len
```

@的左操作数必须位于内存中。使用@这种方式创建的数组值在索引方面表现得和其他数组一样，并在表达式中被强制转换为指针。人工数组最常通过值历史（参见第10.11 [值历史], 页172）出现在表达式中，在打印一个值之后。

创建人工数组的另一种方法是使用强制转换。这会像数组一样重新解释一个值。该值不必在内存中：

```
(gdb) p/x (short[2])0x12345678
$1 = {0x1234, 0x5678}
```

为了方便，如果你省略数组长度（如在 ‘(type[])value’ 中），GDB 将自动计算以填充值的长度（如 ‘sizeof(value)/sizeof(type)’）：

```
(gdb) p/x (short[])0x12345678
$2 = {0x1234, 0x5678}
```

有时人工数组机制并不完全足够；在中等复杂度的数据结构中，感兴趣的元素可能并不相邻——例如，如果你关心数组中指针的值。在这种情况下，一个有用的变通方法是使用一个便利变量（参见第10.12 [便利变量], 页173）作为表达式中的计数器，该表达式用于打印第一个感兴趣的值，然后通过RET重复该表达式。例如，假设你有一个指向结构的指针数组dtab，并且你关心每个结构中字段fv的值。下面是一个你可能输入的例子：

```
set $i = 0
p dtab[$i++]>fv
RET
RET
...
```

10.5 输出格式

默认情况下，GDB 会根据其数据类型打印值。有时这不是你想要的结果。例如，你可能希望以十六进制形式打印一个数字，或以十进制形式打印一个指针。或者，你可能希望将内存中某个地址的数据视为字符串或指令来查看。为此，在打印值时指定输出格式。

The simplest use of output formats is to say how to print a value already computed. This is done by starting the arguments of the `print` command with a slash and a format letter. The format letters supported are:

- x** Print the binary representation of the value in hexadecimal.
- d** Print the binary representation of the value in decimal.
- u** Print the binary representation of the value as an decimal, as if it were unsigned.
- o** Print the binary representation of the value in octal.
- t** Print the binary representation of the value in binary. The letter ‘t’ stands for “two”.¹
- a** Print as an address, both absolute in hexadecimal and as an offset from the nearest preceding symbol. You can use this format used to discover where (in what function) an unknown address is located:

```
(gdb) p/a 0x54320
$3 = 0x54320 <_initialize_vx+396>
```

The command `info symbol 0x54320` yields similar results. See Chapter 16 [Symbols], page 265.
- c** Cast the value to an integer (unlike other formats, this does not just reinterpret the underlying bits) and print it as a character constant. This prints both the numerical value and its character representation. The character representation is replaced with the octal escape ‘\nnn’ for characters outside the 7-bit ASCII range.

Without this format, GDB displays `char`, `unsigned char`, and `signed char` data as character constants. Single-byte members of vectors are displayed as integer data.
- f** Regard the bits of the value as a floating point number and print using typical floating point syntax.
- s** Regard as a string, if possible. With this format, pointers to single-byte data are displayed as null-terminated strings and arrays of single-byte data are displayed as fixed-length strings. Other values are displayed in their natural types.

Without this format, GDB displays pointers to and arrays of `char`, `unsigned char`, and `signed char` as strings. Single-byte members of a vector are displayed as an integer array.
- z** Like ‘x’ formatting, the value is treated as an integer and printed as hexadecimal, but leading zeros are printed to pad the value to the size of the integer type.
- r** Print using the ‘raw’ formatting. By default, GDB will use a Python-based pretty-printer, if one is available (see Section 10.10 [Pretty Printing], page 170). This typically results in a higher-level display of the value’s contents. The ‘r’ format bypasses any Python pretty-printer which might exist.

¹ ‘b’ cannot be used because these format letters are also used with the x command, where ‘b’ stands for “byte”; see Section 10.6 [Examining Memory], page 154.

输出格式的最简单用法是说明如何打印已计算出的值。这通过在打印命令的参数前加上斜杠和一个格式字母来完成。支持的格式字母有：

- x** 打印该值的十六进制二进制表示。
- d** 打印该值的十进制二进制表示。
- u** 以无符号十进制形式打印该值的二进制表示，就好像它是无符号的一样。
- o** 打印该值的八进制二进制表示。
- t** 打印该值的二进制表示。字母 ‘t’ 代表 “两”。¹
- a** 以地址形式打印，同时以十六进制绝对值和相对于最近前一个符号的偏移量显示。您可以使用此格式来发现未知地址位于何处（在哪个函数中）：

```
(gdb) p/a 0x54320
$3 = 0x54320 <_initialize_vx+396>
```

命令信息符号 0x54320 会产生类似的结果。参见第 16[符号],页 265。
- c** 将值转换为整数（与其他格式不同，这不仅仅是重新解释底层的位），并将其作为字符常量打印。这会打印出数值和它的字符表示。对于7位 ASCII范围之外的字符，字符表示会用八进制转义序列 ‘\nnn’ 替换。

如果没有这种格式， GDB 将char、unsigned char和signed char数据显示为字符常量。向量的单字节数组成员显示为整数数据。
- f** 将值的位视为浮点数并使用典型方式打印浮点语法。
- s** 尽可能视为字符串。使用这种格式时，指向单字节数据的指针会显示为空终止字符串，单字节数据数组会显示为定长字符串。其他值会以它们的自然类型显示。

不使用这种格式时，GDB 会将指向char、unsigned char和signed char的指针和数组显示为字符串。向量的单字节成员会显示为整数数组。
- z** 类似于 ‘x’ 格式，值被视为整数并以十六进制打印，但会打印前导零以将值填充到整数类型的大小。
- r** 使用 ‘raw’ 格式进行打印。默认情况下， GDB 将使用基于Python的格式化打印工具，如果可用（参见第 10.10 [格式化打印], 页 170）。这通常会导致显示值内容的更高级形式。The ‘r’ 此格式绕过任何可能存在的 Python 美化打印机。

¹ ‘b’ 不能使用，因为这些格式字符也用于 x 命令，其中 ‘b’ 代表 “字节”；参见第 10.6 [检查内存], 页 154。

For example, to print the program counter in hex (see Section 10.14 [Registers], page 181), type

`p/x $pc`

Note that no space is required before the slash; this is because command names in GDB cannot contain a slash.

To reprint the last value in the value history with a different format, you can use the `print` command with just a format and no expression. For example, `'p/x'` reprints the last value in hex.

10.6 Examining Memory

You can use the command `x` (for “examine”) to examine memory in any of several formats, independently of your program’s data types.

`x/nfu addr`

`x addr`

`x` Use the `x` command to examine memory.

n, *f*, and *u* are all optional parameters that specify how much memory to display and how to format it; *addr* is an expression giving the address where you want to start displaying memory. If you use defaults for *nfu*, you need not type the slash `/`. Several commands set convenient defaults for *addr*.

n, the repeat count

The repeat count is a decimal integer; the default is 1. It specifies how much memory (counting by units *u*) to display. If a negative number is specified, memory is examined backward from *addr*.

f, the display format

The display format is one of the formats used by `print` (`'x'`, `'d'`, `'u'`, `'o'`, `'t'`, `'a'`, `'c'`, `'f'`, `'s'`), `'i'` (for machine instructions) and `'m'` (for displaying memory tags). The default is `'x'` (hexadecimal) initially. The default changes each time you use either `x` or `print`.

u, the unit size

The unit size is any of

- `b` Bytes.
- `h` Halfwords (two bytes).
- `w` Words (four bytes). This is the initial default.
- `g` Giant words (eight bytes).

Each time you specify a unit size with `x`, that size becomes the default unit the next time you use `x`. For the `'i'` format, the unit size is ignored and is normally not written. For the `'s'` format, the unit size defaults to `'b'`, unless it is explicitly given. Use `x /hs` to display 16-bit char strings and `x /ws` to display 32-bit strings. The next use of `x /s` will again display 8-bit strings. Note that the results depend on the programming language of the current compilation unit. If the language is C, the `'s'` modifier will use the UTF-16 encoding while

例如，要在十六进制中打印程序计数器（参见第 10.14 [寄存器],页 181），请输入

`p/x $pc`

请注意，斜杠前不需要空格；这是因为 GDB中的命令名不能包含斜杠。

要使用不同格式重新打印值历史中的最后一个值，您可以使用仅包含格式而不包含表达式的`print`命令。例如，“`p/x`”以十六进制重新打印最后一个值。

10.6 检查内存

您可以使用`x`命令（“检查”的缩写）以多种格式检查内存，而与您的程序数据类型无关。

`x/nfu addr`

`x addr`

`x` 使用`x`命令检查内存。

n、*f* 和 *u* 都是可选参数，用于指定显示多少内存以及如何格式化；*addr* 是一个表达式，给出你希望开始显示内存的地址。如果你使用 *nfu* 的默认值，就不需要输入斜杠 `/`。几个命令为 *addr* 设置了方便的默认值。

n, 重复次数

重复次数是一个十进制整数；默认值为 1。它指定要显示多少内存（以单位 *u* 计数）。如果指定了负数，内存将从 *addr* 向后检查。

f, 显示格式

显示格式是 `print` 使用的格式之一（`'x'`、`'d'`、`'u'`、`'o'`、`'t'`、`'a'`、`'c'`、`'f'`、`'s'`）、`'i'`（用于机器指令）和 `'m'`（用于显示内存标签）。初始默认值是 `'x'`（十六进制）。每次你使用 `x` 或 `print` 时，默认值都会改变。

u, 单位大小

单元大小可以是

- `b` 字节。
- `h` 半字（两个字节）。
- `w` 字（四个字节）。这是初始默认值。
- `g` 巨型字（八个字节）。

每次您使用 `x` 指定单位大小，该大小将成为下次使用 `x` 时的默认单位。对于 `"i"` 格式，单位大小会被忽略，通常不会写入。对于 `"s"` 格式，单位大小默认为 `"b"`，除非明确给出。使用 `x /hs` 显示 16 位字符字符串，使用 `x /ws` 显示 32 位字符串。`x /s` 的下次使用将再次显示 8 位字符串。请注意，结果取决于当前编译单元的编程语言。如果语言是 C，则 `"s"` 修饰符将使用 UTF-16 编码。

‘w’ will use UTF-32. The encoding is set by the programming language and cannot be altered.

addr, starting display address

addr is the address where you want GDB to begin displaying memory. The expression need not have a pointer value (though it may); it is always interpreted as an integer address of a byte of memory. See Section 10.1 [Expressions], page 147, for more information on expressions. The default for *addr* is usually just after the last address examined—but several other commands also set the default address: **info breakpoints** (to the address of the last breakpoint listed), **info line** (to the starting address of a line), and **print** (if you use it to display a value from memory).

For example, ‘x/3uh 0x54320’ is a request to display three halfwords (h) of memory, formatted as unsigned decimal integers (‘u’), starting at address 0x54320. ‘x/4xw \$sp’ prints the four words (‘w’) of memory above the stack pointer (here, ‘\$sp’; see Section 10.14 [Registers], page 181) in hexadecimal (‘x’).

You can also specify a negative repeat count to examine memory backward from the given address. For example, ‘x/-3uh 0x54320’ prints three halfwords (h) at 0x5431a, 0x5431c, and 0x5431e.

Since the letters indicating unit sizes are all distinct from the letters specifying output formats, you do not have to remember whether unit size or format comes first; either order works. The output specifications ‘4xw’ and ‘4wx’ mean exactly the same thing. (However, the count *n* must come first; ‘wx4’ does not work.)

Even though the unit size *u* is ignored for the formats ‘s’ and ‘i’, you might still want to use a count *n*; for example, ‘3i’ specifies that you want to see three machine instructions, including any operands. For convenience, especially when used with the **display** command, the ‘i’ format also prints branch delay slot instructions, if any, beyond the count specified, which immediately follow the last instruction that is within the count. The command **disassemble** gives an alternative way of inspecting machine instructions; see Section 9.6 [Source and Machine Code], page 137.

If a negative repeat count is specified for the formats ‘s’ or ‘i’, the command displays null-terminated strings or instructions before the given address as many as the absolute value of the given number. For the ‘i’ format, we use line number information in the debug info to accurately locate instruction boundaries while disassembling backward. If line info is not available, the command stops examining memory with an error message.

All the defaults for the arguments to **x** are designed to make it easy to continue scanning memory with minimal specifications each time you use **x**. For example, after you have inspected three machine instructions with ‘x/3i *addr*’, you can inspect the next seven with just ‘x/7’. If you use **RET** to repeat the **x** command, the repeat count *n* is used again; the other arguments default as for successive uses of **x**.

When examining machine instructions, the instruction at current program counter is shown with a => marker. For example:

```
(gdb) x/5i $pc-6
0x804837f <main+11>: mov    %esp,%ebp
0x8048381 <main+13>: push   %ecx
0x8048382 <main+14>: sub    $0x4,%esp
```

‘w’ 将使用UTF-32。编码由编程语言设置，无法更改。

addr, 起始显示地址

*addr*是你希望 GDB 开始显示内存的地址。该表达式不必具有指针值（尽管可以）；它始终被解释为内存字节的一个整数地址。有关表达式的更多信息，请参阅第10.1 [表达式],页147。*addr*的默认值通常是检查的最后一个地址之后——但还有其他几个命令也设置默认地址：info breakpoints（设置为列出的最后一个断点的地址），info line（设置为行的起始地址），以及print（如果你用它来显示内存中的值）。

例如，‘x/3uh 0x54320’是一个请求，要求显示三个半字（h）的内存，格式化为无符号十进制整数（‘u’），起始地址为0x54320。‘x/4xw \$sp’打印栈指针（此处为‘\$sp’；参见第10.14[寄存器],页181）上方的四个字（‘w’）的内存，格式为十六进制（‘x’）。

您也可以指定负的重复次数，从给定地址向后检查内存。例如，‘x/-3uh 0x54320’会打印位于0x5431a、0x5431c和0x5431e的三个半字（h）。

由于表示单位大小的字母与指定输出格式的字母完全不同，您不必记住单位大小或格式哪个先出现；两种顺序都可以。输出规范‘4xw’和‘4wx’表示完全相同的意思。（但是，计数*n*必须先出现；‘wx4’无效。）

尽管单位大小*u*对于格式‘s’和‘i’会被忽略，您可能仍然想使用计数*n*；例如，‘3i’指定您想看到三个机器指令，包括任何操作数。为了方便，特别是与显示命令一起使用时，格式‘i’还会打印超出指定计数的分支延迟槽指令（如果有），这些指令紧随在计数范围内的最后一个指令之后。反汇编命令提供了另一种检查机器指令的方式；参见第9.6节[源代码和机器代码],页137。

如果格式‘s’或‘i’指定了负数重复次数，命令会以给定数字的绝对值，在指定地址之前显示空终止字符串或指令。对于‘i’格式，我们在反汇编反向时使用调试信息中的行号信息来精确定位指令边界。如果行信息不可用，命令会显示错误消息并停止检查内存。

x的参数默认值旨在让您每次使用**x**时都能以最少的指定继续扫描内存。例如，在用‘x/3i *addr*’检查了三个机器指令后，您只需用‘x/7’即可检查接下来的七个。如果您使用**RET**重复**x**命令，重复次数*n*会被再次使用；其他参数的默认值与**x**的连续使用相同。

在检查机器指令时，当前程序计数器处的指令会显示一个 => 标记。例如：

```
(gdb) x/5i $pc-6 0x804837f <main+11>: mov
%esp,%ebp 0x8048381 <main+13>: push
%ecx
0x8048382 <main+14>: 子    $0x4,%esp
```



```
=> 0x8048385 <main+17>: movl    $0x8048460, (%esp)
0x804838c <main+24>: call    0x80482d4 <puts@plt>
```

If the architecture supports memory tagging, the tags can be displayed by using ‘m’. See Section 10.7 [Memory Tagging], page 157.

The information will be displayed once per granule size (the amount of bytes a particular memory tag covers). For example, AArch64 has a granule size of 16 bytes, so it will display a tag every 16 bytes.

Due to the way GDB prints information with the `x` command (not aligned to a particular boundary), the tag information will refer to the initial address displayed on a particular line. If a memory tag boundary is crossed in the middle of a line displayed by the `x` command, it will be displayed on the next line.

The ‘m’ format doesn’t affect any other specified formats that were passed to the `x` command.

The addresses and contents printed by the `x` command are not saved in the value history because there is often too much of them and they would get in the way. Instead, GDB makes these values available for subsequent use in expressions as values of the convenience variables `$_` and `$__`. After an `x` command, the last address examined is available for use in expressions in the convenience variable `$_`. The contents of that address, as examined, are available in the convenience variable `$__`.

If the `x` command has a repeat count, the address and contents saved are from the last memory unit printed; this is not the same as the last address printed if several units were printed on the last line of output.

Most targets have an addressable memory unit size of 8 bits. This means that to each memory address are associated 8 bits of data. Some targets, however, have other addressable memory unit sizes. Within GDB and this document, the term *addressable memory unit* (or *memory unit* for short) is used when explicitly referring to a chunk of data of that size. The word *byte* is used to refer to a chunk of data of 8 bits, regardless of the addressable memory unit size of the target. For most systems, addressable memory unit is a synonym of byte.

When you are debugging a program running on a remote target machine (see Chapter 20 [Remote Debugging], page 317), you may wish to verify the program’s image in the remote machine’s memory against the executable file you downloaded to the target. Or, on any target, you may want to check whether the program has corrupted its own read-only sections. The `compare-sections` command is provided for such situations.

`compare-sections` [*section-name*|-r]

Compare the data of a loadable section *section-name* in the executable file of the program being debugged with the same section in the target machine’s memory, and report any mismatches. With no arguments, compares all loadable sections. With an argument of `-r`, compares all loadable read-only sections.

Note: for remote targets, this command can be accelerated if the target supports computing the CRC checksum of a block of memory (see [qCRC packet], page 812).

```
=> 0x8048385 <main+17>: movl    $0x8048460, (%esp)
0x804838c <main+24>: call    0x80482d4 <puts@plt>
```

如果架构支持内存标记，可以通过使用 ‘m’ 来显示标记。参见第10.7 [内存标记],页157。

信息将按粒度大小（特定内存标记覆盖的字节数）每粒度显示一次。例如，AArch64的粒度大小为16字节，因此它将每16字节显示一个标记。

由于 GDB 使用 `x` 命令打印信息的方式（未对齐到特定边界），标记信息将引用特定行上显示的初始地址。如果在 `x` 命令显示的行中间跨越了内存标记边界，它将在下一行显示。

‘m’格式不会影响传递给 `x` 命令的任何其他指定格式。

`x` 命令打印的地址和内容不会保存在值历史记录中，因为它们通常太多，会妨碍查看。相反，GDB 将这些值作为便利变量 `$_` 和 `$__` 的值，供后续在表达式中使用。在 `x` 命令之后，最后检查的地址可用于在便利变量 `$_` 中的表达式中使用。该地址检查的内容可在便利变量 `$__` 中获取。

如果 `x` 命令有重复次数，则保存的地址和内容是最后打印的内存单元；这与最后打印的地址不同，如果有多個单元打印在输出最后一行的。

大多数目标具有 8 位可寻址内存单元大小。这意味着每个内存地址都与 8 位数据相关联。然而，某些目标具有其他可寻址内存单元大小。在 GDB 和本文档中，当明确指代该大小的数据块时，会使用“可寻址内存单元”（或简称为“内存单元”）这一术语。无论目标的可寻址内存单元大小如何，“字节”一词用于指代 8 位数据块。对于大多数系统，“可寻址内存单元”是“字节”的同义词。

当你在远程目标机器上调试程序时（参见第 20 章 [远程调试]，第 317 页），你可能希望将远程机器内存中的程序映像与下载到目标的可执行文件进行验证。或者，在任何目标上，你可能希望检查程序是否已损坏其自身的只读段。为此，提供了 `compare-sections` 命令。

比较节 [*section-name*|-r]

比较正在调试的程序的可加载段 *section-name* 在可执行文件中的数据与目标机器内存中相同段的数据，并报告任何不匹配。没有参数时，比较所有可加载段。

使用 `-r` 参数时，比较所有可加载的只读节。

注意：对于远程目标，如果目标支持计算内存块（内存块）的 CRC 校验和，此命令可以加速（参见 [qCRC 数据包],页 812）。

10.7 Memory Tagging

Memory tagging is a memory protection technology that uses a pair of tags to validate memory accesses through pointers. The tags are integer values usually comprised of a few bits, depending on the architecture.

There are two types of tags that are used in this setup: logical and allocation. A logical tag is stored in the pointers themselves, usually at the higher bits of the pointers. An allocation tag is the tag associated with particular ranges of memory in the physical address space, against which the logical tags from pointers are compared.

The pointer tag (logical tag) must match the memory tag (allocation tag) for the memory access to be valid. If the logical tag does not match the allocation tag, that will raise a memory violation.

Allocation tags cover multiple contiguous bytes of physical memory. This range of bytes is called a memory tag granule and is architecture-specific. For example, AArch64 has a tag granule of 16 bytes, meaning each allocation tag spans 16 bytes of memory.

If the underlying architecture supports memory tagging, like AArch64 MTE or SPARC ADI do, GDB can make use of it to validate pointers against memory allocation tags.

The `print` (see Chapter 10 [Data], page 143) and `x` (see Section 10.6 [Memory], page 154) commands will display tag information when appropriate, and a command prefix of `memory-tag` gives access to the various memory tagging commands.

The `memory-tag` commands are the following:

- memory-tag print-logical-tag *pointer_expression*

Print the logical tag stored in *pointer_expression*.
- memory-tag with-logical-tag *pointer_expression tag_bytes*

Print the pointer given by *pointer_expression*, augmented with a logical tag of *tag_bytes*.
- memory-tag print-allocation-tag *address_expression*

Print the allocation tag associated with the memory address given by *address_expression*.
- memory-tag setatag *starting_address length tag_bytes*

Set the allocation tag(s) for memory range [*starting_address*, *starting_address* + *length*) to *tag_bytes*.
- memory-tag check *pointer_expression*

Check if the logical tag in the pointer given by *pointer_expression* matches the allocation tag for the memory referenced by the pointer.

This essentially emulates the hardware validation that is done when tagged memory is accessed through a pointer, but does not cause a memory fault as it would during hardware validation.

It can be used to inspect potential memory tagging violations in the running process, before any faults get triggered.

10.7 内存标记

内存标记是一种内存保护技术，它使用一对标记来验证通过指针进行的内存访问。这些标记是整数值，通常由几个位组成，具体取决于架构。

在此设置中，有两种类型的标记被使用：逻辑标记和分配标记。逻辑标记存储在指针本身中，通常位于指针的高位。分配标记是与物理地址空间中特定内存范围关联的标记，逻辑标记会与这些指针进行比较。

指针标记（逻辑标记）必须与内存标记（分配标记）匹配，内存访问才有效。如果逻辑标记与分配标记不匹配，就会引发内存违规。

分配标记覆盖多个连续的物理内存字节。这个字节的范围称为内存标记粒度，它是架构特定的。例如，AArch64有一个16字节的标记粒度，这意味着每个分配标记跨越16字节的内存。

如果底层架构支持内存标记，比如AArch64 MTE或SPARC ADI，GDB 可以利用它来验证指针与内存分配标记。

打印（参见第 10 [数据], 章第143页）和 `x`（参见第10.6 [内存],节第154页）命令在适当情况下会显示标签信息，而 `memory-tag` 命令前缀可用于访问各种内存标签命令。

内存标签命令如下：

- memory-tag print-logical-tag *pointer_expression*

Print the logical tag stored in *pointer_expression*.
- memory-tag with-logical-tag *pointer_expression tag_bytes*

打印由指针表达式给出的指针，并附加一个由标签字节组成的逻辑标签。
- memory-tag print-allocation-tag *address_expression*

打印与给定内存地址（由地址表达式指定）关联的分配标签。
- memory-tag 设置分配标签 *starting_address length tag_bytes*

为内存范围 [*起始地址*设置分配标签（s）+ *长度*）以标签字节。
- memory-tag 检查 *pointer_expression*

检查指针表达式给出的指针中的逻辑标签是否与指针引用的内存的分配标签匹配。

这本质上模拟了通过指针访问带标签内存时硬件执行的有效性检查，但不会像硬件有效性检查期间那样引发内存故障。

它可用于在运行进程中的任何错误触发之前检查潜在的内存标记违规。

10.8 Automatic Display

If you find that you want to print the value of an expression frequently (to see how it changes), you might want to add it to the *automatic display list* so that GDB prints its value each time your program stops. Each expression added to the list is given a number to identify it; to remove an expression from the list, you specify that number. The automatic display looks like this:

```
2: foo = 38
3: bar[5] = (struct hack *) 0x3804
```

This display shows item numbers, expressions and their current values. As with displays you request manually using `x` or `print`, you can specify the output format you prefer; in fact, `display` decides whether to use `print` or `x` depending your format specification—it uses `x` if you specify either the ‘i’ or ‘s’ format, or a unit size; otherwise it uses `print`.

display expr

Add the expression `expr` to the list of expressions to display each time your program stops. See Section 10.1 [Expressions], page 147.

`display` does not repeat if you press `RET` again after using it.

display/fmt expr

For `fmt` specifying only a display format and not a size or count, add the expression `expr` to the auto-display list but arrange to display it each time in the specified format `fmt`. See Section 10.5 [Output Formats], page 152.

display/fmt addr

For `fmt` ‘i’ or ‘s’, or including a unit-size or a number of units, add the expression `addr` as a memory address to be examined each time your program stops. Examining means in effect doing ‘`x/fmt addr`’. See Section 10.6 [Examining Memory], page 154.

For example, ‘`display/i $pc`’ can be helpful, to see the machine instruction about to be executed each time execution stops (‘`$pc`’ is a common name for the program counter; see Section 10.14 [Registers], page 181).

undisplay dnums...

delete display dnums...

Remove items from the list of expressions to display. Specify the numbers of the displays that you want affected with the command argument `dnums`. It can be a single display number, one of the numbers shown in the first field of the ‘`info display`’ display; or it could be a range of display numbers, as in 2-4.

`undisplay` does not repeat if you press `RET` after using it. (Otherwise you would just get the error ‘No display number ...’.)

disable display dnums...

Disable the display of item numbers `dnums`. A disabled display item is not printed automatically, but is not forgotten. It may be enabled again later. Specify the numbers of the displays that you want affected with the command argument `dnums`. It can be a single display number, one of the numbers shown in the first field of the ‘`info display`’ display; or it could be a range of display numbers, as in 2-4.

10.8 自动显示

如果你发现需要频繁打印某个表达式的值（以查看其如何变化），你可能希望将其添加到自动显示列表中，以便 GDB 在你的程序停止时每次都打印其值。列表中添加的每个表达式都会被分配一个编号以进行识别；要从列表中移除一个表达式，你需要指定该编号。自动显示如下：

```
2: foo = 38
3: bar[5] = (struct hack *) 0x3804
```

此显示界面会显示项目编号、表达式及其当前值。与您使用 `x` 或 `print` 手动请求的显示界面一样，您可以指定您偏好的输出格式；实际上，显示界面会根据您的格式规范决定使用 `print` 还是 `x`——如果您指定了 ‘i’ 或 ‘s’ 格式，或单位大小，则使用 `x`；否则使用 `print`。

display expr

将表达式 `expr` 添加到每次程序停止时显示的表达式列表中。参见第 10.1 [表达式], 页 147。

如果您在使用 `display` 后再次按下 `RET`，`display` 不会重复显示。

显示/fmt expr

对于 `fmt` 仅指定显示格式而不指定大小或数量，将表达式 `expr` 添加到自动显示列表中，但每次以指定的格式 `fmt` 显示。参见第 10.5 [输出格式], 页 152。

显示/fmt addr

对于 `fmt` 为 ‘i’ 或 ‘s’，或包含单位大小或单位数量，将表达式 `addr` 作为要每次程序停止时检查的内存地址添加。检查意味着实际上执行 ‘`x/fmt addr`’。参见第 10.6 [检查内存], 页 154。

例如，‘`display/i $pc`’ 很有帮助，可以查看每次执行停止时要执行的机器指令（‘`$pc`’ 是程序计数器的常用名称；参见第 10.14 [寄存器], 页 181）。

undisplay dnums...

删除显示 dnums...

从表达式显示列表中移除项目。使用命令参数 `dnums` 指定要受影响的显示编号。可以是单个显示编号，即 “`info display`” 显示的第一个字段中显示的编号之一；也可以是显示编号范围，如 2-4。

`undisplay` 命令在使用后按 `RET` 键不会重复执行。（否则你只会得到 “没有显示编号 ...” 的错误。）

禁用显示 dnums...

禁用显示编号 `dnums` 的项不会自动打印，但不会被遗忘。它之后可以再次启用。使用命令参数 ‘`dnums`’ 指定您希望影响的显示项编号。可以是单个显示编号，也可以是 “`info display`” 显示的第一字段中显示的编号之一；或者可以是显示编号的范围，例如 2-4。

enable display *dnums*...

Enable display of item numbers *dnums*. It becomes effective once again in auto display of its expression, until you specify otherwise. Specify the numbers of the displays that you want affected with the command argument *dnums*. It can be a single display number, one of the numbers shown in the first field of the ‘info display’ display; or it could be a range of display numbers, as in 2-4.

display Display the current values of the expressions on the list, just as is done when your program stops.

info display

Print the list of expressions previously set up to display automatically, each one with its item number, but without showing the values. This includes disabled expressions, which are marked as such. It also includes expressions which would not be displayed right now because they refer to automatic variables not currently available.

If a display expression refers to local variables, then it does not make sense outside the lexical context for which it was set up. Such an expression is disabled when execution enters a context where one of its variables is not defined. For example, if you give the command **display last_char** while inside a function with an argument **last_char**, GDB displays this argument while your program continues to stop inside that function. When it stops elsewhere—where there is no variable **last_char**—the display is disabled automatically. The next time your program stops where **last_char** is meaningful, you can enable the display expression once again.

10.9 Print Settings

GDB provides the following ways to control how arrays, structures, and symbols are printed. These settings are useful for debugging programs in any language:

set print address

set print address on

GDB prints memory addresses showing the location of stack traces, structure values, pointer values, breakpoints, and so forth, even when it also displays the contents of those addresses. The default is **on**. For example, this is what a stack frame display looks like with **set print address on**:

```
(gdb) f
#0  set_quotes (lq=0x34c78 "<<", rq=0x34c88 ">>")
    at input.c:530
530      if (lquote != def_lquote)
```

set print address off

Do not print addresses when displaying their contents. For example, this is the same stack frame displayed with **set print address off**:

```
(gdb) set print addr off
(gdb) f
#0  set_quotes (lq="<<", rq=">>") at input.c:530
530      if (lquote != def_lquote)
```

You can use ‘**set print address off**’ to eliminate all machine dependent displays from the GDB interface. For example, with **print address off**, you

启用显示 *dnums*...

启用显示项编号*dnums*。在自动显示其表达式时再次生效，直到您指定其他设置。使用命令参数*dnums*指定要受影响的显示编号。它可以是单个显示编号，也可以是“信息显示”显示的第一字段中显示的编号之一；或者它可以是显示编号的范围，如2-4。

display 显示列表上表达式的当前值，就像在您的程序停止时所做的。

info display

打印先前设置用于自动显示的表达式列表，每个表达式附带其项编号，但不显示值。这包括禁用的表达式，这些表达式会标记为禁用。它还包括当前无法显示的表达式，因为它们引用了当前不可用的自动变量。

如果一个显示表达式引用了局部变量，那么在它设置的词法上下文之外就没有意义。当执行进入其中一个变量未定义的上下文时，此类表达式会被禁用。例如，如果您在具有参数*last_char*的函数内给出**display last_char**命令，GDB 将在您的程序继续在该函数内停止时显示此参数。当它在其他地方停止——没有*last_char*变量时——显示会自动禁用。下次您的程序在*last_char*有意义的地方停止时，您可以再次启用显示表达式。

10.9 打印设置

GDB 提供了以下方式来控制数组和结构、符号的打印。这些设置适用于任何语言的程序调试：

设置打印地址

设置打印地址开启

GDB 打印内存地址，显示栈跟踪、结构值、指针值、断点等位置，即使它也显示这些地址的内容。默认是开启的。例如，开启设置打印地址时的栈帧显示如下：

```
(gdb) f #0 set_quotes (lq=0x34c78 "<<", rq=0
x34c88 ">>") at input.c:530 530 if (lquote !=def_
lquote)
```

关闭打印地址

在显示其内容时不要打印地址。例如，这是使用关闭打印地址显示的相同栈帧：

```
(gdb) set print addr off (gdb) f #0 set_quotes
(lq="<<", rq=">>") at input.c:530 530 if (lquote !=def_
lquote)
```

您可以使用 ‘**set print address off**’ 来消除 GDB 接口中的所有与机器相关的显示。例如，在关闭打印地址的情况下，您

should get the same text for backtraces on all machines—whether or not they involve pointer arguments.

show print address

Show whether or not addresses are to be printed.

When GDB prints a symbolic address, it normally prints the closest earlier symbol plus an offset. If that symbol does not uniquely identify the address (for example, it is a name whose scope is a single source file), you may need to clarify. One way to do this is with `info line`, for example ‘`info line *0x4537`’. Alternately, you can set GDB to print the source file and line number when it prints a symbolic address:

set print symbol-filename on

Tell GDB to print the source file name and line number of a symbol in the symbolic form of an address.

set print symbol-filename off

Do not print source file name and line number of a symbol. This is the default.

show print symbol-filename

Show whether or not GDB will print the source file name and line number of a symbol in the symbolic form of an address.

Another situation where it is helpful to show symbol filenames and line numbers is when disassembling code; GDB shows you the line number and source file that corresponds to each instruction.

Also, you may wish to see the symbolic form only if the address being printed is reasonably close to the closest earlier symbol:

set print max-symbolic-offset max-offset

set print max-symbolic-offset unlimited

Tell GDB to only display the symbolic form of an address if the offset between the closest earlier symbol and the address is less than *max-offset*. The default is `unlimited`, which tells GDB to always print the symbolic form of an address if any symbol precedes it. Zero is equivalent to `unlimited`.

show print max-symbolic-offset

Ask how large the maximum offset is that GDB prints in a symbolic address.

If you have a pointer and you are not sure where it points, try ‘`set print symbol-filename on`’. Then you can determine the name and source file location of the variable where it points, using ‘`p/a pointer`’. This interprets the address in symbolic form. For example, here GDB shows that a variable `ptt` points at another variable `t`, defined in `hi2.c`:

```
(gdb) set print symbol-filename on
(gdb) p/a ptt
$4 = 0xe008 <t in hi2.c>
```

Warning: For pointers that point to a local variable, ‘`p/a`’ does not show the symbol name and filename of the referent, even with the appropriate `set print` options turned on.

应该在所有机器上获得相同的文本回溯信息——无论是否涉及指针参数。

显示打印地址

显示是否要打印地址。

当 GDB 打印符号地址时，通常会打印最近的先前符号加上偏移量。如果该符号不能唯一标识地址（例如，其作用域仅限于单个源文件），您可能需要澄清。一种方法是通过 `info` 行，例如 “`info 行 *0x4537`”。或者，您可以设置 GDB 在打印符号地址时打印源文件和行号：

设置打印符号-文件名为开启

告诉 GDB 以符号地址形式的地址打印符号的源文件名和行号。

关闭打印符号文件名

不打印符号的源文件名和行号。这是默认设置。

显示打印符号文件名

显示 GDB 是否会在地址的符号形式中打印符号的源文件名和行号。

另一个显示符号文件名和行号很有用的场景是反汇编代码；GDB 会显示与每条指令对应的行号和源文件。

此外，如果您希望在打印的地址与最近的早期符号合理接近时仅显示符号形式，也可以这样做：

设置 print max-symbolic-offset max-offset

set print max-symbolic-offset unlimited

告诉 GDB，只有当地址与最近的先前符号之间的偏移量小于 `max-offset` 时才显示符号形式。默认为无限，这告诉 GDB，如果任何符号在它之前，则始终打印地址的符号形式。零等于无限。

显示 print max-symbolic-offset

询问 GDB 在符号地址中打印的最大偏移量是多少。

如果你有一个指针并且不确定它指向哪里，尝试 ‘`set print symbol-filename on`’。然后你可以使用 ‘`p/a pointer`’ 来确定它指向的变量的名称和源文件位置，该地址以符号形式解释。例如，这里 GDB 显示变量 `ptt` 指向另一个在 `hi2.c` 中定义的变量 `t`：

```
(gdb) set print symbol-filename on
(gdb) p/a ptt
$4 = 0xe008 <t in hi2.c>
```

*警告：*对于指向局部变量的指针，即使启用了适当的打印选项，‘`p/a`’ 也不会显示引用对象的符号名和文件名。

You can also enable ‘/a’-like formatting all the time using ‘set print symbol on’:

set print symbol on
Tell GDB to print the symbol corresponding to an address, if one exists.

set print symbol off
Tell GDB not to print the symbol corresponding to an address. In this mode, GDB will still print the symbol corresponding to pointers to functions. This is the default.

show print symbol
Show whether GDB will display the symbol corresponding to an address.

Other settings control how different kinds of objects are printed:

set print array
set print array on
Pretty print arrays. This format is more convenient to read, but uses more space. The default is off.

set print array off
Return to compressed format for arrays.

show print array
Show whether compressed or pretty format is selected for displaying arrays.

set print array-indexes
set print array-indexes on
Print the index of each element when displaying arrays. May be more convenient to locate a given element in the array or quickly find the index of a given element in that printed array. The default is off.

set print array-indexes off
Stop printing element indexes when displaying arrays.

show print array-indexes
Show whether the index of each element is printed when displaying arrays.

set print nibbles
set print nibbles on
Print binary values in groups of four bits, known as *nibbles*, when using the print command of GDB with the option ‘/t’. For example, this is what it looks like with **set print nibbles on**:

```
(gdb) print val_flags
$1 = 1230
(gdb) print/t val_flags
$2 = 0100 1100 1110
```

set print nibbles off
Don't printing binary values in groups. This is the default.

show print nibbles
Show whether to print binary values in groups of four bits.

您也可以始终使用 ‘set print symbol on’ 启用类似 ‘/a’ 的格式化：

设置打印符号
告诉 GDB 打印与地址对应的符号（如果存在）。

关闭打印符号
告诉 GDB 不打印与地址对应的符号。在此模式下，GDB 仍会打印指向函数的指针对应的符号。这是默认设置。

显示打印符号
显示 GDB 是否将显示对应地址的符号。

其他设置控制不同类型的对象如何打印：

设置打印数组
设置打印数组开启
美化打印数组。这种格式更方便阅读，但会占用更多空间。默认关闭。

设置打印数组关闭
返回数组压缩格式。

显示打印数组
显示数组时选择压缩格式还是美化格式。

设置打印数组索引
设置打印数组索引开启
显示数组时打印每个元素的索引。在数组中定位给定元素或快速找到该打印数组中给定元素的索引可能更方便。默认关闭。

设置关闭打印数组索引
在显示数组时停止打印元素索引。

显示打印数组索引
显示在显示数组时是否打印每个元素的索引。

设置打印字节
设置打印字节开启
使用 GDB 的打印命令并选择 ‘/t’ 选项时，以四位二进制值（称为字节）分组打印。例如，启用打印字节时的显示效果如下：

```
(gdb) print val_flags
$1 = 1230
(gdb) print/t val_flags
$2 = 0100 1100 1110
```

关闭打印字节
不按字节分组打印二进制值。这是默认设置。

显示打印字节
显示是否以四位二进制值为一组进行打印。

`set print characters number-of-characters`
`set print characters elements`
`set print characters unlimited`

Set a limit on how many characters of a string GDB will print. If GDB is printing a large string, it stops printing after it has printed the number of characters set by the `set print characters` command. This equally applies to multi-byte and wide character strings, that is for strings whose character type is `wchar_t`, `char16_t`, or `char32_t` it is the number of actual characters rather than underlying bytes the encoding uses that this setting controls. Setting *number-of-characters* to `elements` means that the limit on the number of characters to print follows one for array elements; see [set print elements], page 162. Setting *number-of-characters* to `unlimited` means that the number of characters to print is unlimited. When GDB starts, this limit is set to `elements`.

`show print characters`

Display the number of characters of a large string that GDB will print.

`set print elements number-of-elements`
`set print elements unlimited`

Set a limit on how many elements of an array GDB will print. If GDB is printing a large array, it stops printing after it has printed the number of elements set by the `set print elements` command. By default this limit also applies to the display of strings; see [set print characters], page 161. When GDB starts, this limit is set to 200. Setting *number-of-elements* to `unlimited` or zero means that the number of elements to print is unlimited.

When printing very large arrays, whose size is greater than `max-value-size` (see [max-value-size], page 195), if the `print elements` is set such that the size of the elements being printed is less than or equal to `max-value-size`, then GDB will print the array (up to the `print elements` limit), and only `max-value-size` worth of data will be added into the value history (see Section 10.11 [Value History], page 172).

`show print elements`

Display the number of elements of a large array that GDB will print.

`set print frame-arguments value`

This command allows to control how the values of arguments are printed when the debugger prints a frame (see Section 8.1 [Frames], page 115). The possible values are:

`all` The values of all arguments are printed.

`scalars` Print the value of an argument only if it is a scalar. The value of more complex arguments such as arrays, structures, unions, etc, is replaced by `...`. This is the default. Here is an example where only scalar arguments are shown:

```
#1 0x08048361 in call_me (i=3, s=..., ss=0xbf8d508c, u=..., e=green)
    at frame-args.c:23
```

设置打印字符 *number-of-characters*
设置打印字符元素
设置打印字符不受限制

设置字符串打印字符数量的上限。如果 GDB 正在打印大字符串，它会在打印了由 `set print characters` 命令设置的字符数量后停止打印。这同样适用于多字节和宽字符字符串，即对于字符类型为 `wchar_t`、`char16_t` 或 `char32_t` 的字符串，此设置控制的是实际字符数量而不是该编码使用的底层字节数量。将字符数量设置为 `elements` 表示打印字符数量的上限遵循数组元素的规则；参见 [set print elements], 页162。将字符数量设置为 `unlimited` 表示打印字符数量无限制。当 GDB 启动时，此限制被设置为 `elements`。

显示打印字符

显示 GDB 将打印的大字符串的字符数量。

设置打印元素 *number-of-elements*
设置打印元素无限制

设置数组 GDB 打印元素数量的限制。如果 GDB 正在打印大型数组，则在打印到由 `set print elements` 命令设置的元素数量后停止打印。默认情况下，此限制也适用于字符串的显示；参见 [set print characters], 页161。当 GDB 启动时，此限制设置为 200。将 *number-of-elements* 设置为无限制或零表示打印元素的数量无限制。

当打印非常大的数组，其大小大于 `max-value-size`（参见 [max-value-size], 页195）时，如果 `print elements` 设置得使得打印的元素大小小于或等于 `max-value-size`，那么 GDB 将打印数组（最多到 `print elements` 限制），并且只有 `max-value-size` 大小的数据将被添加到值历史记录中（参见第10.11 [值历史记录], 页172）。

显示打印元素

显示大数组中将要打印的元素数量。

设置打印帧参数 *value*

此命令允许控制当调试器打印帧时参数值的打印方式（参见第8.1 [帧], 页115）。可能的值有：

`all` 所有参数的值都会被打印。

`scalars` 仅当参数是标量时才打印其值。更复杂的参数（如数组、结构、联合等）的值会被替换为 `...`。这是默认行为。这里是一个示例，

仅显示标量参数：

```
#1 0x08048361 在调用_ (i=3, s=..., ss=0xbf8d508c, u=..., e=green) 时，位于
    frame-args.c:23
```